

Theorie V01–V20

Datei: lessons/V01-Binaeres-Zahlensystem/V01-Binaeres-Zahlensystem_skript.md

V01: Binäres Zahlensystem

[!NOTE] **Lernziele dieser Vorlesung:** - Stellenwertsysteme verstehen und zwischen Dezimal-, Binär-, Hexadezimal- und Oktal-System umrechnen können - Binäre Rechenoperationen (Addition, Subtraktion) durchführen können - Das Zweierkomplement zur Darstellung negativer Zahlen verstehen und anwenden - Bit-Masken und grundlegende Bit-Manipulation beherrschen - Python installieren und die erste Programme mit `print()` und `input()` schreiben - Variablen in Python deklarieren, zuweisen und verwenden - Kommentare zur Code-Dokumentation einsetzen

Teil 1: Theorie - Binäres Zahlensystem

Überblick

Computer arbeiten ausschließlich mit elektrischen Signalen, die nur zwei Zustände kennen: Strom fließt oder Strom fließt nicht. Diese beiden Zustände werden als **Bit** mit den Werten 0 und 1 repräsentiert. Um mit Computern arbeiten zu können, müssen wir verstehen, wie Zahlen in diesem binären System dargestellt werden. Das **binäre Zahlensystem** bildet die fundamentale Grundlage der gesamten digitalen Datenverarbeitung. Ohne dieses Verständnis ist es unmöglich, die Funktionsweise von Prozessoren, Speichern und Algorithmen vollständig zu erfassen.

[!NOTE] Ein **Bit** (Binary Digit) ist die kleinste Informationseinheit in der digitalen Datenverarbeitung und kann genau zwei Zustände annehmen: 0 oder 1.

In der Praxis arbeiten Ingenieure häufig mit verschiedenen Zahlensystemen, da bestimmte Systeme für bestimmte Anwendungen besser geeignet sind. Hexadezimal-Notation vereinfacht beispielsweise die Darstellung von Speicheradressen und Farbcodes erheblich.

Stellenwertsysteme allgemein

Menschen verwenden im Alltag das **Dezimalsystem** (Basis 10), weil wir zehn Finger haben. Computer nutzen das **Binärsystem** (Basis 2), weil elektronische Schaltungen nur zwei stabile Zustände haben. Das Prinzip bleibt jedoch identisch: Jede Position einer Zahl repräsentiert eine Potenz der Basis.

[!NOTE] Ein **Stellenwertsystem** ist eine Methode zur Darstellung von Zahlen, bei der jede Ziffer einen Wert hat, der von ihrer Position abhängt. Der Wert einer Position wird durch eine Potenz der Basis bestimmt.

Im Dezimalsystem hat die Zahl **2468** folgende Struktur:

$$\begin{aligned} 2468_{10} &= 2 \cdot 10^3 + 4 \cdot 10^2 + 6 \cdot 10^1 + 8 \cdot 10^0 \\ &= 2000 + 400 + 60 + 8 = 2468 \end{aligned}$$

Das gleiche Prinzip gilt für alle anderen Stellenwertsysteme. Die wichtigsten für die Informatik sind:

- **Binärsystem (Basis 2)**: Ziffern 0, 1
- **Oktalsystem (Basis 8)**: Ziffern 0-7
- **Dezimalsystem (Basis 10)**: Ziffern 0-9
- **Hexadezimalsystem (Basis 16)**: Ziffern 0-9, A-F (wobei A=10, B=11, C=12, D=13, E=14, F=15)

[!TIP] **Beispiel: Die binäre Zahl 1011 im Detail**

$$\begin{aligned} 1011_2 &= 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 \\ &= 8 + 0 + 2 + 1 = 11_{10} \end{aligned}$$

Die Zweierpotenzen von rechts nach links: 1, 2, 4, 8, 16, 32, 64, 128, 256, ...

[!WARNING] **Häufiger Fehler bei der Notation**: Ohne Angabe der Basis ist unklar, welches Zahlensystem gemeint ist. Die Zahl "101" könnte 101 (einhunderteins im Dezimalsystem) oder 101 (fünf im Binärsystem) bedeuten. Verwende daher immer einen Index zur Kennzeichnung der Basis (z.B. für binär, für dezimal, für hexadezimal) oder ein Präfix (0b für binär, 0x für hexadezimal in vielen Programmiersprachen).

Umrechnung zwischen verschiedenen Zahlensystemen

Die Fähigkeit, schnell zwischen verschiedenen Zahlensystemen umzurechnen, ist eine zentrale Kompetenz für Informatiker und Ingenieure. Dabei gibt es systematische Verfahren, die sich auf alle Basis-Konvertierungen anwenden lassen.

Binär → Dezimal Die Umrechnung von binär nach dezimal erfolgt durch Berechnung der Stellenwerte. Multipliziere jede Binärziffer mit der entsprechenden Zweierpotenz und addiere alle Ergebnisse.

[!TIP] **Beispiel: 11010110 → Dezimal**

Position	7	6	5	4	3	2	1	0
Binärziffer	1	1	0	1	0	1	1	0
Zweierpotenz	2	2	2	2	2 ³	2 ²	2 ¹	2
Wert	128	64	0	16	0	4	2	0

Summe: $128 + 64 + 16 + 4 + 2 = 214$

Dezimal → Binär Die Umrechnung von dezimal nach binär erfolgt durch wiederholte Division durch 2 und Notieren der Reste. Die Reste ergeben von unten nach oben gelesen die Binärdarstellung.

[!TIP] **Beispiel: 214 → Binär (Divisionsverfahren)**

Die Reste von unten nach oben gelesen: **11010110**

Hexadezimal Binär Die Umrechnung zwischen Hexadezimal und Binär ist besonders einfach, weil eine Hexadezimalziffer genau vier Binärziffern entspricht (da $16 = 2^4$).

[!NOTE] **Hexadezimal-Binär Zuordnung:** Jede Hexadezimalziffer repräsentiert exakt 4 Bit (eine sogenannte **Nibble**). Diese 1:4 Beziehung macht Konvertierungen trivial.

[!TIP] **Hexadezimal-Binär Tabelle**

Hex	Binär	Dezimal	Hex	Binär	Dezimal
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

Beispiel Hex → Binär: 3A7F

- 3 = 0011
- A = 1010

- $7 = 0111$
- $F = 1111$

Ergebnis: **0011 1010 0111 1111**

Binäre Rechenoperationen

Computer führen alle Rechenoperationen binär durch. Die grundlegenden Operationen Addition und Subtraktion funktionieren nach den gleichen Prinzipien wie im Dezimalsystem, nur mit der Basis 2.

Binäre Addition Die binäre Addition folgt einfachen Regeln. Wenn die Summe zweier Bits größer oder gleich 2 ist, entsteht ein **Übertrag** (Carry) zur nächsthöheren Stelle.

[!NOTE] **Additionsregeln im Binärsystem:** $0 + 0 = 0$ - $0 + 1 = 1$ - $1 + 0 = 1$ - $1 + 1 = 10$ (0 mit Übertrag 1) - $1 + 1 + 1$ (mit Übertrag) = 11 (1 mit Übertrag 1)

[!TIP] **Beispiel: 10110 + 01011**

Schrittweise von rechts nach links: - Position 0: $0 + 1 = 1$ - Position 1: $1 + 1 = 0$, Übertrag 1 - Position 2: $1 + 0 + \text{Übertrag } 1 = 0$, Übertrag 1 - Position 3: $0 + 1 + \text{Übertrag } 1 = 0$, Übertrag 1 - Position 4: $1 + 0 + \text{Übertrag } 1 = 0$, Übertrag 1 - Position 5: Übertrag 1

Ergebnis: **100001** (Dezimal: $22 + 11 = 33$)

Binäre Subtraktion Die binäre Subtraktion funktioniert analog zur dezimalen Subtraktion mit Borgen (Borrow) von höherwertigen Stellen.

[!NOTE] **Subtraktionsregeln im Binärsystem:** $0 - 0 = 0$ - $1 - 0 = 1$ - $0 - 1 = 0 - 0 - 1 = 1$ mit Borgen von 1 von der nächsthöheren Stelle - $10 - 1 = 1$

[!TIP] **Beispiel: 10110 - 01011**

Ergebnis: **01011** (Dezimal: $22 - 11 = 11$)

[!WARNING] **Negative Ergebnisse:** Die einfache Subtraktion kann keine negativen Zahlen darstellen. Wenn der Subtrahend größer ist als der Minuend, benötigen wir ein System zur Darstellung negativer Zahlen. Dafür verwendet man in der Informatik das **Zweierkomplement**, das im nächsten Abschnitt behandelt wird.

Negative Zahlen im Computer: Das Zweierkomplement

Computer müssen sowohl positive als auch negative Zahlen darstellen können. Das **Zweierkomplement** ist die Standardmethode zur Darstellung negativer Zahlen in Computern, weil es arithmetische Operationen extrem vereinfacht.

[!NOTE] Das **Zweierkomplement** ist eine Methode zur Darstellung negativer ganzer Zahlen in binärer Form. Das höchstwertige Bit (MSB - Most Significant Bit) dient als Vorzeichenbit: 0 bedeutet positiv, 1 bedeutet negativ. Diese Darstellung ermöglicht es, Subtraktion als Addition mit negativen Zahlen durchzuführen, was die Hardware-Implementierung erheblich vereinfacht.

Bildung des Zweierkomplements Um das Zweierkomplement einer Zahl zu bilden, gibt es zwei äquivalente Methoden:

Methode 1: Invertieren und 1 addieren 1. Invertiere alle Bits ($0 \rightarrow 1$, $1 \rightarrow 0$) 2. Addiere 1 zum Ergebnis

Methode 2: Von rechts lesen 1. Kopiere alle Bits von rechts bis einschließlich der ersten 1 2. Invertiere alle übrigen Bits links davon

[!TIP] **Beispiel: Zweierkomplement von 5 (bei 8 Bit)**

Positive Darstellung: 00000101

Methode 1: - Invertieren: 11111010 - Plus 1: 11111011

Ergebnis: 11111011 repräsentiert **-5** im Zweierkomplement

Probe: 00000101 + 11111011 = 100000000

Der 9. Bit (Überlauf) wird ignoriert $\rightarrow 00000000 = 0$

Wertebereich beim Zweierkomplement Mit n Bits können im Zweierkomplement Zahlen von -2^{n-1} bis $2^{n-1} - 1$ dargestellt werden.

[!TIP] **Wertebereiche nach Bit-Anzahl**

Bit-Anzahl	Von	Bis	Anzahl Werte
4 Bit	-8	+7	16
8 Bit	-128	+127	256
16 Bit	-32.768	+32.767	65.536
32 Bit	-	+2.147.483.648	4,3 Mrd.
64 Bit	-	+9.223.372.036.854.775.808	18,4 Mrd.

[!WARNING] **Integer Overflow:** Wenn das Ergebnis einer Operation den darstellbaren Wertebereich überschreitet, tritt ein **Überlauf** (Overflow) auf. Bei 8-Bit-Zahlen im Zweierkomplement würde $127 + 1 = -128$ ergeben, was logisch falsch ist. Moderne Programmiersprachen wie Python verwenden automatisch größere Datentypen, aber in Sprachen wie C/C++ oder in Hardware-naher Programmierung muss der Programmierer explizit auf Überläufe achten.

Praktische Anwendung: Bit-Masken und Bit-Manipulation

Bit-Manipulation bezeichnet Operationen auf einzelnen Bits oder Bit-Gruppen. Diese Techniken sind in der System-Programmierung, Embedded Systems und Performance-kritischen Anwendungen unverzichtbar.

[!NOTE] Eine **Bit-Maske** ist ein Bitmuster, das verwendet wird, um bestimmte Bits zu isolieren, zu setzen oder zu löschen. Bit-Masken nutzen die logischen Bit-Operatoren AND, OR, XOR und NOT.

Bitweise Operatoren Die wichtigsten bitweisen Operatoren sind:

- **AND (&):** Beide Bits müssen 1 sein $\rightarrow$ Ergebnis 1
- **OR (|):** Mindestens ein Bit muss 1 sein $\rightarrow$ Ergebnis 1
- **XOR (^):** Genau ein Bit muss 1 sein $\rightarrow$ Ergebnis 1
- **NOT (~):** Invertiert alle Bits
- **Left Shift («):** Verschiebt Bits nach links (entspricht Multiplikation mit 2)
- **Right Shift (»):** Verschiebt Bits nach rechts (entspricht Division durch 2)

[!TIP] **Beispiel: Typische Anwendungen von Bit-Operationen**

1. Bestimmtes Bit prüfen (Bit n lesen)
2. Bestimmtes Bit setzen (Bit n auf 1 setzen)
3. Bestimmtes Bit löschen (Bit n auf 0 setzen)
4. Bestimmtes Bit umschalten (Toggle)

[!TIP] **Beispiel: Shift-Operationen**

Left Shift (Multiplikation)

Right Shift (Division)

[!WARNING] **Vorzeichenbehaftete Right Shifts:** Bei vorzeichenbehafteten Zahlen (signed integers) gibt es zwei Arten von Right Shift: - **Logischer Shift:** Füllt links mit 0 auf - **Arithmetischer Shift:** Füllt links mit dem Vorzeichenbit auf (erhält das Vorzeichen)

In Python ist der Right Shift immer arithmetisch für negative Zahlen.
In C/C++ hängt das Verhalten vom Compiler ab.

Zusammenfassung Theorie

Das binäre Zahlensystem bildet die Grundlage der digitalen Datenverarbeitung. Computer arbeiten ausschließlich mit Bits (0 und 1), weil elektronische Schaltungen nur zwei stabile Zustände kennen. Die Umrechnung zwischen verschiedenen Stellenwertsystemen (binär, dezimal, hexadezimal, oktal) ist eine fundamentale

Fähigkeit für jeden Informatiker. Besonders die Hexadezimal-Darstellung ist in der Praxis wichtig, da sie eine kompakte Schreibweise für Binärzahlen bietet.

Binäre Rechenoperationen funktionieren nach den gleichen Prinzipien wie im Dezimalsystem, nur mit der Basis 2. Das Zweierkomplement ist die Standardmethode zur Darstellung negativer Zahlen in Computern, da es arithmetische Operationen vereinfacht und keine separate Subtraktions-Hardware erfordert. Bit-Manipulation mit Masken und bitweisen Operatoren ist ein mächtiges Werkzeug für effiziente Programmierung, besonders in hardwarenahen Anwendungen.

Teil 2: Python-Praxis - Python Get Started

[!WARNING] **Python-Konsistenz beachten:** Prüfe ../../python_topics.md für bereits eingeführte Konzepte!

Überblick

Python ist eine der beliebtesten Programmiersprachen weltweit und besonders für Anfänger geeignet. Die Syntax ist klar und lesbar, was Python zur idealen Sprache für den Einstieg in die Programmierung macht. In dieser ersten Python-Lektion lernen wir die Grundlagen: Installation, erste Programme und die fundamentalen Konzepte Variablen, Ein- und Ausgabe.

Python wird in vielen Bereichen eingesetzt: Datenanalyse, Maschinelles Lernen, Webentwicklung, Automatisierung, wissenschaftliches Rechnen und vieles mehr. Als Maschinenbau-Ingenieur wirst du Python häufig für Berechnungen, Simulationen und Datenauswertung verwenden.

[!NOTE] **Python** ist eine interpretierte, objektorientierte Hochsprache mit dynamischer Typisierung. “Interpretiert” bedeutet, dass Python-Code nicht kompiliert werden muss, sondern direkt ausgeführt wird. “Dynamische Typisierung” bedeutet, dass Variablen zur Laufzeit ihren Typ ändern können.

Installation und Entwicklungsumgebungen

Bevor wir programmieren können, müssen wir Python installieren. Die aktuelle Version ist Python 3.12 (Stand Januar 2026). Python 2 ist veraltet und sollte nicht mehr verwendet werden.

Python installieren Windows: 1. Besuche python.org/downloads 2. Lade den neuesten Python 3 Installer herunter 3. Führe den Installer aus und aktiviere die Option “Add Python to PATH” 4. Überprüfe die Installation mit `python --version` in der Kommandozeile

macOS: Python 3 ist auf modernen macOS-Versionen vorinstalliert. Falls nicht, verwende Homebrew:

Linux: Python 3 ist auf den meisten Linux-Distributionen vorinstalliert:

Falls nicht installiert:

Entwicklungsumgebungen Für die Programmierung in Python gibt es verschiedene Entwicklungsumgebungen (IDEs):

IDLE (Python's Integrated Development and Learning Environment) - Kommt mit Python-Installation - Einfach für Anfänger - Begrenzte Funktionalität

Visual Studio Code (VS Code) - Kostenloser, moderner Code-Editor von Microsoft - Exzellente Python-Unterstützung mit Erweiterungen - Empfohlen für diese Vorlesung

PyCharm - Professionelle IDE speziell für Python - Community Edition ist kostenlos - Viele fortgeschrittene Features

Jupyter Notebook - Interaktive Umgebung für Datenanalyse - Mischt Code, Visualisierungen und Text - Besonders für wissenschaftliche Anwendungen

[!TIP] Für diese Vorlesung empfehlen wir **Visual Studio Code** mit der Python-Erweiterung. VS Code bietet eine gute Balance zwischen Einfachheit und Funktionalität.

Erste Programme: Ausgabe mit `print()`

Das erste Programm, das man in jeder Programmiersprache schreibt, ist traditionell "Hello, World!". Es demonstriert die grundlegendste Funktionalität: Text ausgeben.

[!NOTE] Die `print()`-Funktion gibt Text oder Werte auf der Konsole (Terminal) aus. Sie ist eine der wichtigsten Funktionen für Debugging und Benutzerkommunikation.

[!TIP] **Beispiel: Das erste Python-Programm**

Ausgabe:

Die `print()`-Funktion kann verschiedene Datentypen ausgeben und mehrere Argumente verarbeiten:

[!TIP] **Beispiel: Verschiedene Verwendungen von `print()`**

Ausgabe:

[!NOTE] **String (Zeichenkette):** Ein String ist eine Sequenz von Zeichen, die in Anführungszeichen eingeschlossen ist. In Python können sowohl einfache ('...') als auch doppelte ("...") Anführungszeichen verwendet werden. Beide sind äquivalent, aber konsistente Verwendung verbessert die Lesbarkeit.

[!WARNING] **Syntaxfehler bei fehlenden Anführungszeichen:**
Wenn du Text ohne Anführungszeichen verwendest, interpretiert Python dies als Variablennamen oder Schlüsselwort. Dies führt zu einem Fehler:

Variablen deklarieren und zuweisen

Variablen sind benannte Speicherplätze im Arbeitsspeicher, die Werte aufnehmen können. Sie sind fundamental für jede Art von Programmierung, da sie es ermöglichen, Daten zu speichern und später wiederzuverwenden.

[!NOTE] Eine **Variable** ist ein symbolischer Name für einen Wert im Speicher. In Python wird eine Variable durch Zuweisung mit dem Gleichheitszeichen (=) erstellt. Python verwendet **dynamische Typisierung**, was bedeutet, dass der Typ einer Variable automatisch aus dem zugewiesenen Wert abgeleitet wird.

[!TIP] **Beispiel: Variablen erstellen und verwenden**

Ausgabe:

Regeln für Variablennamen Python hat klare Regeln und Konventionen für Variablennamen:

Obligatorische Regeln (Syntaxvorgaben): - Namen dürfen nur Buchstaben (a-z, A-Z), Ziffern (0-9) und Unterstriche (_) enthalten - Namen dürfen nicht mit einer Ziffer beginnen - Namen dürfen keine Python-Schlüsselwörter sein (z.B. `if`, `for`, `while`, `def`) - Groß- und Kleinschreibung wird unterschieden (`alter` `Alter`)

Konventionen (PEP 8 - Python Style Guide): - Verwende aussagekräftige Namen: `geschwindigkeit` statt `g` - Verwende `snake_case` für Variablennamen: `maximale_geschwindigkeit` - Vermeide einzelne Buchstaben außer für Zähler (`i`, `j`, `k`) oder Koordinaten (`x`, `y`, `z`) - Verwende englische Namen für Code, der geteilt wird

[!TIP] **Beispiel: Gültige und ungültige Variablennamen**

[!WARNING] **Überschreiben von Built-in-Namen:** Python hat vordefinierte Funktionen wie `print`, `len`, `sum` etc. Verwende diese Namen nicht als Variablen, da du sonst die ursprüngliche Funktion überschreibst:

Variablen neu zuweisen und überschreiben Variablen können jederzeit neue Werte zugewiesen bekommen. Der alte Wert wird dabei überschrieben.

[!TIP> **Beispiel: Variablen aktualisieren**

Ausgabe:

[!NOTE] **Zuweisungsoperator vs. Gleichheit:** Das Gleichheitszeichen (=) ist in Python der **Zuweisungsoperator**, nicht das mathematische Gleichheitszeichen. `x = 5` bedeutet “weise der Variable `x` den Wert 5 zu”, nicht “`x` ist gleich 5”. Für Gleichheitsvergleiche verwendet man den `==` Operator (wird in V04 behandelt).

Benutzereingaben mit `input()` einlesen

Programme werden erst dann wirklich nützlich, wenn sie mit Benutzern interagieren können. Die `input()`-Funktion ermöglicht es, Daten vom Benutzer einzulesen.

[!NOTE] Die `input()`-Funktion pausiert die Programmausführung und wartet auf eine Eingabe des Benutzers. Der Benutzer tippt Text ein und bestätigt mit Enter. Die Funktion gibt den eingegebenen Text als String zurück.

[!TIP] **Beispiel: Einfache Benutzereingabe**

Interaktion:

[!WARNING] `input()` gibt immer einen String zurück: Auch wenn der Benutzer eine Zahl eingibt, wird diese als String (Text) zurückgegeben. Für Berechnungen muss der String in eine Zahl konvertiert werden:

[!NOTE] **Type Casting (Typkonvertierung):** Python bietet Funktionen zur Konvertierung zwischen Datentypen: - `int()` konvertiert zu Ganzzahl (Integer) - `float()` konvertiert zu Fließkommazahl - `str()` konvertiert zu String (Text)

Diese Funktionen werden ausführlich in V03 behandelt.

[!TIP] **Beispiel: Praktisches Programm mit Eingabe und Berechnung**

Interaktion:

Kommentare für lesbaren Code

Kommentare sind Textzeilen im Code, die vom Python-Interpreter ignoriert werden. Sie dienen ausschließlich der Dokumentation und helfen Menschen, den Code zu verstehen.

[!NOTE] **Kommentare** sind Erklärungen im Code für menschliche Leser, die von Python nicht ausgeführt werden. Einzeilige Kommentare beginnen mit `#`, mehrzeilige Kommentare werden mit dreifachen Anführungszeichen (`"""..."""` oder `'''...'''`) erstellt.

[!TIP] **Beispiel: Verschiedene Arten von Kommentaren**

[!WARNING] **Kommentare sinnvoll einsetzen:** Kommentare sollten erklären **warum** etwas getan wird, nicht **was** getan wird (das sieht man am Code). Schlechte Kommentare sind redundant oder veraltet:

Häufige Fehler und Lösungen

[!WARNING] **Fehler 1: Fehlende Anführungszeichen bei Strings**

Lösung: Strings müssen immer in Anführungszeichen stehen.

[!WARNING] **Fehler 2: Variablenname mit Ziffer beginnen**

Lösung: Variablennamen dürfen nicht mit einer Ziffer beginnen.

[!WARNING] **Fehler 3: Vergessen, `input()` in Zahl zu konvertieren**

Lösung: Verwende `int()` oder `float()` um Eingaben in Zahlen zu konvertieren.

[!WARNING] **Fehler 4: Variablen vor der Zuweisung verwenden**

Lösung: Variablen müssen vor ihrer Verwendung einen Wert zugewiesen bekommen haben.

[!WARNING] **Fehler 5: Tippfehler bei Variablennamen**

Lösung: Achte auf exakte Schreibweise. Moderne IDEs wie VS Code markieren unbekannte Variablennamen.

Zusammenfassung Python

Python ist eine anfängerfreundliche, aber mächtige Programmiersprache, die in vielen Bereichen eingesetzt wird. Die Installation ist einfach, und moderne IDEs wie Visual Studio Code bieten exzellente Unterstützung. Die `print()`-Funktion ermöglicht Ausgaben auf der Konsole und ist essentiell für Debugging und Benutzerinteraktion.

Variablen sind benannte Speicherplätze, die Werte aufnehmen können. Python verwendet dynamische Typisierung, sodass Variablentypen automatisch erkannt werden. Variablennamen müssen bestimmte Regeln befolgen und sollten aussagekräftig gewählt werden. Die `input()`-Funktion liest Benutzereingaben als String ein. Für numerische Berechnungen muss der String mit `int()` oder `float()` konvertiert werden.

Kommentare dokumentieren Code für menschliche Leser und werden von Python ignoriert. Gute Kommentare erklären das “Warum”, nicht das “Was”. Sie sind essentiell für wartbaren, verständlichen Code, besonders in Team-Projekten.

Neue Python-Funktionen/Methoden

Die folgenden Python-Funktionen und Konzepte wurden in dieser Lektion **erstmals eingeführt**:

Built-in Functions

- **print(*objects, sep=' ', end='\n')** (Python 3.0+)
 - Gibt Werte auf der Standardausgabe (Konsole) aus
 - Mehrere Argumente werden durch **sep** (Standard: Leerzeichen) getrennt
 - Nach der Ausgabe wird **end** angehängt (Standard: Zeilenumbruch)
- **input(prompt='')** (Python 3.0+)
 - Liest eine Zeile Text von der Standardeingabe (Tastatur)
 - Zeigt optional einen Prompt-Text an
 - Gibt immer einen String zurück
- **int(x)** (Built-in)
 - Konvertiert einen Wert in eine Ganzzahl (Integer)
 - Kann Strings und Floats konvertieren
 - Wirft **ValueError** bei ungültiger Konvertierung
- **float(x)** (Built-in)
 - Konvertiert einen Wert in eine Fließkommazahl
 - Kann Strings und Integers konvertieren
 - Wirft **ValueError** bei ungültiger Konvertierung
- **str(x)** (Built-in)
 - Konvertiert einen Wert in einen String
 - Funktioniert mit fast allen Datentypen

Operatoren

- **Zuweisungsoperator =**
 - Weist einer Variable einen Wert zu
 - Links steht der Variablenname, rechts der Wert
- **Zusammengesetzte Zuweisungsoperatoren: +=, -=, *=, /=**
 - Kurzschreibweise für $x = x + y \rightarrow x += y$
 - Funktioniert mit allen arithmetischen Operatoren

Konzepte

- **Variablen:** Benannte Speicherplätze für Werte
 - **Dynamische Typisierung:** Typ wird automatisch aus dem Wert abgeleitet
 - **Kommentare:** **#** für einzeilige, **"""..."""** für mehrzeilige Kommentare
-

Weiterführende Ressourcen

Theorie

Bücher und Online-Ressourcen: - *Code: The Hidden Language of Computer Hardware and Software* von Charles Petzold - Exzellente Einführung in binäre Systeme und Computer-Grundlagen - Khan Academy: Computer Science - How Computers Work - Interaktive Lektionen zu Binärsystem und Computern - Binary Tutorial (Stanford CS) - Ausführliches Stanford-Tutorial zu Bits und Bytes

Interaktive Tools: - RapidTables Binary Calculator - Online-Rechner für binäre Operationen - Binary Game - Spielerisches Lernen der Binär-Dezimal-Umrechnung - IEEE 754 Visualizer - Visualisierung von Fließkommazahlen (Vorbereitung auf V02)

Python

Offizielle Dokumentation: - Python Official Tutorial - Das offizielle Python-Tutorial - Python Built-in Functions - Dokumentation aller Built-in-Funktionen

Interaktive Lernplattformen: - Python Tutor - Visualisiert Schritt-für-Schritt, was im Speicher passiert - Codecademy Python Course - Interaktives Python-Tutorial - Real Python Tutorials - Hochwertige Python-Tutorials für alle Levels

Bücher für Anfänger: - *Python Crash Course* von Eric Matthes - Praktische Einführung mit Projekten - *Automate the Boring Stuff with Python* von Al Sweigart - Python für Automatisierung (kostenlos online verfügbar)

YouTube-Kanäle: - Corey Schafer - Exzellente Python-Tutorials - Programming with Mosh - Anfängerfreundliche Python-Videos

Datei: lessons/V02-Fließkommazahlen/V02-Fließkommazahlen_skript.md

V02: Fließkommazahlen

[!NOTE] **Lernziele dieser Vorlesung:** - Den IEEE 754 Standard für Fließkommazahlen verstehen und anwenden können - Die Darstellung von Fließkommazahlen im Binärsystem mit Vorzeichen, Exponent und Mantisse erklären können - Rundungsfehler und Genauigkeitsprobleme bei Fließkommaberechnungen erkennen und vermeiden - Spezielle Werte (NaN, Inf, -Inf) verstehen und korrekt handhaben - Best Practices beim Vergleichen von Fließkommazahlen anwenden können - Verschiedene Methoden der String-Formatierung in Python beherrschen (f-Strings, .format(),

%-Operator) - Die erweiterten Parameter von `print()` nutzen können - Escape-Sequenzen und mehrzeilige Strings korrekt einsetzen - Daten in Dateien schreiben und aus Dateien lesen können

Teil 1: Theorie - Fließkommazahlen

Überblick

In der ersten Vorlesung haben wir uns mit ganzen Zahlen und dem binären Zahlensystem beschäftigt. Doch wie speichert ein Computer Zahlen mit Nachkommastellen wie 3.14159 oder 0.000001? Die naive Idee, einfach einen **Binärpunkt** zwischen zwei Binärzahlen zu setzen (analog zum Dezimalpunkt), funktioniert zwar theoretisch, ist aber in der Praxis ineffizient. Computer verwenden stattdessen **Fließkommazahlen** (englisch: floating-point numbers), bei denen die Position des Kommas variabel ist – daher der Name.

Die Darstellung von Fließkommazahlen folgt einem international standardisierten Format: dem **IEEE 754 Standard**. Dieser Standard definiert, wie reelle Zahlen im Binärsystem approximiert werden, welche Genauigkeit erreicht wird und wie spezielle Werte wie Unendlich oder “Not a Number” repräsentiert werden. Das Verständnis dieses Standards ist fundamental für jeden, der mit numerischen Berechnungen arbeitet – sei es in der Simulation, der Datenanalyse oder der Steuerungstechnik.

[!NOTE] **Fließkommazahlen** sind eine Methode zur Darstellung reeller Zahlen in Computern, bei der die Position des Dezimalpunkts (bzw. Binärpunkts) variabel ist. Der Name “Fließkomma” (floating point) bezieht sich darauf, dass der Punkt in der Zahlendarstellung “schwimmt” und nicht an einer festen Position steht.

Motivation: Warum nicht einfach ein festes Komma?

Stellen Sie sich vor, Sie möchten sowohl die Masse eines Elektrons (ca. 9.109×10^{-31} kg) als auch die Masse der Erde (ca. 5.972×10^{24} kg) speichern. Mit einer **Festkommadarstellung**, bei der das Komma immer an derselben Stelle steht, müssten Sie entweder eine extrem hohe Anzahl von Bits verwenden oder auf Genauigkeit in einem der beiden Bereiche verzichten. Die Fließkommadarstellung löst dieses Problem elegant durch eine **wissenschaftliche Notation**, ähnlich der Exponentialschreibweise in der Mathematik.

Wissenschaftliche Notation als Grundlage

Bevor wir zur Computerdarstellung kommen, betrachten wir die wissenschaftliche Notation im Dezimalsystem:

$$\text{Zahl} = (-1)^s \times m \times 10^e$$

Hierbei ist: - s das **Vorzeichen** (0 für positiv, 1 für negativ) - m die **Mantisse** (auch Signifikand genannt), eine Zahl zwischen 1.0 und 9.999... - e der **Exponent** (eine ganze Zahl)

Beispiel: Die Zahl -6500.0 wird dargestellt als -6.5×10^3 , also $s = 1$, $m = 6.5$, $e = 3$.

[!TIP] **Beispiele für wissenschaftliche Notation im Dezimalsystem:** $-123.45 = 1.2345 \times 10^2$ - $0.00078 = 7.8 \times 10^{-4}$ - $-5000000 = -5.0 \times 10^6$

IEEE 754 Standard: Binäre Fließkommazahlen

Im Computer verwenden wir das **Binärsystem**, sodass die Formel lautet:

$$\text{Zahl} = (-1)^s \times m \times 2^e$$

Der **IEEE 754 Standard** definiert zwei gängige Formate:

Single Precision (32 Bit) Die **Single Precision** (auch **float** genannt) verwendet 32 Bits:

- **1 Bit** für das Vorzeichen s
- **8 Bits** für den Exponenten e
- **23 Bits** für die Mantisse m

Double Precision (64 Bit) Die **Double Precision** (auch **double** genannt) verwendet 64 Bits und bietet höhere Genauigkeit:

- **1 Bit** für das Vorzeichen s
- **11 Bits** für den Exponenten e
- **52 Bits** für die Mantisse m

[!NOTE] **Single Precision** bietet etwa 7 Dezimalstellen Genauigkeit und einen Wertebereich von ca. 10^{-38} bis 10^{38} .

Double Precision bietet etwa 15-16 Dezimalstellen Genauigkeit und einen Wertebereich von ca. 10^{-308} bis 10^{308} .

In Python entspricht der Datentyp **float** standardmäßig der **Double Precision** (64 Bit).

Aufbau und Interpretation

Das Vorzeichenbit Das **Vorzeichenbit** ist das einfachste Element: - 0 bedeutet: Die Zahl ist positiv oder +0 - 1 bedeutet: Die Zahl ist negativ oder -0

[!WARNING] Es gibt tatsächlich zwei verschiedene Null-Werte: +0 und -0. Diese werden in den meisten Berechnungen gleich behandelt,

können aber in Grenzfällen (z.B. bei Division) zu unterschiedlichen Ergebnissen führen.

Der Exponent (Bias-Darstellung) Der Exponent wird **nicht** im Zweierkomplement gespeichert, sondern verwendet eine **Bias-Darstellung**. Dabei wird zu jedem Exponenten ein fester Wert (der Bias) addiert, um negative Exponenten darzustellen:

- **Single Precision:** Bias = 127
- **Double Precision:** Bias = 1023

Der tatsächliche Exponent berechnet sich als: $e_{\text{real}} = e_{\text{gespeichert}} - \text{Bias}$

Beispiel (Single Precision): - Gespeicherter Wert 10000010 (binär) = 130 (dezimal) - Tatsächlicher Exponent = 130 - 127 = 3 - Die Zahl wird mit $2^3 = 8$ multipliziert

[!TIP] **Warum Bias-Darstellung?**

Die Bias-Darstellung hat den Vorteil, dass sich Fließkommazahlen als Ganzzahlen interpretiert in derselben Reihenfolge befinden. Dies vereinfacht Vergleichsoperationen in Hardware erheblich.

Die Mantisse (Normalisierung) Die Mantisse repräsentiert die signifikanten Stellen der Zahl. Um maximale Genauigkeit zu erreichen, wird die **normalisierte Darstellung** verwendet:

$$m = 1.b_1b_2b_3 \dots b_{23}$$

Das führende 1. wird **nicht gespeichert** (implizites Bit), da es bei normalisierten Zahlen immer vorhanden ist. Dies gewinnt ein zusätzliches Bit an Genauigkeit!

Beispiel: Die Mantisse 11001000000000000000000 repräsentiert eigentlich:

$$1.11001000000000000000000_2 = 1 + 2^{-1} + 2^{-2} + 2^{-5} = 1.78125$$

Vollständiges Beispiel: Darstellung von 12.375

Konvertieren wir die Dezimalzahl **12.375** in die IEEE 754 Single Precision Darstellung:

Schritt 1: Vorzeichen bestimmen 12.375 ist positiv $\rightarrow s = 0$

Schritt 2: Binärdarstellung Ganzzahlteil: $12_{10} = 1100_2$
Nachkommateil: $0.375_{10} = 0.011_2$ (da $0.375 = \frac{3}{8} = \frac{1}{4} + \frac{1}{8} = 2^{-2} + 2^{-3}$)
Zusammen: $12.375_{10} = 1100.011_2$

Schritt 3: Normalisierung Verschiebe den Binärpunkt so, dass genau eine 1 links davon steht:

$$1100.011_2 = 1.100011_2 \times 2^3$$

Hier ist der **Exponent** $e = 3$.

Schritt 4: Exponent mit Bias Gespeicherter Exponent $= 3 + 127 = 130 = 10000010_2$

Schritt 5: Mantisse (ohne führende 1) Aus 1.100011 wird nur 100011 gespeichert, aufgefüllt mit Nullen:

$$1000110000000000000000$$

Ergebnis

[!TIP] **Interaktive Online-Tools:**

Um die IEEE 754 Darstellung zu visualisieren und zu experimentieren, gibt es hilfreiche Online-Konverter: - <https://www.h-schmidt.net/FloatConverter/IEEE754.html> - <https://baseconvert.com/ieee-754-floating-point>

Rundungsfehler und Genauigkeitsprobleme

Ein fundamentales Problem von Fließkommazahlen ist, dass nicht alle Dezimalzahlen exakt darstellbar sind. Dies führt zu **Rundungsfehlern**.

Warum 0.1 + 0.2 0.3 Das klassische Beispiel zeigt das Problem eindrucksvoll: Im Dezimalsystem ist $\frac{1}{3} = 0.333 \dots$ eine unendliche Dezimalzahl. Im Binärsystem haben viele dezimal "einfache" Brüche unendliche Binärdarstellungen:

$$0.1_{10} = 0.0001100110011001100110011 \dots_2 \text{ (periodisch)}$$

Da Computer nur endlich viele Bits speichern können, wird gerundet. Die Folge:

[!WARNING] **Niemals Fließkommazahlen mit == vergleichen!**

Aufgrund von Rundungsfehlern führt direkter Vergleich mit == oft zu unerwartetem Verhalten. Stattdessen sollte man prüfen, ob der Unterschied kleiner als eine Toleranzgrenze (Epsilon) ist.

Akkumulation von Rundungsfehlern Bei wiederholten Berechnungen können sich Rundungsfehler aufaddieren:

[!TIP] **Best Practices gegen Rundungsfehler:** - Wenn möglich, mit ganzen Zahlen rechnen und erst am Ende dividieren - Bei Geldbeträgen: Rechnen in Cent (Ganzzahlen) statt Euro (Fließkomma)
- Verwenden Sie spezielle Bibliotheken wie `decimal` für höhere Genauigkeit bei kritischen Anwendungen

Spezielle Werte

Der IEEE 754 Standard definiert mehrere spezielle Werte für außergewöhnliche Situationen:

Positive und negative Unendlichkeit ($\pm\text{Inf}$) **Infinity** tritt auf bei: - Division durch Null: $\frac{5.0}{0.0} = +\infty$ - Überlauf bei Berechnungen: $10^{400} = +\infty$ (außerhalb des Wertebereichs)

Darstellung: - Exponent: alle Bits auf 1 - Mantisse: alle Bits auf 0 - Vorzeichen bestimmt $+\infty$ oder $-\infty$

[!NOTE] **Unendlichkeit (Infinity, Inf)** ist ein spezieller Fließkommawert, der Zahlen repräsentiert, die den maximalen darstellbaren Wertebereich überschreiten. Es gibt positive und negative Unendlichkeit.

Not a Number (NaN) **NaN** (Not a Number) signalisiert ein ungültiges oder undefiniertes Ergebnis: - $\frac{0}{0}$ - $\infty - \infty$ - $\sqrt{-1}$ (ohne komplexe Zahlen)

Darstellung: - Exponent: alle Bits auf 1 - Mantisse: **mindestens ein Bit** auf 1 (beliebige Kombination)

[!NOTE] **NaN (Not a Number)** ist ein spezieller Fließkommawert, der signalisiert, dass das Ergebnis einer Operation mathematisch undefiniert oder nicht darstellbar ist.

[!WARNING] **NaN ist nie gleich zu irgendetwas – auch nicht zu sich selbst!**

`NaN == NaN` ergibt **False**! Um auf NaN zu testen, muss eine spezielle Funktion verwendet werden (z.B. `math.isnan()` in Python).

Null Es gibt **zwei verschiedene Nullen**: `+0` und `-0`. Beide verhalten sich in den meisten Operationen identisch, können aber unterschiedliche Vorzeichen für das Ergebnis produzieren:

Best Practices beim Vergleichen von Fließkommazahlen

Aufgrund von Rundungsfehlern sollten Fließkommazahlen niemals direkt mit `==` verglichen werden. Stattdessen wird geprüft, ob der **absolute Unterschied** kleiner als eine **Toleranzgrenze** (Epsilon, ε) ist:

$$|a - b| < \varepsilon$$

Typische Werte für ε : - **1e-9** (0.000000001) für Double Precision bei “normalen” Zahlen - **1e-6** (0.000001) für weniger kritische Anwendungen

[!TIP] **Fließkomma-Vergleich in Pseudocode:**

Für sehr große oder sehr kleine Zahlen sollte zusätzlich ein **relativer Vergleich** durchgeführt werden:

$$\frac{|a - b|}{\max(|a|, |b|)} < \varepsilon$$

Dies stellt sicher, dass der Vergleich sowohl für kleine Zahlen (10^{-10}) als auch für große Zahlen (10^{10}) funktioniert.

Zusammenfassung Theorie

Die wichtigsten Erkenntnisse über Fließkommazahlen:

1. **IEEE 754** ist der Industriestandard für die binäre Darstellung von Fließkommazahlen mit Vorzeichen, Exponent (Bias-Darstellung) und Mantisse (normalisiert mit implizitem führenden Bit).
2. **Rundungsfehler** sind unvermeidlich, da nicht alle Dezimalzahlen im Binärsystem exakt darstellbar sind. Dies gilt besonders für dezimal “einfache” Brüche wie 0.1, 0.2, 0.3.
3. **Spezielle Werte** wie $\pm\infty$ und NaN ermöglichen die Repräsentation von Überlauf, Unterlauf und undefinierten Operationen ohne Programmabsturz.
4. **Vergleiche** zwischen Fließkommazahlen sollten nie mit `==` durchgeführt werden, sondern immer mit einer Toleranzgrenze (Epsilon-Vergleich).
5. **Single Precision** (32 Bit) bietet ca. 7 Dezimalstellen, **Double Precision** (64 Bit) ca. 15-16 Dezimalstellen Genauigkeit. Für die meisten Anwendungen ist Double Precision ausreichend und der Standard in Python.

Teil 2: Python-Praxis - Eingaben/Ausgaben & Formatierung

[!WARNING] **Python-Konsistenz beachten:** Prüfe ../../python_topics.md für bereits eingeführte Konzepte!

Überblick

In der ersten Vorlesung haben wir die grundlegende Verwendung von `print()` und `input()` kennengelernt. In dieser Lektion vertiefen wir die Möglichkeiten der **Ein- und Ausgabe** erheblich. Wir lernen verschiedene Methoden zur **String-Formatierung**, die es ermöglichen, Zahlen, Variablen und Text elegant und präzise zu formatieren. Dies ist besonders wichtig beim Arbeiten mit Fließkommazahlen, da wir oft die Anzahl der Dezimalstellen kontrollieren oder Zahlen in wissenschaftlicher Notation darstellen möchten.

Zusätzlich erkunden wir die erweiterten Parameter von `print()`, die Verwendung von **Escape-Sequenzen** für Spezialzeichen und die grundlegende **Datei-Ein- und Ausgabe**, um Programme zu erstellen, die persistente Daten speichern und lesen können.

String-Formatierung: Drei Methoden im Vergleich

Python bietet mehrere Methoden zur String-Formatierung. Wir behandeln die drei wichtigsten: **f-Strings** (modern und empfohlen), die **.format()-Methode** (flexibel und weit verbreitet) und den **%-Operator** (legacy, aber noch in altem Code zu finden).

f-Strings (Formatted String Literals) – Empfohlen! f-Strings sind seit Python 3.6 verfügbar und die modernste, lesbarste Form der String-Formatierung. Sie werden durch ein **f** vor dem öffnenden Anführungszeichen gekennzeichnet und erlauben es, Variablen und Ausdrücke direkt in geschweiften Klammern `{}` einzubetten.

[!NOTE] **f-Strings** (Formatted String Literals) sind eine moderne Methode zur String-Formatierung in Python, bei der Ausdrücke in geschweiften Klammern innerhalb eines Strings mit dem Präfix **f** direkt ausgewertet werden.

[!TIP] **Grundlegende f-String Verwendung:**

Ausdrücke in f-Strings: Innerhalb der geschweiften Klammern können beliebige Python-Ausdrücke stehen:

Zahlenformatierung in f-Strings: Die eigentliche Stärke von f-Strings zeigt sich bei der Kontrolle über die Ausgabeformatierung. Nach der Variable bzw. dem Ausdruck folgt ein Doppelpunkt `:` und dann die **Formatspezifikation**.

Dezimalstellen bei Fließkommazahlen Die Syntax `:.2f` bedeutet: - : leitet die Formatspezifikation ein - .2 gibt die Anzahl der Dezimalstellen an - f steht für “fixed-point notation” (Festkommadarstellung)

[!NOTE] **Formatspezifikation** ist der Teil nach dem Doppelpunkt in f-Strings oder `.format()`, der definiert, wie ein Wert dargestellt werden soll (z.B. Anzahl der Dezimalstellen, Breite, Ausrichtung).

Wissenschaftliche Notation Für sehr große oder sehr kleine Zahlen ist die **wissenschaftliche Notation** (Exponentialschreibweise) praktisch:

Die Syntax `:.3e` bedeutet: - .3 gibt die Anzahl der Dezimalstellen in der Mantisse an - e steht für “exponential notation”

Breite und Ausrichtung Oft möchte man Ausgaben in Spalten formatieren. Die Breite eines Feldes wird vor dem Dezimalpunkt angegeben:

Die Syntax: - `{z:10.2f}` – rechtsbündig (Standard für Zahlen), Breite 10 - `{z:<10.2f}` – linksbündig - `{z:^10.2f}` – zentriert

Tausender-Trennzeichen Für bessere Lesbarkeit großer Zahlen:

- , verwendet Komma als Tausender-Trennzeichen (US-amerikanisch)
- _ verwendet Unterstrich als Tausender-Trennzeichen (Python-Style)

Prozentangaben Die Syntax `:.1%` multipliziert den Wert mit 100 und fügt ein Prozentzeichen hinzu, mit 1 Dezimalstelle.

[!TIP] **f-String Formatierungs-Spickzettel:**

Format	Bedeutung	Beispiel	Ausgabe
<code>{x:.2f}</code>	2 Dezimalstellen	<code>x = 3.14159</code>	<code>3.14</code>
<code>{x:.3e}</code>	Exponentialnotation	<code>x = 1234.5</code>	<code>1.235e+03</code>
<code>{x:10.2f}</code>	Breite 10, rechtsbündig	<code>x = 42.1</code>	<code>42.10</code>
<code>{x:<10.2f}</code>	Breite 10, linksbündig	<code>x = 42.1</code>	<code>42.10</code>
<code>{x:^10.2f}</code>	Breite 10, zentriert	<code>x = 42.1</code>	<code>42.10</code>
<code>{x:,}</code>	Tausender-Trennung (Komma)	<code>x = 1000000</code>	<code>1,000,000</code>
<code>{x:_}</code>	Tausender-Trennung (Unterstrich)	<code>x = 1000000</code>	<code>1_000_000</code>
<code>{x:.1%}</code>	Prozent	<code>x = 0.8642</code>	<code>86.4%</code>
<code>{x:+.2f}</code>	Mit Vorzeichen	<code>x = 42.1</code>	<code>+42.10</code>
<code>{x:08.2f}</code>	Mit Nullen auffüllen	<code>x = 42.1</code>	<code>00042.10</code>

Die `.format()`-Methode Die `.format()`-Methode ist die Vorgänger-Technologie zu f-Strings, aber immer noch weit verbreitet und in manchen Situationen nützlich (z.B. wenn der Format-String wiederverwendet wird).

[!NOTE] **Die `.format()`-Methode** ist eine String-Methode, die Platzhalter in geschweiften Klammern durch die als Argumente übergebenen Werte ersetzt.

Grundlegende Verwendung:

Mit Positionsangabe:

Mit benannten Platzhaltern:

Formatierung wie bei f-Strings:

[!WARNING] **f-Strings vs. `.format()`:**

f-Strings sind in den meisten Fällen vorzuziehen, da sie lesbarer und schneller sind. Verwenden Sie `.format()` nur, wenn Sie den Format-String mehrfach mit unterschiedlichen Werten verwenden möchten.

Der %-Operator (Legacy) Der %-Operator ist die älteste Methode zur String-Formatierung in Python und stammt aus der C-Programmiersprache. Er wird heute nicht mehr empfohlen, ist aber in älterem Code häufig zu finden.

[!NOTE] **Der %-Operator** ist eine veraltete Methode zur String-Formatierung, die Platzhalter wie `%s`, `%d`, `%f` verwendet und Werte aus einem Tupel einsetzt.

Wichtige Platzhalter: - `%s` – String - `%d` – Integer (decimal) - `%f` – Float - `%.2f` – Float mit 2 Dezimalstellen

[!WARNING] **Legacy-Methode – nicht empfohlen!**

Verwenden Sie den %-Operator nur, wenn Sie alten Code warten. Für neuen Code sind f-Strings die deutlich bessere Wahl.

Erweiterte `print()`-Parameter

Die `print()`-Funktion akzeptiert mehrere optionale Parameter, die wir in V01 noch nicht behandelt haben.

Der `sep`-Parameter Standardmäßig trennt `print()` mehrere Argumente durch ein Leerzeichen. Mit `sep` kann dies angepasst werden:

Der `end`-Parameter Standardmäßig fügt `print()` am Ende einen Zeilenumbruch `\n` hinzu. Mit `end` kann dies geändert werden:

Praktisches Beispiel – Fortschrittsbalken:

Der `flush`-Parameter Normalerweise werden Ausgaben **gepuffert** (buffered), d.h. sie werden erst geschrieben, wenn der Puffer voll ist oder ein Zeilenumbruch kommt. Mit `flush=True` wird die Ausgabe sofort geschrieben:

Ohne `flush=True` würde "Warte 5 Sekunden" erst nach den 5 Sekunden erscheinen.

[!TIP] `print()`-Parameter kombinieren:

Escape-Sequenzen und Spezialzeichen

Escape-Sequenzen beginnen mit einem Backslash `\` und repräsentieren Spezialzeichen, die nicht direkt getippt werden können oder besondere Bedeutung haben.

[!NOTE] **Escape-Sequenzen** sind spezielle Zeichenkombinationen, die mit einem Backslash `\` beginnen und Spezialzeichen oder Steuerzeichen in Strings repräsentieren.

Die wichtigsten Escape-Sequenzen

Escape-Sequenz	Bedeutung	Beispiel
<code>\n</code>	Zeilenumbruch (Newline)	"Zeile 1\nZeile 2"
<code>\t</code>	Tabulator (Tab)	"Name:\tAda"
<code>\\</code>	Backslash selbst	"Pfad: C:\\Users"
<code>\'</code>	Einfaches Anführungszeichen	'It\'s okay'
<code>\"</code>	Doppeltes Anführungszeichen	"Er sagte \"Hallo\""
<code>\r</code>	Carriage Return (Wagenrücklauf)	"Überschreibe\rText"
<code>\b</code>	Backspace (Rückschritt)	"abc\b d" → "abd"

[!TIP] **Escape-Sequenzen in Aktion:**

Raw Strings (r-Strings) Wenn Sie viele Backslashes in einem String haben (z.B. bei Windows-Pfaden oder regulären Ausdrücken), können Sie **Raw Strings** verwenden, die durch ein `r` vor dem Anführungszeichen gekennzeichnet sind. In Raw Strings werden Escape-Sequenzen **nicht** interpretiert:

[!NOTE] **Raw Strings** sind Strings mit dem Präfix `r`, in denen Backslashes nicht als Escape-Zeichen interpretiert werden, sondern als literale Backslashes.

Mehrzeilige Strings

Für längere Texte über mehrere Zeilen können **triple quotes** (dreifache Anführungszeichen) verwendet werden:

Mehrzeilige Strings respektieren alle Zeilenumbrüche und Einrückungen:

[!TIP] **Mehrzeilige f-Strings:**

f-Strings können auch mehrzeilig sein:

Dateien lesen und schreiben

Bisher haben wir nur mit der Konsole interagiert. Für persistente Datenspeicherung müssen wir **Dateien** verwenden.

Dateien schreiben Mit der `open()`-Funktion öffnen wir eine Datei, schreiben mit der `.write()`-Methode und schließen mit `.close()`:

[!NOTE] `open(filename, mode)` öffnet eine Datei. Der `mode` bestimmt, ob gelesen, geschrieben oder angehängt wird: - `"r"` – Read (Lesen, Standard) - `"w"` – Write (Schreiben, überschreibt existierende Datei) - `"a"` – Append (Anhängen, fügt am Ende hinzu) - `"r+"` – Read/Write (Lesen und Schreiben)

[!WARNING] **Mode “w” überschreibt existierende Dateien!**

Wenn Sie `open("datei.txt", "w")` aufrufen und die Datei bereits existiert, wird ihr Inhalt **komplett gelöscht**. Verwenden Sie `"a"` (append), um am Ende anzuhängen.

Dateien lesen Zeile für Zeile lesen:

Alle Zeilen als Liste lesen:

Context Manager: with-Statement Das manuelle Öffnen und Schließen von Dateien ist fehleranfällig (was passiert bei einem Fehler vor `.close()`?). Python bietet das **with-Statement**, das sicherstellt, dass die Datei automatisch geschlossen wird:

[!NOTE] **Das with-Statement** ist ein Kontext-Manager, der sicherstellt, dass Ressourcen (wie Dateien) korrekt freigegeben werden, auch wenn ein Fehler auftritt. Die Datei wird automatisch geschlossen, wenn der `with`-Block verlassen wird.

[!TIP] **Best Practice: Immer with verwenden!**

Das `with`-Statement ist die empfohlene Methode zum Arbeiten mit Dateien:

Praktisches Beispiel: Messdaten speichern

[!WARNING] **Dateipfade unter Windows:**

Unter Windows verwenden Pfade Backslashes `\`, die in Python Escape-Zeichen sind. Verwenden Sie entweder: - Raw Strings: `r"C:\Users\Ada\file.txt"` - Forward Slashes: `"C:/Users/Ada/file.txt"`

(funktioniert auch unter Windows!) - Doppelte Backslashes:
`"C:\\Users\\Ada\\file.txt"`

Häufige Fehler und Lösungen

[!WARNING] Fehler 1: Datei nicht geschlossen

Problem: Datei wird mit `open()` geöffnet, aber nie mit `.close()` geschlossen. Dies kann zu Datenverlust führen.

Lösung: Verwenden Sie **immer** das `with`-Statement für automatisches Schließen.

[!WARNING] Fehler 2: Dezimalstellen bei `print()` statt in Format-String

Problem:

Lösung: Verwenden Sie f-Strings zur Kontrolle:

[!WARNING] Fehler 3: Fehlende `\n` bei `.write()`

Problem: `.write()` fügt **keine** automatischen Zeilenumbrüche hinzu (im Gegensatz zu `print()`).

Lösung: Manuell `\n` hinzufügen:

[!WARNING] Fehler 4: Vergessener Mode-Parameter beim Anhängen

Problem: Wollen an Datei anhängen, verwenden aber `"w"` statt `"a"` und löschen versehentlich alle Daten.

Lösung:

Zusammenfassung Python

Die wichtigsten Python-Konzepte dieser Lektion:

1. **f-Strings** sind die modernste und empfohlene Methode zur String-Formatierung. Sie ermöglichen direkte Einbettung von Variablen mit `f"{variable}"` und mächtige Formatierungsoptionen wie `{x:.2f}` für Dezimalstellen oder `{x:10.2f}` für Breite und Ausrichtung.
2. Die **`.format()`-Methode** ist eine flexible Alternative, die besonders nützlich ist, wenn der Format-String wiederverwendet wird. Der **%-Operator** ist veraltet und sollte in neuem Code vermieden werden.
3. **`print()`** akzeptiert die Parameter **`sep`** (Trennzeichen zwischen Argumenten), **`end`** (was am Ende ausgegeben wird) und **`flush`** (sofortiges Schreiben ohne Pufferung).

4. **Escape-Sequenzen** wie `\n` (Zeilenumbruch), `\t` (Tab) und `\\` (Backslash) ermöglichen die Darstellung von Spezialzeichen. **Raw Strings** mit `r"..."` deaktivieren Escape-Sequenzen.
5. **Dateien** werden mit `open(filename, mode)` geöffnet. Der Mode bestimmt, ob gelesen ("**r**"), geschrieben ("**w**") oder angehängt ("**a**") wird. Das **with-Statement** ist die empfohlene Methode, da es automatisches Schließen garantiert.

Neue Python-Funktionen/Methoden

In dieser Lektion wurden folgende Python-Funktionen und -Konzepte **neu** eingeführt:

String-Formatierung

- **f-Strings** (Python 3.6+): `f"Text {variable}"` mit Formatspezifikationen wie `{x:.2f}`, `{x:10.2f}`, `{x:.3e}`, `{x:,}`, `{x:.1%}`
- **.format()-Methode**: `"Text {}".format(value)` mit Positions- und benannten Argumenten
- **%-Operator** (legacy): `"Text %s" % (value,)` mit Platzhaltern wie `%s`, `%d`, `%f`

Erweiterte print()-Parameter

- **sep-Parameter**: `print(a, b, sep="-")` – Trennzeichen zwischen Argumenten
- **end-Parameter**: `print(text, end="")` – was am Ende ausgegeben wird
- **flush-Parameter**: `print(text, flush=True)` – sofortiges Schreiben ohne Pufferung

Escape-Sequenzen und Strings

- **Escape-Sequenzen**: `\n` (newline), `\t` (tab), `\\` (backslash), `\'`, `\"`
- **Raw Strings**: `r"C:\Users"` – Backslashes werden nicht als Escape interpretiert
- **Mehrzeilige Strings**: `"""Text über\nmehrere Zeilen"""`

Datei-Ein-/Ausgabe

- **open(filename, mode)** (Built-in): Öffnet eine Datei
 - Modi: `"r"` (read), `"w"` (write), `"a"` (append)
- **.write(string)**: Schreibt String in Datei (ohne automatischen Zeilenumbruch)
- **.read()**: Liest gesamten Dateinhalt als String
- **.readlines()**: Liest alle Zeilen als Liste von Strings
- **with-Statement**: Context Manager für automatisches Schließen von Ressourcen

Weiterführende Ressourcen

Theorie

- **IEEE 754 Standard (offizielles Dokument):** <https://standards.ieee.org/standard/754-2019.html> – Der vollständige Standard (sehr technisch)
- **What Every Computer Scientist Should Know About Floating-Point Arithmetic** von David Goldberg: https://docs.oracle.com/cd/E19957-01/806-3568/ngc_goldberg.html – Klassischer Artikel zum Thema
- **Floating Point Visually Explained:** https://fabiensanglard.net/floating_point_visually_explained/ – Interaktive Visualisierung des IEEE 754 Formats
- **Online IEEE 754 Converter:** <https://www.h-schmidt.net/FloatConverter/IEEE754.html> – Tool zum Experimentieren mit Fließkommazahlen

Python

- **Python String Formatting (offizielle Dokumentation):** <https://docs.python.org/3/library/string.html> – Vollständige Referenz aller Formatierungsoptionen
- **Python f-Strings Guide:** <https://realpython.com/python-f-strings/> – Umfassender Artikel zu f-Strings
- **Python File I/O Tutorial:** <https://realpython.com/read-write-files-python/> – Ausführliche Anleitung zum Arbeiten mit Dateien
- **Python Format Specification Mini-Language:** <https://docs.python.org/3/library/string.html#format-specification-mini-language> – Details zur Formatspezifikation
- **PEP 498 – Literal String Interpolation:** <https://www.python.org/dev/peps/pep-0498/> – Die offizielle Spezifikation für f-Strings

Datei: `lessons/V03-Boolsche-Algebra-Teil1/V03-Boolsche-Algebra-Teil1_skript.md`

V03: Boolsche Algebra & Logische Schaltungen – Teil 1

[!NOTE] **Lernziele dieser Vorlesung:** - Die drei grundlegenden logischen Operatoren **AND**, **OR** und **NOT** verstehen und anwenden können - **Wahrheitstabellen** für logische Ausdrücke erstellen und interpretieren - Einfache **logische Schaltungen** analysieren und deren Funktionsweise nachvollziehen - Python-**Datentypen** (`int`, `float`, `str`, `bool`) unterscheiden und gezielt einsetzen - **Type Casting** und **Type Checking** mit `type()` und `isinstance()` durchführen - Den Unterschied zwischen **unveränderlichen** (immutable)

und **veränderlichen** (mutable) Typen erklären - **Variablen-Scope**
(lokal vs. global) verstehen und korrekt anwenden

Teil 1: Theorie - Boolesche Algebra & Logische Schaltungen

Überblick

Die **Boolesche Algebra** bildet das mathematische Fundament der digitalen Elektronik und damit aller modernen Computer. Der englische Mathematiker George Boole entwickelte im 19. Jahrhundert ein algebraisches System, das nur zwei Zustände kennt: **wahr** (1, true, high) und **falsch** (0, false, low). Dieses binäre System ermöglicht es, komplexe logische Zusammenhänge präzise zu beschreiben und technisch umzusetzen.

In dieser Vorlesung lernen Sie die drei fundamentalen logischen Operatoren kennen, mit denen sich jede beliebige logische Funktion aufbauen lässt. Diese Operatoren bilden die Grundlage für Prozessor-Architekturen, Speicherbausteine und alle digitalen Schaltungen in Computern, Smartphones und Industriesteuerungen.

[!NOTE] **Boolesche Algebra:** Ein algebraisches System mit zwei Wahrheitswerten (0 und 1) und logischen Operationen, das die Grundlage der digitalen Elektronik bildet.

Die drei fundamentalen logischen Operatoren

Die Boolesche Algebra kennt drei grundlegende Operatoren, aus denen sich alle komplexeren logischen Funktionen zusammensetzen lassen. Diese Operatoren verknüpfen einen oder mehrere Eingangswerte zu einem Ausgangswert.

Der AND-Operator (Logisches UND) Der **AND-Operator** (auch **Konjunktion** genannt) verknüpft zwei Eingangswerte und liefert nur dann den Wert 1 (wahr), wenn **beide** Eingänge den Wert 1 haben. In allen anderen Fällen ist das Ergebnis 0 (falsch).

[!NOTE] **AND-Operator:** Eine logische Verknüpfung, die nur dann wahr ist, wenn alle Eingänge wahr sind. Symbol: (mathematisch), & (Programmierung), • (Schaltungslogik).

Die **Wahrheitstabelle** für den AND-Operator zeigt alle möglichen Eingangskombinationen und die zugehörigen Ausgänge:

A	B	A	B
0	0	0	
0	1	0	
1	0	0	

A	B	A	B
1	1	1	

[!TIP] **Alltagsbeispiel für AND:** Ein Auto startet nur, wenn der Schlüssel eingesteckt **UND** die Bremse getreten ist. Beide Bedingungen müssen erfüllt sein.

In der **Digitaltechnik** wird der AND-Operator durch ein **AND-Gatter** realisiert. Das Schaltsymbol nach IEEE-Standard (amerikanische Notation) zeigt eine charakteristische D-förmige Linie:

Die **logische Gleichung** lautet: $Y = A \cdot B$ oder häufig auch $Y = A \cdot B$ (Punkt-Schreibweise aus der klassischen Algebra).

[!WARNING] **Häufiger Denkfehler:** Anfänger verwechseln oft AND mit OR. Merke: Bei AND müssen **alle** Bedingungen erfüllt sein – wie bei einer Tür mit mehreren Schlössern, die alle aufgeschlossen werden müssen.

Der OR-Operator (Logisches ODER) Der **OR-Operator** (auch **Disjunktion** genannt) liefert den Wert 1 (wahr), wenn **mindestens einer** der Eingänge den Wert 1 hat. Nur wenn beide Eingänge 0 sind, ist auch das Ergebnis 0.

[!NOTE] **OR-Operator:** Eine logische Verknüpfung, die wahr ist, wenn mindestens ein Eingang wahr ist. Symbol: $\vee$ (mathematisch), $|$ (Programmierung), $+$ (Schaltungslogik).

Die **Wahrheitstabelle** für den OR-Operator:

A	B	A	B
0	0	0	
0	1	1	
1	0	1	
1	1	1	

[!TIP] **Alltagsbeispiel für OR:** Eine Lampe leuchtet, wenn der Wandschalter eingeschaltet **ODER** der Fernbedienungsschalter aktiviert ist. Es reicht, wenn eine der beiden Bedingungen erfüllt ist.

Das **OR-Gatter** in der Digitaltechnik wird durch ein Symbol mit geschwungener Eingangsseite dargestellt:

Die **logische Gleichung** lautet: $Y = A \vee B$ oder $Y = A + B$ (Plus-Schreibweise, aber Achtung: **nicht** die gewöhnliche arithmetische Addition!).

[!WARNING] **Wichtige Unterscheidung:** In der Booleschen Algebra gilt $1 + 1 = 1$ (nicht 2!), da OR bereits bei einem einzigen wahren Eingang wahr ist.

Der NOT-Operator (Logische Negation) Der **NOT-Operator** (auch **Negation** oder **Inversion** genannt) ist ein **einstelliger** (unärer) Operator. Er kehrt den Eingangswert um: Aus 0 wird 1, aus 1 wird 0.

[!NOTE] **NOT-Operator:** Ein logischer Operator, der den Eingangswert invertiert (umkehrt). Symbol: $\neg$ (mathematisch), $\sim$ (Programmierung), Überstrich (Schaltungslogik: $\bar{A}$).

Die **Wahrheitstabelle** für den NOT-Operator ist besonders einfach:

A	$\neg A$
0	1
1	0

[!TIP] **Alltagsbeispiel für NOT:** Ein Türöffner ist aktiv, wenn der Not-Aus-Schalter **NICHT** gedrückt ist. Die Negation kehrt den Zustand um.

Das **NOT-Gatter** (auch **Inverter** genannt) wird durch ein Dreieck-Symbol mit einem kleinen Kreis am Ausgang dargestellt. Der Kreis symbolisiert die Invertierung:

Die **logische Gleichung** lautet: $Y = \neg A$ oder $Y = \bar{A}$ (A mit Überstrich) oder manchmal $Y = A'$ (A mit Apostroph).

Wahrheitstabellen: Systematische Analyse logischer Ausdrücke

Wahrheitstabellen sind das zentrale Werkzeug zur systematischen Analyse und Darstellung logischer Funktionen. Sie zeigen für alle möglichen Eingangskombinationen das jeweilige Ausgangsergebnis.

[!NOTE] **Wahrheitstabelle:** Eine tabellarische Übersicht, die für alle möglichen Eingangskombinationen den zugehörigen Ausgangswert einer logischen Funktion angibt.

Anzahl der Zeilen in einer Wahrheitstabelle Bei n Eingangsvariablen gibt es 2^n mögliche Kombinationen. Jede Kombination entspricht einer Zeile in der Wahrheitstabelle:

- 1 Variable: $2^1 = 2$ Zeilen (0, 1)
- 2 Variablen: $2^2 = 4$ Zeilen (00, 01, 10, 11)
- 3 Variablen: $2^3 = 8$ Zeilen (000, 001, 010, ..., 111)
- 4 Variablen: $2^4 = 16$ Zeilen

[!TIP] **Systematisches Vorgehen:** Beginne bei der ersten Spalte und wechsele zwischen 0 und 1 nach jeweils 2^{n-1} Zeilen. In der zweiten Spalte nach 2^{n-2} Zeilen, usw. So entsteht automatisch die richtige Reihenfolge.

Beispiel: Wahrheitstabelle für $A \cdot (B \vee C)$ Betrachten wir den zusammengesetzten Ausdruck $A \cdot (B \vee C)$. Diese Funktion hat drei Eingänge (A, B, C) und benötigt daher 8 Zeilen:

A	B	C	$B \vee C$	$A \cdot (B \vee C)$
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	1	0
1	0	0	0	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Vorgehen beim Erstellen: 1. Alle Eingangskombinationen systematisch auflisten (8 Zeilen für 3 Variablen) 2. Zwischenergebnis $B \vee C$ berechnen (OR-Verknüpfung von B und C) 3. Endergebnis $A \cdot (B \vee C)$ durch AND-Verknüpfung von A mit dem Zwischenergebnis

[!WARNING] **Klammerung beachten:** Die Reihenfolge der Auswertung ist entscheidend! $A \cdot (B \vee C)$ ist **nicht** dasselbe wie $(A \cdot B) \vee C$. Arbeite immer von innen nach außen, wie bei mathematischen Klammern.

Einfache logische Schaltungen

Logische Operatoren werden in der Digitaltechnik durch **Gatter** (engl. Gates) realisiert. Ein Gatter ist eine elektronische Schaltung, die eine logische Funktion umsetzt. Mehrere Gatter lassen sich zu komplexeren Schaltungen kombinieren.

Beispiel 1: Serienschaltung zweier Schalter Zwei **in Serie geschaltete** Schalter entsprechen einem AND-Gatter. Der Strom fließt nur, wenn **beide** Schalter geschlossen sind.

Logische Funktion: $\text{Lampe} = A \cdot B$

Die Lampe leuchtet nur, wenn sowohl Schalter A als auch Schalter B geschlossen sind.

[!TIP] **Technische Anwendung:** Sicherheitsschaltungen in Maschinen verwenden oft AND-Logik. Die Maschine startet nur, wenn alle Schutztüren geschlossen sind **UND** der Startknopf gedrückt wird.

Beispiel 2: Parallelschaltung zweier Schalter Zwei **parallel geschaltete** Schalter entsprechen einem OR-Gatter. Der Strom fließt, wenn **mindestens einer** der Schalter geschlossen ist.

Logische Funktion: Lampe = A $\vee$ B

Die Lampe leuchtet, wenn Schalter A oder Schalter B (oder beide) geschlossen sind.

[!TIP] **Technische Anwendung:** Wechselschaltungen in Häusern erlauben es, eine Lampe von mehreren Stellen aus zu steuern. Dies basiert auf OR-Logik (bzw. deren Erweiterungen).

Beispiel 3: Kombinierte Schaltung Betrachten wir eine Schaltung mit der Funktion $Y = (A \wedge B) \vee C$:

Diese Schaltung besteht aus einem AND-Gatter (A und B) gefolgt von einem OR-Gatter, das das AND-Ergebnis mit C verknüpft.

Wahrheitstabelle:

A	B	C	A	B	(A $\wedge$ B)	C
0	0	0	0		0	
0	0	1	0		0	1
0	1	0	0	0	0	
0	1	1	0	0	0	1
1	0	0	0	0	0	
1	0	1	0	0	0	1
1	1	0	1	1	1	
1	1	1	1	1	1	1

Interpretation: Der Ausgang Y ist wahr, wenn entweder (A und B beide wahr sind) oder C wahr ist.

[!NOTE] **Schaltungsentwurf:** Komplexe logische Funktionen werden systematisch in Gatter-Schaltungen umgesetzt. Jeder logische Operator entspricht einem Gatter, und die Gatter werden entsprechend der Klammersetzung verschaltet.

Praktische Anwendungen Boolescher Algebra

Die Boolesche Algebra und logische Schaltungen finden sich überall in der modernen Technik:

1. **Prozessor-Architektur:** Jede Rechenoperation in einem Prozessor basiert auf Millionen miteinander verschalteten Gattern. Eine einfache Addition wird durch komplexe Kombinationen von AND-, OR- und NOT-Gattern realisiert.

2. **Speicherbausteine:** RAM und Flash-Speicher verwenden logische Gatter, um Daten zu speichern und abzurufen. Ein einzelnes Bit im Speicher wird durch eine Schaltung aus Gattern gehalten.
3. **Steuerungstechnik:** Industrielle SPS (Speicherprogrammierbare Steuerungen) verwenden Boolesche Logik, um Fertigungsanlagen zu steuern. Sensorsignale werden logisch verknüpft, um Aktoren zu schalten.
4. **Datenübertragung:** Fehlererkennungs- und Fehlerkorrektur-Codes in Netzwerken und Speichermedien basieren auf Boolescher Algebra (z.B. Paritätsbits, CRC-Prüfsummen).
5. **Software-Entwicklung:** Bedingungen in Programmiersprachen (`if`, `while`) basieren direkt auf Boolescher Algebra. Jede Verzweigung im Programmablauf entspricht einer logischen Verknüpfung.

[!TIP] **Vom Gatter zum Prozessor:** Ein moderner Prozessor enthält mehrere Milliarden Transistoren, die zu logischen Gattern zusammenschaltet sind. Die fundamentalen Operationen AND, OR und NOT bilden die Basis für alle höheren Funktionen.

Zusammenfassung Theorie

Die wichtigsten Erkenntnisse dieses Theorieteils:

- Die **Boolesche Algebra** kennt nur zwei Zustände (0 und 1, falsch und wahr) und bildet das Fundament der Digitaltechnik.
- Die drei **fundamentalen Operatoren** sind **AND** (wahr nur wenn alle Eingänge wahr), **OR** (wahr wenn mindestens ein Eingang wahr) und **NOT** (Invertierung).
- **Wahrheitstabellen** stellen alle möglichen Eingangskombinationen und die zugehörigen Ausgänge systematisch dar. Bei n Eingängen gibt es 2^n Zeilen.
- **Logische Schaltungen** setzen Boolesche Ausdrücke mit elektronischen Gattern um. Serienschaltungen entsprechen AND, Parallelschaltungen entsprechen OR.
- Die Anwendungen reichen von einzelnen Schaltungen bis zu komplexen Prozessoren mit Milliarden von Gattern.

Teil 2: Python-Praxis - Variablen Management & Datentypen

[!WARNING] **Python-Konsistenz beachten:** Prüfe `../python_topics.md` für bereits eingeführte Konzepte!

Überblick

In den vorherigen Vorlesungen haben wir Variablen bereits verwendet und mit verschiedenen Datentypen gearbeitet. In dieser Lektion vertiefen wir das Verständnis für Pythons **Typsystem** und lernen, wie wir Variablen gezielt verwalten, Typen konvertieren und überprüfen können.

Python ist eine **dynamisch typisierte** Sprache: Der Typ einer Variable wird zur Laufzeit automatisch aus dem zugewiesenen Wert abgeleitet. Dies unterscheidet Python von **statisch typisierten** Sprachen wie C oder Java, wo Variablen einen festen, vorab deklarierten Typ haben. Dynamische Typisierung macht Python flexibel und einfach zu schreiben, erfordert aber ein gutes Verständnis der verschiedenen Datentypen und ihrer Eigenschaften.

Die vier grundlegenden Datentypen

Python bietet mehrere eingebaute Datentypen (Built-in Types). Die vier wichtigsten für den Einstieg sind `int`, `float`, `str` und `bool`.

Integer (`int`) – Ganzzahlen Der Datentyp **Integer** (`int`) repräsentiert **ganze Zahlen** ohne Nachkommastellen. In Python 3 haben Integers keine Größenbeschränkung – sie können beliebig groß werden, begrenzt nur durch den verfügbaren Arbeitsspeicher.

[!NOTE] **Integer (`int`):** Ein Datentyp für ganze Zahlen (positiv, negativ oder null) ohne Nachkommastellen. Python 3 unterstützt beliebig große Integers.

[!TIP]

Eigenschaften von Integers: - Keine Nachkommastellen - Unbegrenzte Größe in Python 3 (in Python 2 gab es `int` und `long`) - Arithmetische Operationen ergeben wieder Integers, außer Division (/)

[!WARNING] **Division erzeugt Float:** Die Division mit / erzeugt **immer** einen Float, auch wenn das Ergebnis ganzzahlig ist: `10 / 2` ergibt `5.0`, nicht `5`. Für ganzzahlige Division verwende `//` (Floor Division).

Float (`float`) – Fließkommazahlen Der Datentyp **Float** (`float`) repräsentiert **Fließkommazahlen** mit Nachkommastellen. Python verwendet intern den IEEE 754 Double Precision Standard (64 Bit), den wir in V02 ausführlich behandelt haben.

[!NOTE] **Float (`float`):** Ein Datentyp für Zahlen mit Dezimalstellen (Fließkommazahlen). Python verwendet IEEE 754 Double Precision (64 Bit).

[!TIP]

Eigenschaften von Floats: - Erlauben Nachkommastellen - Wissenschaftliche Notation möglich: $1.5e3 = 1.5 \times 10^3 = 1500.0$ - Begrenzte Präzision (ca. 15-17 Dezimalstellen) - Rundungsfehler möglich (siehe V02)

[!WARNING] **Rundungsfehler:** Floats haben begrenzte Präzision. $0.1 + 0.2$ ergibt 0.30000000000000004 , nicht exakt 0.3 . Für Geld-Berechnungen verwende das `decimal`-Modul!

String (str) – Zeichenketten Der Datentyp **String (str)** repräsentiert **Zeichenketten** (Text). Strings sind in Python **unveränderlich** (immutable) – einmal erzeugt, kann ihr Inhalt nicht mehr geändert werden.

[!NOTE] **String (str):** Ein Datentyp für Textdaten (Zeichenketten). Strings sind unveränderlich (immutable) und werden mit Anführungszeichen definiert.

[!TIP]

Eigenschaften von Strings: - Definiert mit `"..."` oder `'...'` (beides gleichwertig) - Mehrzeilige Strings mit `"""..."""` oder `'''...'''` - Unveränderlich: `text[0] = 'X'` führt zu einem Fehler - Umfangreiche String-Methoden verfügbar (`.upper()`, `.lower()`, `.split()`, etc.)

[!WARNING] **Unveränderlichkeit:** Strings können nicht verändert werden. Operationen wie `text.upper()` erzeugen einen **neuen** String, der originale bleibt unverändert. Weise das Ergebnis einer Variable zu: `text = text.upper()`.

Boolean (bool) – Wahrheitswerte Der Datentyp **Boolean (bool)** repräsentiert **Wahrheitswerte**. Es gibt nur zwei mögliche Werte: **True** (wahr) und **False** (falsch). Booleans sind das Herzstück von Bedingungen und logischen Ausdrücken.

[!NOTE] **Boolean (bool):** Ein Datentyp für Wahrheitswerte mit nur zwei möglichen Werten: **True** (wahr) und **False** (falsch). Wichtig: Großschreibung beachten!

[!TIP]

Eigenschaften von Booleans: - Nur zwei Werte: **True** und **False** (Großschreibung zwingend!) - Entstehen aus Vergleichen: `==`, `!=`, `<`, `>`, `<=`, `>=` - Intern sind Booleans Subtypen von `int`: `True = 1`, `False = 0` - Verwenden in Bedingungen (`if`) und Schleifen (`while`)

[!WARNING] **Großschreibung beachten:** `True` und `False` müssen großgeschrieben werden. `true` oder `false` sind **keine** gültigen Python-Werte und führen zu `NameError`.

Type Casting: Datentypen konvertieren

Oft müssen wir Werte von einem Datentyp in einen anderen umwandeln. Python bietet hierfür die **Type Casting**-Funktionen `int()`, `float()`, `str()` und `bool()`.

[!NOTE] **Type Casting:** Die explizite Konvertierung eines Wertes von einem Datentyp in einen anderen. Python führt manche Konvertierungen automatisch durch (implizites Casting), andere müssen explizit angefordert werden.

String zu Zahl: `int()` und `float()` Die Funktionen `int()` und `float()` wandeln Strings in Zahlen um. Dies ist besonders wichtig, da `input()` immer einen String zurückgibt.

[!TIP]

[!WARNING] **ValueError bei ungültiger Konvertierung:** Wenn der String keine gültige Zahl enthält, wirft Python einen `ValueError`:

Lösung: Prüfe die Eingabe vorher oder fange den Fehler mit `try-except` ab (kommt in V09).

Zahl zu String: `str()` Die Funktion `str()` wandelt Zahlen (und andere Typen) in Strings um. Dies ist nützlich für String-Konkatenation und formatierte Ausgaben.

[!TIP]

[!WARNING] **Typ-Mismatch bei Konkatenation:** Man kann Strings nicht direkt mit Zahlen konkatenieren:

Lösung: Verwende `str(25)` oder (besser) f-Strings: `f"Alter: {25}"`.

Float zu Integer: Vorsicht bei Rundung Die Konvertierung von Float zu Integer mit `int()` schneidet die Nachkommastellen **ab** (Floor-Operation für positive Zahlen, Ceiling für negative). Es findet **keine** kaufmännische Rundung statt!

[!TIP]

[!WARNING] **`int()` schneidet ab, rundet nicht:** Für mathematisch korrektes Runden verwende die `round()`-Funktion. `int()` entfernt einfach die Nachkommastellen.

Konvertierung zu Boolean: `bool()` Die Funktion `bool()` konvertiert Werte zu Booleans. Python hat klare Regeln, welche Werte als **False** gelten (**Falsy Values**) und welche als **True** (**Truthy Values**).

[!NOTE] **Falsy Values:** Werte, die in Booleschem Kontext als **False** interpretiert werden: 0, 0.0, "" (leerer String), [] (leere Liste), None.

Truthy Values: Alle anderen Werte gelten als **True**.

[!TIP]

Type Checking: Datentypen überprüfen

Um den Typ einer Variable zur Laufzeit zu überprüfen, bietet Python zwei Hauptmethoden: `type()` und `isinstance()`.

type() – Typ einer Variable ermitteln Die Funktion `type()` gibt den **exakten Typ** eines Objekts zurück.

[!NOTE] **type(obj):** Gibt den Datentyp des übergebenen Objekts zurück. Signatur: `type(obj) → type`.

[!TIP]

isinstance() – Überprüfung mit Vererbung Die Funktion `isinstance()` prüft, ob ein Objekt eine **Instanz** eines bestimmten Typs (oder einer seiner Oberklassen) ist. Sie ist flexibler als `type()`, da sie **Vererbung** berücksichtigt.

[!NOTE] **isinstance(obj, classinfo):** Prüft, ob `obj` eine Instanz von `classinfo` ist. Berücksichtigt Vererbung. Signatur: `isinstance(obj, classinfo) → bool`.

[!TIP]

[!WARNING] **type() vs. isinstance():** Verwende `isinstance()` für Typ-Checks, besonders wenn Vererbung eine Rolle spielt. `bool` ist ein Subtyp von `int`, daher ist `isinstance(True, int) → True`, aber `type(True) == int → False`.

Unveränderliche vs. veränderliche Typen

Ein zentrales Konzept in Python ist die Unterscheidung zwischen **unveränderlichen** (immutable) und **veränderlichen** (mutable) Datentypen.

[!NOTE] **Immutable (unveränderlich):** Objekte, deren Wert nach der Erzeugung nicht mehr geändert werden kann. Operationen erzeugen neue Objekte. Beispiele: `int`, `float`, `str`, `tuple`, `bool`.

Mutable (veränderlich): Objekte, deren Inhalt nach der Erzeugung verändert werden kann. Operationen modifizieren das Objekt direkt. Beispiele: `list`, `dict`, `set`.

Unveränderliche Typen: `int`, `float`, `str`, `bool` Alle grundlegenden Datentypen (`int`, `float`, `str`, `bool`) sind **unveränderlich**. Wenn man sie “verändert”, erzeugt Python im Hintergrund ein neues Objekt.

[!TIP]

Warum ist das wichtig? - **Performance:** Unveränderliche Objekte können effizient geteilt werden (String Interning) - **Sicherheit:** Unveränderliche Objekte können als Dictionary-Keys verwendet werden - **Funktionsparameter:** Keine unerwarteten Seiteneffekte bei Funktionsaufrufen

[!WARNING] **String-Modifikation ist ineffizient:** Viele String-Konkationen in Schleifen sind langsam, da jedes Mal ein neues Objekt erzeugt wird. Verwende stattdessen `''.join(liste)` oder f-Strings.

Veränderliche Typen: `list`, `dict`, `set` Die Datenstrukturen `list`, `dict` und `set` (kommen in V08) sind **veränderlich**. Sie können nach ihrer Erzeugung modifiziert werden, ohne dass ein neues Objekt entsteht.

[!TIP]

Konsequenzen: - **Seiteneffekte möglich:** Änderungen an einer Liste wirken sich auf alle Referenzen aus - **Nicht als Dictionary-Keys:** Veränderliche Objekte können nicht als Keys verwendet werden - **Funktionsparameter:** Vorsicht bei Funktionen, die Listen modifizieren!

Variablen-Scope: Lokal vs. Global

Der **Scope** (Gültigkeitsbereich) einer Variable bestimmt, wo im Code auf sie zugegriffen werden kann. Python unterscheidet zwischen **lokalen** und **globalen** Variablen.

[!NOTE] **Scope (Gültigkeitsbereich):** Der Bereich im Code, in dem eine Variable sichtbar und verwendbar ist.

Globale Variable: Eine Variable, die außerhalb aller Funktionen definiert ist und im gesamten Modul sichtbar ist.

Lokale Variable: Eine Variable, die innerhalb einer Funktion definiert ist und nur dort sichtbar ist.

Globale Variablen Variablen, die **außerhalb** von Funktionen definiert werden, sind **global** und können überall im Modul gelesen werden.

[!TIP]

Lokale Variablen Variablen, die **innerhalb** einer Funktion definiert werden, sind **lokal** und nur innerhalb dieser Funktion sichtbar.

[!TIP]

[!WARNING] **Namenskonflikte:** Wenn eine lokale Variable denselben Namen hat wie eine globale, **überdeckt** die lokale Variable die globale innerhalb der Funktion:

Das global-Keyword Um eine **globale Variable innerhalb einer Funktion zu modifizieren**, muss sie mit dem `global`-Keyword deklariert werden.

[!NOTE] **global-Keyword:** Deklariert, dass eine Variable innerhalb einer Funktion die globale Variable mit diesem Namen referenziert, nicht eine neue lokale Variable.

[!TIP]

[!WARNING] **global sparsam verwenden:** Globale Variablen, die von Funktionen verändert werden, machen Code schwer verständlich und fehleranfällig. Bevorzuge **Funktionsparameter und Rückgabewerte** statt globaler Variablen. Verwende `global` nur, wenn wirklich notwendig (z.B. für Zähler oder Konfiguration).

Multiple Assignment und Value Unpacking

Python erlaubt elegante Kurzschreibweisen für die Zuweisung mehrerer Variablen gleichzeitig.

Multiple Assignment Man kann **mehreren Variablen gleichzeitig denselben Wert** zuweisen:

[!TIP]

[!WARNING] **Vorsicht bei mutable Typen:** Bei veränderlichen Typen (Listen) zeigen alle Variablen auf **dasselbe** Objekt:

Value Unpacking (Tuple Unpacking) Python erlaubt das **gleichzeitige Zuweisen mehrerer Werte** an mehrere Variablen in einer Zeile:

[!NOTE] **Value Unpacking:** Das Zuweisen mehrerer Werte gleichzeitig durch komma-getrennte Notation. Intern arbeitet Python mit Tupeln.

[!TIP]

[!WARNING] **Anzahl muss übereinstimmen:** Die Anzahl der Variablen links muss mit der Anzahl der Werte rechts übereinstimmen:

Häufige Fehler und Lösungen

Fehler 1: Type Mismatch bei Operationen

[!WARNING] **Fehler:** Versuch, inkompatible Typen zu verknüpfen:

Lösung: Verwende `str()` zur Konvertierung oder (besser) f-Strings:

Fehler 2: Division durch Null

[!WARNING] **Fehler:** Division durch Null ist mathematisch undefiniert:

Lösung: Prüfe den Nenner vor der Division:

Fehler 3: Falsche Annahme über Unveränderlichkeit

[!WARNING] **Fehler:** Annahme, dass String-Methoden den originalen String ändern:

Lösung: Weise das Ergebnis der Variable zu:

Fehler 4: Verwechslung von = und ==

[!WARNING] **Fehler:** Zuweisung (=) statt Vergleich (==) in Bedingungen:

Lösung: Verwende == für Vergleiche:

Zusammenfassung Python

Die wichtigsten Erkenntnisse zu Variablen und Datentypen in Python:

- Die **vier grundlegenden Datentypen** sind `int` (Ganzzahlen), `float` (Fließkommazahlen), `str` (Strings) und `bool` (Wahrheitswerte).
- **Type Casting** mit `int()`, `float()`, `str()`, `bool()` konvertiert zwischen Datentypen. Achtung: `int()` schneidet Nachkommastellen ab, rundet nicht!
- **Type Checking** mit `type()` ermittelt den exakten Typ, `isinstance()` prüft unter Berücksichtigung von Vererbung.
- **Unveränderliche Typen** (`int`, `float`, `str`, `bool`, `tuple`) können nach Erzeugung nicht modifiziert werden. Operationen erzeugen neue Objekte.
- **Veränderliche Typen** (`list`, `dict`, `set`) können modifiziert werden, ohne dass neue Objekte entstehen.
- **Variablen-Scope:** Globale Variablen sind überall sichtbar, lokale nur innerhalb ihrer Funktion. `global`-Keyword ermöglicht Modifikation globaler Variablen in Funktionen.
- **Multiple Assignment** und **Value Unpacking** erlauben elegante Mehrfachzuweisungen: `x, y = 1, 2` oder Variablen-Tausch: `a, b = b, a`.

Neue Python-Funktionen/Methoden

In dieser Lektion wurden folgende Python-Funktionen **neu eingeführt**:

- **type(obj)** (Built-in): Gibt den Datentyp des Objekts zurück. Signatur: `type(obj) → type`.
 - **isinstance(obj, classinfo)** (Built-in): Prüft, ob `obj` eine Instanz von `classinfo` ist (berücksichtigt Vererbung). Signatur: `isinstance(obj, classinfo) → bool`.
 - **bool(x)** (Built-in): Konvertiert Wert in Boolean (bereits in V01 erwähnt als Datentyp, hier als Casting-Funktion).
 - **round(number, ndigits=None)** (Built-in): Rundet eine Zahl auf `ndigits` Dezimalstellen (Banker's Rounding). Signatur: `round(number, ndigits=None) → float` oder `int`.
 - **id(obj)** (Built-in): Gibt die Identität (Speicheradresse) eines Objekts zurück. Signatur: `id(obj) → int`.
 - **global-Keyword**: Deklariert Variable als global innerhalb einer Funktion.
 - **min(iterable) / max(iterable)** (Built-in): Gibt das kleinste/größte Element zurück. Signatur: `min(iterable) → Element`, `max(iterable) → Element`.
-

Weiterführende Ressourcen

Theorie

- **Buch**: “Digital Design” von Morris Mano & Michael Ciletti (Kapitel 2: Boolesche Algebra)
- **Online**: “Boolean Algebra Tutorial” auf Electronics Tutorials: https://www.electronics-tutorials.ws/boolean/bool_1.html
- **Video**: “But what is a GPT?” von 3Blue1Brown (Bezug zu logischen Gattern): <https://www.youtube.com/watch?v=wjZofJX0v4M>

Python

- **Python-Dokumentation**: Built-in Types: <https://docs.python.org/3/library/stdtypes.html>
 - **Real Python Tutorial**: “Basic Data Types in Python”: <https://realpython.com/python-data-types/>
 - **Python-Dokumentation**: Type Hierarchy: <https://docs.python.org/3/reference/datamodel.html#the-standard-type-hierarchy>
 - **Tutorial**: “Python's type() and isinstance(): Validate Data Types”: <https://realpython.com/python-type-checking/>
-

Datei: lessons/V04-Boolsche-Algebra-Teil2/V04-Boolsche-Algebra-Teil2_skript.md

V04: Boolsche Algebra & Logische Schaltungen – Teil 2

[!NOTE] **Lernziele dieser Vorlesung:** - Erweiterte logische Operatoren **XOR**, **NAND** und **NOR** verstehen und anwenden können - **De Morgan'sche Gesetze** zur Vereinfachung logischer Ausdrücke einsetzen - Komplexe logische Schaltungen wie **Halbaddierer** und **Volladdierer** analysieren und konstruieren - **Multiplexer** und **Demultiplexer** als fundamentale digitale Bausteine verstehen - Python-**Vergleichsoperatoren** (**==**, **!=**, **<**, **>**, **<=**, **>=**) sicher verwenden - Logische Operatoren in Python (**and**, **or**, **not**) zur Formulierung komplexer Bedingungen nutzen - **Kurzschlussauswertung** verstehen und gezielt einsetzen - Unterschied zwischen **Truthy** und **Falsy** Values in Python kennen

Teil 1: Theorie - Boolsche Algebra & Logische Schaltungen – Teil 2

Überblick

In der vorherigen Vorlesung haben wir die grundlegenden logischen Operatoren **AND**, **OR** und **NOT** kennengelernt. Diese Operatoren bilden das Fundament der digitalen Logik und ermöglichen es uns, einfache logische Entscheidungen zu treffen. In dieser Vorlesung erweitern wir unser Repertoire um leistungsfähigere Operatoren und lernen, wie diese in praktischen digitalen Schaltungen eingesetzt werden. Die **Boolsche Algebra** ist nicht nur theoretisches Fundament der Informatik, sondern bildet die Grundlage für jeden modernen Computer, jedes Smartphone und jede digitale Steuerung in Maschinen und Anlagen. Maschinenbau-Ingenieure begegnen digitaler Logik in Steuerungssystemen, Sensornetzwerken und der Automatisierungstechnik. Die in dieser Vorlesung gelernten Konzepte ermöglichen es, komplexe Steuerlogiken zu verstehen, zu analysieren und selbst zu entwerfen.

Erweiterte Logische Operatoren

XOR – Exklusives ODER Der **XOR-Operator** (gesprochen “Ex-Or”, von “Exclusive OR”) ist ein zweistelliger logischer Operator, der nur dann den Wert 1 (wahr) liefert, wenn genau einer der beiden Eingänge 1 ist, aber nicht beide gleichzeitig. Der XOR-Operator wird durch das Symbol $\oplus$ dargestellt und ist fundamental für viele digitale Schaltungen, insbesondere in der Arithmetik und Verschlüsselung.

[!NOTE] **XOR (Exklusives ODER)**: Ein logischer Operator, der genau dann 1 liefert, wenn die Anzahl der Eingänge mit Wert 1 ungerade ist. Bei zwei Eingängen bedeutet dies: Genau einer ist 1, der andere 0.

Die **Wahrheitstabelle** für XOR ($A \oplus B$) lautet:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Eine wichtige Eigenschaft: XOR erkennt **Ungleichheit**. $A \oplus B = 1$ bedeutet A $\neq$ B.

[!TIP] **Praktisches Beispiel – Lichtschalter-Schaltung**: Stellen Sie sich zwei Lichtschalter vor, die eine Lampe steuern (Wechselschaltung). Die Lampe soll leuchten, wenn genau ein Schalter gedrückt ist. Dies entspricht exakt einem XOR-Gatter: Schalter 1 = A, Schalter 2 = B, Lampe = $A \oplus B$. Diese Schaltung findet sich in jedem Treppenhaus, wo man das Licht von oben oder unten an- und ausschalten kann.

Mathematische Darstellung durch Grundoperatoren: XOR lässt sich durch AND, OR und NOT ausdrücken:

$$A \oplus B = (A \wedge \neg B) \vee (\neg A \wedge B)$$

Dies bedeutet: XOR ist wahr, wenn (A ist wahr UND B ist falsch) ODER (A ist falsch UND B ist wahr).

Wichtige XOR-Eigenschaften: - **Kommutativität**: $A \oplus B = B \oplus A$ - **Assoziativität**: $(A \oplus B) \oplus C = A \oplus (B \oplus C)$ - **Identitätselement**: $A \oplus 0 = A$ (XOR mit 0 lässt Wert unverändert) - **Selbstinverse**: $A \oplus A = 0$ (XOR mit sich selbst ergibt 0) - **Doppelte Negation**: $A \oplus B \oplus B = A$ (zweimal XOR mit B hebt sich auf)

Diese Eigenschaften machen XOR besonders nützlich für Verschlüsselung und Fehlerkorrektur.

NAND – Nicht-UND Der **NAND-Operator** (von “NOT AND”) ist die Negation des AND-Operators. NAND liefert nur dann 0, wenn beide Eingänge 1 sind – in allen anderen Fällen liefert NAND den Wert 1. NAND hat eine besondere Bedeutung in der digitalen Elektronik, da es **funktional vollständig** ist: Jede beliebige logische Funktion lässt sich ausschließlich durch NAND-Gatter realisieren.

[!NOTE] **NAND (Nicht-UND)**: Ein logischer Operator, der die Negation von AND darstellt. Symbolisch: $\overline{A \wedge B}$. NAND ist funktional vollständig, das heißt, alle anderen logischen Operatoren (AND, OR, NOT, XOR) können ausschließlich durch NAND-Verknüpfungen konstruiert werden.

Die **Wahrheitstabelle** für NAND ($A \uparrow B$ oder $\neg(A \wedge B)$):

A	B	$A \uparrow B$
0	0	1
0	1	1
1	0	1
1	1	0

[!TIP] **Warum ist NAND so wichtig?** In der Halbleitertechnologie sind NAND-Gatter technisch einfacher und schneller zu realisieren als andere Gatter. Deshalb werden in der Praxis oft komplexe Schaltungen ausschließlich mit NAND-Gattern aufgebaut, auch wenn theoretisch andere Operatoren direkter wären. Dies reduziert die Produktionskosten und verbessert die Performance.

Funktionale Vollständigkeit von NAND – Konstruktion aller Grundoperatoren:

1. **NOT aus NAND**: $\neg A = \overline{A \wedge A}$ (beide Eingänge gleich)
2. **AND aus NAND**: $A \wedge B = \overline{\overline{A \wedge B}}$ (doppelte Negation)
3. **OR aus NAND**: $A \vee B = \overline{(\neg A) \wedge (\neg B)} = \overline{\overline{A} \wedge \overline{B}}$

Diese Eigenschaft macht NAND zum universellen Baustein digitaler Schaltungen.

NOR – Nicht-ODER Der **NOR-Operator** (von “NOT OR”) ist die Negation des OR-Operators. NOR liefert nur dann 1, wenn beide Eingänge 0 sind – ansonsten liefert NOR den Wert 0. NOR ist ebenso wie NAND **funktional vollständig**, das heißt, alle logischen Operationen lassen sich ausschließlich durch NOR-Gatter konstruieren.

[!NOTE] **NOR (Nicht-ODER)**: Ein logischer Operator, der die Negation von OR darstellt. Symbolisch: $\overline{A \vee B}$. NOR ist funktional vollständig und wird oft in speziellen Schaltungsfamilien eingesetzt.

Die **Wahrheitstabelle** für NOR ($A \downarrow B$ oder $\neg(A \vee B)$):

A	B	$A \downarrow B$
0	0	1
0	1	0

A	B	$A \downarrow B$
1	0	0
1	1	0

Funktionale Vollständigkeit von NOR – Konstruktion aller Grundoperatoren:

1. **NOT aus NOR:** $\neg A = \overline{A \downarrow A}$
2. **OR aus NOR:** $A \vee B = \overline{\overline{A \downarrow B}}$ (doppelte Negation)
3. **AND aus NOR:** $A \wedge B = \overline{(\neg A) \vee (\neg B)} = \overline{\overline{A \downarrow A} \vee \overline{B \downarrow B}}$

[!WARNING] **Verwechslungsgefahr:** Anfänger verwechseln oft NAND und NOR, weil beide “negierte” Operatoren sind. Merkhilfe: NAND ist das Gegenteil von AND (beide Eingänge müssen wahr sein für Falsch bei NAND), während NOR das Gegenteil von OR ist (mindestens ein Eingang muss wahr sein für Falsch bei NOR).

De Morgan’sche Gesetze

Die **De Morgan’schen Gesetze** sind fundamentale Umformungsregeln der Booleschen Algebra, benannt nach dem britischen Mathematiker Augustus De Morgan (1806–1871). Diese Gesetze beschreiben, wie die Negation einer Konjunktion (UND) bzw. Disjunktion (ODER) in äquivalente Ausdrücke transformiert werden kann. Sie sind unverzichtbar für die Vereinfachung logischer Schaltungen und die Optimierung digitaler Designs.

[!NOTE] **De Morgan’sche Gesetze:** Zwei fundamentale Äquivalenzregeln der Booleschen Algebra: 1. $\neg(A \wedge B) = \neg A \vee \neg B$ – Die Negation eines UND ist ein ODER der negierten Eingänge 2. $\neg(A \vee B) = \neg A \wedge \neg B$ – Die Negation eines ODER ist ein UND der negierten Eingänge

Erstes De Morgan’sches Gesetz:

$$\neg(A \wedge B) = \neg A \vee \neg B$$

Dies besagt: “Nicht (A und B)” ist äquivalent zu “(nicht A) oder (nicht B)”. Anschaulich: Wenn nicht beide Bedingungen gleichzeitig erfüllt sind, dann ist mindestens eine Bedingung nicht erfüllt.

Beweis durch Wahrheitstabelle:

A	B	A	B	$\neg(A \wedge B)$	$\neg A$	$\neg B$	$\neg A \vee \neg B$
0	0	0	0	1	1	1	1
0	1	0	1	1	1	0	1
1	0	1	0	1	0	1	1

A	B	A	B	$\neg(A \wedge B)$	$\neg A$	$\neg B$	$\neg A$	$\neg B$
1	1	1		0	0	0	0	

Die Spalten für $\neg(A \wedge B)$ und $\neg A \vee \neg B$ sind identisch – die Ausdrücke sind äquivalent.

Zweites De Morgan'sches Gesetz:

$$\neg(A \vee B) = \neg A \wedge \neg B$$

Dies besagt: “Nicht (A oder B)” ist äquivalent zu “(nicht A) und (nicht B)”. Anschaulich: Wenn keine der beiden Bedingungen erfüllt ist, dann sind beide Bedingungen nicht erfüllt.

Beweis durch Wahrheitstabelle:

A	B	A	B	$\neg(A \vee B)$	$\neg A$	$\neg B$	$\neg A$	$\neg B$
0	0	0		1	1	1	1	
0	1	1		0	1	0	0	
1	0	1		0	0	1	0	
1	1	1		0	0	0	0	

Auch hier sind die Spalten für $\neg(A \vee B)$ und $\neg A \wedge \neg B$ identisch.

[!TIP] **Merkhilfe für De Morgan:** Beim Negieren einer Klammer werden aus UND-Verknüpfungen OR-Verknüpfungen (und umgekehrt), und jeder Operand wird einzeln negiert. Stellen Sie sich vor, Sie “verteilen” die Negation nach innen und “flippen” den Operator: - $\neg(A \text{ AND } B) \rightarrow (\neg A) \text{ OR } (\neg B)$ - $\neg(A \text{ OR } B) \rightarrow (\neg A) \text{ AND } (\neg B)$

Praktische Anwendung – Schaltungsvereinfachung:

Die De Morgan'schen Gesetze erlauben es, komplexe logische Ausdrücke zu vereinfachen, was in der Praxis zu weniger Gattern und damit günstigeren, schnelleren und energieeffizienteren Schaltungen führt.

Beispiel: Vereinfache $\neg(A \wedge \neg B \wedge C)$

Anwendung des ersten De Morgan'schen Gesetzes:

$$\neg(A \wedge \neg B \wedge C) = \neg A \vee \neg(\neg B) \vee \neg C = \neg A \vee B \vee \neg C$$

Die ursprünglich komplexe dreifache UND-Verknüpfung mit Negation wird zu einer einfacheren ODER-Verknüpfung.

[!WARNING] **Häufiger Fehler bei De Morgan:** Studierende vergessen oft, beim Anwenden von De Morgan auch die einzelnen Operanden zu negieren. Falsch wäre: $\neg(A \wedge B) = A \vee B$ (Negation fehlt auf A und B!). Richtig ist: $\neg(A \wedge B) = \neg A \vee \neg B$.

Logische Schaltungen – Addierer

Digitale Computer arbeiten intern mit binären Zahlen, und die grundlegendste arithmetische Operation ist die Addition. **Addierer** sind logische Schaltungen, die binäre Zahlen addieren. Sie sind fundamentale Bausteine in der **Arithmetisch-Logischen Einheit (ALU)** eines jeden Prozessors. Wir betrachten zwei Arten von Addierern: den **Halbaddierer** (Half Adder) und den **Volladdierer** (Full Adder).

Halbaddierer (Half Adder) Ein **Halbaddierer** addiert zwei einzelne Bits A und B und erzeugt zwei Ausgänge: die **Summe S** (engl. Sum) und den **Übertrag C** (engl. Carry). Der Halbaddierer berücksichtigt keinen Übertrag von einer vorherigen Addition, daher der Name “Halb”-Addierer.

[!NOTE] **Halbaddierer (Half Adder):** Eine digitale Schaltung, die zwei Bits addiert und eine Summe sowie einen Übertrag erzeugt. Der Halbaddierer hat zwei Eingänge (A, B) und zwei Ausgänge (Summe S, Übertrag C).

Wahrheitstabelle des Halbaddierers:

A	B	S (Summe)	C (Übertrag)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Analyse der Ausgänge:

- **Summe S:** Die Summe ist genau dann 1, wenn genau ein Eingang 1 ist – dies entspricht der XOR-Funktion: $S = A \oplus B$
- **Übertrag C:** Ein Übertrag tritt nur auf, wenn beide Eingänge 1 sind – dies entspricht der AND-Funktion: $C = A \wedge B$

Logische Gleichungen:

$$S = A \oplus B$$

$$C = A \wedge B$$

Schaltplan des Halbaddierers:

[!TIP] **Binäre Addition verstehen:** Bei der Addition von $1 + 1$ im Binärsystem erhalten wir 10 (dezimal 2). Die niedrigste Stelle ist 0 (Summe S), und wir haben einen Übertrag 1 zur nächsthöheren Stelle (Carry C). Der Halbaddierer modelliert genau diese Operation für einzelne Bits.

Beispiel – Rechnung mit dem Halbaddierer:

Addition von $A = 1$ und $B = 1$: - Summe: $S = 1 \oplus 1 = 0$ - Übertrag: $C = 1 \wedge 1 = 1$ - Ergebnis: 10 (binär) = 2 (dezimal)

Volladdierer (Full Adder) Ein **Volladdierer** erweitert den Halbaddierer, indem er einen zusätzlichen Eingang für einen **Übertrag von einer vorherigen Stelle** (Cin, Carry-In) berücksichtigt. Der Volladdierer ist die Grundlage für die Addition mehrstelliger binärer Zahlen, da er Überträge zwischen den Stellen propagieren kann.

[!NOTE] **Volladdierer (Full Adder):** Eine digitale Schaltung, die drei Bits addiert (A, B und ein Übertrag Cin) und eine Summe (S) sowie einen neuen Übertrag (Cout) erzeugt. Der Volladdierer hat drei Eingänge (A, B, Cin) und zwei Ausgänge (S, Cout).

Wahrheitstabelle des Volladdierers:

A	B	Cin	S (Summe)	Cout (Übertrag)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Analyse der Ausgänge:

- **Summe S:** Die Summe ist 1, wenn eine ungerade Anzahl der drei Eingänge 1 ist. Dies entspricht einer dreifachen XOR-Verknüpfung:

$$S = A \oplus B \oplus Cin$$
- **Übertrag Cout:** Ein Übertrag tritt auf, wenn mindestens zwei der drei Eingänge 1 sind. Dies lässt sich durch folgende Formel ausdrücken:

$$Cout = (A \wedge B) \vee (A \wedge Cin) \vee (B \wedge Cin)$$

Logische Gleichungen:

$$S = A \oplus B \oplus Cin$$

$$Cout = (A \wedge B) \vee (A \wedge Cin) \vee (B \wedge Cin)$$

Alternative kompaktere Form für Cout (unter Verwendung der XOR-Eigenschaft):

$$Cout = (A \wedge B) \vee ((A \oplus B) \wedge Cin)$$

Konstruktion aus zwei Halbaddierern:

Ein Volladdierer lässt sich elegant aus zwei Halbaddierern und einem OR-Gatter konstruieren:

1. **Erster Halbaddierer** addiert A und B:
 - Zwischensumme: $S_1 = A \oplus B$
 - Zwischenübertrag: $C_1 = A \wedge B$
2. **Zweiter Halbaddierer** addiert S_1 und Cin:
 - Endgültige Summe: $S = S_1 \oplus Cin = A \oplus B \oplus Cin$
 - Zweiter Zwischenübertrag: $C_2 = S_1 \wedge Cin$
3. **OR-Gatter** verknüpft die beiden Überträge:
 - Endgültiger Übertrag: $Cout = C_1 \vee C_2$

Schaltplan des Volladdierers (aus Halbaddierern):

[!TIP] **Ripple-Carry-Addierer**: Um mehrstellige Binärzahlen zu addieren (z.B. zwei 8-Bit-Zahlen), kettet man Volladdierer hintereinander. Der Übertrag (Cout) jeder Stelle wird zum Carry-In (Cin) der nächsthöheren Stelle. Diese Architektur heißt **Ripple-Carry-Addierer**, da der Übertrag sich von der niedrigsten zur höchsten Stelle “durchrippet”. Nachteil: Bei breiten Zahlen (z.B. 64 Bit) ist dies langsam, da jede Stelle auf den Übertrag der vorherigen warten muss. Schnellere Designs (Carry-Lookahead-Adder) existieren für High-Performance-Prozessoren.

Beispiel – 4-Bit-Addition:

Addiere 1011 (binär, dezimal 11) und 0110 (binär, dezimal 6):

Schritt-für-Schritt: - Stelle 0: A=1, B=0, Cin=0 → S=1, Cout=0 - Stelle 1: A=1, B=1, Cin=0 → S=0, Cout=1 - Stelle 2: A=0, B=1, Cin=1 → S=0, Cout=1 - Stelle 3: A=1, B=0, Cin=1 → S=0, Cout=1 - Stelle 4: Cout=1 (finaler Übertrag)

Multiplexer und Demultiplexer

Multiplexer (MUX) und **Demultiplexer** (DEMUX) sind fundamentale digitale Bausteine zur Datenweiterleitung. Sie werden in nahezu jedem digitalen System eingesetzt, von Prozessoren über Kommunikationssysteme bis hin zu Speicherarchitekturen.

Multiplexer (MUX) Ein **Multiplexer** ist eine digitale Schaltung, die aus mehreren Eingangssignalen ein einzelnes Ausgangssignal auswählt. Die Auswahl erfolgt über ein oder mehrere **Steuer-** oder **Select-Leitungen (S)**. Ein Multiplexer funktioniert wie ein steuerbarer digitaler Schalter.

[!NOTE] **Multiplexer (MUX)**: Eine Kombinationsschaltung, die aus 2^n Dateneingängen einen auswählt und zum Ausgang durchschaltet. Die Auswahl erfolgt über n Select-Leitungen. Ein 2:1-MUX hat 2 Dateneingänge und 1 Select-Leitung, ein 4:1-MUX hat 4 Dateneingänge und 2 Select-Leitungen, usw.

Beispiel: 2:1-Multiplexer (2 Eingänge, 1 Select-Leitung):

Eingänge: I0, I1 (Dateneingänge), S (Select)

Ausgang: Y

Wahrheitstabelle:

S	Y
0	I0
1	I1

Bedeutung: Wenn $S = 0$, wird I0 zum Ausgang durchgeschaltet; wenn $S = 1$, wird I1 durchgeschaltet.

Logische Gleichung:

$$Y = (\neg S \wedge I0) \vee (S \wedge I1)$$

Anschaulich: Der Ausgang ist I0 UND (S ist 0) ODER I1 UND (S ist 1).

Schaltplan 2:1-MUX:

Beispiel: 4:1-Multiplexer (4 Eingänge, 2 Select-Leitungen):

Eingänge: I0, I1, I2, I3, S1, S0 (zwei Select-Leitungen)

Ausgang: Y

Wahrheitstabelle:

S1	S0	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

Mit zwei Select-Leitungen kann man $2^2 = 4$ verschiedene Eingänge auswählen.

Logische Gleichung:

$$Y = (\neg S1 \wedge \neg S0 \wedge I0) \vee (\neg S1 \wedge S0 \wedge I1) \vee (S1 \wedge \neg S0 \wedge I2) \vee (S1 \wedge S0 \wedge I3)$$

[!TIP] **Anwendung von Multiplexern – CPU-Datenbus:** In einem Prozessor werden Multiplexer verwendet, um verschiedene Datenquellen (Register, Speicher, ALU-Ergebnis) auf einen gemeinsamen Bus zu schalten. Die CPU-Steuerung wählt über Select-Leitungen, welche Datenquelle gerade aktiv ist. Dies spart Leitungen und reduziert die Komplexität.

Demultiplexer (DEMUX) Ein **Demultiplexer** ist das Gegenstück zum Multiplexer: Er nimmt ein einziges Eingangssignal und leitet es zu einem von mehreren Ausgängen weiter. Die Auswahl des Ausgangs erfolgt ebenfalls über Select-Leitungen.

[!NOTE] **Demultiplexer (DEMUX):** Eine Kombinationsschaltung, die ein Eingangssignal auf einen von 2^n Ausgängen verteilt. Die Auswahl erfolgt über n Select-Leitungen. Ein 1:2-DEMUX hat 1 Dateneingang und 2 Ausgänge, ein 1:4-DEMUX hat 1 Dateneingang und 4 Ausgänge.

Beispiel: 1:2-Demultiplexer:

Eingang: I (Dateneingang), S (Select)

Ausgänge: Y0, Y1

Wahrheitstabelle:

S	Y0	Y1
0	I	0
1	0	I

Bedeutung: Wenn $S = 0$, wird I auf Y0 geleitet (und $Y1 = 0$); wenn $S = 1$, wird I auf Y1 geleitet (und $Y0 = 0$).

Logische Gleichungen:

$$Y0 = \neg S \wedge I$$

$$Y1 = S \wedge I$$

Schaltplan 1:2-DEMUX:

[!TIP] **Anwendung von Demultiplexern – Adressdekodierung:**

In Speichersystemen werden Demultiplexer zur Adressdekodierung verwendet. Eine Adresse (Select-Leitungen) aktiviert genau eine Speicherzelle (Ausgang). Dies ermöglicht den gezielten Zugriff auf einzelne Speicherstellen in einem großen Speicherarray.

[!WARNING] **Verwechslungsgefahr MUX vs. DEMUX:** Anfänger verwechseln oft die Richtung: - **Multiplexer:** Viele Eingänge → Ein Ausgang (mehrere Quellen zu einer Senke) - **Demultiplexer:** Ein Eingang → Viele Ausgänge (eine Quelle zu mehreren Senken)

Merkhilfe: “Multi” deutet auf viele Eingänge hin, “De-Multi” auf das “Aufteilen” (De-multiplizieren) auf viele Ausgänge.

Zusammenfassung Theorie

Die zentralen Konzepte dieser Vorlesung zusammengefasst:

1. **Erweiterte Operatoren:** XOR, NAND und NOR sind leistungsfähige logische Operatoren. XOR erkennt Ungleichheit, NAND und NOR sind funktional vollständig und bilden die Grundlage für praktische Schaltungsdesigns.
2. **De Morgan’sche Gesetze:** Die Negation von AND wird zu OR (und umgekehrt), wobei alle Operanden negiert werden. Diese Gesetze sind essentiell für die Vereinfachung und Optimierung logischer Schaltungen.
3. **Addierer:** Der Halbaddierer addiert zwei Bits, der Volladdierer drei Bits (unter Berücksichtigung eines Übertrags). Durch Kaskadierung von Volladdierern können mehrstellige Binärzahlen addiert werden – dies ist das Fundament jeder CPU-Arithmetik.
4. **Multiplexer und Demultiplexer:** MUX wählt einen von mehreren Eingängen aus, DEMUX verteilt einen Eingang auf mehrere Ausgänge. Beide sind fundamentale Bausteine für Datenweiterleitung und -steuerung in digitalen Systemen.
5. **Praktische Relevanz:** Alle behandelten Konzepte sind nicht nur theoretische Konstrukte, sondern werden in realen Computern, Steuerungssystemen und digitalen Schaltungen täglich milliardenfach eingesetzt.

Teil 2: Python-Praxis - Logische Ausdrücke (Boolsche Algebra)

[!WARNING] **Python-Konsistenz beachten:** Prüfe ../python_topics.md für bereits eingeführte Konzepte! Die Grundlagen zu `bool`, `True`, `False` wurden bereits in V03 eingeführt. In dieser Vorlesung vertiefen wir die praktische Anwendung logischer Ausdrücke.

Überblick

Nachdem wir in der Theorie erweiterte logische Operatoren und Schaltungen kennengelernt haben, wenden wir uns nun der praktischen Umsetzung logischer Ausdrücke in Python zu. Die **Vergleichsoperatoren** und **logischen Operatoren** in Python entsprechen direkt den Konzepten der Booleschen Algebra. Sie ermöglichen es uns, Bedingungen zu formulieren, Entscheidungen zu treffen und komplexe logische Zusammenhänge auszudrücken. Diese Konzepte bilden die Grundlage für Verzweigungen (die wir in V05 ausführlich behandeln) und sind essentiell für jedes nicht-triviale Python-Programm. Maschinenbau-Ingenieure nutzen diese Operatoren zur Formulierung von Steuerungsbedingungen, zur Validierung von Sensordaten und zur Implementierung von Sicherheitslogik in automatisierten Systemen.

Vergleichsoperatoren

Python bietet sechs fundamentale **Vergleichsoperatoren**, die zwei Werte miteinander vergleichen und einen Boolean-Wert (**True** oder **False**) zurückgeben. Diese Operatoren funktionieren nicht nur mit Zahlen, sondern auch mit Strings, Listen und anderen Datentypen.

[!NOTE] **Vergleichsoperatoren:** Operatoren, die zwei Werte miteinander vergleichen und einen Boolean-Wert zurückgeben. Python bietet sechs Vergleichsoperatoren: `==` (gleich), `!=` (ungleich), `<` (kleiner), `>` (größer), `<=` (kleiner-gleich), `>=` (größer-gleich).

Gleichheit und Ungleichheit

- `==` (**Gleichheit**): Prüft, ob zwei Werte **gleich** sind
- `!=` (**Ungleichheit**): Prüft, ob zwei Werte **unterschiedlich** sind

[!TIP]

[!WARNING] **Häufiger Fehler:** `==` vs. `=`

Anfänger verwechseln oft den Zuweisungsoperator `=` mit dem Vergleichsoperator `==`:
- `x = 5` → **Zuweisung:** Variable `x` bekommt den Wert 5
- `x == 5` → **Vergleich:** Prüft, ob `x` den Wert 5 hat

Größer, Kleiner und ihre Varianten

- `<` (**Kleiner als**): Prüft, ob linker Wert **kleiner** als rechter ist
- `>` (**Größer als**): Prüft, ob linker Wert **größer** als rechter ist
- `<=` (**Kleiner oder gleich**): Prüft, ob linker Wert **kleiner oder gleich** rechter ist
- `>=` (**Größer oder gleich**): Prüft, ob linker Wert **größer oder gleich** rechter ist

[!TIP]

[!WARNING] **Float-Vergleiche mit Vorsicht!**

Aufgrund von Rundungsfehlern bei Fließkommazahlen (siehe V02) sollte man Floats nicht direkt mit `==` vergleichen:

Logische Operatoren

Python bietet drei fundamentale **logische Operatoren**, die Boolean-Werte verknüpfen: **and**, **or** und **not**. Diese entsprechen den logischen Operatoren AND, OR und NOT aus der Booleschen Algebra.

[!NOTE] **Logische Operatoren in Python:** Operatoren zur Verknüpfung von Boolean-Ausdrücken: - **and** – Logisches UND (beide müssen wahr sein) - **or** – Logisches ODER (mindestens einer muss wahr sein) - **not** – Logische Negation (kehrt Wahrheitswert um)

AND-Operator Der **and-Operator** gibt **True** zurück, wenn **beide** Operanden **True** sind. Ansonsten gibt er **False** zurück.

[!TIP]

OR-Operator Der **or-Operator** gibt **True** zurück, wenn **mindestens einer** der Operanden **True** ist. Er gibt nur dann **False** zurück, wenn **alle** Operanden **False** sind.

[!TIP]

NOT-Operator Der **not-Operator** kehrt den Boolean-Wert um: **True** wird zu **False** und umgekehrt.

[!TIP]

Verkettete Vergleiche (Chained Comparisons)

Python erlaubt als besonderes Feature **verkettete Vergleiche**, die mehrere Vergleichsoperatoren elegant kombinieren. Dies entspricht der mathematischen Notation und ist deutlich lesbarer als mehrfache **and**-Verknüpfungen.

[!NOTE] **Verkettete Vergleiche (Chained Comparisons):** Ein Python-spezifisches Feature, das mehrere Vergleiche ohne explizite **and**-Verknüpfung erlaubt. Syntax: `a < b < c` statt `a < b and b < c`. Python garantiert, dass der Mittelwert nur einmal ausgewertet wird.

[!TIP]

[!WARNING] **Vorsicht bei ungewöhnlichen Ketten**

Verkettete Vergleiche funktionieren bei allen Vergleichsoperatoren, aber nicht alle Kombinationen ergeben semantisch Sinn:

Nutze verkettete Vergleiche für Bereichsprüfungen (`a < b < c`) und Gleichheitsprüfungen (`a == b == c`).

Kurzschlussauswertung (Short-Circuit Evaluation)

Python nutzt **Kurzschlussauswertung** (auch Lazy Evaluation genannt) bei logischen Operatoren. Das bedeutet: Python wertet Ausdrücke von links nach rechts aus und stoppt, sobald das Endergebnis feststeht. Dies spart Performance und kann zur Fehlervermeidung genutzt werden.

[!NOTE] **Kurzschlussauswertung (Short-Circuit Evaluation):** Python wertet logische Ausdrücke von links nach rechts aus und stoppt, sobald das Gesamtergebnis feststeht: - Bei **and**: Wenn der linke Operand **False** ist, wird der rechte **nicht** ausgewertet - Bei **or**: Wenn der linke Operand **True** ist, wird der rechte **nicht** ausgewertet

Kurzschluss bei AND Bei `a and b`: Wenn `a` bereits **False** ist, kann das Gesamtergebnis nur **False** sein – `b` wird nicht ausgewertet.

[!TIP]

Kurzschluss bei OR Bei `a or b`: Wenn `a` bereits **True** ist, kann das Gesamtergebnis nur **True** sein – `b` wird nicht ausgewertet.

[!TIP]

[!WARNING] **Performance-Tipp: Reihenfolge bei AND**

Setze “billige” Checks vor “teure” Checks bei **and**:

Operator-Präzedenz (Vorrangregeln)

Python wertet logische und Vergleichsoperatoren in einer festgelegten Reihenfolge aus. Die **Operator-Präzedenz** bestimmt, welche Operationen zuerst ausgeführt werden.

[!NOTE] **Operator-Präzedenz:** Die Reihenfolge, in der Operatoren ausgewertet werden (von höchster zu niedrigster Priorität):
1. **Vergleichsoperatoren** (`==`, `!=`, `<`, `>`, `<=`, `>=`) 2. **not** (Negation)
3. **and** (Konjunktion) 4. **or** (Disjunktion)

[!TIP]

[!WARNING] **Best Practice: Klammern für Lesbarkeit!**

Auch wenn Python klare Präzedenzregeln hat, verbessern Klammern die Lesbarkeit erheblich:

Truthy und Falsy Values (Truth Value Testing)

Eine Besonderheit von Python: **Jeder Wert** kann in einem Boolean-Kontext als **True** oder **False** interpretiert werden. Man spricht von **truthy** (wird als **True** interpretiert) und **falsey** (wird als **False** interpretiert) Values.

[!NOTE] **Truthy und Falsy Values:** In Python kann jedes Objekt in einem Boolean-Kontext ausgewertet werden: - **Falsy Values** (werden als **False** interpretiert): **False**, **None**, **0**, **0.0**, **""** (leerer String), **[]** (leere Liste), **()** (leeres Tupel), **{}** (leeres Dictionary), **set()** (leeres Set) - **Truthy Values** (werden als **True** interpretiert): Alle anderen Werte!

[!TIP]

Praktische Anwendung – Kompakte Existenzprüfungen:

[!TIP]

[!WARNING] **Vorsicht: `if x:` vs. `if x is not None:`**

Diese beiden Checks sind **nicht** identisch!

Regel: Nutze `if x:` für Leer-/Existenzprüfungen. Nutze `if x is not None:` wenn du explizit auf **None** testen willst.

Komplexe Logische Ausdrücke

In der Praxis kombiniert man oft mehrere Vergleichs- und logische Operatoren zu komplexen Bedingungen. Hier sind Best Practices für lesbare und wartbare Ausdrücke.

[!TIP]

[!WARNING] **Häufige Fehler bei komplexen Bedingungen**

Fehler 1: Mehrfachvergleich falsch formuliert

Fehler 2: De Morgan nicht korrekt angewendet

Häufige Fehler und Lösungen

[!WARNING] **Fehler 1: `==` vs. `=` verwechseln**

Problem: Zuweisungsoperator statt Vergleichsoperator verwendet

[!WARNING] **Fehler 2: Float-Vergleiche ohne Toleranz**

Problem: Direkter Vergleich von Floats schlägt wegen Rundungsfehlern fehl

[!WARNING] **Fehler 3: Truthy/Falsy missverstehen**

Problem: String `"0"` wird als **truthy** interpretiert (weil nicht leer!)

[!WARNING] Fehler 4: Mehrfachvergleich mit or falsch

Problem: `x == 1 or 2` funktioniert nicht wie erwartet

Zusammenfassung Python

Die wichtigsten Python-Konzepte dieser Vorlesung:

1. **Vergleichsoperatoren** (`==`, `!=`, `<`, `>`, `<=`, `>=`) vergleichen Werte und liefern Boolean-Ergebnisse. Sie funktionieren mit Zahlen, Strings und vielen anderen Datentypen.
2. **Logische Operatoren** (`and`, `or`, `not`) verknüpfen Boolean-Ausdrücke nach den Regeln der Booleschen Algebra.
3. **Verkettete Vergleiche** sind ein Python-Feature, das elegante Bereichsprüfungen ermöglicht: `a < b < c`.
4. **Kurzschlussauswertung** spart Performance und verhindert Fehler, indem Ausdrücke nur so weit ausgewertet werden, wie nötig.
5. **Truthy/Falsy Values:** Jeder Python-Wert hat einen impliziten Boolean-Wert. Leere Collections, Null und `None` sind falsy, alles andere ist truthy.
6. **Operator-Präzedenz:** Vergleiche vor `not` vor `and` vor `or`. Klammern nutzen für Klarheit!
7. **Best Practices:** Floats mit Toleranz vergleichen, `isinstance()` für Typ-Checks, explizite Klammern für komplexe Ausdrücke.

Neue Python-Funktionen/Methoden in dieser Vorlesung

Folgende Python-Konzepte wurden in V04 **neu eingeführt**:

Operatoren

- **Vergleichsoperatoren:** `==`, `!=`, `<`, `>`, `<=`, `>=` – Vergleichen Werte, liefern Boolean
- **Logische Operatoren:** `and`, `or`, `not` – Verknüpfen Boolean-Ausdrücke
- **Bitweiser XOR-Operator:** `^` – Für Integer (nur erwähnt als Theorie-Bezug)

Konzepte

- **Verkettete Vergleiche (Chained Comparisons):** `a < b < c` (Python-spezifisch)
- **Kurzschlussauswertung (Short-Circuit Evaluation):** Bei `and` und `or`
- **Operator-Präzedenz:** Reihenfolge der Auswertung von Operatoren
- **Truthy und Falsy Values:** Implizite Boolean-Interpretation aller Werte

Built-in Funktionen (erwähnt/vorschau)

- **abs(x)** (Built-in, bereits in V03 kurz erwähnt): Absolutbetrag für Float-Toleranzprüfungen
- **any(iterable)** (Built-in, Vorschau): Gibt **True** zurück, wenn mindestens ein Element truthy ist (wird in V08 ausführlich behandelt)

[!NOTE] Die detaillierte Dokumentation aller neu eingeführten Konzepte findet sich in ../python_topics.md unter der Sektion V04.

Weiterführende Ressourcen

Theorie

- **Wikipedia: Boolesche Algebra** – Umfassende Übersicht über Grundlagen und erweiterte Konzepte
https://de.wikipedia.org/wiki/Boolesche_Algebra
- **Wikipedia: Addierer** – Detaillierte Erklärung von Halb- und Volladdierern
<https://de.wikipedia.org/wiki/Addierer>
- **Nand2Tetris** – Kostenloses Projekt: Bau eines Computers von NAND-Gattern aufwärts
<https://www.nand2tetris.org/>
- **Digital Logic Design Tutorial** – Interaktive Tutorials zu logischen Schaltungen
<https://www.electronics-tutorials.ws/logic/>

Python

- **Python Official Documentation: Comparisons** – Offizielle Dokumentation zu Vergleichsoperatoren
<https://docs.python.org/3/reference/expressions.html#comparisons>
- **Python Official Documentation: Boolean Operations** – Offizielle Dokumentation zu logischen Operatoren
<https://docs.python.org/3/library/stdtypes.html#boolean-operations-and-or-not>
- **Python Official Documentation: Truth Value Testing** – Truthy/Falsy Values
<https://docs.python.org/3/library/stdtypes.html#truth-value-testing>
- **Real Python: Operators and Expressions in Python** – Tutorial zu allen Python-Operatoren
<https://realpython.com/python-operators-expressions/>

- **PEP 8: Style Guide for Python Code** – Empfehlungen für lesbare Python-Bedingungen
<https://www.python.org/dev/peps/pep-0008/>
-

Datei: lessons/V05-Programm-Ablauf-Plaene-Teil1/V05-Programm-Ablauf-Plaene-Teil1_skript.md

V05: Programm-Ablauf-Pläne – Teil 1

[!NOTE] **Lernziele dieser Vorlesung:** - Verstehen, was **Programm-Ablauf-Pläne (PAPs)** sind und wofür sie verwendet werden - Die **Symbole und Notation** nach DIN 66001 / ISO 5807 kennen und anwenden können - Die drei **Grundstrukturen** (Sequenz, Verzweigung, Schleife) identifizieren und unterscheiden - Einfache **PAPs erstellen und lesen** können - **Python if-Anweisungen** verstehen und schreiben können - **if-else** für binäre Entscheidungen einsetzen - **if-elif-else** für mehrfache Verzweigungen verwenden - **Verschachtelte Bedingungen** lesbar formulieren - Den **ternären Operator** für kompakte Ausdrücke nutzen

Teil 1: Theorie - Programm-Ablauf-Pläne

Überblick

In der Softwareentwicklung ist es essenziell, komplexe Abläufe und Algorithmen zu visualisieren, bevor man mit der Implementierung beginnt. **Programm-Ablauf-Pläne (PAPs)**, auch **Flussdiagramme** oder **Flowcharts** genannt, sind eine grafische Darstellungsmethode, die den Ablauf eines Programms oder Algorithmus Schritt für Schritt zeigt. Sie wurden bereits in den 1960er Jahren standardisiert und sind bis heute ein wichtiges Werkzeug in der Softwareentwicklung, Prozessmodellierung und im Requirements Engineering.

PAPs bieten mehrere zentrale Vorteile. Sie ermöglichen eine **sprachunabhängige Kommunikation** über Algorithmen, da die Symbole weltweit standardisiert sind. Dadurch können Entwickler, Projektmanager und Stakeholder ohne tiefe Programmierkenntnisse den Ablauf verstehen. PAPs helfen auch beim **Strukturieren des Denkprozesses** – das Zeichnen eines PAPs zwingt dazu, alle Entscheidungen und Schritte explizit zu durchdenken, bevor man Code schreibt. Zudem unterstützen sie die **Fehlersuche**, da logische Fehler im Ablauf oft im Diagramm leichter zu erkennen sind als im Code. Schließlich dienen PAPs als **Dokumentation**, die auch Jahre später noch verständlich bleibt, selbst wenn der ursprüngliche Code nicht mehr verfügbar ist.

[!NOTE] **Programm-Ablauf-Plan (PAP)**: Eine grafische Darstellung eines Algorithmus oder Programmablaufs durch standardisierte Symbole, die die Reihenfolge der Operationen, Entscheidungen und Datenflüsse zeigt.

[!NOTE] **Algorithmus**: Eine präzise, endliche Folge von Anweisungen zur Lösung eines Problems oder zur Durchführung einer Aufgabe. Algorithmen sind die Grundlage jeder Programmierung.

Standardisierung: DIN 66001 und ISO 5807

PAPs sind international standardisiert, um eine einheitliche Verständigung zu gewährleisten. Die wichtigsten Standards sind **DIN 66001** (deutscher Standard, erstmals 1966 veröffentlicht) und **ISO 5807** (internationaler Standard von 1985). Beide Standards definieren die **Form der Symbole**, die **Bedeutung jedes Symbols** und die **Richtlinien für die Anordnung**.

Die Standards legen fest, dass Pfeile den **Kontrollfluss** zeigen, also die Reihenfolge der Ausführung. Der Ablauf erfolgt üblicherweise **von oben nach unten** und **von links nach rechts**, sofern Pfeile keine andere Richtung angeben. Symbole werden mit geraden Linien verbunden, und Verzweigungen werden klar beschriftet (z.B. “Ja” / “Nein” bei Entscheidungen). Diese Standardisierung stellt sicher, dass ein in Deutschland erstellter PAP auch von Ingenieuren in Japan oder den USA problemlos verstanden wird.

Die wichtigsten PAP-Symbole

Startsymbol und Endsymbol Das **Startsymbol** markiert den Einstiegspunkt eines Programms oder Algorithmus. Es wird als abgerundetes Rechteck (auch Oval oder “Terminator” genannt) dargestellt und enthält typischerweise das Wort “Start” oder “Anfang”. Jeder PAP hat genau ein Startsymbol. Das **Endsymbol** markiert den Austrittspunkt und signalisiert das Ende des Programms oder Algorithmus. Es hat die gleiche Form wie das Startsymbol und enthält üblicherweise “Ende”, “Stop” oder “Halt”. Ein PAP kann mehrere Endsymbole haben, wenn es verschiedene Abbruchbedingungen gibt (z.B. bei Fehlerbehandlung).

[!NOTE] **Startsymbol / Endsymbol**: Abgerundete Rechtecke, die den Beginn und das Ende eines Programmablaufs kennzeichnen. Sie dienen als Ein- und Ausstiegspunkte.

Prozess-Box (Operationen/Anweisungen) Die **Prozess-Box** ist ein Rechteck und repräsentiert eine einzelne Anweisung oder Operation. Typische Inhalte sind **Zuweisungen** (z.B. “ $x = 5$ ”), **Berechnungen** (z.B. “Summe = $a + b$ ”), **Funktionsaufrufe** (z.B. “Berechne Wurzel von x ”) oder **allgemeine Aktionen** (z.B. “Schalte Motor ein”). Prozess-Boxen enthalten keine Entscheidungen oder Verzweigungen – diese werden durch separate Symbole dargestellt.

[!NOTE] **Prozess-Box:** Ein Rechteck, das eine einzelne Operation oder Anweisung darstellt. Es beschreibt eine Aktion, die ausgeführt wird, ohne Verzweigung oder Wiederholung.

Entscheidungssymbol (Verzweigung) Das **Entscheidungssymbol** ist eine Raute (Diamant-Form) und stellt eine Bedingung oder Frage dar, die zu einer Verzweigung im Programmablauf führt. Die Raute enthält eine **Ja/Nein-Frage** oder **Bedingung** (z.B. “ $x > 0$?”, “Passwort korrekt?”). Aus der Raute führen mindestens zwei Pfeile heraus, üblicherweise beschriftet mit “Ja” / “Nein”, “True” / “False” oder “Wahr” / “Falsch”. Entscheidungssymbole sind der Kern von Verzweigungslogik und ermöglichen es, verschiedene Ausführungspfade basierend auf Bedingungen zu modellieren.

[!NOTE] **Entscheidungssymbol:** Eine Raute, die eine Bedingung darstellt, die zu unterschiedlichen Ausführungspfaden führt. Die Ausgänge werden mit “Ja” und “Nein” beschriftet.

[!TIP] **Formulierung von Bedingungen:** Stelle Bedingungen als **geschlossene Fragen** dar, die mit Ja/Nein beantwortet werden können. Beispiel: Nicht “Wert von x prüfen”, sondern “ $x > 10$?”.

Ein-/Ausgabe-Symbol Das **Ein-/Ausgabe-Symbol** ist ein Parallelogramm und kennzeichnet Operationen, bei denen Daten in das Programm eingelesen oder aus dem Programm ausgegeben werden. Typische Verwendungen sind **Benutzereingaben** (z.B. “Eingabe: Name”, “Lies Temperatur”), **Ausgaben** (z.B. “Ausgabe: Ergebnis”, “Zeige Meldung”) oder **Dateioperationen** (z.B. “Lies Datei config.txt”, “Schreibe in log.txt”). Das Parallelogramm unterscheidet sich visuell deutlich von der Prozess-Box und macht sofort klar, dass hier eine Interaktion mit der Außenwelt stattfindet.

[!NOTE] **Ein-/Ausgabe-Symbol:** Ein Parallelogramm, das Operationen zum Einlesen von Daten (Input) oder Ausgeben von Daten (Output) darstellt. Es markiert die Schnittstelle zur Außenwelt.

Schleifensymbol (Wiederholung) Schleifen werden in PAPs durch eine Kombination aus **Entscheidungssymbol** und **Rückwärtspfeil** dargestellt. Die Raute enthält die **Abbruchbedingung** der Schleife (z.B. “Zähler ≤ 10 ?”). Ein Pfeil führt bei erfüllter Bedingung zum **Schleifenrumpf** (die zu wiederholenden Anweisungen), und von dort führt ein Pfeil zurück zur Entscheidungsraute. Wenn die Bedingung nicht mehr erfüllt ist, führt der andere Pfeil aus der Raute heraus und das Programm setzt nach der Schleife fort. Dieses Pattern entspricht einer **while-Schleife** in den meisten Programmiersprachen.

[!NOTE] **Schleife:** Eine Kontrollstruktur, die einen Codeblock wiederholt ausführt, solange eine Bedingung erfüllt ist. In PAPs wird dies durch eine Raute mit Rückwärtspfeil dargestellt.

Die drei Grundstrukturen der strukturierten Programmierung

Die **strukturierte Programmierung** basiert auf drei fundamentalen Kontrollstrukturen, die sich in jedem Programm wiederfinden. Diese wurden in den 1960er Jahren von Böhm und Jacopini mathematisch bewiesen als ausreichend, um jeden berechenbaren Algorithmus darzustellen.

1. Sequenz (Folge) Die **Sequenz** ist die einfachste Struktur. Anweisungen werden **nacheinander in festgelegter Reihenfolge** ausgeführt. Es gibt keine Verzweigungen oder Wiederholungen. Die Sequenz entspricht dem natürlichen Lesen eines Programmcodes von oben nach unten. Beispiel: Erst wird eine Variable initialisiert, dann wird ein Wert berechnet, dann wird das Ergebnis ausgegeben. Jeder Schritt wird genau einmal ausgeführt.

[!NOTE] **Sequenz:** Die lineare Ausführung von Anweisungen in einer festen Reihenfolge, ohne Verzweigung oder Wiederholung. Jede Anweisung wird genau einmal ausgeführt.

2. Verzweigung (Selektion) Die **Verzweigung** ermöglicht **bedingte Ausführung**. Basierend auf einer Bedingung wird einer von mehreren Ausführungspfaden gewählt. Die einfachste Form ist die **binäre Verzweigung** (if-else), bei der zwischen zwei Pfaden gewählt wird. Es existieren auch **mehrfache Verzweigungen** (if-elif-else), bei denen aus mehr als zwei Pfaden ausgewählt wird. Verzweigungen sind essenziell für intelligentes Verhalten von Programmen, da sie Reaktionen auf unterschiedliche Eingaben oder Zustände ermöglichen.

[!NOTE] **Verzweigung (Selektion):** Eine Kontrollstruktur, die basierend auf einer Bedingung einen von mehreren Ausführungspfaden auswählt. Sie ermöglicht bedingte Logik.

3. Wiederholung (Iteration/Schleife) Die **Wiederholung** ermöglicht **wiederholte Ausführung** eines Codeblocks, solange eine Bedingung erfüllt ist. Dies vermeidet redundanten Code und ermöglicht die Verarbeitung großer Datenmengen. Es gibt verschiedene Arten von Schleifen. Die **kopfgesteuerte Schleife** (while) prüft die Bedingung vor jedem Durchlauf – wenn die Bedingung initial falsch ist, wird der Schleifenrumpf nie ausgeführt. Die **fußgesteuerte Schleife** (do-while, in Python selten) prüft die Bedingung nach jedem Durchlauf – der Schleifenrumpf wird mindestens einmal ausgeführt. Die **Zählschleife** (for) wiederholt eine festgelegte Anzahl von Durchläufen.

[!NOTE] **Wiederholung (Iteration):** Eine Kontrollstruktur, die einen Codeblock mehrfach ausführt, solange eine Bedingung erfüllt ist. Sie ermöglicht effiziente Verarbeitung wiederkehrender Aufgaben.

Richtlinien für gute PAPs

Beim Erstellen von PAPs sollten mehrere Best Practices beachtet werden. PAPs sollten **von oben nach unten** fließen, da dies der natürlichen Leserichtung entspricht. Verzweigungen sollten **eindeutig beschriftet** sein mit “Ja” / “Nein” oder “Wahr” / “Falsch”. Pfade sollten sich nach einer Verzweigung oder Schleife wieder zusammenführen, um den Ablauf klar zu strukturieren.

Kreuzungen von Linien sollten vermieden werden, da sie die Lesbarkeit stark beeinträchtigen – bei komplexen Diagrammen können Konnektoren (Kreise mit Buchstaben) verwendet werden. **Jede Box sollte genau eine Aufgabe** beschreiben (Single Responsibility Principle). Verwende **klare, präzise Beschriftungen** – statt “Verarbeitung” schreibe “Berechne Durchschnitt aus Werten”. PAPs sollten **hierarchisch strukturiert** sein – bei komplexen Algorithmen können Teilprozesse in eigene PAPs ausgelagert und durch “Unterprogramm: Name” referenziert werden.

[!WARNING] **Vermeidung von Spaghetti-Code:** PAPs mit vielen Rücksprüngen und Kreuzungen sind schwer zu lesen und deuten auf schlechte Strukturierung hin. Refaktoriere solche Diagramme durch Aufteilung in Unterprogramme.

Praktische Anwendungen von PAPs

PAPs werden in vielen Bereichen eingesetzt. Im **Requirements Engineering** helfen sie, Geschäftsprozesse zu visualisieren und mit Stakeholdern zu diskutieren, bevor Code geschrieben wird. Bei **Algorithmen-Design** wird die Logik vor der Implementierung durchdacht und optimiert. In der **Fehlersuche und Debugging** können PAPs helfen, logische Fehler zu identifizieren, indem man den Ablauf Schritt für Schritt nachvollzieht. Als **Dokumentation** dienen PAPs als zeitlose Referenz, die auch ohne Kenntnis der ursprünglichen Programmiersprache verständlich bleibt. In **Schulung und Lehre** vermitteln PAPs algorithmisches Denken, bevor Syntaxdetails einer Programmiersprache gelernt werden.

Von PAPs zu Code und zurück

Ein wesentlicher Vorteil von PAPs ist ihre **Sprachunabhängigkeit**. Ein PAP kann in Python, Java, C++ oder jede andere Programmiersprache übersetzt werden. Die Übersetzung folgt klaren Regeln. Eine **Prozess-Box** wird zu einer einfachen Anweisung im Code. Ein **Entscheidungssymbol** wird zu einer if-Anweisung. Eine **Schleife** (Raute mit Rückwärtspeil) wird zu einer while- oder for-Schleife. **Ein-/Ausgabe-Symbole** werden zu Input- oder Print-Anweisungen.

Umgekehrt kann man aus bestehendem Code einen PAP erstellen, um die Logik zu visualisieren. Dies ist besonders hilfreich beim **Verstehen von Legacy-Code**, beim **Code-Review** (Logikfehler sind im Diagramm oft offensichtlicher)

und bei der **Optimierung** (Redundante Pfade oder unnötige Verzweigungen werden sichtbar).

[!TIP] **Workflow-Empfehlung:** Erstelle zuerst einen groben PAP, bevor du mit dem Programmieren beginnst. Dies hilft, die Struktur zu klären. Verfeinere den PAP iterativ, während du implementierst. Nutze den finalen PAP als Dokumentation.

Zusammenfassung Theorie

Programm-Ablauf-Pläne sind ein unverzichtbares Werkzeug in der Softwareentwicklung und im Engineering. Sie bieten eine **standardisierte, visuelle Darstellung** von Algorithmen, die **sprachunabhängig** und für verschiedene Zielgruppen verständlich ist. Die wichtigsten Symbole sind **Startsymbol/Endsymbol** (abgerundete Rechtecke), **Prozess-Box** (Rechteck für Anweisungen), **Entscheidungssymbol** (Raute für Verzweigungen) und **Ein-/Ausgabe-Symbol** (Parallelogramm für I/O-Operationen).

Die drei **Grundstrukturen** Sequenz, Verzweigung und Wiederholung sind ausreichend, um jeden Algorithmus darzustellen. PAPs helfen beim **Entwurf, Verständnis, Debugging und der Dokumentation** von Software. Sie zwingen zu **strukturiertem Denken** und machen Logikfehler oft früher sichtbar als im Code. Die Einhaltung von **Standards (DIN 66001, ISO 5807)** gewährleistet internationale Verständlichkeit.

Teil 2: Python-Praxis - Verzweigungen (if, if-elif-else)

[!WARNING] **Python-Konsistenz beachten:** Prüfe ../../python_topics.md für bereits eingeführte Konzepte!

Überblick

Nachdem wir in Teil 1 die theoretischen Grundlagen von Verzweigungen in Programm-Ablauf-Plänen kennengelernt haben, setzen wir diese nun in Python-Code um. **Verzweigungen** sind eine der fundamentalen Kontrollstrukturen in jeder Programmiersprache. Sie ermöglichen es, Code **bedingt auszuführen** – das heißt, basierend auf Bedingungen unterschiedliche Aktionen durchzuführen.

In Python werden Verzweigungen durch **if-Anweisungen** realisiert. Diese erlauben es, Codeblöcke nur dann auszuführen, wenn bestimmte Bedingungen erfüllt sind. Ohne Verzweigungen könnten Programme nur sequenziell ablaufen und nicht auf unterschiedliche Eingaben oder Situationen reagieren. Verzweigungen sind essenziell für **Validierung von Eingaben, Fehlerbehandlung, Geschäftslogik** (z.B. “Wenn Benutzer Premium-Mitglied, dann gewähre Zugriff”), und **Algorithmen** (z.B. Sortieren, Suchen).

Die einfache if-Anweisung

Die einfachste Form der Verzweigung ist die **if-Anweisung ohne else-Zweig**. Sie führt einen Codeblock nur dann aus, wenn eine Bedingung erfüllt ist. Ist die Bedingung nicht erfüllt, wird der Block übersprungen und das Programm setzt nach dem if-Block fort.

Syntax:

Die **Bedingung** ist ein Ausdruck, der zu einem Boolean-Wert (**True** oder **False**) evaluiert wird. Dies kann ein **Vergleich** sein (z.B. `x > 10`), ein **logischer Ausdruck** (z.B. `a and b`), oder direkt ein **Boolean-Wert** (z.B. `is_active`). Der **Codeblock** nach dem Doppelpunkt muss **engerückt** sein – Python verwendet Einrückung zur Definition von Codeblöcken (üblicherweise 4 Leerzeichen). Alle Zeilen mit gleicher Einrückung gehören zum selben Block.

[!NOTE] **if-Anweisung:** Eine Kontrollstruktur, die einen Codeblock nur dann ausführt, wenn eine Bedingung als **True** evaluiert wird. Die Syntax erfordert einen Doppelpunkt nach der Bedingung und Einrückung des Codeblocks.

[!TIP]

Ausgabe:

Wenn `alter < 18` wäre, würde nur “Programmende” ausgegeben.

[!WARNING] **Einrückung ist in Python verpflichtend!**
Vergessene oder falsche Einrückung führt zu einem **IndentationError**.
Verwende konsistent entweder 4 Leerzeichen (Standard) oder Tabs, aber nie beides gemischt.

Die if-else-Anweisung (binäre Verzweigung)

Die **if-else-Anweisung** erweitert die einfache if-Anweisung um einen **Alternative-Zweig**. Wenn die Bedingung **True** ist, wird der if-Block ausgeführt. Wenn die Bedingung **False** ist, wird stattdessen der else-Block ausgeführt. Es wird **immer genau einer der beiden Blöcke** ausgeführt, niemals beide und niemals keiner.

Syntax:

Die if-else-Struktur entspricht der **binären Verzweigung** in PAPs – die Raute mit zwei Ausgängen (“Ja” und “Nein”). Sie wird verwendet, wenn **genau zwei Ausführungspfade** existieren, die sich gegenseitig ausschließen.

[!NOTE] **if-else-Anweisung:** Eine Kontrollstruktur mit zwei sich gegenseitig ausschließenden Pfaden. Wenn die Bedingung **True** ist, wird der if-Block ausgeführt, andernfalls der else-Block.

[!TIP]

Eingabe: 7

Ausgabe:

PAP-Darstellung:

Die if-elif-else-Anweisung (mehrfache Verzweigung)

Wenn mehr als zwei Ausführungspfade existieren, wird die **if-elif-else-Anweisung** verwendet. Das Schlüsselwort **elif** (kurz für “else if”) ermöglicht das Testen mehrerer Bedingungen nacheinander. Die Bedingungen werden von oben nach unten geprüft. **Sobald eine Bedingung True ist, wird der zugehörige Block ausgeführt und alle weiteren Bedingungen werden übersprungen.** Wenn keine Bedingung erfüllt ist, wird der else-Block ausgeführt (falls vorhanden).

Syntax:

Die Anzahl der elif-Zweige ist unbegrenzt. Der else-Zweig ist optional – wenn er fehlt und keine Bedingung erfüllt ist, wird keiner der Blöcke ausgeführt. Die if-elif-else-Struktur ist effizienter als mehrere separate if-Anweisungen, da nach dem ersten **True-Match** alle weiteren Prüfungen übersprungen werden.

[!NOTE] **elif (else if):** Ein Schlüsselwort in Python, das eine zusätzliche Bedingung nach einem **if** einführt. Es ermöglicht mehrfache Verzweigungen und wird nur geprüft, wenn alle vorherigen Bedingungen **False** waren.

[!TIP]

Eingabe: 85

Ausgabe:

PAP-Darstellung:

[!WARNING] **Reihenfolge der Bedingungen beachten!** Bei if-elif-else wird nur die **erste erfüllte Bedingung** ausgeführt. Wenn du **punkte >= 60** vor **punkte >= 90** prüfst, würden alle Werte >= 60 als “Ausreichend” klassifiziert, auch wenn sie eigentlich “Sehr gut” wären. Prüfe immer **von spezifisch zu allgemein** oder **von groß zu klein**.

Verschachtelte if-Anweisungen

Verschachtelung bedeutet, dass ein if-Block innerhalb eines anderen if-Blocks liegt. Dies ermöglicht **komplexere Entscheidungslogik** mit mehreren Ebenen. Allerdings kann tiefe Verschachtelung die Lesbarkeit stark beeinträchtigen.

[!NOTE] **Verschachtelte Bedingungen:** if-Anweisungen innerhalb anderer if-Anweisungen, die mehrstufige Entscheidungslogik ermöglichen. Jede Ebene wird durch zusätzliche Einrückung dargestellt.

[!TIP]

Ausgabe:

PAP-Darstellung verschachtelter Bedingungen:

[!WARNING] **Zu viele Verschachtelungsebenen vermeiden!**
Mehr als 2-3 Ebenen machen Code schwer lesbar. Verwende stattdessen **logische Operatoren** (**and**, **or**) oder **frühe Returns** (wenn innerhalb von Funktionen, wird in V10 behandelt).

Alternative mit logischen Operatoren:

Diese Variante ist flacher und leichter zu erfassen, da die Bedingungen parallel statt verschachtelt formuliert sind.

Der ternäre Operator (Conditional Expression)

Der **ternäre Operator** ist eine kompakte Schreibweise für einfache if-else-Anweisungen. Er ermöglicht es, eine Zuweisung basierend auf einer Bedingung in einer einzigen Zeile zu formulieren. Der ternäre Operator ist besonders nützlich für **einfache Zuweisungen** oder **inline-Entscheidungen** in Funktionsaufrufen.

Syntax:

Die Bedingung wird in der Mitte platziert, davor steht der Wert für den True-Fall, danach der Wert für den False-Fall. Das Ergebnis des Ausdrucks kann einer Variable zugewiesen oder direkt verwendet werden.

[!NOTE] **Ternärer Operator (Conditional Expression):**
Eine kompakte Syntax für einfache if-else-Zuweisungen in einer Zeile. Die Form ist: `wert_wenn_wahr if Bedingung else wert_wenn_falsch`.

[!TIP]

Ausgabe:

Weitere Beispiele:

[!WARNING] **Lesbarkeit vs. Kompaktheit:** Der ternäre Operator sollte nur für **einfache, einzeilige Entscheidungen** verwendet werden. Bei komplexen Bedingungen oder mehreren Anweisungen pro Zweig wird der Code schnell unlesbar. In solchen Fällen ist klassisches if-else vorzuziehen.

Pass-Statement (Platzhalter für leere Blöcke)

Manchmal möchte man die Struktur einer if-Anweisung definieren, aber den Inhalt eines Blocks noch nicht implementieren. Python erlaubt keine leeren

Blöcke – das würde einen `IndentationError` verursachen. Für solche Fälle gibt es das **pass-Statement**.

Das **pass-Statement** ist eine Null-Operation, die nichts tut. Es dient als **syntaktischer Platzhalter** und signalisiert “hier soll später Code stehen”. Pass kann in if-Blöcken, Schleifen, Funktionen und Klassen verwendet werden.

[!NOTE] **pass-Statement:** Ein Schlüsselwort, das als Platzhalter für syntaktisch erforderliche, aber noch nicht implementierte Codeblöcke dient. Es führt keine Aktion aus.

[!TIP]

Pass wird häufig während der Entwicklung verwendet, wenn die Struktur bereits klar ist, aber Teile der Implementierung noch fehlen. Es ist auch nützlich in Ausnahmefällen, wenn explizit “nichts tun” die gewünschte Aktion ist.

Häufige Fehler und Lösungen

[!WARNING] **Fehler 1: Vergessener Doppelpunkt nach if/elif/else**

Fehler: `SyntaxError: invalid syntax`

Lösung: Doppelpunkt nach der Bedingung nicht vergessen:

[!WARNING] **Fehler 2: Fehlende oder inkonsistente Einrückung**

Fehler: `IndentationError: expected an indented block`

Lösung: Alle Zeilen im if-Block müssen gleich eingerückt sein (4 Leerzeichen):

[!WARNING] **Fehler 3: Verwechslung von = und ==**

Fehler: `SyntaxError: invalid syntax`

Lösung: Verwende `==` für Vergleiche, `=` nur für Zuweisungen:

[!WARNING] **Fehler 4: Falsche Reihenfolge bei if-elif-else**

Problem: Die erste Bedingung `punkte >= 50` ist bereits bei 95 erfüllt, also wird “Bestanden” ausgegeben und die zweite Bedingung nie geprüft.

Lösung: Spezifischere Bedingungen zuerst prüfen:

[!WARNING] **Fehler 5: Mehrere separate if statt if-elif-else**

Problem: Alle drei Bedingungen werden immer geprüft, auch wenn die erste bereits zutrifft.

Lösung: Verwende if-elif-else für sich gegenseitig ausschließende Fälle:

Zusammenfassung Python

Verzweigungen sind fundamentale Kontrollstrukturen, die bedingtes Ausführen von Code ermöglichen. Die **einfache if-Anweisung** führt einen Block nur bei erfüllter Bedingung aus. Die **if-else-Anweisung** bietet zwei sich ausschließende Pfade. Die **if-elif-else-Anweisung** ermöglicht mehrfache Verzweigungen mit beliebig vielen Bedingungen.

Verschachtelte if-Anweisungen erlauben mehrstufige Logik, sollten aber nicht zu tief werden. Der **ternäre Operator** ist eine kompakte Alternative für einfache Zuweisungen. Das **pass-Statement** dient als Platzhalter für noch nicht implementierte Blöcke.

Wichtige Best Practices sind **Doppelpunkt nicht vergessen, konsistente Einrückung** (4 Leerzeichen), **== für Vergleiche, = für Zuweisungen, spezifische Bedingungen vor allgemeinen** prüfen und **elif statt mehrerer if** für sich ausschließende Fälle verwenden.

Neue Python-Funktionen/Methoden

In dieser Lektion wurden folgende Python-Konzepte **neu eingeführt**:

- **if-Anweisung**: Bedingte Ausführung eines Codeblocks
 - Syntax: `if Bedingung:`
 - Erfordert Doppelpunkt und Einrückung
 - **else-Klausel**: Alternativer Pfad, wenn if-Bedingung False
 - Syntax: `else:`
 - Kein Bedingungsausdruck
 - **elif-Klausel**: Zusätzliche Bedingung nach if
 - Syntax: `elif Bedingung:`
 - Ermöglicht mehrfache Verzweigungen
 - **Ternärer Operator (Conditional Expression)**: Kompakte if-else-Syntax
 - Syntax: `wert_wenn_wahr if Bedingung else wert_wenn_falsch`
 - Für einfache inline-Entscheidungen
 - **pass-Statement**: Platzhalter für leere Codeblöcke
 - Syntax: `pass`
 - Keine Operation, syntaktischer Platzhalter
-

Weiterführende Ressourcen

Theorie

- **DIN 66001**: Offizielle deutsche Norm für Programm-Ablauf-Pläne (verfügbar über Beuth Verlag)
- **ISO 5807**: Internationale Norm für Flussdiagramme (Information processing — Documentation symbols and conventions for data, program

and system flowcharts)

- **Structured Programming (Dijkstra, 1968)**: Grundlegendes Paper zur strukturierten Programmierung
- **Flowchart.js**: Web-basiertes Tool zum Erstellen von Flussdiagrammen mit Code
- **Lucidchart** / **Draw.io**: Professionelle Tools für PAP-Erstellung

Python

- **Python Official Documentation - Control Flow**: <https://docs.python.org/3/tutorial/controlflow.html>
- **Real Python - Conditional Statements**: <https://realpython.com/python-conditional-statements/>
- **PEP 308 - Conditional Expressions**: <https://www.python.org/dev/peps/pep-0308/> (Design-Begründung für ternären Operator)
- **Python Style Guide (PEP 8)**: Empfohlene Konventionen für if-Anweisungen und Einrückung

Datei: lessons/V06-Programm-Ablauf-Plaene-Teil2/V06-Programm-Ablauf-Plaene-Teil2_skript.md

V06: Programm-Ablauf-Pläne – Teil 2 & Schleifen (for, while) – Teil 1

[!NOTE] **Lernziele dieser Vorlesung**: - PAPs in Pseudocode übersetzen und umgekehrt - Komplexe Algorithmen mit Schleifen in PAPs visualisieren - Verschiedene Darstellungsformen von Algorithmen vergleichen - **for**-Schleifen mit **range()** verstehen und anwenden - **while**-Schleifen mit Abbruchbedingungen implementieren - Endlos-Schleifen erkennen und vermeiden - Iterieren über Strings und andere Iterables

Teil 1: Theorie - Programm-Ablauf-Pläne – Teil 2

Überblick

In der vorherigen Vorlesung (V05) haben Sie die Grundlagen der **Programm-Ablauf-Pläne** (PAPs) kennengelernt: Symbole, Notation nach DIN 66001/ISO 5807, und die drei Grundstrukturen der strukturierten Programmierung (Sequenz, Verzweigung, Schleife). In dieser zweiten Teil vertiefen wir die praktische Anwendung von PAPs und lernen, wie wir zwischen verschiedenen Darstellungsformen wechseln können. Der Fokus liegt dabei auf der Transformation zwischen PAPs, Pseudocode und realem Programmcode sowie auf der Dokumentation komplexer Algorithmen.

Die Fähigkeit, zwischen verschiedenen Abstraktionsebenen zu wechseln, ist eine Kernkompetenz in der Softwareentwicklung. PAPs bieten eine visuelle, sprachunabhängige Darstellung, während Pseudocode eine semi-formale Beschreibung liefert, die näher am tatsächlichen Code liegt. Beide Darstellungsformen haben ihre Berechtigung je nach Zielgruppe und Dokumentationszweck.

Von PAP zu Pseudocode

Pseudocode ist eine informelle Beschreibung eines Algorithmus, die Elemente natürlicher Sprache mit programmiersprachenähnlicher Syntax kombiniert. Der große Vorteil von Pseudocode liegt in seiner Flexibilität: Er ist präziser als natürliche Sprache, aber weniger streng als echter Programmcode. Dadurch eignet er sich hervorragend für die Kommunikation zwischen Entwicklern, für Entwurfsdokumente und zur Vorbereitung der tatsächlichen Implementierung.

[!NOTE] **Pseudocode** ist eine strukturierte, aber nicht formal definierte Notation zur Beschreibung von Algorithmen. Im Gegensatz zu echtem Programmcode muss Pseudocode nicht syntaktisch korrekt sein oder von einem Compiler interpretiert werden können. Er dient primär der menschlichen Kommunikation und Planung.

Die Transformation von einem PAP zu Pseudocode folgt klaren Regeln. Jedes Symbol im PAP entspricht einer bestimmten Pseudocode-Struktur. Diese Zuordnung ist nicht eindeutig standardisiert, sondern orientiert sich an gängigen Konventionen aus der Informatik und Algorithmen-Lehrbüchern.

Mapping von PAP-Symbolen zu Pseudocode-Konstrukten:

PAP-Symbol	Pseudocode-Entsprechung	Beispiel
Terminator (Start/Ende)	BEGIN / END oder START / STOP	BEGIN Algorithmus
Prozess (Rechteck)	Zuweisung oder Anweisung	$x \leftarrow x + 1$ oder AUSGABE(x)
Verzweigung (Raute)	IF-THEN-ELSE	IF $x > 0$ THEN ... ELSE ... ENDIF
Schleife (verschiedene Symbole)	WHILE, FOR, REPEAT-UNTIL	WHILE $x < 10$ DO ... ENDWHILE
Ein-/Ausgabe (Parallelogramm)	EINGABE(), AUSGABE()	AUSGABE("Ergebnis: ", summe)

[!TIP] **Beispiel: Einfacher Algorithmus zur Berechnung der Summe zweier Zahlen**

PAP (verbal beschrieben): 1. START 2. EINGABE: a, b 3.
PROZESS: $\text{summe} \leftarrow a + b$ 4. AUSGABE: summe 5. ENDE

Pseudocode-Darstellung:

Beachten Sie, dass Pseudocode verschiedene Stile erlaubt. Manche bevorzugen mathematische Notation wie $\leftarrow$ für Zuweisungen, andere verwenden $:=$ oder $=$. Wichtig ist die Konsistenz innerhalb eines Dokuments und die Verständlichkeit für die Zielgruppe.

Komplexe Algorithmen mit Schleifen in PAPs

Schleifen sind die dritte Grundstruktur der strukturierten Programmierung und ermöglichen die wiederholte Ausführung von Codeblöcken. In PAPs werden Schleifen durch spezielle Symbole und Pfeile dargestellt, die den Kontrollfluss zurück zu einem früheren Punkt lenken. Es gibt verschiedene Schleifentypen, die sich in der Position der Bedingungsprüfung unterscheiden.

[!NOTE] **Schleife** ist eine Kontrollstruktur, die einen Codeblock wiederholt ausführt, solange eine Bedingung erfüllt ist (oder bis sie erfüllt wird). Man unterscheidet zwischen **kopfgesteuerten Schleifen** (Bedingung wird vor dem Schleifenkörper geprüft) und **fußgesteuerten Schleifen** (Bedingung wird nach dem Schleifenkörper geprüft).

Typen von Schleifen in PAPs:

1. **Kopfgesteuerte Schleife (while-Schleife)**
 - Die Bedingung wird **vor** jedem Durchlauf geprüft
 - Der Schleifenkörper wird möglicherweise **null Mal** ausgeführt
 - Darstellung: Raute (Bedingung) $\rightarrow$ bei "Ja" zum Schleifenkörper $\rightarrow$ Rücksprung zur Raute
2. **Fußgesteuerte Schleife (do-while/repeat-until)**
 - Die Bedingung wird **nach** jedem Durchlauf geprüft
 - Der Schleifenkörper wird **mindestens einmal** ausgeführt
 - Darstellung: Schleifenkörper $\rightarrow$ Raute (Bedingung) $\rightarrow$ bei "Ja" zurück zum Schleifenkörper
3. **Zählschleife (for-Schleife)**
 - Spezialfall: Wiederholung mit festgelegter Anzahl oder über Elemente einer Sequenz
 - Darstellung: Oft als spezielle Symbol-Kombination oder mit Schleifenzähler im Prozess-Symbol

[!WARNING] **Endlos-Schleifen** entstehen, wenn die Abbruchbedingung niemals erfüllt wird. Dies ist ein häufiger Programmierfehler, der dazu führt, dass das Programm "hängen bleibt" und nie terminiert. In PAPs erkennt man potenzielle Endlos-Schleifen daran, dass kein Pfad zur Bedingungsänderung innerhalb der Schleife führt.

Beispiel: PAP für Summation von 1 bis n (Zählschleife)

Zugehöriger Pseudocode:

Die Schleife wird durch die Bedingung $i \leq n$ gesteuert. Solange i kleiner oder gleich n ist, wird der Schleifenkörper ausgeführt. Nach jedem Durchlauf wird i erhöht. Sobald i größer als n wird, verlässt der Algorithmus die Schleife und gibt das Ergebnis aus.

Verschachtelte Schleifen in PAPs

Verschachtelte Schleifen (Nested Loops) treten auf, wenn eine Schleife innerhalb einer anderen Schleife platziert wird. Dies ist notwendig für Algorithmen, die über mehrdimensionale Datenstrukturen iterieren oder bei denen für jeden Durchlauf der äußeren Schleife mehrere Wiederholungen erforderlich sind.

[!NOTE] **Verschachtelte Schleifen** sind Schleifen innerhalb von Schleifen. Die **äußere Schleife** führt die **innere Schleife** vollständig für jeden ihrer Durchläufe aus. Die Gesamtzahl der Durchläufe ist das Produkt der Durchläufe beider Schleifen.

Beispiel: Multiplikationstabelle (1 bis 3)

Wir möchten eine kleine Multiplikationstabelle erstellen, die für jede Zahl von 1 bis 3 die Produkte mit 1 bis 3 ausgibt.

Zugehöriger Pseudocode:

Die äußere Schleife iteriert über i von 1 bis 3. Für jeden Wert von i wird die innere Schleife vollständig durchlaufen, die über j von 1 bis 3 iteriert. Das bedeutet, dass die innere Schleife 3 Mal (für jeden Durchlauf der äußeren Schleife) mit jeweils 3 Durchläufen ausgeführt wird, insgesamt also $3 \times 3 = 9$ Ausgaben.

[!WARNING] **Komplexität verschachtelter Schleifen:** Die Laufzeit steigt multiplikativ an. Zwei ineinander verschachtelte Schleifen mit n Durchläufen haben eine Zeitkomplexität von $O(n^2)$. Bei drei verschachtelten Schleifen erreicht man $O(n^3)$. Dies kann bei großen Datenmengen zu erheblichen Performance-Problemen führen.

Von Pseudocode zu PAP

Die Rücktransformation von Pseudocode zu PAP ist ebenso wichtig wie die Hinrichtung. Sie ist besonders nützlich, wenn ein Algorithmus bereits in textueller Form vorliegt und visuell kommuniziert werden soll, etwa in Präsentationen oder Dokumentationen für nicht-technisches Personal.

Schritte zur Transformation:

1. **Identifiziere die Kontrollstrukturen:** Markiere alle IF-THEN-ELSE, WHILE, FOR, etc.

2. **Ordne jedem Element das passende PAP-Symbol zu:** Zuweisungen → Rechteck, Bedingungen → Raute, Ein-/Ausgabe → Parallelogramm
3. **Zeichne den Kontrollfluss:** Verbinde die Symbole mit Pfeilen gemäß der Ausführungsreihenfolge
4. **Prüfe auf Vollständigkeit:** Jeder Pfad muss zu einem Ende-Symbol führen

[!TIP] **Beispiel: Maximum von drei Zahlen finden**

Pseudocode:

PAP:

Dokumentation von Programmabläufen

Die Wahl der Darstellungsform hängt vom Kontext und der Zielgruppe ab. PAPs sind visuell ansprechend und sprachunabhängig, erfordern aber mehr Platz und sind bei sehr komplexen Algorithmen schwer lesbar. Pseudocode ist kompakter und lässt sich leichter in Textdokumente integrieren, setzt aber ein gewisses technisches Verständnis voraus.

Vergleich der Darstellungsformen:

Kriterium	PAP (Flussdiagramm)	Pseudocode	Programmcode
Abstraktionsebene	Hoch (visuell)	Mittel (semi-formal)	Niedrig (formal)
Sprachunabhängigkeit	Hoch	Weitgehend	Nein
Präzision	Mittel	Mittel	Hoch
Lesbarkeit für Nicht-Programmierer	Sehr gut	Gut	Schlecht
Eignung für komplexe Algorithmen	Begrenzt (wird unübersichtlich)	Gut	Sehr gut
Werkzeugunterstützung	Stark (Visio, Mermaid)	Texteditor	IDE
Wartbarkeit	Aufwändig (manuelles Neuzeichnen)	Einfach (Text bearbeiten)	Automatisiert (Refactoring)

[!NOTE] **Best Practice:** Verwende PAPs für die initiale Planung und Kommunikation mit Nicht-Technikern. Nutze Pseudocode für detaillierte Algorithmen-Entwürfe und Code-Reviews. Schreibe echten Programmcode erst, wenn die Logik vollständig durchdacht ist.

In modernen Software-Projekten werden PAPs oft nur noch für didaktische Zwecke oder bei der Dokumentation von Schnittstellen verwendet. Für die eigentliche Entwicklung haben sich Pseudocode und direkte Code-Kommentare durchgesetzt, da sie sich besser in die Entwicklungsumgebung integrieren lassen.

Vergleich verschiedener Darstellungsformen

Neben PAPs und Pseudocode gibt es weitere Darstellungsformen für Algorithmen, die je nach Kontext ihre Berechtigung haben.

Weitere Darstellungsformen:

1. **Struktogramme (Nassi-Shneiderman-Diagramme)**
 - Rechteckige Blöcke statt Symbole und Pfeile
 - Verschachtelung durch Unterteilung der Blöcke
 - Zwingen zu strukturierter Programmierung (keine Sprünge möglich)
 - Vorteile: Kompakter als PAPs, erzwingen klare Struktur
 - Nachteile: Weniger intuitiv für Nicht-Informatiker
2. **Aktivitätsdiagramme (UML)**
 - Teil der Unified Modeling Language
 - Ähnlich wie PAPs, aber mit zusätzlichen Konzepten (Swimlanes, Synchronisation)
 - Vorteile: Standard in der Software-Engineering-Industrie, Werkzeugunterstützung
 - Nachteile: Höhere Komplexität, Lernkurve
3. **Zustandsautomaten (State Machines)**
 - Für ereignisgesteuerte Systeme
 - Zustände und Übergänge zwischen Zuständen
 - Vorteile: Ideal für reaktive Systeme, Hardware-Design
 - Nachteile: Nicht geeignet für sequenzielle Algorithmen
4. **Entscheidungstabellen**
 - Tabellarische Darstellung von Bedingungen und Aktionen
 - Vorteile: Übersichtlich bei vielen Verzweigungen
 - Nachteile: Nicht für Schleifen geeignet

[!TIP] **Wahl der Darstellungsform:** - **PAPs:** Initiale Planung, Lehre, Kommunikation mit Nicht-Technikern - **Pseudocode:** Detaillierte Algorithmen-Entwürfe, Code-Reviews - **Struktogramme:** Wenn strukturierte Programmierung erzwungen werden soll - **UML-Aktivitätsdiagramme:** Große Software-Projekte, Teamarbeit - **Zustandsautomaten:** Ereignisgesteuerte Systeme, Hardware-nahe Programmierung - **Entscheidungstabellen:** Komplexe Verzweigungslogik (z.B. Business Rules)

In der Praxis werden häufig mehrere Darstellungsformen kombiniert. Ein Projektbericht könnte beispielsweise einen übergeordneten PAP für den Gesamtablauf enthalten, detaillierte Algorithmen in Pseudocode beschreiben und kritische Abschnitte durch Entscheidungstabellen oder Zustandsautomaten

verdeutlichen.

Zusammenfassung Theorie

- **Pseudocode** ist eine semi-formale Notation zur Beschreibung von Algorithmen, die flexibler als Programmcode, aber präziser als natürliche Sprache ist
 - Die **Transformation zwischen PAP und Pseudocode** folgt klaren Regeln: Jedes PAP-Symbol entspricht einer Pseudocode-Struktur
 - **Schleifen** in PAPs ermöglichen wiederholte Ausführung und werden durch Rückwärtspfeile dargestellt. Man unterscheidet kopfgesteuerte, fußgesteuerte und Zählschleifen
 - **Verschachtelte Schleifen** führen zu multiplikativer Zunahme der Durchläufe und erfordern besondere Aufmerksamkeit bei der Komplexitätsanalyse
 - Die **Wahl der Darstellungsform** (PAP, Pseudocode, Programmcode, UML, etc.) hängt von Zielgruppe, Komplexität und Projektstadium ab
 - **Best Practice:** Verwende PAPs für visuelle Kommunikation, Pseudocode für Algorithmen-Entwürfe und echten Code für die Implementierung
-

Teil 2: Python-Praxis - Schleifen (for, while) – Teil 1

[!WARNING] **Python-Konsistenz beachten:** Prüfe ../../python_topics.md für bereits eingeführte Konzepte!

Überblick

Nachdem Sie in V05 Verzweigungen kennengelernt haben, erweitern wir nun Ihr Repertoire um **Schleifen** – die dritte Grundstruktur der strukturierten Programmierung. Schleifen ermöglichen es, Codeblöcke mehrfach auszuführen, ohne ihn wiederholt hinschreiben zu müssen. Dies ist unverzichtbar für die Verarbeitung von Listen, die Implementierung von Algorithmen und die Automatisierung repetitiver Aufgaben.

Python bietet zwei Haupttypen von Schleifen: Die **for-Schleife** für Iterationen über Sequenzen und die **while-Schleife** für Wiederholungen mit Bedingung. In diesem ersten Teil konzentrieren wir uns auf die Grundlagen beider Schleifentypen, während fortgeschrittene Konzepte wie **break**, **continue** und List Comprehensions in V07 behandelt werden.

Die Verbindung zur Theorie ist offensichtlich: Die Schleifen-Konstrukte in Python entsprechen direkt den Schleifensymbolen in PAPs. Eine **while**-Schleife in Python ist eine kopfgesteuerte Schleife, während die **for**-Schleife in vielen Fällen einer Zählschleife entspricht.

Die **for**-Schleife mit `range()`

Die **for**-Schleife in Python iteriert über Sequenzen (Listen, Strings, Ranges, etc.). Im einfachsten Fall verwenden wir die `range()`-Funktion, um eine Zahlenfolge zu erzeugen, über die wir iterieren.

[!NOTE] **for**-Schleife ist eine Kontrollstruktur, die über alle Elemente einer Sequenz (Iterable) iteriert. In jedem Durchlauf wird das aktuelle Element einer Variablen zugewiesen, und der Schleifenkörper wird ausgeführt. Nach dem letzten Element endet die Schleife automatisch.

Syntax der **for**-Schleife:

- **variable**: Die Schleifenvariable, die in jedem Durchlauf den Wert des aktuellen Elements erhält
- **sequenz**: Ein Iterable (z.B. `range()`, Liste, String)
- Der Schleifenkörper muss eingerückt sein (4 Leerzeichen)

Die `range()`-Funktion

[!NOTE] `range(start=0, stop, step=1)` (Built-in, Python 3.0+) erzeugt eine Sequenz von Ganzzahlen. In Python 3 ist `range()` ein Typ (kein Generator), der Speicher-effizient arbeitet und Zahlen nur bei Bedarf generiert. Die erzeugte Sequenz beginnt bei **start** (inklusive) und endet bei **stop** (exklusive) mit Schrittweite **step**.

Drei Verwendungsformen von `range()`:

1. `range(stop)`: Erzeugt Zahlen von 0 bis `stop - 1`
2. `range(start, stop)`: Erzeugt Zahlen von `start` bis `stop - 1`
3. `range(start, stop, step)`: Erzeugt Zahlen von `start` bis `stop - 1` mit Schrittweite `step`

[!WARNING] **Häufiger Fehler**: Die Obergrenze (**stop**) ist **exklusive**. `range(10)` erzeugt Zahlen von 0 bis 9, nicht bis 10. Um bis 10 (inklusive) zu zählen, verwende `range(11)`.

[!TIP] **Beispiel: Summe der Zahlen von 1 bis 100 berechnen**

Dieser Code entspricht dem PAP aus dem Theorie-Teil, angepasst für `n=100`.

Negative Schrittweiten: Mit negativem **step** kann rückwärts gezählt werden.

[!WARNING] **Achtung bei negativem Step**: Wenn **step** negativ ist, muss `start > stop` sein, sonst wird die Schleife nicht ausgeführt. `range(0, 10, -1)` erzeugt eine leere Sequenz.

Iterieren über Strings

Die **for**-Schleife kann über jedes Iterable iterieren, nicht nur über Zahlenfolgen. Strings sind in Python Sequenzen von Zeichen, über die wir direkt iterieren können.

Jedes Zeichen des Strings wird nacheinander der Variable **buchstabe** zugewiesen. Diese Fähigkeit macht die **for**-Schleife extrem vielseitig.

[!TIP] **Beispiel: Vokale in einem String zählen**

Die while-Schleife

Die **while**-Schleife wiederholt einen Codeblock, solange eine Bedingung wahr ist. Im Gegensatz zur **for**-Schleife, die über eine vorgegebene Sequenz iteriert, läuft die **while**-Schleife, bis die Bedingung **False** wird.

[!NOTE] **while**-Schleife ist eine kopfgesteuerte Schleife, die einen Codeblock wiederholt ausführt, solange eine Bedingung **True** ist. Die Bedingung wird vor jedem Durchlauf geprüft. Wenn sie beim ersten Check bereits **False** ist, wird der Schleifenkörper niemals ausgeführt.

Syntax der while-Schleife:

- **bedingung**: Ein boolescher Ausdruck, der in jedem Durchlauf ausgewertet wird
- Der Schleifenkörper muss eingerückt sein

[!TIP] **Beispiel: Countdown mit while**

Abbruchbedingungen richtig setzen

Die Abbruchbedingung einer **while**-Schleife muss irgendwann **False** werden, sonst entsteht eine **Endlos-Schleife** (Infinite Loop). Dies ist einer der häufigsten Programmierfehler bei Schleifen.

[!WARNING] **Endlos-Schleife**: Eine Schleife, deren Abbruchbedingung niemals **False** wird, läuft unendlich weiter. Das Programm “hängt” und muss manuell abgebrochen werden (z.B. mit Strg+C in der Konsole oder durch Schließen des Terminals).

Beispiel für eine Endlos-Schleife (FEHLER!):

Korrigierte Version:

[!TIP] **Checkliste zur Vermeidung von Endlos-Schleifen**: 1. Wird die Schleifenvariable im Schleifenkörper modifiziert? 2. Führt die Modifikation dazu, dass die Bedingung irgendwann **False** wird? 3. Gibt es einen garantierten Abbruch (z.B. durch Benutzereingabe)? 4. Ist die Abbruchbedingung korrekt formuliert (nicht **x < 10** wenn **x++** gemeint war)?

while vs. for: Wann welche Schleife?

Beide Schleifentypen können oft für dieselbe Aufgabe verwendet werden, aber es gibt typische Anwendungsfälle für jede.

Verwendung von for: - Wenn die Anzahl der Durchläufe bekannt oder vorhersehbar ist - Beim Iterieren über Sequenzen (Listen, Strings, Ranges) - Wenn ein Schleifenzähler benötigt wird

Verwendung von while: - Wenn die Anzahl der Durchläufe nicht im Voraus bekannt ist - Wenn die Schleife durch eine Bedingung beendet wird, die sich dynamisch ändert - Bei Benutzereingaben, die validiert werden müssen - Bei Algorithmen, die bis zur Konvergenz laufen

[!TIP] **Beispiel: Dieselbe Aufgabe mit for und while**

Aufgabe: Zahlen von 1 bis 5 ausgeben

Mit for:

Mit while:

Die **for**-Version ist hier idiomatischer und kompakter. Verwende **for**, wenn du über eine bekannte Sequenz iterierst.

Eingabevalidierung mit while

Ein klassischer Anwendungsfall für **while**-Schleifen ist die Validierung von Benutzereingaben. Die Schleife läuft so lange, bis eine gültige Eingabe erfolgt.

[!TIP] **Beispiel: Eingabe einer Zahl zwischen 1 und 10**

Die Schleife läuft, bis **zahl** im Bereich `[1, 10]` liegt. Ungültige Eingaben führen zu Fehlermeldungen und erneutem Prompt.

Akkumulation in Schleifen

Ein häufiges Muster in Schleifen ist die **Akkumulation** (Accumulation), bei der über mehrere Durchläufe ein Wert aufgebaut wird (z.B. Summe, Produkt, String-Konkatenation).

[!NOTE] **Akkumulation** bezeichnet das schrittweise Aufbauen eines Ergebnisses über mehrere Schleifendurchläufe. Typischerweise wird eine Variable vor der Schleife initialisiert (z.B. `summe = 0`) und in jedem Durchlauf aktualisiert (z.B. `summe += wert`).

Akkumulations-Muster:

1. **Summation:**
2. **Produkt:**
3. **String-Konkatenation:**

[!WARNING] **String-Konkatenation in Schleifen ist ineffizient:** Strings sind in Python immutable. Jede Konkatenation erzeugt ein neues String-Objekt. Bei vielen Durchläufen ist es effizienter, eine Liste zu verwenden und am Ende mit `.join()` zu verbinden (wird in V08 behandelt).

Zählen in Schleifen

Ein weiteres häufiges Muster ist das **Zählen**, bei dem die Häufigkeit eines Ereignisses gezählt wird.

[!TIP] **Beispiel: Gerade Zahlen von 1 bis 100 zählen**

Der Modulo-Operator `%` gibt den Rest der Division zurück. Eine Zahl ist gerade, wenn `zahl % 2 == 0`.

Iterieren über Listen (Vorschau)

Obwohl Listen erst in V08 ausführlich behandelt werden, können wir bereits jetzt über einfache Listen iterieren, da Listen ebenfalls Iterables sind.

Die `for`-Schleife greift automatisch auf jedes Element der Liste zu und weist es der Variable `frucht` zu.

`enumerate()` für Index und Wert

Manchmal benötigt man sowohl den Index als auch den Wert eines Elements. Die `enumerate()`-Funktion liefert beide.

[!NOTE] `enumerate(iterable, start=0)` (Built-in) gibt ein Iterable zurück, das Tupel aus Index und Wert erzeugt. Der Parameter `start` legt fest, bei welchem Index die Zählung beginnt (Standard: 0).

Mit `start=1` kann die Zählung bei 1 beginnen:

Häufige Fehler und Lösungen

[!WARNING] **Fehler 1: Modifikation der Schleifenvariable in `for` wirkt sich nicht auf die Iteration aus**

Lösung: Die Schleifenvariable in `for` sollte nicht im Schleifenkörper geändert werden. Sie wird in jedem Durchlauf neu zugewiesen. Verwende eine separate Variable, falls nötig.

[!WARNING] **Fehler 2: `range()` erzeugt keine Liste, sondern ein Range-Objekt**

Lösung: Verwende `list(range(5))`, wenn du eine Liste benötigst. Für Schleifen ist das Range-Objekt aber effizienter und sollte bevorzugt werden.

[!WARNING] **Fehler 3: Vergessen, die Schleifenvariable in while zu aktualisieren**

Lösung: Stelle sicher, dass die Bedingung irgendwann **False** wird, indem du die relevanten Variablen im Schleifenkörper aktualisierst.

Zusammenfassung Python

- Die **for-Schleife** iteriert über Sequenzen (Iterables) und eignet sich für bekannte Anzahl von Durchläufen oder Iterationen über Sammlungen
- **range(start, stop, step)** erzeugt Zahlenfolgen, wobei **stop** exklusive ist. Häufigste Form: **range(n)** für 0 bis n-1
- Die **while-Schleife** wiederholt Code, solange eine Bedingung **True** ist. Wichtig: Abbruchbedingung muss irgendwann **False** werden, sonst entsteht Endlos-Schleife
- **Akkumulation** (Aufbauen von Summen, Produkten, Strings) und **Zählen** sind typische Muster in Schleifen
- **enumerate(iterable)** liefert Index und Wert gleichzeitig, nützlich für nummerierte Ausgaben
- **Wann for, wann while?** Verwende **for** für Iterationen über Sequenzen und bekannte Durchläufe, **while** für dynamische Bedingungen und unbekannte Durchläufe

Neue Python-Funktionen/Methoden

[!NOTE] Diese Funktionen/Methoden wurden in V06 **neu eingeführt**:

Kontrollstrukturen

- **for variable in iterable:** (Python Keyword)
 - Iteriert über alle Elemente eines Iterables
 - Signatur: **for variable in iterable:**
 - Beispiel: **for i in range(5): print(i)**
- **while bedingung:** (Python Keyword)
 - Wiederholt Codeblock, solange Bedingung **True** ist
 - Signatur: **while condition:**
 - Beispiel: **while x < 10: x += 1**

Built-in Funktionen

- **range(start=0, stop, step=1)** (Built-in, Python 3.0+)
 - Erzeugt Sequenz von Ganzzahlen
 - Parameter:
 - * **start**: Startwert (inklusive, Standard: 0)
 - * **stop**: Endwert (exklusive, erforderlich)
 - * **step**: Schrittweite (Standard: 1)

- Signatur: `range(stop)` oder `range(start, stop, step=1)` → `range`
 - Beispiel: `range(5)` → 0, 1, 2, 3, 4
 - Beispiel: `range(2, 10, 2)` → 2, 4, 6, 8
 - **`enumerate(iterable, start=0)`** (Built-in)
 - Gibt Iterator zurück, der Tupel aus Index und Wert erzeugt
 - Parameter:
 - * **`iterable`**: Die Sequenz, über die iteriert wird
 - * **`start`**: Startwert für den Index (Standard: 0)
 - Signatur: `enumerate(iterable, start=0)` → `enumerate`
 - Beispiel: `for i, wert in enumerate(['a', 'b', 'c']):`
`print(i, wert)`
-

Weiterführende Ressourcen

Theorie

- DIN 66001 / ISO 5807: Offizielle Standards für Programm-Ablauf-Pläne
- Cormen et al.: “Introduction to Algorithms” (4. Auflage) – Kapitel zu Algorithmen-Notation
- Robert Sedgewick: “Algorithms” – Enthält viele PAPs und Pseudocode-Beispiele
- UML 2.5 Specification – Für Aktivitätsdiagramme und Verhaltensmodellierung

Python

- Python Documentation: Control Flow - <https://docs.python.org/3/tutorial/controlflow.html>
 - Real Python: Python “for” Loops - <https://realpython.com/python-for-loop/>
 - Real Python: Python “while” Loops - <https://realpython.com/python-while-loop/>
 - Python Documentation: Built-in Functions (range, enumerate) - <https://docs.python.org/3/library/functions.html>
 - PEP 3136 – Labeled break and continue (rejected, aber interessant für Verständnis von Schleifen-Kontrolle)
-

Datei: `lessons/V07-Software-Engineering-KISS-DRY-SRP/V07-Software-Engineering-KISS-DRY-SRP_skript.md`

V07: Software Engineering (KISS, DRY, SRP)

[!NOTE] **Lernziele dieser Vorlesung:** - Die drei grundlegenden

Software-Engineering-Prinzipien KISS, DRY und SRP verstehen und anwenden können - Code-Qualität anhand objektiver Kriterien bewerten können - Typische Code-Smells erkennen und durch Refactoring beseitigen - **break** und **continue** zur präzisen Schleifensteuerung einsetzen - Verschachtelte Schleifen korrekt implementieren und deren Komplexität verstehen - Die **else**-Klausel bei Schleifen für elegante Suchalgorithmen nutzen - List Comprehensions als kompakte Alternative zu expliziten Schleifen schreiben

Teil 1: Theorie - Software Engineering Prinzipien

Überblick

Software Engineering beschäftigt sich mit der systematischen Entwicklung, Wartung und Verwaltung von Softwaresystemen. Während in den bisherigen Vorlesungen die korrekte Funktionsweise von Code im Vordergrund stand, geht es nun um die Qualität des Codes selbst. Guter Code ist nicht nur Code, der funktioniert, sondern Code, der lesbar, wartbar, erweiterbar und robust ist. Die drei Prinzipien KISS, DRY und SRP bilden das Fundament für qualitativ hochwertigen Code und sind in der professionellen Softwareentwicklung unverzichtbar.

Diese Prinzipien sind sprachunabhängig und gelten für alle Programmierparadigmen. Sie helfen dabei, technische Schulden zu vermeiden und die Langlebigkeit von Software sicherzustellen. Technische Schulden entstehen, wenn kurzfristige Lösungen die langfristige Wartbarkeit beeinträchtigen. Je früher diese Prinzipien verinnerlicht werden, desto natürlicher werden sie in der täglichen Programmierpraxis angewendet.

Das KISS-Prinzip: Keep It Simple, Stupid

[!NOTE] **KISS (Keep It Simple, Stupid)**: Ein Software-Design-Prinzip, das besagt, dass Systeme am besten funktionieren, wenn sie einfach gehalten werden, anstatt unnötig komplex zu sein.

Das KISS-Prinzip wurde in den 1960er Jahren von der US-Navy entwickelt und ist heute eines der fundamentalsten Prinzipien im Software Engineering. Die zentrale Aussage lautet: Einfachheit sollte ein primäres Designziel sein, und unnötige Komplexität sollte aktiv vermieden werden. Dies bedeutet nicht, dass Software simpel oder primitiv sein soll, sondern dass jede Komplexität gerechtfertigt sein muss.

Komplexität in Software entsteht auf verschiedenen Ebenen. Die **algorithmische Komplexität** beschreibt, wie aufwändig ein Algorithmus ist. Die **strukturelle Komplexität** bezieht sich auf die Architektur und Organisation des

Codes. Die **kognitive Komplexität** misst, wie schwierig es ist, den Code zu verstehen. KISS zielt primär darauf ab, die kognitive Komplexität zu minimieren, da diese die Wartbarkeit am stärksten beeinflusst.

Warum Einfachheit wichtig ist Einfacher Code bietet zahlreiche Vorteile, die sich über den gesamten Softwarelebenszyklus bemerkbar machen. Die **Verständlichkeit** verbessert sich dramatisch, da neue Teammitglieder den Code schneller erfassen können und die Einarbeitungszeit sinkt. Die **Wartbarkeit** profitiert davon, dass Bugs leichter zu finden und zu beheben sind. Die **Erweiterbarkeit** wird erleichtert, weil neue Features ohne große Umbauten integriert werden können. Schließlich führt Einfachheit zu weniger Bugs, da weniger Komplexität weniger potenzielle Fehlerquellen bedeutet.

Komplexer Code hingegen birgt erhebliche Risiken. Versteckte Bugs treten häufiger auf, da sie in verschachtelten Strukturen schwerer zu entdecken sind. Die Entwicklungsgeschwindigkeit sinkt, weil Änderungen mehr Zeit und Aufwand erfordern. Refactoring wird schwieriger, da die Auswirkungen von Änderungen schwer abzuschätzen sind. Zudem führt komplexer Code zu Frustration im Team, was die Motivation und Produktivität beeinträchtigt.

KISS in der Praxis Die Anwendung des KISS-Prinzips erfordert bewusste Entscheidungen auf allen Ebenen der Softwareentwicklung. Bei der **Namenswahl** sollten Variablen, Funktionen und Klassen selbsterklärende Namen erhalten, die ihre Funktion klar beschreiben. Vermeide kryptische Abkürzungen, die nur dem ursprünglichen Autor verständlich sind. Statt `calc_res_for_usr_inp` ist `calculate_result_for_user_input` oder noch besser `process_user_calculation` viel klarer.

Bei **Funktionen und Methoden** gilt die Regel: Eine Funktion sollte eine klar definierte Aufgabe erfüllen. Funktionen, die mehrere nicht zusammenhängende Dinge tun, sollten aufgeteilt werden. Eine Funktion mit 200 Zeilen Code ist fast immer ein Zeichen dafür, dass sie gegen KISS verstößt. Als Faustregel gilt: Wenn eine Funktion nicht auf einen Bildschirm passt, ist sie wahrscheinlich zu komplex.

Die **Verschachtelungstiefe** sollte begrenzt werden. Mehr als drei Ebenen von verschachtelten `if`-Statements oder Schleifen machen Code schwer lesbar. In solchen Fällen sollte der Code durch frühe Returns, Hilfsfunktionen oder Guard Clauses umstrukturiert werden.

Bei **Algorithmen** bedeutet KISS nicht, ineffiziente Lösungen zu wählen. Vielmehr geht es darum, den einfachsten Algorithmus zu wählen, der die Anforderungen erfüllt. Vorzeitige Optimierung ist oft die Wurzel unnötiger Komplexität. Erst wenn ein Performanzproblem tatsächlich gemessen wurde, sollte optimiert werden.

[!TIP]

[!WARNING] **Häufiger Fehler:** KISS bedeutet nicht, auf Abstraktionen oder Design Patterns zu verzichten, wenn sie angemessen sind. Es bedeutet, keine unnötigen Abstraktionen einzuführen. Die Kunst liegt darin, die richtige Balance zu finden.

Wann ist Code zu einfach? Es gibt Situationen, in denen übermäßige Vereinfachung problematisch sein kann. Wenn ein einfacher Algorithmus signifikante Performance-Probleme verursacht, die durch eine etwas komplexere Lösung behoben werden können, ist die Komplexität gerechtfertigt. Wenn Domänen-Logik inhärent komplex ist (z.B. in Finanzsystemen oder wissenschaftlichen Berechnungen), sollte diese Komplexität nicht künstlich verschleiert werden. Wenn wichtige Fehlerbehandlung oder Validierung weggelassen wird, um den Code “einfach” zu halten, ist dies ein Fehler. KISS bedeutet nicht, auf notwendige Robustheit zu verzichten.

Das DRY-Prinzip: Don’t Repeat Yourself

[!NOTE] **DRY (Don’t Repeat Yourself):** Ein Software-Entwicklungsprinzip, das besagt, dass jede Information in einem System eine einzige, eindeutige und autoritative Repräsentation haben sollte.

Das DRY-Prinzip wurde von Andrew Hunt und David Thomas in ihrem Buch “The Pragmatic Programmer” (1999) formuliert und ist heute ein zentrales Konzept moderner Softwareentwicklung. Die Kernidee ist, dass Code-Duplikation aktiv vermieden werden sollte, weil sie zu Inkonsistenzen, erhöhtem Wartungsaufwand und Fehleranfälligkeit führt.

Wichtig ist zu verstehen, dass DRY sich nicht nur auf identischen Code bezieht. Auch konzeptionelle Duplikation, bei der dasselbe Konzept an mehreren Stellen auf unterschiedliche Weise implementiert wird, verstößt gegen DRY. Ebenso fällt die Duplikation von Wissen oder Business-Logik unter dieses Prinzip.

Warum Code-Duplikation problematisch ist Code-Duplikation führt zu zahlreichen Problemen im Softwarelebenszyklus. Bei der **Wartung** entsteht das Problem, dass bei Änderungen alle duplizierten Stellen gefunden und konsistent aktualisiert werden müssen. Übersieht man eine Stelle, entstehen Inkonsistenzen. Bugs müssen an mehreren Stellen gefixt werden, was zeitaufwändig und fehleranfällig ist.

Die **Konsistenz** leidet, weil verschiedene Versionen desselben Codes sich im Laufe der Zeit auseinanderentwickeln können. Dies führt zu unerwartetem Verhalten und macht das System unvorhersehbar. Die **Codegröße** wächst unnötig, was die Übersichtlichkeit reduziert und die Build-Zeit erhöht. Bei der **Fehler-suche** wird es schwieriger, die Ursache von Bugs zu lokalisieren, wenn derselbe fehlerhafte Code an mehreren Stellen existiert.

Arten von Code-Duplikation Es gibt verschiedene Formen von Duplikation, die erkannt werden sollten. Die **exakte Duplikation** liegt vor, wenn identischer oder nahezu identischer Code mehrfach vorkommt. Dies ist die offensichtlichste Form und am leichtesten zu beheben.

Die **strukturelle Duplikation** tritt auf, wenn derselbe Algorithmus oder dieselbe Logik mit leicht unterschiedlichen Variablen oder Parametern mehrfach implementiert wird. Diese Form ist subtiler und erfordert Abstraktion durch Funktionen oder Klassen.

Die **konzeptionelle Duplikation** entsteht, wenn dasselbe Problem auf verschiedene Weisen gelöst wird. Beispielsweise könnte Datums-Formatierung an einer Stelle mit `strftime()` und an anderer Stelle manuell mit String-Operationen erfolgen. Beide implementieren dasselbe Konzept, aber nicht konsistent.

Die **Datenduplikation** bezieht sich auf redundante Datenhaltung, bei der dieselbe Information an mehreren Stellen gespeichert wird, ohne dass eine Single Source of Truth existiert.

DRY in der Praxis Die Anwendung des DRY-Prinzips erfolgt durch verschiedene Techniken. **Funktionen und Methoden** sind das primäre Werkzeug zur Vermeidung von Code-Duplikation. Wiederholter Code sollte in Funktionen extrahiert werden. Diese Funktionen sollten mit klaren, aussagekräftigen Namen versehen werden, die ihre Aufgabe beschreiben.

Konstanten und Konfigurationen helfen dabei, Magic Numbers und wiederholte Werte zu vermeiden. Statt die Zahl `3.14159` an mehreren Stellen zu verwenden, sollte eine Konstante `PI = 3.14159` definiert werden. Dies gilt auch für Schwellwerte, Limits und Konfigurationswerte.

Schleifen und Datenstrukturen können oft genutzt werden, um repetitiven Code zu eliminieren. Anstatt denselben Code für verschiedene Eingaben zu kopieren, sollte eine Schleife über eine Datenstruktur verwendet werden.

Abstraktionen und Design Patterns ermöglichen es, wiederkehrende Lösungsmuster elegant zu implementieren. Template Method, Strategy oder Factory Patterns sind Beispiele für Abstraktionen, die Duplikation auf höherer Ebene vermeiden.

[!TIP]

[!WARNING] **Häufiger Fehler:** Zwei Code-Stücke, die zufällig momentan identisch sind, aber unterschiedliche Konzepte repräsentieren, sollten nicht “getrocknet” werden. Die Ähnlichkeit könnte zufällig sein, und eine gemeinsame Abstraktion würde künstliche Kopplung erzeugen. Die “Rule of Three” besagt: Erst beim dritten Vorkommen refaktorisieren, nicht beim zweiten.

Wann ist DRY zu weit getrieben? Es gibt Situationen, in denen strikte Anwendung von DRY kontraproduktiv sein kann. Wenn die Abstraktion komplexer ist als die ursprüngliche Duplikation, ist DRY übertrieben. Eine einfache Wiederholung von zwei Zeilen kann klarer sein als eine generische Funktion mit vielen Parametern.

Wenn zwei Code-Stücke nur zufällig ähnlich sind, aber unterschiedliche Verantwortlichkeiten haben, sollten sie nicht zusammengefasst werden. Dies würde unerwünschte Kopplung erzeugen. Wenn eine Änderung an der gemeinsamen Abstraktion eine der Verwendungsstellen betrifft, aber nicht die andere, ist die Abstraktion falsch gewählt.

In Performance-kritischen Situationen kann manchmal bewusste Duplikation akzeptabel sein, wenn die Abstraktion zu einem messbaren Performance-Verlust führt. Dies sollte aber die Ausnahme sein und durch Profiling belegt werden.

Das SRP: Single Responsibility Principle

[NOTE] **SRP (Single Responsibility Principle):** Ein Prinzip aus der objektorientierten Programmierung, das besagt, dass jede Klasse, Funktion oder Modul genau eine Verantwortlichkeit haben sollte und nur einen Grund zur Änderung haben sollte.

Das Single Responsibility Principle wurde von Robert C. Martin (“Uncle Bob”) als Teil der SOLID-Prinzipien formuliert und ist fundamental für modulare, wartbare Software. Die zentrale Idee ist, dass jede Softwareeinheit eine klar definierte Aufgabe haben sollte. Dies reduziert Komplexität, verbessert die Testbarkeit und erleichtert Änderungen.

Robert C. Martin definiert eine “Verantwortlichkeit” als “einen Grund zur Änderung”. Wenn eine Klasse oder Funktion aus mehreren unterschiedlichen Gründen geändert werden muss, verletzt sie das SRP. Beispielsweise sollte eine Funktion, die sowohl Daten aus einer Datei liest als auch Berechnungen durchführt, aufgeteilt werden, da Änderungen am Dateiformat und Änderungen an der Berechnungslogik unterschiedliche Verantwortlichkeiten darstellen.

Warum SRP wichtig ist Das Single Responsibility Principle bietet zahlreiche Vorteile für die Softwarequalität. Die **Wartbarkeit** verbessert sich, weil Änderungen lokal bleiben und sich nicht über das gesamte System ausbreiten. Wenn eine Funktion nur eine Aufgabe hat, sind die Auswirkungen von Änderungen leicht vorhersehbar.

Die **Testbarkeit** profitiert enorm von SRP. Kleine, fokussierte Funktionen sind einfach zu testen. Unit Tests können präzise geschrieben werden, und Edge Cases sind leichter zu identifizieren. Testdaten sind weniger komplex, wenn Funktionen nur eine Sache tun.

Die **Wiederverwendbarkeit** steigt, da spezialisierte Funktionen in verschiedenen Kontexten eingesetzt werden können. Eine Funktion, die nur eine Aufgabe

erfüllt, ist wahrscheinlicher an anderer Stelle nützlich als eine Funktion, die mehrere Dinge tut.

Die **Verständlichkeit** wird deutlich besser. Code, der SRP befolgt, liest sich wie gut strukturierte Prosa. Jede Funktion hat einen klaren Namen, der ihre Aufgabe beschreibt, und der Code innerhalb der Funktion fokussiert sich auf diese eine Aufgabe.

SRP in der Praxis Die Anwendung des SRP erfordert sorgfältige Planung und kontinuierliches Refactoring. Bei **Funktionen** bedeutet SRP, dass jede Funktion genau eine klar definierte Aufgabe erfüllen sollte. Eine Funktion namens `berechne_und_speichere_ergebnis()` verletzt vermutlich SRP, da sie zwei Dinge tut: berechnen und speichern. Besser wäre es, dies in `berechne_ergebnis()` und `speichere_ergebnis()` zu trennen.

Bei **Klassen** (die in späteren Vorlesungen behandelt werden) sollte jede Klasse eine kohärente Menge von Funktionen bereitstellen, die alle derselben übergeordneten Verantwortlichkeit dienen. Eine Klasse `BenutzerManager`, die gleichzeitig Benutzer authentifiziert, E-Mails versendet und Datenbank-Backups erstellt, verletzt SRP massiv.

Bei **Modulen und Dateien** sollte die Dateistruktur die Verantwortlichkeiten widerspiegeln. Eine Python-Datei sollte zusammenhängende Funktionalität enthalten. Eine Datei `utils.py`, die zufällige Hilfsfunktionen sammelt, verletzt oft SRP, weil sie keine klare Verantwortlichkeit hat.

Wie man Verantwortlichkeiten identifiziert Die Identifikation von Verantwortlichkeiten erfordert Übung und Erfahrung. Ein praktischer Ansatz ist die **“Reason to Change”-Methode**. Stelle dir die Frage: “Aus welchen unterschiedlichen Gründen könnte dieser Code geändert werden müssen?” Wenn du mehr als einen Grund findest, liegt wahrscheinlich eine SRP-Verletzung vor.

Die **Beschreibungsmethode** hilft ebenfalls. Versuche, die Aufgabe einer Funktion in einem Satz zu beschreiben, ohne “und” zu verwenden. Wenn du “und” brauchst, macht die Funktion wahrscheinlich mehrere Dinge.

Die **Kohäsionsanalyse** betrachtet, wie stark die verschiedenen Teile einer Funktion oder Klasse zusammenhängen. Wenn Variablen und Operationen logisch zusammengehören und aufeinander aufbauen, ist die Kohäsion hoch. Wenn verschiedene Teile unabhängig voneinander sind, ist die Kohäsion niedrig und SRP wird wahrscheinlich verletzt.

[!TIP]

[!WARNING] **Häufiger Fehler:** Über-Engineering durch zu starke Aufteilung. Nicht jede Zeile Code muss in eine eigene Funktion. Die Aufteilung sollte logische Verantwortlichkeiten widerspiegeln, nicht willkürlich erfolgen. Eine Funktion mit 5-10 Zeilen, die eine kohärente Aufgabe erfüllt, ist oft besser als fünf Einzeilen-Funktionen.

Wann ist SRP zu strikt? Es gibt Situationen, in denen eine strikte Anwendung von SRP kontraproduktiv sein kann. Bei sehr kleinen Funktionen (3-5 Zeilen), die eine offensichtlich zusammenhängende Aufgabe erfüllen, ist weitere Aufteilung oft übertrieben. Wenn die Abstraktion die Lesbarkeit verschlechtert, weil man ständig zwischen Funktionen hin- und herspringen muss, ist SRP zu weit getrieben.

In Performance-kritischen Abschnitten kann es sinnvoll sein, verwandte Operationen zusammenzufassen, um Funktionsaufruf-Overhead zu vermeiden. Dies sollte aber durch Profiling belegt sein und dokumentiert werden.

Bei prototypischer Entwicklung oder Proof-of-Concepts ist es oft pragmatischer, zunächst funktionierenden Code zu schreiben und erst später zu refaktorisieren. Verfrühte Abstraktion kann die Entwicklung verlangsamen, ohne echten Nutzen zu bringen.

Code-Qualität und Wartbarkeit

Die drei Prinzipien KISS, DRY und SRP bilden zusammen ein starkes Fundament für qualitativ hochwertigen Code. Sie ergänzen sich gegenseitig und überlappen in vielen Bereichen. KISS sorgt dafür, dass Code verständlich bleibt. DRY verhindert Redundanz und Inkonsistenz. SRP stellt sicher, dass Code modular und änderbar ist.

Code-Qualität ist nicht nur eine ästhetische Frage, sondern hat direkte Auswirkungen auf den Projekterfolg. Schlechte Code-Qualität führt zu erhöhten Wartungskosten, längeren Entwicklungszeiten und höherer Fehlerrate. Gute Code-Qualität ermöglicht es Teams, nachhaltig produktiv zu bleiben und flexibel auf Änderungen zu reagieren.

Code-Smells und Refactoring **Code-Smells** sind Symptome im Code, die auf tieferliegende Probleme hindeuten. Sie sind keine Bugs, aber Warnsignale, dass der Code verbessert werden sollte. Typische Code-Smells umfassen lange Funktionen (mehr als 20-30 Zeilen), viele Parameter (mehr als 3-4), tief verschachtelte Strukturen (mehr als 3 Ebenen), duplizierter Code, magische Zahlen ohne Erklärung, unklare Namensgebung und Funktionen mit Seiteneffekten.

Refactoring ist der Prozess, Code zu verbessern, ohne sein externes Verhalten zu ändern. Refactoring sollte in kleinen, sicheren Schritten erfolgen. Jeder Schritt sollte getestet werden, bevor der nächste erfolgt. Idealerweise existieren automatisierte Tests, die sicherstellen, dass Refactoring keine Funktionalität bricht.

Typische Refactoring-Operationen umfassen das Extrahieren von Funktionen (Extract Method), das Umbenennen von Variablen (Rename), das Zusammenfassen von dupliziertem Code (Extract Method oder Superclass), das Aufteilen von langen Funktionen (Extract Method), das Einführen von Konstanten für Magic Numbers und das Vereinfachen von Bedingungen.

Code Reviews und Best Practices Code Reviews sind ein essentieller Bestandteil professioneller Softwareentwicklung. Sie dienen dazu, Fehler frühzeitig zu finden, Wissen im Team zu teilen, Code-Qualität zu sichern und Best Practices zu etablieren. Bei einem Code Review prüft ein anderer Entwickler den Code, bevor er ins Hauptrepository integriert wird.

Effektive Code Reviews konzentrieren sich auf die Lesbarkeit des Codes, die Einhaltung von Coding Standards, die Anwendung von KISS, DRY und SRP, potenzielle Bugs und Edge Cases, die Korrektheit von Tests und die Dokumentation.

Best Practices für Code Reviews umfassen konstruktives Feedback mit konkreten Verbesserungsvorschlägen, die Fokussierung auf den Code statt auf die Person, das Anbieten von Alternativen statt bloßer Kritik, die zeitnahe Durchführung von Reviews und die Begrenzung der Review-Größe auf überschaubare Mengen (typischerweise 200-400 Zeilen Code).

Zusammenfassung Theorie

Die drei Software-Engineering-Prinzipien KISS, DRY und SRP bilden das Fundament für qualitativ hochwertigen, wartbaren Code. Das **KISS-Prinzip** fordert, Komplexität nur dort einzuführen, wo sie notwendig ist, und Einfachheit als primäres Designziel zu verfolgen. Das **DRY-Prinzip** eliminiert Code-Duplikation und stellt sicher, dass jede Information im System eine einzige, autoritative Repräsentation hat. Das **SRP** garantiert, dass jede Code-Einheit eine klar definierte Verantwortlichkeit hat und nur aus einem Grund geändert werden muss.

Diese Prinzipien sind nicht dogmatisch zu verstehen, sondern als Leitlinien, die mit Erfahrung und Kontext angewendet werden sollten. Die Balance zwischen Einfachheit und Funktionalität, zwischen DRY und Pragmatismus, zwischen SRP und Lesbarkeit ist eine Kunst, die mit der Zeit entwickelt wird. Code-Qualität ist ein kontinuierlicher Prozess, kein einmaliges Ziel. Durch regelmäßiges Refactoring, Code Reviews und bewusste Anwendung dieser Prinzipien entsteht langfristig wartbare Software, die den Anforderungen moderner Entwicklung gerecht wird.

Teil 2: Python-Praxis - Schleifen (for, while) – Teil 2

[!WARNING] **Python-Konsistenz beachten:** Die Grundlagen von `for` und `while` wurden in V06 eingeführt. Hier erweitern wir das Wissen um fortgeschrittene Schleifensteuerung.

Überblick

In V06 wurden die Grundlagen von Schleifen behandelt: **for**-Schleifen zum Iterieren über Sequenzen und **while**-Schleifen für bedingte Wiederholungen. Diese Vorlesung erweitert das Schleifen-Repertoire um drei mächtige Konzepte: Die präzise Steuerung mit **break** und **continue**, die elegante **else**-Klausel für Schleifen und die kompakte List Comprehension-Syntax. Diese Werkzeuge ermöglichen es, komplexe Iterationslogik elegant und pythonisch zu formulieren.

Schleifen sind fundamentale Kontrollstrukturen, und ihre Beherrschung ist essentiell für effektive Programmierung. Die hier vorgestellten Konzepte sind typisch für Python und heben sich von vielen anderen Programmiersprachen ab. Die **else**-Klausel bei Schleifen ist beispielsweise ein Python-spezifisches Feature, das in Sprachen wie Java oder C++ nicht existiert.

Schleifensteuerung mit **break** und **continue**

Python bietet zwei Keywords zur feingranularen Steuerung von Schleifen: **break** und **continue**. Beide beeinflussen den Kontrollfluss innerhalb von Schleifen, erfüllen aber unterschiedliche Zwecke.

Das **break**-Statement

[!NOTE] **break**: Ein Python-Keyword, das die Ausführung der umgebenden Schleife sofort beendet und die Kontrolle an die nächste Anweisung nach der Schleife übergibt.

Das **break**-Statement wird verwendet, wenn eine Schleife vorzeitig beendet werden soll, weil eine bestimmte Bedingung erfüllt ist. Nach der Ausführung von **break** wird die Schleife verlassen, und das Programm fährt mit der ersten Anweisung nach der Schleife fort. **break** funktioniert sowohl in **for**-Schleifen als auch in **while**-Schleifen.

Typische Anwendungsfälle für **break** umfassen die Suche in Listen oder Strings, bei der die Schleife beendet werden soll, sobald das gesuchte Element gefunden wurde. Eingabevalidierung nutzt **break**, um aus einer Eingabeschleife auszubrechen, wenn gültige Daten eingegeben wurden. Bei der Verarbeitung von Dateien oder Datenströmen wird **break** verwendet, wenn ein Abbruchkriterium (z.B. ein Sentinel-Wert) erkannt wird. In Spielprogrammierung dient **break** dazu, Spiel-Loops bei Game Over oder Pause zu beenden.

[!TIP]

Das **break**-Statement beeinflusst nur die innerste umgebende Schleife. Bei verschachtelten Schleifen beendet **break** nur die Schleife, in der es steht, nicht die äußeren Schleifen.

[!TIP]

Das `continue`-Statement

[!NOTE] **continue:** Ein Python-Keyword, das den aktuellen Schleifendurchlauf sofort beendet und mit dem nächsten Durchlauf fortsetzt, ohne die Schleife zu verlassen.

Das `continue`-Statement überspringt den Rest des aktuellen Schleifendurchlaufs und springt direkt zur nächsten Iteration. Bei `for`-Schleifen wird die Schleifenvariable auf den nächsten Wert gesetzt. Bei `while`-Schleifen wird die Bedingung erneut geprüft. `continue` wird verwendet, wenn bestimmte Elemente übersprungen werden sollen, ohne die gesamte Schleife zu beenden.

Typische Anwendungsfälle für `continue` umfassen das Filtern von Elementen während der Verarbeitung, das Überspringen ungültiger oder fehlerhafter Daten, das Ignorieren von Sonderfällen und das Vermeiden tief verschachtelter `if`-Statements durch frühe Fortsetzung.

[!TIP]

Vergleich: `break` vs. `continue` Die Unterschiede zwischen `break` und `continue` sind fundamental für ihr Verständnis. `break` beendet die gesamte Schleife sofort, während `continue` nur den aktuellen Durchlauf beendet. Nach `break` wird die Schleife nicht mehr fortgesetzt, nach `continue` geht es mit dem nächsten Element weiter. `break` wird für vorzeitigen Abbruch bei erfüllter Bedingung verwendet, `continue` zum Überspringen einzelner Iterationen. Bei Schleifen mit `else`-Klausel verhindert `break` die Ausführung der `else`-Klausel, `continue` nicht.

[!TIP]

[!WARNING] **Häufige Fehler mit `break` und `continue`:**

Fehler 1: `continue` in Kombination mit Inkrement-Operationen bei `while`-Schleifen vergessen

Fehler 2: `break` verwenden, wenn `continue` gemeint war (oder umgekehrt)

Best Practices für `break` und `continue` Die effektive Nutzung von `break` und `continue` erfordert Sorgfalt. Vermeide übermäßigen Einsatz, der die Lesbarkeit beeinträchtigt. Zu viele `break` oder `continue` in einer Schleife machen den Kontrollfluss schwer nachvollziehbar. Überlege, ob die Logik nicht klarer mit besseren Bedingungen formuliert werden kann.

Bevorzuge klare Schleifenbedingungen über `break`, wenn möglich. Eine `while`-Schleife mit aussagekräftiger Bedingung ist oft verständlicher als `while True` mit `break`. Nutze `continue` zur Reduzierung von Verschachtelung. Statt tief verschachtelter `if`-Statements kann `continue` verwendet werden, um frühe Exits zu implementieren (Guard Clauses).

Dokumentiere komplexe Verwendungen von **break** und **continue** mit Kommentaren, damit andere Entwickler (oder dein zukünftiges Ich) die Intention verstehen. In verschachtelten Schleifen verwende aussagekräftige Variablennamen und strukturiere den Code so, dass klar ist, welche Schleife von **break** oder **continue** betroffen ist.

Die **else**-Klausel bei Schleifen

Python bietet eine einzigartige Ergänzung zu Schleifen: die **else**-Klausel. Dieses Feature existiert in den meisten anderen Programmiersprachen nicht und ist eine der eleganten Besonderheiten von Python.

Funktionsweise der Schleifen-**else**-Klausel

[!NOTE] **Schleifen-else-Klausel:** Ein Python-Feature, bei dem ein **else**-Block nach einer Schleife ausgeführt wird, wenn die Schleife vollständig durchlaufen wurde, ohne dass **break** aufgerufen wurde.

Die **else**-Klausel kann sowohl an **for**-Schleifen als auch an **while**-Schleifen angehängt werden. Der Code im **else**-Block wird nur ausgeführt, wenn die Schleife normal beendet wurde, das heißt ohne Unterbrechung durch **break**. Wenn **break** aufgerufen wird, wird der **else**-Block übersprungen.

Diese Semantik mag zunächst kontraintuitiv erscheinen, da **else** typischerweise “sonst” bedeutet. In diesem Kontext ist es besser als “dann” oder “abschließend, falls nicht abgebrochen” zu interpretieren. Eine alternative Benennung wäre **nobreak** gewesen, aber Python verwendet **else** für Konsistenz.

[!TIP]

Praktische Anwendungsfälle Die **else**-Klausel bei Schleifen ist besonders nützlich für Suchalgorithmen. Wenn nach einem Element gesucht wird und es nicht gefunden wurde, wird der **else**-Block ausgeführt. Dies vermeidet die Notwendigkeit zusätzlicher Boolean-Flags.

Bei der Validierung von Daten kann die **else**-Klausel verwendet werden, um anzuzeigen, dass alle Elemente eine Bedingung erfüllen. Wenn ein Element die Bedingung nicht erfüllt, wird **break** aufgerufen und **else** wird übersprungen.

Für Benutzer-Dialoge mit Abbruchmöglichkeit zeigt die **else**-Klausel an, dass der Benutzer die Interaktion nicht vorzeitig abgebrochen hat. In Algorithmen, die nach der ersten Lösung suchen, signalisiert **else**, dass keine Lösung gefunden wurde.

[!TIP]

Alternative ohne **else-Klausel** Bevor die **else**-Klausel bei Schleifen eingeführt wurde (und in Sprachen, die sie nicht haben), wurde typischerweise ein

Boolean-Flag verwendet. Dieser Ansatz funktioniert, ist aber weniger elegant und erfordert zusätzlichen Code.

[!TIP]

[!WARNING] **Häufiger Fehler:** Die `else`-Klausel wird nicht als “falls Schleife nicht durchlaufen” interpretiert (was falsch ist), sondern als “falls `break` nicht aufgerufen wurde” (was richtig ist). Die Schleife kann vollständig durchlaufen werden, und `else` wird trotzdem ausgeführt – solange kein `break` vorkam.

Verschachtelte Schleifen

Verschachtelte Schleifen sind Schleifen innerhalb von Schleifen. Sie werden verwendet, wenn mehrdimensionale Strukturen verarbeitet werden müssen, wie Matrizen, Koordinatensysteme oder verschachtelte Listen.

Grundlagen verschachtelter Schleifen Bei verschachtelten Schleifen wird für jede Iteration der äußeren Schleife die innere Schleife vollständig durchlaufen. Dies führt zu einer multiplikativen Anzahl von Durchläufen. Wenn die äußere Schleife n -mal läuft und die innere m -mal, werden insgesamt $n \times m$ Durchläufe ausgeführt.

[!TIP]

Zeitkomplexität verschachtelter Schleifen Die Zeitkomplexität verschachtelter Schleifen ist ein wichtiges Konzept, das in V10 (Laufzeitanalyse) ausführlich behandelt wird. Zwei ineinander verschachtelte Schleifen, die jeweils n -mal laufen, haben eine Zeitkomplexität von $O(n^2)$. Drei verschachtelte Schleifen führen zu $O(n^3)$, und so weiter.

Diese exponentielle Steigerung macht verschachtelte Schleifen potenziell teuer. Bei großen Datenmengen kann die Laufzeit schnell problematisch werden. Daher sollte die Notwendigkeit verschachtelter Schleifen kritisch hinterfragt und nach effizienteren Alternativen gesucht werden.

[!TIP]

Praktische Beispiele Verschachtelte Schleifen finden Anwendung in vielen realen Szenarien. Bei der Matrixverarbeitung wird die äußere Schleife für Zeilen und die innere für Spalten verwendet. Koordinatensysteme nutzen verschachtelte Schleifen für x - und y -Koordinaten. Mustergenerierung erstellt 2D-Muster oder ASCII-Art. Kombinatorik erzeugt alle Paare, Tripel oder Permutationen. Algorithmen wie Bubble Sort, Selection Sort und Matrix-Multiplikation verwenden verschachtelte Schleifen.

[!TIP]

[!WARNING] **Performance-Fallstrick:** Verschachtelte Schleifen können bei großen Datenmengen sehr langsam werden. Wenn möglich, verwende effizientere Algorithmen oder Datenstrukturen. In vielen Fällen können verschachtelte Schleifen durch Hash-Tabellen (Dictionaries) oder Set-Operationen ersetzt werden, was die Komplexität von $O(n^2)$ auf $O(n)$ reduziert.

break und continue in verschachtelten Schleifen In verschachtelten Schleifen beeinflussen **break** und **continue** nur die innerste Schleife, in der sie stehen. Um die äußere Schleife zu beeinflussen, sind zusätzliche Mechanismen nötig.

[!TIP]

List Comprehensions

List Comprehensions sind eine kompakte und pythonische Methode, um Listen zu erstellen. Sie kombinieren die Funktionalität von Schleifen, Bedingungen und Listenerstellung in einem einzigen, lesbaren Ausdruck.

Grundsyntax der List Comprehension

[!NOTE] **List Comprehension:** Eine syntaktische Konstruktion in Python, die es ermöglicht, Listen durch eine kompakte Notation zu erzeugen, die eine Iteration und optional eine Bedingung enthält.

Die grundlegende Syntax einer List Comprehension ist: `[ausdruck for element in iterable]`. Dies ist äquivalent zu einer **for**-Schleife, die eine Liste aufbaut:

List Comprehensions sind nicht nur kürzer, sondern in den meisten Fällen auch schneller als äquivalente Schleifen, da sie intern optimiert sind.

[!TIP]

List Comprehensions mit Bedingungen List Comprehensions können eine optionale **if**-Bedingung enthalten, um Elemente zu filtern. Die Syntax ist: `[ausdruck for element in iterable if bedingung]`. Nur Elemente, für die die Bedingung **True** ist, werden in die resultierende Liste aufgenommen.

[!TIP]

Verschachtelte List Comprehensions List Comprehensions können verschachtelt werden, ähnlich wie verschachtelte Schleifen. Die Syntax wird von links nach rechts gelesen, ähnlich wie verschachtelte Schleifen von außen nach innen.

`[ausdruck for outer in outer_iterable for inner in inner_iterable]` entspricht:

[!TIP]

[!WARNING] **Lesbarkeit beachten:** Verschachtelte List Comprehensions können schnell unlesbar werden. Als Faustregel gilt: Wenn eine List Comprehension mehr als zwei Ebenen der Verschachtelung hat oder nicht auf eine Zeile passt, ist eine explizite Schleife wahrscheinlich lesbarer.

Bedingte Ausdrücke in List Comprehensions List Comprehensions können auch den ternären Operator enthalten, um verschiedene Werte basierend auf einer Bedingung zu erzeugen. Die Syntax ist: `[ausdruck_wenn_wahr if bedingung else ausdruck_wenn_falsch for element in iterable]`.

[!TIP]

Wann List Comprehensions verwenden List Comprehensions sind ideal für einfache Transformationen und Filterungen von Listen. Sie sollten verwendet werden, wenn die Logik in eine Zeile passt und leicht verständlich ist, wenn keine komplexe Fehlerbehandlung erforderlich ist, wenn die resultierende Liste tatsächlich benötigt wird (nicht nur Iteration) und wenn die Performance eine Rolle spielt (List Comprehensions sind oft schneller).

List Comprehensions sollten vermieden werden, wenn die Logik komplex ist und mehrere Verschachtelungen erfordert, wenn Seiteneffekte gewünscht sind (z.B. Ausgaben, Datei-Operationen), wenn die Lesbarkeit durch die kompakte Form leidet oder wenn die Fehlerbehandlung wichtig ist (try-except ist in List Comprehensions nicht möglich).

[!WARNING] **Häufiger Fehler:** List Comprehensions für Seiteneffekte verwenden

Set und Dictionary Comprehensions Neben List Comprehensions gibt es auch Set Comprehensions und Dictionary Comprehensions, die nach demselben Prinzip funktionieren. Diese werden in V08 ausführlicher behandelt, wenn Sets und Dictionaries eingeführt werden.

[!TIP]

Häufige Fehler und Lösungen

[!WARNING] **Fehler 1:** Schleifenvariable in List Comprehension außerhalb verwenden

Lösung: Verlasse dich nicht darauf, dass Schleifenvariablen nach der Schleife einen bestimmten Wert haben. Verwende explizite Variablen, wenn der Wert benötigt wird.

[!WARNING] **Fehler 2:** Modifikation der iterierten Liste während der Iteration

Lösung: Iteriere über eine Kopie oder erstelle eine neue Liste mit den gewünschten Elementen.

[!WARNING] **Fehler 3:** Unendliche Schleifen mit `while True` ohne `break`

Lösung: Stelle immer sicher, dass eine Abbruchbedingung existiert.

Zusammenfassung Python

Die erweiterten Schleifenkonzepte aus V07 vervollständigen das Werkzeugset für Iterationen in Python. Das **break-Statement** ermöglicht den vorzeitigen Abbruch von Schleifen, wenn eine Bedingung erfüllt ist, und ist besonders nützlich für Such-Algorithmen. Das **continue-Statement** überspringt den aktuellen Durchlauf und setzt die Schleife mit dem nächsten Element fort, was zur Filterung und Reduzierung von Verschachtelungen dient.

Die **else-Klausel bei Schleifen** ist ein Python-spezifisches Feature, das ausgeführt wird, wenn die Schleife vollständig durchlaufen wurde, ohne dass `break` aufgerufen wurde. Dies eliminiert die Notwendigkeit für Boolean-Flags in vielen Such- und Validierungs-Szenarien.

Verschachtelte Schleifen ermöglichen die Verarbeitung mehrdimensionaler Datenstrukturen, bringen aber eine Zeitkomplexität von $O(n^2)$ oder höher mit sich. Die Performance sollte bei großen Datenmengen berücksichtigt werden, und Alternativen wie Hash-Tabellen sollten in Betracht gezogen werden.

List Comprehensions bieten eine kompakte, pythonische Syntax zur Erstellung von Listen durch Iteration und optionale Filterung. Sie sind lesbarer und oft performanter als äquivalente Schleifen, sollten aber bei komplexer Logik zugunsten expliziter Schleifen vermieden werden.

Die Verbindung zwischen den Software-Engineering-Prinzipien aus Teil 1 und den Python-Konzepten aus Teil 2 ist klar erkennbar. KISS fordert, Schleifen einfach zu halten und keine unnötig verschachtelten Strukturen zu erstellen. DRY wird durch Funktionen und List Comprehensions unterstützt, die redundanten Code eliminieren. SRP gilt auch für Schleifenkörper: Jede Schleife sollte eine klare Aufgabe haben.

Neue Python-Funktionen/Methoden

In dieser Vorlesung wurden folgende neue Python-Konzepte eingeführt:

- **break** (Python Keyword, Python 1.0+): Beendet die umgebende Schleife sofort und übergibt die Kontrolle an die nächste Anweisung nach der Schleife.
- **continue** (Python Keyword, Python 1.0+): Beendet den aktuellen Schleifendurchlauf sofort und springt zur nächsten Iteration.

- **Schleifen-else-Klausel** (Python 1.0+): Ein `else`-Block nach einer Schleife wird ausgeführt, wenn die Schleife normal beendet wurde (ohne `break`).
- **List Comprehensions** (Python 2.0+, PEP 202): Kompakte Syntax zur Erstellung von Listen: `[ausdruck for element in iterable if bedingung]`.
- **Verschachtelte List Comprehensions** (Python 2.0+): List Comprehensions mit mehreren `for`-Klauseln für mehrdimensionale Iterationen.
- **Set Comprehensions** (Python 2.7+, PEP 274, Vorschau): `{ausdruck for element in iterable}` - ausführliche Behandlung in V08.
- **Dictionary Comprehensions** (Python 2.7+, PEP 274, Vorschau): `{key: value for element in iterable}` - ausführliche Behandlung in V08.

Weiterführende Ressourcen

Theorie

- **“The Pragmatic Programmer” von Andrew Hunt und David Thomas**: Klassisches Werk über Software-Qualität und DRY-Prinzip
- **“Clean Code” von Robert C. Martin**: Umfassendes Buch über Code-Qualität, KISS, SRP und SOLID-Prinzipien
- **“Code Complete” von Steve McConnell**: Detailliertes Werk über Code-Konstruktion und Best Practices
- **“Refactoring” von Martin Fowler**: Standardwerk über Code-Verbesserung und Refactoring-Patterns
- **PEP 8 - Style Guide for Python Code**: <https://www.python.org/dev/peps/pep-0008/>
- **“The Zen of Python” (PEP 20)**: <https://www.python.org/dev/peps/pep-0020/>

Python

- **Python Dokumentation - Control Flow**: <https://docs.python.org/3/tutorial/controlflow.html>
- **PEP 202 - List Comprehensions**: <https://www.python.org/dev/peps/pep-0202/>
- **PEP 274 - Dict Comprehensions**: <https://www.python.org/dev/peps/pep-0274/>
- **Real Python - Python “for” Loops**: <https://realpython.com/python-for-loop/>
- **Real Python - Python “while” Loops**: <https://realpython.com/python-while-loop/>
- **Real Python - List Comprehensions**: <https://realpython.com/list-comprehension-python/>

Datei: lessons/V08-Listen-und-Datenstrukturen-Teil1/V08-Listen-und-Datenstrukturen-Teil1_skript.md

V08: Listen und Datenstrukturen – Teil 1

[!NOTE] **Lernziele dieser Vorlesung:** - Grundlegende Datenstrukturen verstehen: Arrays, verkettete Listen, Stacks, Queues - LIFO- und FIFO-Prinzipien erkennen und anwenden können - Vor- und Nachteile verschiedener Datenstrukturen analysieren - Python-Listen erstellen, manipulieren und effizient nutzen - Unterschiede zwischen Listen und Tupeln verstehen und korrekt einsetzen - List Slicing und fortgeschrittene Indexierungstechniken beherrschen

Teil 1: Theorie - Listen und Datenstrukturen

Überblick

Datenstrukturen bilden das Rückgrat jeder Software. Die richtige Wahl der Datenstruktur entscheidet über Effizienz, Wartbarkeit und Korrektheit eines Programms. Diese Vorlesung behandelt fundamentale sequenzielle Datenstrukturen, die in nahezu jedem Softwareprojekt zum Einsatz kommen.

Die Wahl der passenden Datenstruktur hängt von mehreren Faktoren ab: Wie häufig werden Elemente hinzugefügt oder entfernt? Muss der Zugriff auf beliebige Positionen schnell sein? Ist die Reihenfolge wichtig? Werden Duplikate zugelassen? Diese Fragen führen uns zu unterschiedlichen Datenstrukturen mit jeweils spezifischen Stärken und Schwächen.

Arrays: Die Basis sequenzieller Datenstrukturen

Ein **Array** ist eine zusammenhängende Folge von Elementen gleichen Typs im Speicher. Jedes Element kann über seinen **Index** (Position) direkt angesprochen werden. Arrays sind die fundamentalste Form sequenzieller Datenspeicherung in der Informatik.

[!NOTE] Ein **Array** ist eine Datenstruktur mit fester oder variabler Größe, die Elemente in zusammenhängendem Speicher ablegt. Der Zugriff auf ein Element erfolgt über einen numerischen Index in konstanter Zeit $O(1)$.

Eigenschaften von Arrays Die zentrale Stärke von Arrays liegt im **direkten Zugriff** (auch Random Access genannt). Wenn das Array an Speicheradresse A beginnt und jedes Element s Bytes groß ist, liegt das Element an

Index i an der Adresse $A + i * s$. Diese Berechnung benötigt konstante Zeit, unabhängig von der Array-Größe.

Arrays haben eine feste Größe, die bei der Erzeugung festgelegt wird. In Sprachen wie C oder Java muss diese Größe im Voraus bekannt sein. Moderne Sprachen wie Python oder Java verwenden **dynamische Arrays**, die automatisch wachsen können, wenn mehr Platz benötigt wird. Intern wird dabei ein neues, größeres Array angelegt und alle Elemente kopiert.

[!TIP] **Visualisierung eines Arrays:**

Zeitkomplexität von Array-Operationen Die **Zeitkomplexität** beschreibt, wie die Laufzeit einer Operation mit der Größe der Datenstruktur wächst. Bei Arrays ergeben sich folgende Komplexitäten:

- **Zugriff auf Element an Index i :** $O(1)$ – konstante Zeit, da die Adresse direkt berechnet werden kann
- **Suchen eines Elements:** $O(n)$ – im schlimmsten Fall muss das gesamte Array durchlaufen werden
- **Einfügen am Ende** (bei dynamischen Arrays mit freiem Platz): $O(1)$ amortisiert
- **Einfügen am Anfang oder in der Mitte:** $O(n)$ – alle nachfolgenden Elemente müssen verschoben werden
- **Löschen am Ende:** $O(1)$ – einfach die Größe reduzieren
- **Löschen am Anfang oder in der Mitte:** $O(n)$ – alle nachfolgenden Elemente müssen nachrutschen

[!WARNING] **Häufiger Denkfehler:** Viele Anfänger glauben, dass Einfügen in ein Array immer $O(1)$ ist. Das stimmt nur für das Ende bei dynamischen Arrays mit freiem Platz. Einfügen in der Mitte erfordert das Verschieben aller nachfolgenden Elemente und ist daher $O(n)$.

Vor- und Nachteile von Arrays Vorteile: - Sehr schneller Zugriff auf beliebige Elemente durch Index - Speicher-effizient durch zusammenhängenden Speicherblock - Cache-freundlich durch lokale Speicheranordnung - Einfache Implementierung und Verwendung

Nachteile: - Feste Größe bei statischen Arrays, teure Größenanpassung bei dynamischen Arrays - Einfügen und Löschen in der Mitte ist ineffizient - Keine flexible Größenanpassung ohne Kopieren - Bei dynamischen Arrays kann ungenutzter Speicher verschwendet werden

Verkettete Listen: Flexible Alternative zu Arrays

Eine **verkettete Liste** (Linked List) speichert Elemente nicht zusammenhängend, sondern verstreut im Speicher. Jedes Element (Knoten) enthält neben

den Daten einen Verweis (Pointer) auf den nächsten Knoten. Die Liste wird durch Verfolgen dieser Verweise traversiert.

[!NOTE] Eine **verkettete Liste** ist eine Datenstruktur, bei der jedes Element (Knoten) einen Datenwert und einen Verweis auf das nächste Element enthält. Die Liste beginnt mit einem Kopf-Verweis (Head) auf das erste Element.

Einfach verkettete Liste Bei einer **einfach verketteten Liste** hat jeder Knoten genau einen Verweis: auf den Nachfolger. Das letzte Element verweist auf `None` (oder `NULL`).

[!TIP] **Visualisierung einer einfach verketteten Liste:**

Struktur eines Knotens:

Doppelt verkettete Liste Eine **doppelt verkettete Liste** erweitert die einfache Liste um Rückwärtsverweise. Jeder Knoten hat zwei Verweise: einen auf den Nachfolger und einen auf den Vorgänger. Dies ermöglicht bidirektionales Traversieren.

[!TIP] **Visualisierung einer doppelt verketteten Liste:**

Struktur eines Knotens:

Zeitkomplexität verketteter Listen

- **Zugriff auf Element an Index i :** $O(n)$ – Liste muss von Anfang bis Index durchlaufen werden
- **Suchen eines Elements:** $O(n)$ – gesamte Liste muss durchlaufen werden
- **Einfügen am Anfang:** $O(1)$ – neuer Knoten wird vor Head eingefügt
- **Einfügen am Ende:** $O(n)$ bei einfacher Liste ohne Tail-Pointer, $O(1)$ mit Tail-Pointer
- **Einfügen in der Mitte** (bei bekanntem Vorgängerknoten): $O(1)$
- **Löschen am Anfang:** $O(1)$
- **Löschen am Ende:** $O(n)$ bei einfacher Liste, $O(1)$ bei doppelt verketteter Liste mit Tail-Pointer
- **Löschen in der Mitte** (bei bekanntem Knoten): $O(1)$ bei doppelt verketteter Liste, $O(n)$ bei einfacher Liste

[!WARNING] **Wichtige Einschränkung:** Bei verketteten Listen ist der Zugriff auf beliebige Positionen ineffizient. Es gibt keinen direkten Zugriff wie bei Arrays. Die Liste muss immer vom Anfang (oder bei doppelt verketteten Listen auch vom Ende) durchlaufen werden.

Vor- und Nachteile verketteter Listen **Vorteile:** - Effizientes Einfügen und Löschen am Anfang und (bei doppelt verketteten Listen) am Ende - Dynamische Größe ohne Kopieren des gesamten Inhalts - Kein verschwendeter

Speicher durch Überdimensionierung - Einfügen/Löschen in der Mitte ohne Verschieben vieler Elemente (wenn Knoten bekannt)

Nachteile: - Kein direkter Zugriff auf Elemente (kein Random Access) - Zusätzlicher Speicher für Verweise (Pointer Overhead) - Schlechtere Cache-Lokalität durch verstreute Speicherung - Komplexere Implementierung als Arrays - Anfälliger für Programmierfehler (z.B. verlorene Verweise, Memory Leaks)

Stacks: Das LIFO-Prinzip

Ein **Stack** (Stapel, Kellerspeicher) ist eine lineare Datenstruktur, die das **LIFO-Prinzip** (Last In, First Out) umsetzt. Das zuletzt hinzugefügte Element wird als erstes wieder entfernt – wie ein Stapel Teller, bei dem nur der oberste genommen werden kann.

[!NOTE] Ein **Stack** ist eine abstrakte Datenstruktur mit zwei Hauptoperationen: - **Push**: Fügt ein Element oben auf den Stack hinzu
- **Pop**: Entfernt das oberste Element vom Stack und gibt es zurück

Zugriff ist nur auf das oberste Element (Top) möglich.

Grundoperationen eines Stacks Ein Stack bietet typischerweise folgende Operationen:

1. **Push(element)**: Legt ein neues Element auf den Stack ($O(1)$)
2. **Pop()**: Entfernt das oberste Element und gibt es zurück ($O(1)$)
3. **Peek()** / **Top()**: Gibt das oberste Element zurück, ohne es zu entfernen ($O(1)$)
4. **IsEmpty()**: Prüft, ob der Stack leer ist ($O(1)$)
5. **Size()**: Gibt die Anzahl der Elemente zurück ($O(1)$ bei Zähler-Variable)

[!TIP] **Visualisierung eines Stacks:**

Anwendungsfälle von Stacks Stacks sind in der Informatik allgegenwärtig:

Funktionsaufrufe (Call Stack): Jedes laufende Programm verwendet einen Stack, um Funktionsaufrufe zu verwalten. Beim Aufruf einer Funktion wird ein neuer **Stack Frame** mit lokalen Variablen und Rücksprungadresse auf den Call Stack gelegt. Bei Rückkehr wird dieser Frame vom Stack entfernt.

Rückgängig-Funktion (Undo): Textverarbeitungen und Bildbearbeitungsprogramme speichern Änderungen auf einem Stack. Bei “Rückgängig” wird die letzte Änderung vom Stack genommen und umgekehrt.

Auswertung arithmetischer Ausdrücke: Die **Reverse Polish Notation** (RPN) nutzt einen Stack zur Auswertung von Ausdrücken ohne Klammern. Der **Shunting-Yard-Algorithmus** konvertiert Infix-Notation (z.B. $3 + 4$) in RPN mithilfe eines Stacks.

Tiefensuche (DFS) in Graphen: Der Algorithmus verwendet einen Stack, um den aktuellen Pfad zu verfolgen und bei Sackgassen zurückzukehren.

Parser und Compiler: Die **Syntaxanalyse** von Programmiersprachen nutzt Stacks, um verschachtelte Strukturen (Klammern, Blöcke) zu verarbeiten.

Browser-Historie: Die Zurück-Funktion eines Browsers kann als Stack implementiert werden (obwohl moderne Browser komplexere Strukturen verwenden).

[!WARNING] **Stack Overflow:** Wird ein Stack zu groß (z.B. bei zu tiefer Rekursion), kann der verfügbare Speicher überschritten werden. Dies führt zum berüchtigten **Stack Overflow Error**, der das Programm zum Absturz bringt.

Queues: Das FIFO-Prinzip

Eine **Queue** (Warteschlange) ist eine lineare Datenstruktur, die das **FIFO-Prinzip** (First In, First Out) umsetzt. Das zuerst hinzugefügte Element wird als erstes wieder entfernt – wie eine Warteschlange an der Kasse.

[!NOTE] Eine **Queue** ist eine abstrakte Datenstruktur mit zwei Hauptoperationen: - **Enqueue:** Fügt ein Element am Ende der Queue hinzu - **Dequeue:** Entfernt das vorderste Element der Queue und gibt es zurück

Zugriff ist nur auf das vorderste Element (Front) und das hinterste Element (Rear) möglich.

Grundoperationen einer Queue Eine Queue bietet typischerweise folgende Operationen:

1. **Enqueue(element):** Fügt ein Element am Ende der Queue hinzu ($O(1)$)
2. **Dequeue():** Entfernt das vorderste Element und gibt es zurück ($O(1)$)
3. **Front() / Peek():** Gibt das vorderste Element zurück, ohne es zu entfernen ($O(1)$)
4. **IsEmpty():** Prüft, ob die Queue leer ist ($O(1)$)
5. **Size():** Gibt die Anzahl der Elemente zurück ($O(1)$ bei Zähler-Variable)

[!TIP] **Visualisierung einer Queue:**

Implementierungsvarianten von Queues **Array-basierte Queue:** Verwendet ein Array mit zwei Zeigern (Front und Rear). Problem: Beim Dequeue entsteht am Anfang ungenutzter Platz. Lösung: **Zirkuläres Array** (Ringpuffer), bei dem die Zeiger modulo Array-Länge inkrementiert werden.

Verkettete Liste als Queue: Nutzt eine einfach verkettete Liste mit Head (Front) und Tail (Rear). Enqueue fügt am Tail hinzu, Dequeue entfernt am Head. Beide Operationen sind $O(1)$.

Double-Ended Queue (Deque): Verallgemeinerung, die Einfügen und Entfernen an beiden Enden erlaubt. Kombiniert Stack- und Queue-Funktionalität.

Anwendungsfälle von Queues Queues sind essentiell für viele Algorithmen und Systeme:

Betriebssysteme: Die **Prozess-Scheduler** verwenden Queues, um Ready-Prozesse zu verwalten. Der Scheduler entnimmt den ersten Prozess der Queue, führt ihn aus und reiht ihn ggf. wieder ein (Round-Robin-Scheduling).

Netzwerk-Kommunikation: Datenpakete werden in Queues gepuffert, wenn die Netzwerkbandbreite nicht ausreicht, um alle sofort zu senden.

Druckerwarteschlangen: Druckaufträge werden in einer Queue verwaltet und in der Reihenfolge des Eingangs abgearbeitet.

Breitensuche (BFS) in Graphen: Der Algorithmus verwendet eine Queue, um Knoten Ebene für Ebene zu durchlaufen.

Asynchrone Datenverarbeitung: Message Queues (z.B. RabbitMQ, Apache Kafka) entkoppeln Sender und Empfänger in verteilten Systemen.

Simulation: Warteschlangen-Modelle simulieren Systeme mit begrenzter Kapazität (z.B. Supermarkt-Kassen, Call-Center).

[!TIP] **Mermaid-Diagramm: Vergleich Stack vs. Queue**

Vergleich der Datenstrukturen

Die folgende Tabelle fasst die wichtigsten Eigenschaften zusammen:

Datenstruktur	Zugriff	Suche	Einfügen Anfang	Einfügen Ende	Löschen Anfang	Löschen Ende	Hauptvorteil
Array	$O(1)$	$O(n)$	$O(n)$	$O(1)^*$	$O(n)$	$O(1)$	Schneller direkter Zugriff
Verkettete Liste	$O(n)$	$O(n)$	$O(1)$	$O(1)^{**}$	$O(1)$	$O(1)^{***}$	Effizientes Einfügen/Löschen
Stack	$O(1)^{****}$	$O(n)$	$O(1)$	–	$O(1)$	–	LIFO-Semantik
Queue	$O(1)^{*****}$	$O(n)$	–	$O(1)$	$O(1)$	–	FIFO-Semantik

Bei dynamischen Arrays amortisiert

Bei einfacher Liste mit Tail-Pointer

Bei doppelt verketteter Liste mit Tail-Pointer

****Nur auf Top-Element
*****Nur auf Front-Element

Wann welche Datenstruktur verwenden?

Die Wahl der richtigen Datenstruktur ist eine zentrale Design-Entscheidung:

Verwende Arrays/Listen, wenn: - Häufiger Zugriff auf beliebige Positionen nötig ist - Die Reihenfolge der Elemente wichtig ist - Iterieren über alle Elemente die Hauptoperation ist - Die Größe weitgehend konstant ist oder Größenänderungen selten sind

Verwende verkettete Listen, wenn: - Häufiges Einfügen und Löschen am Anfang oder in der Mitte erfolgt - Die Größe stark variiert - Kein direkter Zugriff auf Positionen benötigt wird - Speicher-Fragmentierung vermieden werden soll

Verwende Stacks, wenn: - LIFO-Semantik benötigt wird (letztes Element zuerst) - Rückgängig-Funktionalität implementiert werden soll - Rekursion iterativ simuliert werden soll - Auswertung von Ausdrücken oder Parsing erfolgt

Verwende Queues, wenn: - FIFO-Semantik benötigt wird (erstes Element zuerst) - Aufgaben in der Reihenfolge ihrer Ankunft bearbeitet werden sollen - Pufferung von Datenströmen erforderlich ist - Breitensuche oder Level-Order-Traversierung durchgeführt wird

Zusammenfassung Theorie

Die fundamentalen Datenstrukturen Arrays, verkettete Listen, Stacks und Queues bilden die Basis für komplexere Strukturen. Arrays bieten schnellen Zugriff über Indizes, während verkettete Listen flexibles Einfügen und Löschen ermöglichen. Stacks implementieren das LIFO-Prinzip für Situationen, in denen das zuletzt hinzugefügte Element zuerst verarbeitet wird. Queues folgen dem FIFO-Prinzip und eignen sich für Warteschlangen und Pufferung.

Die Zeitkomplexität verschiedener Operationen unterscheidet sich erheblich zwischen den Strukturen. Arrays ermöglichen $O(1)$ -Zugriff, aber $O(n)$ -Einfügen in der Mitte. Verkettete Listen ermöglichen $O(1)$ -Einfügen am Anfang, aber $O(n)$ -Zugriff auf beliebige Positionen. Die Wahl der richtigen Datenstruktur basiert auf den häufigsten Operationen im konkreten Anwendungsfall.

Das Verständnis dieser grundlegenden Strukturen ist essentiell für die Entwicklung effizienter Software. In der nächsten Vorlesung werden wir komplexere Strukturen wie Bäume und Hash-Tabellen behandeln, die auf diesen Grundlagen aufbauen.

Teil 2: Python-Praxis - Listen & Datenstrukturen

[!WARNING] **Python-Konsistenz beachten:** Prüfe .././python

__topics.md für bereits eingeführte Konzepte!

Überblick

Python bietet mit **Listen** eine vielseitige und mächtige eingebaute Datenstruktur, die Arrays, Stacks und Queues kombiniert. Listen sind dynamisch, können Elemente unterschiedlichen Typs enthalten und bieten eine Fülle von Methoden zur Manipulation. In diesem Praxis-Teil lernen wir Listen von Grund auf kennen und nutzen sie für verschiedene Anwendungsfälle.

Zusätzlich behandeln wir **Tupel** als unveränderliche Alternative zu Listen und erkunden fortgeschrittene Techniken wie Slicing, Unpacking und Aliasing. Das Verständnis dieser Konzepte ist fundamental für professionelle Python-Entwicklung.

Listen in Python: Grundlagen

Eine **Liste** in Python ist eine geordnete, veränderbare Sammlung von Elementen beliebigen Typs. Listen werden mit eckigen Klammern `[]` erstellt und können wachsen und schrumpfen.

Listen erstellen

[!NOTE] **list-Datentyp (Built-in):** - Veränderbare (mutable) Sequenz beliebiger Python-Objekte - Erstellen mit `[]` oder `list(iterable)` - Syntax: `[element1, element2, ...]` - Unterstützt gemischte Datentypen - Signatur: `list(iterable=None) → list`

[!TIP]

Zugriff auf Listen-Elemente Listen unterstützen Indizierung wie Strings. Indizes beginnen bei 0 und negative Indizes zählen vom Ende.

[!TIP]

[!WARNING] **IndexError vermeiden:** Prüfe vor Zugriff, ob der Index gültig ist:

Oder nutze **try-except** für Fehlerbehandlung (wird in V09 behandelt).

Listen-Methoden: Elemente hinzufügen

Python-Listen bieten mehrere Methoden zum Hinzufügen von Elementen.

.append() – Element am Ende hinzufügen

[!NOTE] **list.append(element)** (List-Methode): - Fügt ein einzelnes Element am Ende der Liste hinzu - Modifiziert die Liste **in-place** (kein Rückgabewert, gibt **None** zurück) - Zeitkomplexität: $O(1)$ amortisiert - Signatur: **list.append(x)** → **None**

[!TIP]

[!WARNING] **Häufiger Fehler:** **append()** gibt **None** zurück, nicht die modifizierte Liste!

.insert() – Element an bestimmter Position einfügen

[!NOTE] **list.insert(index, element)** (List-Methode): - Fügt ein Element an Position **index** ein - Alle nachfolgenden Elemente rutschen nach rechts - Zeitkomplexität: $O(n)$ – alle nachfolgenden Elemente müssen verschoben werden - Signatur: **list.insert(i, x)** → **None**

[!TIP]

.extend() – Mehrere Elemente hinzufügen

[!NOTE] **list.extend(iterable)** (List-Methode): - Fügt alle Elemente aus einem Iterable am Ende hinzu - Entspricht mehrfachem **append()**, aber effizienter - Zeitkomplexität: $O(k)$, wobei k die Anzahl hinzugefügter Elemente ist - Signatur: **list.extend(iterable)** → **None**

[!TIP]

Listen verketteten mit +-Operator

[!NOTE] **+-Operator für Listen:** - Erzeugt eine **neue** Liste durch Konkatenation - Ursprüngliche Listen bleiben unverändert - Zeitkomplexität: $O(n + m)$, wobei n und m die Längen sind - Syntax: **liste1 + liste2** → neue Liste

[!TIP]

[!WARNING] **Performance-Unterschied:** **extend()** modifiziert die Liste in-place (effizienter), während **+** eine neue Liste erstellt (benötigt mehr Speicher und Zeit):

Listen-Methoden: Elemente entfernen

.remove() – Erstes Vorkommen entfernen

[!NOTE] **list.remove(value)** (List-Methode): - Entfernt das **erste** Vorkommen des Werts aus der Liste - Wirft **ValueError**, wenn der Wert nicht gefunden wird - Zeitkomplexität: $O(n)$ – Liste muss durchsucht werden - Signatur: **list.remove(x)** → **None**

[!TIP]

[!WARNING] **ValueError abfangen:** Prüfe vorher, ob das Element existiert:

.pop() – Element an Index entfernen

[!NOTE] **list.pop(index=-1)** (List-Methode): - Entfernt das Element an **index** und gibt es zurück - Standard: **index=-1** (letztes Element) - Wirft **IndexError**, wenn Index ungültig ist - Zeitkomplexität: $O(n)$ für beliebigen Index, $O(1)$ für letztes Element - Signatur: **list.pop(i=-1) → Element**

[!TIP]

.clear() – Alle Elemente entfernen

[!NOTE] **list.clear()** (List-Methode): - Entfernt alle Elemente aus der Liste - Liste wird leer, aber das Listenobjekt bleibt bestehen - Zeitkomplexität: $O(n)$ - Signatur: **list.clear() → None**

[!TIP]

del-Statement – Elemente oder Slices löschen

[!NOTE] **del-Statement:** - Löscht Elemente an bestimmten Indizes oder Slices - Kann auch die gesamte Variable löschen - Zeitkomplexität: $O(n)$ für Element in der Mitte, $O(1)$ für letztes Element - Syntax: **del liste[index]** oder **del liste[start:stop]**

[!TIP]

Listen durchsuchen und abfragen

.index() – Position eines Elements finden

[!NOTE] **list.index(value, start=0, stop=len)** (List-Methode): - Gibt den Index des **ersten** Vorkommens von **value** zurück - Wirft **ValueError**, wenn der Wert nicht gefunden wird - Optional: Suche in Teilbereich **[start:stop]** - Zeitkomplexität: $O(n)$ - Signatur: **list.index(x, start=0, stop=len) → int**

[!TIP]

.count() – Häufigkeit eines Elements zählen

[!NOTE] **list.count(value)** (List-Methode): - Zählt, wie oft **value** in der Liste vorkommt - Gibt 0 zurück, wenn Element nicht vorhanden - Zeitkomplexität: $O(n)$ - Signatur: **list.count(x) → int**

[!TIP]

in-Operator – Prüfen, ob Element vorhanden ist

[!NOTE] **in-Operator für Listen:** - Prüft, ob ein Wert in der Liste enthalten ist - Gibt `True` oder `False` zurück - Zeitkomplexität: $O(n)$
- Syntax: `value in liste → bool`

[!TIP]

Listen sortieren und umkehren

.sort() – Liste in-place sortieren

[!NOTE] **list.sort(key=None, reverse=False)** (List-Methode):
- Sortiert die Liste **in-place** (modifiziert Original) - **key**: Optionale Funktion zur Berechnung des Sortierschlüssels - **reverse=True**: Absteigende Sortierung - Zeitkomplexität: $O(n \log n)$ (Timsort-Algorithmus) - Signatur: `list.sort(key=None, reverse=False) → None`

[!TIP]

sorted() – Neue sortierte Liste erstellen

[!NOTE] **sorted(iterable, key=None, reverse=False)** (Built-in Funktion): - Erstellt eine **neue** sortierte Liste - Original bleibt unverändert - Funktioniert mit jedem Iterable (Liste, String, Tuple, etc.) - Zeitkomplexität: $O(n \log n)$ - Signatur: `sorted(iterable, key=None, reverse=False) → list`

[!TIP]

[!WARNING] **Unterschied sort() vs. sorted():** - `.sort()` modifiziert die Liste direkt und gibt `None` zurück - `sorted()` gibt eine neue Liste zurück und lässt das Original unverändert

.reverse() – Liste umkehren

[!NOTE] **list.reverse()** (List-Methode): - Kehrt die Reihenfolge der Elemente **in-place** um - Gibt `None` zurück - Zeitkomplexität: $O(n)$ - Signatur: `list.reverse() → None`

[!TIP]

List Slicing: Teilbereiche extrahieren

Slicing erlaubt den Zugriff auf Teilbereiche einer Liste. Die Syntax ist `liste[start:stop:step]`.

[!NOTE] **List Slicing**: - Extrahiert einen Teilbereich: `liste[start:stop:step]`
- **start**: Startindex (inklusive, Standard: 0) - **stop**: Endindex (exklusive, Standard: `len(liste)`) - **step**: Schrittweite (Standard: 1) - Erzeugt eine **neue** Liste (Shallow Copy) - Signatur: `liste[start:stop:step] → neue list`

[!TIP]

[!WARNING] **Slicing erstellt Shallow Copy**: Bei Listen mit veränderbaren Objekten werden nur die Referenzen kopiert:

Für Deep Copy: `import copy; kopie = copy.deepcopy(original)`

Slicing zum Modifizieren von Listen Slicing kann auch auf der linken Seite einer Zuweisung verwendet werden, um Teilbereiche zu ersetzen.

[!TIP]

Tupel: Unveränderliche Listen

Tupel sind wie Listen, aber **immutable** (unveränderlich). Nach der Erzeugung können keine Elemente hinzugefügt, entfernt oder geändert werden.

[!NOTE] **tuple-Datentyp (Built-in)**: - Unveränderbare (immutable) Sequenz beliebiger Python-Objekte - Erstellen mit `()` oder einfach durch Komma-Trennung - Syntax: `(element1, element2, ...)` oder `element1, element2, ...` - Unterstützt Indizierung und Slicing (schreibgeschützt) - Signatur: `tuple(iterable=None) → tuple`

[!TIP]

[!WARNING] **Tupel sind unveränderlich**:

Um ein Tupel zu “ändern”, muss ein neues erstellt werden:

Wann Tupel statt Listen verwenden? Verwende Tupel, wenn: - Daten unveränderlich sein sollen (z.B. Koordinaten, Konfigurationswerte) - Als Dictionary-Keys (Listen sind nicht hashable, Tupel schon) - Für Funktionsrückgaben mehrerer Werte - Speicher-Effizienz wichtig ist (Tupel sind leichter als Listen) - Schutz vor versehentlicher Modifikation gewünscht ist

[!TIP]

List Unpacking und Tuple Unpacking

Unpacking erlaubt das Entpacken von Sequenzen in mehrere Variablen.

[!NOTE] **Sequence Unpacking**: - Weist Elemente einer Sequenz mehreren Variablen zu - Anzahl der Variablen muss mit Anzahl der

Elemente übereinstimmen (oder `*` verwenden) - Funktioniert mit Listen, Tupeln, Strings, etc. - Syntax: `var1, var2, var3 = sequenz`

[!TIP]

[!WARNING] **ValueError** bei falscher Anzahl:

Listen-Aliasing und Kopieren

Wenn eine Liste einer anderen Variable zugewiesen wird, wird nur eine **Referenz** kopiert, nicht die Liste selbst.

[!NOTE] **Aliasing** (Referenz-Zuweisung): - Zuweisung `liste2 = liste1` erstellt **keine** Kopie - Beide Variablen zeigen auf dasselbe Listenobjekt - Änderungen über eine Variable beeinflussen die andere

[!TIP]

Listen kopieren Um eine echte Kopie zu erstellen, gibt es mehrere Möglichkeiten:

[!TIP]

[!WARNING] **Shallow Copy vs. Deep Copy**: Die oben genannten Methoden erstellen nur **Shallow Copies**. Bei Listen mit veränderbaren Objekten werden nur die Referenzen kopiert:

Für vollständige Kopien (Deep Copy):

List Comprehensions (Wiederholung und Vertiefung)

List Comprehensions wurden in V07 eingeführt, hier vertiefen wir sie im Kontext von Listen-Operationen.

[!TIP]

Weitere nützliche Built-in Funktionen für Listen

len() – **Länge der Liste** Bereits in V01 eingeführt, hier im Kontext von Listen:

[!TIP]

sum() – **Summe aller Elemente**

[!NOTE] **sum(iterable, start=0)** (Built-in): - Berechnet die Summe aller Elemente - **start**: Startwert (wird zur Summe addiert) - Nur für numerische Werte - Signatur: `sum(iterable, start=0)` → **number**

[!TIP]

min() und **max()** – **Minimum und Maximum** Bereits in V03 eingeführt, hier mit Listen:

[!TIP]

all() und **any()** – Logische Prüfungen

[!NOTE] **all(iterable)** (Built-in): - Gibt **True** zurück, wenn **alle** Elemente truthy sind - Leere Iterables geben **True** zurück (Vakuum-Wahrheit) - Signatur: **all(iterable)** → **bool**

[!NOTE] **any(iterable)** (Built-in): - Gibt **True** zurück, wenn **mindestens ein** Element truthy ist - Leere Iterables geben **False** zurück - Signatur: **any(iterable)** → **bool** - Bereits in V04 als Vorschau erwähnt, hier vollständige Einführung

[!TIP]

zip() – Mehrere Iterables parallel durchlaufen

[!NOTE] **zip(*iterables)** (Built-in): - Kombiniert mehrere Iterables zu Tupeln - Gibt Iterator zurück, der Tupel aus parallelen Elementen erzeugt - Stoppt bei kürzestem Iterable - Signatur: **zip(*iterables)** → **zip object** (Iterator)

[!TIP]

Stack-Operationen mit Listen

Python-Listen können direkt als Stack verwendet werden.

[!TIP]

Queue-Operationen mit Listen (ineffizient!)

Listen können als Queue verwendet werden, aber **nicht effizient**.

[!WARNING] **Performance-Problem:** `.pop(0)` ist $O(n)$, da alle Elemente nach links verschoben werden müssen. Für effiziente Queues verwende `collections.deque` (wird in späteren Vorlesungen behandelt):

[!TIP]

Häufige Fehler und Lösungen

[!WARNING] **Fehler 1: In-place Methoden geben None zurück**

Viele List-Methoden (`.append()`, `.sort()`, `.reverse()`, `.clear()`) modifizieren die Liste direkt und geben `None` zurück.

Falsch:

Richtig:

[!WARNING] **Fehler 2: Liste während Iteration modifizieren**

Niemals eine Liste modifizieren, während man über sie iteriert!

Falsch:

Richtig (Methode 1: Über Kopie iterieren):

Richtig (Methode 2: List Comprehension):

[!WARNING] **Fehler 3: Aliasing statt Kopieren**

Zuweisung erstellt keine Kopie, sondern nur eine Referenz.

Falsch:

Richtig:

[!WARNING] **Fehler 4: Index außerhalb des Bereichs**

Lösung: Prüfe Länge oder verwende `.get()` bei Dictionaries (V08, Teil 2):

[!WARNING] **Fehler 5: Verwechslung `append()` vs. `extend()`**

Zusammenfassung Python

Python-Listen sind mächtige, dynamische Datenstrukturen, die Arrays, Stacks und (ineffizient) Queues kombinieren. Listen unterstützen Indizierung, Slicing und zahlreiche Methoden zum Hinzufügen, Entfernen, Sortieren und Durchsuchen von Elementen. Wichtige Methoden sind `.append()` ($O(1)$), `.insert()` ($O(n)$), `.remove()` ($O(n)$), `.pop()` ($O(1)$ am Ende, $O(n)$ am Anfang), `.sort()` ($O(n \log n)$) und `.reverse()` ($O(n)$).

Tupel sind unveränderliche Alternativen zu Listen und eignen sich für Daten, die nicht modifiziert werden sollen, sowie als Dictionary-Keys. List Slicing ermöglicht eleganten Zugriff auf Teilbereiche und das Erstellen von Kopien. List Comprehensions bieten kompakte Syntax für Transformationen und Filter.

Wichtige Konzepte sind Aliasing (Referenzen statt Kopien), Shallow vs. Deep Copy und die Unterscheidung zwischen in-place Methoden (geben `None` zurück) und Funktionen, die neue Objekte erstellen. Built-in Funktionen wie `sum()`, `all()`, `any()` und `zip()` erweitern die Möglichkeiten der Listen-Verarbeitung erheblich.

Neue Python-Funktionen/Methoden

In dieser Vorlesung wurden folgende Python-APIs **neu** eingeführt:

Datentypen

- **list**: Veränderbare Sequenz, erstellt mit `[]` oder `list(iterable)`
- **tuple**: Unveränderbare Sequenz, erstellt mit `()` oder `tuple(iterable)`

List-Methoden

- **list.append(x)**: Element am Ende hinzufügen ($O(1)$)
- **list.insert(i, x)**: Element an Index *i* einfügen ($O(n)$)
- **list.extend(iterable)**: Mehrere Elemente hinzufügen ($O(k)$)
- **list.remove(x)**: Erstes Vorkommen von *x* entfernen ($O(n)$)
- **list.pop(i=-1)**: Element an Index entfernen und zurückgeben ($O(n)$, $O(1)$ am Ende)
- **list.clear()**: Alle Elemente entfernen ($O(n)$)
- **list.index(x, start=0, stop=len)**: Index des ersten Vorkommens ($O(n)$)
- **list.count(x)**: Anzahl der Vorkommen von *x* ($O(n)$)
- **list.sort(key=None, reverse=False)**: In-place sortieren ($O(n \log n)$)
- **list.reverse()**: Reihenfolge umkehren ($O(n)$)
- **list.copy()**: Shallow Copy erstellen ($O(n)$)

Built-in Funktionen

- **sorted(iterable, key=None, reverse=False)**: Neue sortierte Liste erstellen
- **sum(iterable, start=0)**: Summe aller Elemente
- **all(iterable)**: True, wenn alle Elemente *truthy*
- **any(iterable)**: True, wenn mindestens ein Element *truthy* (vollständige Einführung, in V04 als Vorschau)
- **zip(*iterables)**: Mehrere Iterables zu Tupeln kombinieren

Operatoren

- **+** für Listen: Verkettung (neue Liste)
- ***** für Listen: Wiederholung (z.B. `[1, 2] * 3` → `[1, 2, 1, 2, 1, 2]`)
- **in** für Listen: Mitgliedschaftstest ($O(n)$)
- **del liste[index]**: Element(e) löschen

Konzepte

- **List Slicing**: `liste[start:stop:step]`
- **Sequence Unpacking**: `a, b, c = [1, 2, 3]` oder `a, *rest, b = liste`
- **Aliasing vs. Kopieren**: Referenzen vs. echte Kopien
- **Shallow Copy vs. Deep Copy**: `copy.deepcopy()` für verschachtelte Strukturen

Weiterführende Ressourcen

Theorie

- **“Introduction to Algorithms” (CLRS):** Kapitel über elementare Datenstrukturen
- **Wikipedia:** Artikel zu Arrays, Linked Lists, Stacks, Queues
- **Visualgo:** Interaktive Visualisierung von Datenstrukturen (<https://visualgo.net/>)
- **Big-O Cheat Sheet:** Komplexitäten verschiedener Datenstrukturen (<https://www.bigocheatsheet.com/>)

Python

- **Python Documentation:**
 - Built-in Types - List (<https://docs.python.org/3/library/stdtypes.html#list>)
 - Built-in Types - Tuple (<https://docs.python.org/3/library/stdtypes.html#tuple>)
 - **Real Python:** “Python Lists and Tuples” Tutorial
 - **Python Tutor:** Code-Visualisierung (<https://pythontutor.com/>) – hervorragend für Aliasing/Referenzen
 - **Official Python Tutorial:** Data Structures (<https://docs.python.org/3/tutorial/datastructures.html>)
-

Datei: lessons/V09-Listen-und-Datenstrukturen-Teil2/V09-Listen-und-Datenstrukturen-Teil2_skript.md

V09: Listen und Datenstrukturen – Teil 2

[!NOTE] **Lernziele dieser Vorlesung:** - Binärbäume und Binäre Suchbäume verstehen und anwenden können - Verschiedene Traversierungsalgorithmen (Inorder, Preorder, Postorder) kennenlernen - Hash-Tabellen und deren Funktionsweise nachvollziehen - Kollisionsbehandlungsstrategien (Chaining, Open Addressing) verstehen - Exceptions in Python gezielt behandeln und eigene werfen können - `try-except-else-finally` korrekt einsetzen - Best Practices für robuste Fehlerbehandlung anwenden

Teil 1: Theorie - Listen und Datenstrukturen – Teil 2

Überblick

In der vorherigen Vorlesung haben wir grundlegende Datenstrukturen wie Arrays, verkettete Listen, Stacks und Queues kennengelernt. Diese Vorlesung erweitert das Repertoire um hierarchische und hash-basierte Datenstrukturen. Bäume ermöglichen effiziente Such-, Einfüge- und Löschoperationen in sortierten Daten, während Hash-Tabellen nahezu konstanten Zugriff auf

Elemente über Schlüssel bieten. Das Verständnis dieser Strukturen ist fundamental für die Entwicklung effizienter Algorithmen und bildet die Grundlage moderner Datenbanksysteme, Compiler und Netzwerk-Routing-Protokolle.

Bäume: Grundkonzepte und Terminologie

Ein **Baum** ist eine hierarchische Datenstruktur, die aus **Knoten** (Nodes) besteht, die durch **Kanten** (Edges) verbunden sind. Bäume finden Anwendung in Dateisystemen, Entscheidungsbäumen in der KI, Syntaxbäumen in Compilern und vielen weiteren Bereichen.

[!NOTE] **Baum (Tree)**: Eine hierarchische Datenstruktur mit einem **Wurzelknoten** (Root), von dem aus alle anderen Knoten erreichbar sind. Ein Baum enthält keine Zyklen – es gibt genau einen Pfad zwischen zwei beliebigen Knoten.

Fundamentale Begriffe Jeder Baum besteht aus mehreren Komponenten, die durch präzise Terminologie beschrieben werden:

Knoten (Node): Ein **Knoten** ist ein einzelnes Element im Baum, das einen Wert (Datum) speichert und Referenzen zu seinen Kind-Knoten enthält. Der oberste Knoten heißt **Wurzel** (Root). Knoten ohne Kinder heißen **Blätter** (Leaves).

Kante (Edge): Eine **Kante** ist die Verbindung zwischen einem Elternknoten und einem seiner Kindknoten. Die Anzahl der Kanten im Baum ist immer **Anzahl Knoten - 1**.

Pfad (Path): Ein **Pfad** ist eine Folge von Knoten, bei der jeder Knoten über eine Kante mit dem nächsten verbunden ist. Die **Pfadlänge** ist die Anzahl der Kanten im Pfad.

Tiefe (Depth): Die **Tiefe** eines Knotens ist die Länge des Pfads von der Wurzel zu diesem Knoten. Die Wurzel hat Tiefe 0.

Höhe (Height): Die **Höhe** eines Baums ist die maximale Tiefe aller Blätter. Ein Baum mit nur einem Knoten (der Wurzel) hat Höhe 0.

Grad (Degree): Der **Grad** eines Knotens ist die Anzahl seiner Kindknoten. Der **Grad eines Baums** ist der maximale Grad aller Knoten.

[!TIP] **Visualisierung eines Beispielbaums**:

In diesem Baum: - **Wurzel**: Knoten 10 (Tiefe 0) - **Blätter**: Knoten 3, 7, 12, 20 (rot markiert) - **Höhe**: 2 (längster Pfad von Wurzel zu einem Blatt) - **Grad des Baums**: 2 (jeder Knoten hat maximal 2 Kinder)

Binärbäume

Ein **Binärbaum** ist ein Baum, bei dem jeder Knoten maximal zwei Kinder hat. Diese werden als **linkes Kind** und **rechtes Kind** bezeichnet. Binärbäume sind die Grundlage für viele spezialisierte Varianten wie Binäre Suchbäume, AVL-Bäume und Heaps.

[!NOTE] **Binärbaum (Binary Tree)**: Ein Baum, bei dem jeder Knoten maximal zwei Kindknoten hat. Die Position der Kinder ist relevant: Es wird zwischen linkem und rechtem Kind unterschieden.

Eigenschaften von Binärbäumen Binärbäume können verschiedene strukturelle Eigenschaften aufweisen, die ihre Effizienz und Anwendbarkeit beeinflussen:

Vollständiger Binärbaum (Full Binary Tree): Jeder Knoten hat entweder 0 oder 2 Kinder (keine Knoten mit nur einem Kind).

Perfekter Binärbaum (Perfect Binary Tree): Alle inneren Knoten haben genau 2 Kinder, und alle Blätter befinden sich auf derselben Ebene. Dies ist die kompakteste Form und hat genau $2^{(h+1)} - 1$ Knoten bei Höhe h .

Kompletter Binärbaum (Complete Binary Tree): Alle Ebenen sind vollständig gefüllt, außer möglicherweise die letzte, die von links nach rechts gefüllt ist. Diese Eigenschaft wird in Heaps ausgenutzt, um effiziente Array-Repräsentationen zu ermöglichen.

Degenerierter Baum (Degenerate Tree): Jeder Knoten hat nur ein Kind, wodurch der Baum zu einer linearen Liste degeneriert. Dies ist der ungünstigste Fall für Such-Operationen.

[!TIP] **Vergleich verschiedener Binärbaum-Formen:**

Perfekter Baum: Alle Ebenen vollständig gefüllt, maximale Balance.

Degenerierter Baum: Entspricht einer verketteten Liste, keine Vorteile gegenüber linearen Strukturen.

Binäre Suchbäume (BST)

Ein **Binärer Suchbaum** (Binary Search Tree, BST) ist ein Binärbaum mit einer entscheidenden Ordnungseigenschaft: Für jeden Knoten gilt, dass alle Werte im linken Teilbaum kleiner und alle Werte im rechten Teilbaum größer sind als der Wert des Knotens selbst. Diese Eigenschaft ermöglicht effiziente Such-, Einfüge- und Löschoperationen.

[!NOTE] **Binärer Suchbaum (BST)**: Ein Binärbaum, bei dem für jeden Knoten gilt: - Alle Werte im **linken Teilbaum** sind **kleiner** als der Wert des Knotens - Alle Werte im **rechten Teilbaum** sind

größer als der Wert des Knotens - Diese Eigenschaft gilt rekursiv für alle Teilbäume

Operationen auf Binären Suchbäumen Die Operationen auf BSTs nutzen die Ordnungseigenschaft, um effizient zu arbeiten:

Suchen (Search): Um einen Wert zu finden, startet man an der Wurzel und vergleicht den gesuchten Wert mit dem aktuellen Knoten. Ist der gesuchte Wert kleiner, wird im linken Teilbaum weitergesucht, andernfalls im rechten. Dieser Prozess wiederholt sich, bis der Wert gefunden wird oder ein Blatt erreicht ist.

Zeitkomplexität: - **Best/Average Case:** $O(\log n)$ bei balanciertem Baum - **Worst Case:** $O(n)$ bei degeneriertem Baum (ähnlich linearer Liste)

Einfügen (Insert): Ein neuer Wert wird eingefügt, indem man zunächst die Position sucht, an der er stehen müsste (wie beim Suchen). Sobald ein Blatt erreicht ist, wird der neue Knoten als Kind hinzugefügt.

Zeitkomplexität: Analog zur Suche – $O(\log n)$ im Average Case, $O(n)$ im Worst Case.

Löschen (Delete): Das Löschen ist die komplexeste Operation und umfasst drei Fälle: 1. **Knoten ist Blatt:** Einfach entfernen 2. **Knoten hat ein Kind:** Knoten durch sein Kind ersetzen 3. **Knoten hat zwei Kinder:** Finde den **Inorder-Nachfolger** (kleinster Wert im rechten Teilbaum) oder **Inorder-Vorgänger** (größter Wert im linken Teilbaum), kopiere dessen Wert in den zu löschenden Knoten und lösche den Nachfolger/Vorgänger rekursiv

Zeitkomplexität: $O(\log n)$ im Average Case, $O(n)$ im Worst Case.

[!TIP] **Beispiel eines Binären Suchbaums:**

Eigenschaften: - Linker Teilbaum von 50: {20, 30, 40} – alle < 50
- Rechter Teilbaum von 50: {60, 70, 80} – alle > 50 - **Suche nach 40:** $50 \rightarrow 30 \rightarrow 40$ (3 Vergleiche) - **Suche nach 80:** $50 \rightarrow 70 \rightarrow 80$ (3 Vergleiche)

[!WARNING] **Balance ist entscheidend:** Ein unbalancierter BST (degeneriert) verliert seine Effizienzvorteile. In der Praxis werden daher **selbstbalancierende Bäume** wie AVL-Bäume oder Rot-Schwarz-Bäume verwendet, die automatisch die Balance nach Einfüge- und Löschoptionen wiederherstellen.

Traversierung von Bäumen

Die **Traversierung** eines Baums bezeichnet den systematischen Besuch aller Knoten in einer definierten Reihenfolge. Es gibt drei fundamentale Traversierungsstrategien für Binärbäume, die sich darin unterscheiden, wann der Wurzelknoten relativ zu seinen Kindern besucht wird.

[!NOTE] **Traversierung (Tree Traversal):** Der Prozess, alle Knoten eines Baums genau einmal in einer bestimmten Reihenfolge zu besuchen. Die Wahl der Traversierungsart hängt vom Anwendungsfall ab (z.B. sortierte Ausgabe, Auswertung von Ausdrücken, Strukturanalyse).

Inorder-Traversierung (LWR) Bei der **Inorder-Traversierung** wird zunächst der linke Teilbaum besucht, dann die Wurzel, dann der rechte Teilbaum. Die Reihenfolge ist: **Links** → **Wurzel** → **Rechts** (LWR).

Algorithmus: 1. Traversiere rekursiv den linken Teilbaum 2. Besuche die Wurzel (verarbeite den aktuellen Knoten) 3. Traversiere rekursiv den rechten Teilbaum

Besonderheit bei BST: Die Inorder-Traversierung eines Binären Suchbaums liefert die Werte in **aufsteigend sortierter Reihenfolge**. Dies macht sie besonders nützlich für Sortier- und Validierungsoperationen.

Zeitkomplexität: $O(n)$ – jeder Knoten wird genau einmal besucht.

[!TIP] **Inorder-Traversierung Beispiel:**

Gegeben ist folgender BST:

Inorder-Reihenfolge: $20 \rightarrow 30 \rightarrow 40 \rightarrow 50 \rightarrow 60 \rightarrow 70 \rightarrow 80$

Beachte: Aufsteigend sortiert!

Preorder-Traversierung (WLR) Bei der **Preorder-Traversierung** wird zunächst die Wurzel besucht, dann der linke Teilbaum, dann der rechte Teilbaum. Die Reihenfolge ist: **Wurzel** → **Links** → **Rechts** (WLR).

Algorithmus: 1. Besuche die Wurzel (verarbeite den aktuellen Knoten) 2. Traversiere rekursiv den linken Teilbaum 3. Traversiere rekursiv den rechten Teilbaum

Anwendung: Die Preorder-Traversierung wird verwendet, um die Struktur eines Baums zu kopieren oder einen Baum in eine Darstellung zu serialisieren, die später rekonstruiert werden kann. Sie entspricht der Reihenfolge, in der man einen Baum “von oben nach unten” aufbaut.

Zeitkomplexität: $O(n)$

[!TIP] **Preorder-Traversierung Beispiel:**

Derselbe BST wie oben:

Preorder-Reihenfolge: $50 \rightarrow 30 \rightarrow 20 \rightarrow 40 \rightarrow 70 \rightarrow 60 \rightarrow 80$

Verwendung: Baum-Kopie, Serialisierung, Präfix-Notation

Postorder-Traversierung (LRW) Bei der **Postorder-Traversierung** wird zunächst der linke Teilbaum besucht, dann der rechte Teilbaum, dann die Wurzel. Die Reihenfolge ist: **Links** → **Rechts** → **Wurzel** (LRW).

Algorithmus: 1. Traversiere rekursiv den linken Teilbaum 2. Traversiere rekursiv den rechten Teilbaum 3. Besuche die Wurzel (verarbeite den aktuellen Knoten)

Anwendung: Die Postorder-Traversierung wird verwendet, um einen Baum zu löschen (Kinder zuerst, dann Eltern) oder um Ausdrücke in Postfix-Notation auszuwerten. Sie garantiert, dass ein Knoten erst verarbeitet wird, nachdem alle seine Nachkommen verarbeitet wurden.

Zeitkomplexität: $O(n)$

[!TIP] **Postorder-Traversierung Beispiel:**

Derselbe BST wie oben:

Postorder-Reihenfolge: $20 \rightarrow 40 \rightarrow 30 \rightarrow 60 \rightarrow 80 \rightarrow 70 \rightarrow 50$

Verwendung: Baum-Löschung, Postfix-Notation, Speicherfreigabe

Level-Order-Traversierung (Breitensuche) Die **Level-Order-Traversierung** (auch **Breitensuche** oder BFS – Breadth-First Search genannt) besucht die Knoten ebenenweise von oben nach unten und innerhalb jeder Ebene von links nach rechts.

Algorithmus (iterativ mit Queue): 1. Füge die Wurzel in eine Queue ein 2. Solange die Queue nicht leer ist: - Entferne den vordersten Knoten aus der Queue - Besuche diesen Knoten (verarbeite ihn) - Füge seine Kindknoten (links, dann rechts) zur Queue hinzu

Anwendung: Level-Order-Traversierung wird verwendet, um die Struktur eines Baums ebenenweise zu analysieren oder den kürzesten Pfad in ungewichteten Graphen zu finden.

Zeitkomplexität: $O(n)$

Speicherkomplexität: $O(w)$, wobei w die maximale Breite des Baums ist

[!TIP] **Level-Order-Traversierung Beispiel:**

Derselbe BST wie oben:

Level-Order-Reihenfolge: $50 \rightarrow 30 \rightarrow 70 \rightarrow 20 \rightarrow 40 \rightarrow 60 \rightarrow 80$

Verwendung: Ebenenweise Ausgabe, Breitensuche, Strukturanalyse

[!WARNING] **Wahl der Traversierung:** Die Wahl der Traversierungsart hängt stark vom Anwendungsfall ab: - **Inorder:** Sortierte Ausgabe bei BST - **Preorder:** Baum-Kopie, Serialisierung - **Postorder:** Baum-Löschung, Ausdrucks-Auswertung - **Level-Order:** Ebenenweise Verarbeitung, Breitensuche

Hash-Tabellen: Grundprinzip

Eine **Hash-Tabelle** (Hash Table oder Hash Map) ist eine Datenstruktur, die **Schlüssel-Wert-Paare** speichert und nahezu konstanten Zugriff ($O(1)$ im Average Case) ermöglicht. Sie ist eine der wichtigsten Datenstrukturen in der Informatik und wird in Datenbanken, Caches, Compilern und vielen weiteren Anwendungen eingesetzt.

[!NOTE] **Hash-Tabelle (Hash Table)**: Eine Datenstruktur, die Schlüssel über eine **Hash-Funktion** auf Indizes in einem Array abbildet. Dies ermöglicht schnellen Zugriff, Einfügen und Löschen von Elementen anhand ihrer Schlüssel.

Hash-Funktionen Das Herzstück jeder Hash-Tabelle ist die **Hash-Funktion**. Sie nimmt einen Schlüssel als Eingabe und berechnet daraus einen ganzzahligen **Hash-Wert**, der als Index in ein Array verwendet wird.

[!NOTE] **Hash-Funktion**: Eine Funktion $h(\text{key}) \rightarrow \text{index}$, die einen Schlüssel deterministisch auf einen Array-Index abbildet. Eine gute Hash-Funktion verteilt die Schlüssel gleichmäßig über den verfügbaren Speicherbereich, um Kollisionen zu minimieren.

Eigenschaften einer guten Hash-Funktion: 1. **Determinismus**: Derselbe Schlüssel erzeugt immer denselben Hash-Wert 2. **Gleichverteilung**: Schlüssel werden gleichmäßig über den Hash-Bereich verteilt 3. **Effizienz**: Die Berechnung des Hash-Werts ist schnell ($O(1)$) 4. **Minimierung von Kollisionen**: Verschiedene Schlüssel sollten möglichst unterschiedliche Hash-Werte erzeugen

Einfache Hash-Funktionen: - **Modulo-Methode**: $h(\text{key}) = \text{key} \% \text{table_size}$ (für ganzzahlige Schlüssel) - **String-Hash**: Summiere ASCII-Werte der Zeichen, multipliziert mit Potenzen einer Primzahl - **Multiplikative Methode**: $h(\text{key}) = \text{floor}(\text{table_size} * (\text{key} * A \bmod 1))$, wobei A eine Konstante (z.B. 0.618) ist

[!TIP] **Beispiel einer einfachen Hash-Funktion für Strings**:

Die Zahl 31 ist eine beliebte Wahl (Primzahl, effiziente Berechnung als $x \ll 5 - x$).

Kollisionen Eine **Kollision** tritt auf, wenn zwei unterschiedliche Schlüssel denselben Hash-Wert (Index) erzeugen. Kollisionen sind unvermeidlich, da der Schlüsselraum in der Regel größer ist als die Hash-Tabellen-Größe (Schubfachprinzip).

[!NOTE] **Kollision (Collision)**: Situation, in der zwei verschiedene Schlüssel k_1 und k_2 denselben Hash-Wert erzeugen: $h(k_1) = h(k_2)$. Kollisionsbehandlung ist ein zentrales Problem beim Design von Hash-Tabellen.

Kollisionsbehandlung: Chaining

Chaining (Verkettung) ist die klassische Methode zur Kollisionsbehandlung. Jede Zelle der Hash-Tabelle enthält nicht direkt ein Element, sondern eine **verkettete Liste** (oder eine andere Datenstruktur wie einen Baum), die alle Elemente mit demselben Hash-Wert speichert.

[!NOTE] **Chaining:** Kollisionsbehandlungsstrategie, bei der jede Zelle der Hash-Tabelle eine verkettete Liste enthält. Alle Elemente mit demselben Hash-Wert werden in dieser Liste gespeichert.

Funktionsweise: - **Einfügen:** Berechne $\text{index} = h(\text{key})$, füge das Element am Anfang oder Ende der Liste bei `table[index]` hinzu - **Suchen:** Berechne $\text{index} = h(\text{key})$, durchsuche die Liste bei `table[index]` linear nach dem Schlüssel - **Löschen:** Berechne $\text{index} = h(\text{key})$, finde das Element in der Liste und entferne es

Komplexität: - **Best Case:** $O(1)$ – keine Kollision, Liste hat Länge 1 - **Average Case:** $O(1 + \dots)$, wobei $\dots = n/m$ (Anzahl Elemente / Tabellengröße) der **Ladefaktor** ist - **Worst Case:** $O(n)$ – alle Elemente kollidieren, eine einzige Liste der Länge n

Vorteile: - Einfach zu implementieren - Tabelle muss nie vollständig sein, kann über 100% Auslastung gehen - Geeignet für dynamisch wachsende Datenmengen

Nachteile: - Zusätzlicher Speicher für Listenzeiger - Cache-ineffizient durch Pointer-Dereferenzierung - Performance degradiert bei hohem Ladefaktor

[!TIP] **Chaining-Visualisierung:**

Annahme: $h(\text{key}) = \text{key} \% 10$

Kollision bei Index 1: Schlüssel 21 und 31 haben beide $h(k) = 1$

Kollisionsbehandlung: Open Addressing

Bei **Open Addressing** (offene Adressierung) werden alle Elemente direkt in der Hash-Tabelle selbst gespeichert, nicht in separaten Listen. Bei einer Kollision wird nach einem alternativen freien Platz in der Tabelle gesucht, indem eine **Sondierungssequenz** (Probing Sequence) verwendet wird.

[!NOTE] **Open Addressing:** Kollisionsbehandlungsstrategie, bei der alle Elemente direkt in der Hash-Tabelle gespeichert werden. Bei einer Kollision wird systematisch nach einem freien Platz gesucht.

Linear Probing **Linear Probing** ist die einfachste Open-Addressing-Strategie. Bei einer Kollision wird linear durch die Tabelle gegangen: $h(\text{key})$, $h(\text{key})+1$, $h(\text{key})+2$, ..., bis ein freier Platz gefunden wird.

Sondierungsformel: $h'(\text{key}, i) = (h(\text{key}) + i) \% \text{table_size}$, wobei $i = 0, 1, 2, \dots$

Vorteile: - Sehr cache-freundlich (sequenzieller Speicherzugriff) - Einfach zu implementieren - Keine zusätzlichen Zeiger erforderlich

Nachteile: - **Primary Clustering:** Es entstehen lange zusammenhängende Bereiche belegter Zellen, was die Suchzeit erhöht - Tabelle darf nicht zu voll werden (üblich: Ladefaktor < 0.7)

Quadratic Probing Quadratic Probing verwendet eine quadratische Funktion zur Sondierung: $h(\text{key}), h(\text{key})+1^2, h(\text{key})+2^2, h(\text{key})+3^2, \dots$

Sondierungsformel: $h'(\text{key}, i) = (h(\text{key}) + c \cdot i + c \cdot i^2) \% \text{table_size}$, wobei c und c Konstanten sind

Vorteile: - Reduziert Primary Clustering - Bessere Verteilung als Linear Probing

Nachteile: - **Secondary Clustering:** Schlüssel mit demselben Hash-Wert folgen derselben Sondierungssequenz - Kompliziertere Implementierung - Nicht alle Zellen werden garantiert untersucht

Double Hashing Double Hashing verwendet eine zweite Hash-Funktion zur Berechnung der Schrittweite: $h(\text{key}), h(\text{key})+h(\text{key}), h(\text{key})+2 \cdot h(\text{key}), \dots$

Sondierungsformel: $h'(\text{key}, i) = (h(\text{key}) + i \cdot h(\text{key})) \% \text{table_size}$

Anforderungen an h : - $h(\text{key})$ darf nie 0 sein - $h(\text{key})$ sollte relativ prim zu table_size sein (oft: table_size ist Primzahl)

Vorteile: - Eliminiert sowohl Primary als auch Secondary Clustering - Beste Verteilung unter den Open-Addressing-Methoden - Annähernd gleichmäßige Sondierung

Nachteile: - Etwas komplexere Implementierung - Zusätzliche Hash-Funktion muss berechnet werden

[!TIP] **Vergleich der Probing-Strategien:**

Strategie	Clustering	Cache-Effizienz	Komplexität
Linear Probing	Stark (Primary)	Sehr gut	Einfach
Quadratic Probing	Mittel (Secondary)	Gut	Mittel
Double Hashing	Minimal	Mittel	Komplex

[!WARNING] **Ladefaktor bei Open Addressing:** Open-Addressing-Tabellen müssen rechtzeitig vergrößert werden (**Rehashing**). Ein Ladefaktor > 0.7 führt zu drastischem Performance-Verlust. Bei $\rightarrow 1$ (Tabelle fast voll) steigt die erwartete Suchzeit exponentiell.

Operationen und Komplexitätsanalyse

Die Effizienz von Hash-Tabellen hängt stark von der Qualität der Hash-Funktion, der Kollisionsbehandlung und dem Ladefaktor ab.

Einfügen (Insert) Chaining: 1. Berechne $\text{index} = h(\text{key})$ 2. Füge das Element zur Liste bei $\text{table}[\text{index}]$ hinzu (Anfang oder Ende)

Zeitkomplexität: $O(1)$ im Average Case, $O(n)$ im Worst Case (alle Elemente in einer Liste)

Open Addressing: 1. Berechne $\text{index} = h(\text{key})$ 2. Sondiere, bis ein freier Platz gefunden wird 3. Füge das Element an der gefundenen Position ein

Zeitkomplexität: $O(1/(1-\alpha))$ im Average Case, wobei α der Ladefaktor ist

Suchen (Search) Chaining: 1. Berechne $\text{index} = h(\text{key})$ 2. Durchsuche die Liste bei $\text{table}[\text{index}]$ linear

Zeitkomplexität: $O(1 + \alpha)$ im Average Case, $O(n)$ im Worst Case

Open Addressing: 1. Berechne $\text{index} = h(\text{key})$ 2. Sondiere, bis der Schlüssel gefunden oder ein leerer Platz erreicht wird

Zeitkomplexität: $O(1/(1-\alpha))$ im Average Case

Löschen (Delete) Chaining: 1. Berechne $\text{index} = h(\text{key})$ 2. Finde und entferne das Element aus der Liste bei $\text{table}[\text{index}]$

Zeitkomplexität: $O(1 + \alpha)$ im Average Case, $O(n)$ im Worst Case

Open Addressing: 1. Berechne $\text{index} = h(\text{key})$ 2. Sondiere, bis der Schlüssel gefunden wird 3. Markiere die Zelle als **gelöscht** (nicht leer), um die Sondierungssequenz nicht zu unterbrechen

Problem: Gelöschte Zellen können die Tabelle "verschmutzen". Lösung: Periodisches Rehashing.

Zeitkomplexität: $O(1/(1-\alpha))$ im Average Case

[!TIP] **Zusammenfassung Zeitkomplexitäten:**

Operation	Chaining (Average)	Open Addressing (Average)	Worst Case (beide)
Insert	$O(1)$	$O(1/(1-\alpha))$	$O(n)$
Search	$O(1 + \alpha)$	$O(1/(1-\alpha))$	$O(n)$
Delete	$O(1 + \alpha)$	$O(1/(1-\alpha))$	$O(n)$

α = **Ladefaktor** (n/m , Anzahl Elemente / Tabellengröße)

Zusammenfassung Theorie

Die fortgeschrittenen Datenstrukturen dieser Vorlesung erweitern das Werkzeugrepertoire erheblich. **Binärbäume** und **Binäre Suchbäume** ermöglichen hierarchische Organisation und effiziente Suchoperationen in $O(\log n)$, sofern der Baum balanciert ist. Die **Traversierungsalgorithmen** (Inorder, Preorder, Postorder, Level-Order) bieten verschiedene Möglichkeiten, Bäume systematisch zu durchlaufen, wobei jede Methode spezifische Anwendungsfälle hat.

Hash-Tabellen bieten nahezu konstanten Zugriff ($O(1)$ im Average Case) auf Elemente über Schlüssel, was sie zu einer der effizientesten Datenstrukturen für Lookup-Operationen macht. Die beiden Hauptstrategien zur Kollisionsbehandlung – **Chaining** und **Open Addressing** – haben unterschiedliche Vor- und Nachteile. Chaining ist robuster bei hoher Auslastung, während Open Addressing speichereffizienter und cache-freundlicher ist.

Die Wahl der richtigen Datenstruktur hängt vom konkreten Anwendungsfall ab: Benötigt man sortierte Daten? Wie häufig werden Einfüge-, Such- und Löschoptionen durchgeführt? Wie groß ist der Datensatz? Ein fundiertes Verständnis dieser Strukturen und ihrer Komplexitäten ist essentiell für die Entwicklung performanter Software.

Teil 2: Python-Praxis - Try-Catch (Fehlerbehandlung)

[!WARNING] **Python-Konsistenz beachten:** Prüfe ../../python_topics.md für bereits eingeführte Konzepte!

Überblick

Fehlerbehandlung ist ein fundamentaler Aspekt robuster Software-Entwicklung. In Python werden Laufzeitfehler als **Exceptions** (Ausnahmen) behandelt, die den normalen Programmablauf unterbrechen und eine strukturierte Fehlerbehandlung erfordern. Ohne geeignete Fehlerbehandlung stürzt ein Programm bei unerwarteten Eingaben oder Zuständen ab, was in produktiven Systemen inakzeptabel ist. Das **try-except**-Konstrukt ermöglicht es, potenzielle Fehlerquellen zu identifizieren, Fehler gezielt abzufangen und darauf angemessen zu reagieren – sei es durch alternative Programmlogik, Benutzerbenachrichtigung oder kontrolliertes Herunterfahren.

In dieser Vorlesung lernen wir, wie Python Exceptions strukturiert, wie wir sie abfangen und behandeln, und welche Best Practices für robuste und wartbare Fehlerbehandlung gelten.

Was sind Exceptions?

Eine **Exception** ist ein Ereignis, das während der Programmausführung auftritt und den normalen Ablauf unterbricht. Exceptions werden **geworfen** (raised),

wenn Python auf einen Fehler stößt, den es nicht automatisch beheben kann.

[!NOTE] **Exception (Ausnahme)**: Ein Objekt, das einen Fehler oder eine außergewöhnliche Situation während der Programmausführung repräsentiert. Python hat eine Hierarchie vordefinierter Exception-Typen, die verschiedene Fehlerarten kategorisieren.

Häufige Exception-Typen Python bietet eine umfangreiche Hierarchie von Exception-Typen. Die wichtigsten für Einsteiger sind:

ValueError: Wird geworfen, wenn eine Funktion ein Argument mit dem richtigen Typ, aber unangemessenem Wert erhält.

[!TIP]

TypeError: Wird geworfen, wenn eine Operation auf einen ungeeigneten Datentyp angewendet wird.

[!TIP]

IndexError: Wird geworfen, wenn versucht wird, auf einen Index zuzugreifen, der außerhalb des gültigen Bereichs einer Sequenz liegt.

[!TIP]

KeyError: Wird geworfen, wenn versucht wird, auf einen nicht existierenden Dictionary-Schlüssel zuzugreifen.

[!TIP]

FileNotFoundError: Wird geworfen, wenn versucht wird, eine nicht existierende Datei zu öffnen.

[!TIP]

ZeroDivisionError: Wird geworfen, wenn versucht wird, durch Null zu teilen.

[!TIP]

AttributeError: Wird geworfen, wenn versucht wird, auf ein nicht existierendes Attribut eines Objekts zuzugreifen.

[!TIP]

[!NOTE] Alle Exception-Typen erben von der Basisklasse **BaseException**. Die meisten benutzerdefinierten Exceptions sollten von **Exception** erben, nicht direkt von **BaseException**.

Das try-except-Konstrukt

Das **try-except**-Konstrukt ist das Herzstück der Fehlerbehandlung in Python. Code, der möglicherweise eine Exception werfen könnte, wird in einen **try**-Block eingeschlossen. Wenn eine Exception auftritt, wird der **try**-Block unterbrochen und der entsprechende **except**-Block ausgeführt.

[!NOTE] **try-except-Block:** Ein Kontrollstrukt, das es ermöglicht, potenziell fehlerhafte Code-Abschnitte zu überwachen und auf Fehler zu reagieren, ohne dass das Programm abstürzt.

Syntax:

Grundlegende Verwendung Die einfachste Form fängt eine spezifische Exception ab:

[!TIP]

Erklärung: - Der **try**-Block enthält Code, der bei ungültiger Eingabe oder Division durch Null fehlschlagen kann - Wenn **int()** keine Zahl konvertieren kann, wird **ValueError** abgefangen - Wenn der Benutzer 0 eingibt, wird **ZeroDivisionError** abgefangen - Andere Exceptions würden nicht abgefangen und das Programm würde abstürzen

[!WARNING] **Vermeide blanket except:** Ein nacktes **except:** ohne Angabe eines Exception-Typs fängt **alle** Exceptions ab, einschließlich **KeyboardInterrupt** (Strg+C). Dies macht Debugging extrem schwierig und sollte vermieden werden. Nutze stattdessen spezifische Exception-Typen oder **except Exception:** für generische Fehlerbehandlung.

Mehrere Exception-Typen abfangen

Ein **try**-Block kann mehrere **except**-Klauseln haben, um verschiedene Fehlertypen unterschiedlich zu behandeln.

[!TIP]

Mehrere Exceptions in einer Klausel Wenn mehrere Exception-Typen identisch behandelt werden sollen, können sie als Tupel gruppiert werden:

[!TIP]

Dies ist eine kompaktere Alternative zu separaten **except**-Blöcken mit identischem Code.

Zugriff auf das Exception-Objekt

Python ermöglicht es, das Exception-Objekt selbst zu erfassen, um detaillierte Fehlerinformationen zu extrahieren:

[!NOTE] **Syntax:** **except ExceptionType as variable_name**

Das Exception-Objekt enthält Informationen wie die Fehlermeldung und kann für Logging oder detaillierte Benutzerbenachrichtigung verwendet werden.

[!TIP]

Output-Beispiel:

Die **else**-Klausel im **try-except**

Die **else**-Klausel wird ausgeführt, wenn der **try**-Block **keine Exception** geworfen hat. Dies ist nützlich, um Code zu platzieren, der nur bei Erfolg ausgeführt werden soll, aber selbst keine Exception-Behandlung benötigt.

[!NOTE] **else-Klausel:** Ein optionaler Block nach allen **except**-Klauseln, der nur ausgeführt wird, wenn keine Exception aufgetreten ist.

Syntax:

[!TIP]

Vorteil: Der **else**-Block signalisiert klar, dass dieser Code nur im Erfolgsfall ausgeführt wird. Es trennt die “normale” Logik von der Fehlerbehandlung.

Die **finally**-Klausel

Die **finally**-Klausel wird **immer** ausgeführt, unabhängig davon, ob eine Exception aufgetreten ist oder nicht. Sie ist ideal für Aufräumarbeiten wie das Schließen von Dateien, Netzwerkverbindungen oder das Freigeben von Ressourcen.

[!NOTE] **finally-Klausel:** Ein optionaler Block, der **garantiert** ausgeführt wird, egal ob eine Exception auftrat oder nicht. Selbst wenn ein **return**-Statement im **try** oder **except** steht, wird **finally** vorher ausgeführt.

Syntax:

[!TIP]

Hinweis: In modernem Python sollte man für Dateien das **with**-Statement verwenden, das automatisch aufräumt. **finally** ist aber universell für andere Ressourcen nützlich.

Vollständige **try-except-else-finally**-Struktur

Die vollständige Struktur kombiniert alle Elemente:

[!TIP]

Eigene Exceptions werfen mit **raise**

Python ermöglicht es, Exceptions explizit zu werfen, um Fehlerzustände im eigenen Code zu signalisieren.

[!NOTE] **raise-Statement:** Wirft eine Exception, unterbricht den aktuellen Code-Fluss und löst die Fehlerbehandlung aus.

Syntax:

[!TIP]

Output:

Exception Re-Raise Manchmal möchte man eine Exception abfangen, partiell behandeln und dann erneut werfen:

[!TIP]

Output:

Benutzerdefinierte Exceptions

Für komplexere Anwendungen ist es oft sinnvoll, eigene Exception-Typen zu definieren. Dies verbessert die Lesbarkeit und ermöglicht präzise Fehlerbehandlung.

[!NOTE] **Benutzerdefinierte Exception:** Eine eigene Exception-Klasse, die von `Exception` oder einem anderen Exception-Typ erbt. Sie kann zusätzliche Attribute und Methoden enthalten.

Syntax:

[!TIP]

Output:

Häufige Fehler und Lösungen

[!WARNING] **Fehler 1: Zu breite except-Klauseln**

Problem:

Warum problematisch: Fängt **alle** Exceptions ab, auch `KeyboardInterrupt` (Strg+C) und `SystemExit`. Verhindert kontrolliertes Beenden und erschwert Debugging.

Lösung: Verwende spezifische Exception-Typen oder mindestens `except Exception::`

[!WARNING] **Fehler 2: Exceptions verschlucken**

Problem:

Warum problematisch: Der Fehler wird komplett unterdrückt. Niemand erfährt, dass etwas schiefgelaufen ist.

Lösung: Mindestens loggen, oder den Benutzer informieren:

[!WARNING] **Fehler 3: Ausnahmen für Kontrollfluss missbrauchen**

Problem:

Warum problematisch: Exceptions sind für Ausnahmesituationen gedacht, nicht für normalen Kontrollfluss. Dies ist langsam und erschwert die Lesbarkeit.

Lösung: Nutze `.get()` bei Dictionaries:

Zusammenfassung Python

Die Fehlerbehandlung in Python mit dem `try-except-else-finally`-Konstrukt ist ein mächtiges Werkzeug, um robuste und fehlertolerante Programme zu schreiben. Die wichtigsten Prinzipien sind:

1. **Spezifische Exceptions abfangen:** Verwende konkrete Exception-Typen, nicht blanket `except`:
2. **Fehler nicht verschlucken:** Informiere den Benutzer oder logge den Fehler
3. **finally für Aufräumarbeiten:** Garantiert Ausführung, ideal für Ressourcen-Freigabe
4. **else für Erfolgs-Logik:** Trennt normale Ausführung von Fehlerbehandlung
5. **Eigene Exceptions werfen:** Nutze `raise` für Validierung und Fehlersignalisierung
6. **Benutzerdefinierte Exceptions:** Verbessern Code-Klarheit bei komplexen Systemen

Mit diesen Werkzeugen können Programme elegant auf unerwartete Situationen reagieren, anstatt unkontrolliert abzustürzen.

Neue Python-Funktionen/Methoden

In dieser Lektion wurden folgende Python-Konzepte **neu eingeführt**:

Exception-Handling: - `try` – Markiert einen Block mit potenziell fehlerhaftem Code - `except ExceptionType` – Fängt spezifische Exceptions ab - `except ExceptionType as e` – Fängt Exception ab und bindet Objekt an Variable - `except (Type1, Type2)` – Fängt mehrere Exception-Typen ab - `else` (in `try-except`) – Wird ausgeführt, wenn keine Exception auftrat - `finally` – Wird immer ausgeführt, auch bei Exceptions oder `return` - `raise ExceptionType("message")` – Wirft eine Exception - `raise` (ohne Argument) – Wirft die aktuell behandelte Exception erneut

Vordefinierte Exception-Typen (Auswahl): - `ValueError` – Wert hat falschen Inhalt/Format - `TypeError` – Operation auf falschen Datentyp angewendet - `IndexError` – Index außerhalb des gültigen Bereichs - `KeyError` – Dictionary-Schlüssel existiert nicht - `FileNotFoundError` – Datei nicht

gefunden - `ZeroDivisionError` – Division durch Null - `AttributeError` – Attribut existiert nicht - `Exception` – Basisklasse für die meisten Exceptions

Benutzerdefinierte Exceptions: - Eigene Exception-Klassen durch Vererbung von `Exception`

Weiterführende Ressourcen

Theorie

Bücher und Tutorials: - “Introduction to Algorithms” (Cormen et al.) – Kapitel zu Bäumen und Hash-Tabellen - “Data Structures and Algorithm Analysis” (Mark Allen Weiss) – Umfassende Darstellung von BST, Balancierung und Hashing - Visualgo – Interaktive Visualisierung von Datenstrukturen und Algorithmen

Wissenschaftliche Papers: - “A Method for the Construction of Minimum-Redundancy Codes” (Huffman, 1952) – Grundlage für Huffman-Bäume - “Self-Adjusting Binary Search Trees” (Sleator & Tarjan, 1985) – Splay Trees

Online-Ressourcen: - GeeksforGeeks: Binary Search Tree - Brilliant.org: Hash Tables

Python

Offizielle Dokumentation: - Python Tutorial: Errors and Exceptions - Built-in Exceptions

Tutorials und Best Practices: - Real Python: Python Exceptions - PEP 8: Exception Handling

Video-Tutorials: - Corey Schafer: “Python Exception Handling - Try, Except, Else, Finally” - Python Official: Exception Handling

Datei: lessons/V10-Laufzeitanalyse-und-Algorithmik/V10-Laufzeitanalyse-und-Algorithmik_skript.md

V10: Laufzeitanalyse & Algorithmik

[!NOTE] **Lernziele dieser Vorlesung:** - Verstehen, wie man die Effizienz von Algorithmen misst und vergleicht - Die O-Notation (Big-O) sicher anwenden können, um Zeitkomplexität zu beschreiben - Unterschiede zwischen Best-Case, Average-Case und Worst-Case Szenarien erkennen - Einfache Algorithmen analysieren und deren Laufzeitverhalten bestimmen - Grundlegende Sortieralgorithmen (Bubble Sort, Quick Sort, Merge Sort) verstehen - Funktionen

in Python definieren und mit Parametern sowie Rückgabewerten arbeiten - Default-Parameter verwenden, um flexible Funktionsschnittstellen zu schaffen - Funktionen als First-Class-Objekte verstehen und einsetzen können

Teil 1: Theorie - Laufzeitanalyse & Algorithmik

Überblick

Die Informatik beschäftigt sich nicht nur mit der Frage, *ob* ein Problem lösbar ist, sondern auch damit, *wie effizient* eine Lösung ist. Zwei Programme können dieselbe Aufgabe erfüllen, aber eines davon benötigt vielleicht Sekunden, während das andere Stunden läuft. Besonders bei großen Datenmengen, wie sie im Maschinenbau bei Simulationen oder Sensorauswertungen anfallen, ist die Effizienz entscheidend.

Die **Laufzeitanalyse** hilft dabei, die Effizienz von Algorithmen zu bewerten. Ein **Algorithmus** ist eine präzise, endliche Folge von Anweisungen zur Lösung eines Problems. Die Laufzeitanalyse ermöglicht es, Algorithmen unabhängig von der Hardware oder Programmiersprache zu vergleichen. Zentrale Werkzeuge dafür sind die O-Notation (Big-O), die beschreibt, wie sich die Laufzeit eines Algorithmus mit wachsender Eingabegröße verhält, sowie die Unterscheidung zwischen Best-Case, Average-Case und Worst-Case Szenarien.

[!NOTE] **Algorithmus:** Eine eindeutige Handlungsvorschrift zur Lösung eines Problems. Ein Algorithmus besteht aus einer endlichen Anzahl von Schritten, die in einer festgelegten Reihenfolge ausgeführt werden, und transformiert Eingabedaten in Ausgabedaten.

Warum Laufzeitanalyse?

Bei kleinen Datenmengen (z.B. 10 Elemente) spielt die Effizienz eines Algorithmus oft keine große Rolle. Ein langsamer Algorithmus benötigt vielleicht 0,001 Sekunden, ein schneller 0,0001 Sekunden – der Unterschied ist kaum spürbar. Bei großen Datenmengen (z.B. 1 Million Elemente) kann jedoch ein ineffizienter Algorithmus Stunden oder Tage benötigen, während ein effizienter nur Sekunden braucht.

Die Laufzeitanalyse erlaubt es, das Verhalten von Algorithmen bei wachsenden Eingabegrößen vorherzusagen. Statt konkrete Zeitmessungen durchzuführen, die von Hardware, Betriebssystem und Implementierung abhängen, betrachtet man die Anzahl der elementaren Operationen (z.B. Vergleiche, Zuweisungen, Additionen) in Abhängigkeit von der Eingabegröße n .

[!TIP] **Beispiel:** Ein Sortieralgorithmus, der bei 100 Elementen 10.000 Vergleiche benötigt, wird bei 1.000 Elementen nicht einfach

100.000 Vergleiche benötigen – je nach Algorithmus kann die Zahl drastisch höher oder niedriger sein. Die O-Notation hilft, dieses Wachstumsverhalten präzise zu beschreiben.

Die O-Notation (Big-O)

Die **O-Notation** (auch Big-O genannt) beschreibt die obere Schranke des Wachstums der Laufzeit eines Algorithmus in Abhängigkeit von der Eingabegröße n . Sie abstrahiert von konstanten Faktoren und kleineren Termen, um das wesentliche Wachstumsverhalten zu erfassen.

[!NOTE] **O-Notation (Big-O)**: Eine mathematische Notation zur Beschreibung des asymptotischen Verhaltens von Funktionen. Für eine Funktion $f(n)$ bedeutet $f(n) = O(g(n))$, dass es Konstanten $c > 0$ und $n_0 > 0$ gibt, sodass $f(n) \leq c \cdot g(n)$ für alle $n \geq n_0$ gilt. In der Praxis beschreibt $O(g(n))$ die maximale Wachstumsrate der Laufzeit.

Warum konstante Faktoren ignoriert werden: Die O-Notation interessiert sich für das asymptotische Verhalten, d.h. das Verhalten bei sehr großen n . Konstante Faktoren (z.B. ob ein Algorithmus $5n$ oder $10n$ Operationen benötigt) werden nicht berücksichtigt, da sie bei wachsendem n weniger relevant sind als das grundlegende Wachstumsmuster (z.B. linear vs. quadratisch).

Warum kleinere Terme ignoriert werden: Bei einer Funktion wie $f(n) = 3n^2 + 5n + 10$ dominiert der Term $3n^2$ für große n alle anderen Terme. Die O-Notation fokussiert auf den dominierenden Term und schreibt $O(n^2)$.

Häufige Komplexitätsklassen Die folgende Tabelle zeigt die wichtigsten Komplexitätsklassen, sortiert von am effizientesten zu am ineffizientesten:

O-Notation	Name	Beispiel	Beschreibung
$O(1)$	Konstant	Zugriff auf Array-Element <code>arr[5]</code>	Laufzeit unabhängig von n
$O(\log n)$	Logarithmisch	Binäre Suche	Halbiert Suchbereich in jedem Schritt
$O(n)$	Linear	Durchsuchen einer Liste	Laufzeit wächst proportional zu n
$O(n \log n)$	Linearithmisch	Merge Sort, Quick Sort (average)	Effiziente Sortieralgorithmen
$O(n^2)$	Quadratisch	Bubble Sort, verschachtelte Schleifen	Laufzeit wächst quadratisch

O-Notation	Name	Beispiel	Beschreibung
$O(n^3)$	Kubisch	Matrix-Multiplikation (naiv)	Drei verschachtelte Schleifen
$O(2^n)$	Exponentiell	Rekursive Fibonacci (naiv)	Unpraktikabel für große n
$O(n!)$	Faktoriell	Brute-Force Traveling Salesman	Nur für sehr kleine n

[!TIP] **Veranschaulichung bei $n = 1.000$:** - $O(1)$: 1 Operation - $O(\log n)$: ca. 10 Operationen - $O(n)$: 1.000 Operationen - $O(n \log n)$: ca. 10.000 Operationen - $O(n^2)$: 1.000.000 Operationen - $O(2^n)$: 10^{301} Operationen (Universum hat nicht genug Atome!)

Weitere Notationen: Omega und Theta

Neben der O-Notation gibt es auch **Omega-Notation** (Ω) und **Theta-Notation** (Θ):

[!NOTE] **Omega-Notation** (Ω): Beschreibt die untere Schranke der Laufzeit. $f(n) = \Omega(g(n))$ bedeutet, dass $f(n)$ mindestens so schnell wächst wie $g(n)$. Dies ist das Best-Case-Szenario.

[!NOTE] **Theta-Notation** (Θ): Beschreibt eine asymptotisch enge Schranke. $f(n) = \Theta(g(n))$ bedeutet, dass $f(n)$ sowohl $O(g(n))$ als auch $\Omega(g(n))$ ist. Die Laufzeit wächst exakt wie $g(n)$, sowohl im Best-Case als auch im Worst-Case.

In der Praxis wird meist die O-Notation verwendet, da sie das Worst-Case-Verhalten beschreibt, welches für die Systemauslegung am relevantesten ist.

Best-Case, Average-Case, Worst-Case

Bei der Analyse von Algorithmen unterscheidet man drei Szenarien:

[!NOTE] **Best-Case:** Das günstigste Eingabe-Szenario, bei dem der Algorithmus die wenigsten Operationen benötigt. Beispiel: Bei einer linearen Suche ist der Best-Case, dass das gesuchte Element an erster Position steht – nur eine Vergleichsoperation.

[!NOTE] **Average-Case:** Das durchschnittliche Verhalten über alle möglichen Eingaben hinweg. Dies erfordert oft eine statistische Analyse und Annahmen über die Verteilung der Eingaben. Der Average-

Case ist realistischer als der Best-Case, aber schwieriger zu berechnen.

[!NOTE] **Worst-Case:** Das ungünstigste Eingabe-Szenario, bei dem der Algorithmus die meisten Operationen benötigt. Beispiel: Bei einer linearen Suche ist der Worst-Case, dass das gesuchte Element nicht vorhanden ist oder an letzter Position steht – n Vergleichsoperationen.

Die Worst-Case-Analyse ist in der Praxis am wichtigsten, da sie garantiert, dass der Algorithmus niemals langsamer ist als angegeben. Für kritische Systeme (z.B. Echtzeitsysteme im Maschinenbau) muss sichergestellt sein, dass ein Algorithmus auch im Worst-Case schnell genug ist.

[!TIP] **Beispiel - Lineare Suche:**

Analyse einfacher Algorithmen

Um die Laufzeit eines Algorithmus zu bestimmen, zählt man die elementaren Operationen in Abhängigkeit von der Eingabegröße n . Betrachten wir einige Beispiele:

Beispiel 1: Einfache Schleife **Analyse:** - Initialisierung: 1 Operation - Schleife läuft n -mal - Pro Iteration: 2 Operationen - Rückgabe: 1 Operation

Gesamt: $1 + n \cdot 2 + 1 = 2n + 2$ Operationen

O-Notation: $O(n)$ (konstante Faktoren und Terme ignoriert)

Beispiel 2: Verschachtelte Schleifen **Analyse:** - Äußere Schleife: n Iterationen - Innere Schleife: n Iterationen pro äußerer Iteration - Gesamt: $n \cdot n = n^2$ Iterationen

O-Notation: $O(n^2)$

Beispiel 3: Binäre Suche (Halbierungsstrategie) **Analyse:** - In jeder Iteration wird der Suchbereich halbiert - Nach k Iterationen: Suchbereich = $n/2^k$ - Algorithmus endet, wenn Suchbereich = 1: $n/2^k = 1 \Rightarrow k = \log_2(n)$

O-Notation: $O(\log n)$

[!WARNING] **Häufiger Fehler:** Verwechslung von $O(n)$ und $O(\log n)$. Lineare Suche durchläuft alle Elemente (schlimmstenfalls), binäre Suche halbiert den Suchbereich in jedem Schritt. Bei 1.000 Elementen bedeutet das: Linear = 1.000 Vergleiche, Binär = ca. 10 Vergleiche.

Rekursion und Rekursive Komplexität

Rekursion ist eine Programmier Technik, bei der eine Funktion sich selbst aufruft. Rekursive Algorithmen können elegante Lösungen für Probleme liefern, haben aber oft eine höhere Laufzeitkomplexität als iterative Ansätze.

[!NOTE] **Rekursion:** Eine Funktion, die sich selbst aufruft, um ein Problem in kleinere Teilprobleme derselben Art zu zerlegen. Rekursion benötigt eine Abbruchbedingung (Basisfall), sonst entsteht eine Endlosschleife.

Beispiel: Fibonacci-Zahlen (naiv rekursiv) Analyse: - Für `fibonacci(n)` werden `fibonacci(n-1)` und `fibonacci(n-2)` aufgerufen - Jeder Aufruf verzweigt sich in zwei weitere Aufrufe - Ergibt einen binären Baum mit Höhe n - Anzahl der Aufrufe: ca. 2^n

O-Notation: $O(2^n)$ – exponentiell!

[!WARNING] **Exponentielles Wachstum bei naiver Rekursion:** Die naive rekursive Fibonacci-Implementierung ist extrem ineffizient. Für $n=40$ werden bereits über 1 Milliarde Funktionsaufrufe benötigt! Optimierte Varianten (z.B. mit Memoization oder iterativ) erreichen $O(n)$.

Sortieralgorithmen im Vergleich

Sortieralgorithmen sind ein klassisches Beispiel zur Illustration unterschiedlicher Komplexitätsklassen. Wir betrachten drei grundlegende Algorithmen:

Bubble Sort **Bubble Sort** vergleicht benachbarte Elemente und vertauscht sie, wenn sie in falscher Reihenfolge sind. Dieser Prozess wird wiederholt, bis keine Vertauschungen mehr nötig sind.

Analyse: - Äußere Schleife: n Durchläufe - Innere Schleife: Im ersten Durchlauf $n - 1$ Vergleiche, im zweiten $n - 2$, etc. - Gesamt: $(n - 1) + (n - 2) + \dots + 1 = \frac{n(n-1)}{2} = \frac{n^2-n}{2}$

Komplexität: - **Best-Case:** $O(n)$ (Liste bereits sortiert, mit Optimierung) - **Average-Case:** $O(n^2)$ - **Worst-Case:** $O(n^2)$

[!NOTE] **Bubble Sort:** Ein einfacher, aber ineffizienter Sortieralgorithmus. Der Name kommt daher, dass große Elemente wie Blasen nach oben “blubbern”. Bubble Sort ist stabil (erhält die Reihenfolge gleicher Elemente) und benötigt keinen zusätzlichen Speicher (in-place).

Quick Sort **Quick Sort** wählt ein Pivot-Element und partitioniert die Liste in Elemente kleiner und größer als das Pivot. Anschließend wird rekursiv auf beide Teilbereiche Quick Sort angewendet.

Analyse: - Im besten Fall wird die Liste immer in zwei gleich große Hälften geteilt - Rekursionstiefe: $\log n$ - Pro Ebene: $O(n)$ Operationen (Partitionierung)
- Gesamt: $O(n \log n)$

Komplexität: - **Best-Case:** $O(n \log n)$ (Pivot teilt Liste optimal) - **Average-Case:** $O(n \log n)$ - **Worst-Case:** $O(n^2)$ (Pivot ist immer das kleinste/größte Element, z.B. bei bereits sortierter Liste mit schlechter Pivot-Wahl)

[!NOTE] **Quick Sort:** Ein sehr effizienter Sortieralgorithmus, der im Durchschnitt $O(n \log n)$ erreicht. Quick Sort ist nicht stabil und arbeitet in-place (kann mit geringem zusätzlichem Speicher implementiert werden). Die Wahl des Pivots beeinflusst die Performance stark – gute Implementierungen verwenden randomisierte Pivots oder den Median-of-Three-Ansatz.

Merge Sort **Merge Sort** teilt die Liste rekursiv in zwei Hälften, sortiert diese und verschmilzt (merged) sie anschließend.

Analyse: - Rekursionstiefe: $\log n$ (Liste wird immer halbiert) - Pro Ebene: $O(n)$ Operationen (Mergen) - Gesamt: $O(n \log n)$

Komplexität: - **Best-Case:** $O(n \log n)$ - **Average-Case:** $O(n \log n)$ - **Worst-Case:** $O(n \log n)$ (garantiert!)

[!NOTE] **Merge Sort:** Ein stabiler Sortieralgorithmus mit garantierter $O(n \log n)$ Laufzeit in allen Fällen. Der Nachteil ist der zusätzliche Speicherbedarf von $O(n)$ für die temporären Listen beim Mergen. Merge Sort ist die Basis für viele moderne Sortieralgorithmen (z.B. Timsort in Python).

Vergleich der Sortieralgorithmen

Algorithmus	Best-Case	Average-Case	Worst-Case	Speicher	Stabil
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Ja
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Nein
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Ja

[!TIP] **Praktische Empfehlung:** - Für kleine Datenmengen (< 50 Elemente): Einfache Algorithmen wie Insertion Sort sind ausreichend - Für große Datenmengen: Verwende Merge Sort oder Quick Sort - In Python: Nutze `sorted()` oder `.sort()` – diese verwenden Timsort, einen hybriden Algorithmus (Merge Sort + Insertion Sort) mit $O(n \log n)$ im Worst-Case

Praktische Bedeutung für den Maschinenbau

Im Maschinenbau fallen häufig große Datenmengen an:

- **Simulation:** Finite-Elemente-Analysen generieren Millionen von Datenpunkten
- **Sensorik:** Messdaten von Sensoren müssen in Echtzeit verarbeitet werden
- **Optimierung:** Parameteroptimierung erfordert das Testen vieler Kombinationen
- **Bildverarbeitung:** Kamerabilder in der Qualitätskontrolle müssen schnell analysiert werden

Die Wahl des richtigen Algorithmus kann den Unterschied zwischen praktischer Nutzbarkeit und Unpraktikabilität bedeuten. Ein ineffizienter Algorithmus mit $O(n^2)$ kann bei 1 Million Datenpunkten Stunden benötigen, während ein effizienter mit $O(n \log n)$ nur Sekunden braucht.

[!WARNING] **Vorzeitige Optimierung vermeiden:** Donald Knuth sagte: “Premature optimization is the root of all evil.” Optimierte erst, wenn ein Performance-Problem identifiziert ist. Schreibe zuerst lesbaren, korrekten Code. Profiling-Tools helfen, Engpässe zu identifizieren.

Zusammenfassung Theorie

Die Laufzeitanalyse ist ein zentrales Werkzeug zur Bewertung der Effizienz von Algorithmen. Die O-Notation abstrahiert von Hardware-Details und ermöglicht den Vergleich von Algorithmen unabhängig von der konkreten Implementierung. Konstante Faktoren werden ignoriert, da das asymptotische Wachstum bei großen Eingabegrößen dominiert.

Die wichtigsten Komplexitätsklassen sind $O(1)$ (konstant), $O(\log n)$ (logarithmisch), $O(n)$ (linear), $O(n \log n)$ (linearithmisch), $O(n^2)$ (quadratisch) und $O(2^n)$ (exponentiell). Effiziente Algorithmen haben meist $O(n \log n)$ oder besser, während $O(n^2)$ und höher bei großen Datenmengen problematisch werden.

Best-Case, Average-Case und Worst-Case beschreiben das Verhalten unter verschiedenen Eingabebedingungen. Die Worst-Case-Analyse ist in der Praxis am wichtigsten, da sie garantierte obere Schranken liefert.

Sortieralgorithmen illustrieren die Unterschiede zwischen Komplexitätsklassen: Bubble Sort ist einfach, aber ineffizient ($O(n^2)$), während Quick Sort und Merge Sort im Durchschnitt $O(n \log n)$ erreichen. Merge Sort garantiert diese Laufzeit auch im Worst-Case, benötigt aber zusätzlichen Speicher.

Rekursive Algorithmen können elegant sein, bergen aber Risiken: Naive rekursive Implementierungen (z.B. Fibonacci) können exponentielles Wachstum aufweisen. Optimierungen wie Memoization oder iterative Varianten sind dann notwendig.

Die Wahl des richtigen Algorithmus ist entscheidend für die praktische Nutzbarkeit, insbesondere bei großen Datenmengen im Maschinenbau. Profiling-Tools helfen, Performance-Engpässe zu identifizieren, bevor vorzeitige Optimierung betrieben wird.

Teil 2: Python-Praxis - Methoden/Funktionen – Teil 1

[!WARNING] **Python-Konsistenz beachten:** Prüfe ../../python_topics.md für bereits eingeführte Konzepte!

Überblick

Funktionen sind eines der fundamentalsten Werkzeuge der Programmierung. Sie ermöglichen es, Code zu strukturieren, wiederverwendbar zu machen und komplexe Probleme in kleinere, verständliche Einheiten zu zerlegen. In dieser Lektion lernen wir, eigene Funktionen in Python zu definieren und anzuwenden.

Eine **Funktion** ist ein benannter Codeblock, der eine spezifische Aufgabe erfüllt. Funktionen können Parameter (Eingaben) entgegen nehmen, Operationen durchführen und Ergebnisse (Ausgaben) zurückgeben. Durch die Verwendung von Funktionen wird Code lesbarer, wartbarer und testbarer. Statt denselben Code mehrfach zu schreiben, definiert man ihn einmal in einer Funktion und ruft diese bei Bedarf auf – das entspricht dem DRY-Prinzip (“Don’t Repeat Yourself”) aus V07.

In dieser ersten von zwei Vorlesungen über Funktionen konzentrieren wir uns auf die Grundlagen: Funktionsdefinition, Parameter, Rückgabewerte und Default-Parameter. In V11 werden wir fortgeschrittenere Konzepte wie ***args**, ****kwargs**, Lambda-Funktionen und Type Hints behandeln.

[!NOTE] **Funktion (Function):** Ein wiederverwendbarer Codeblock, der eine spezifische Aufgabe erfüllt. Funktionen werden mit dem Schlüsselwort **def** definiert, haben einen Namen, können Parameter entgegennehmen und Werte zurückgeben.

Funktionen definieren mit **def**

In Python werden Funktionen mit dem Schlüsselwort **def** (define) definiert. Die grundlegende Syntax lautet:

Bestandteile: 1. **def:** Schlüsselwort zur Funktionsdefinition 2. **Funktionsname:** Benenne Funktionen aussagekräftig mit **snake_case** 3. **Parameter:** Eingabewerte in Klammern (optional, kann leer sein) 4. **Doppelpunkt:** Leitet den Funktionskörper ein 5. **Docstring:** Optional, aber empfohlen – dokumentiert die Funktion 6. **Funktionskörper:** Eingerückter Code (4 Leerzeichen) 7. **return:** Optional – gibt einen Wert zurück

[!TIP] **Beispiel: Einfache Funktion ohne Parameter**

[!TIP] **Beispiel: Funktion mit Parametern**

Wichtige Regeln: - Funktionsnamen folgen denselben Regeln wie Variablennamen (Buchstaben, Ziffern, Unterstrich; nicht mit Ziffer beginnen) - Verwende aussagekräftige Namen: **berechne_volumen()** statt **func1()** - Konvention: **snake_case** für Funktionsnamen - Der Funktionskörper muss eingerückt sein

[!WARNING] **Häufiger Fehler:** Funktionsdefinition ohne Doppelpunkt führt zu **SyntaxError**:

Parameter und Argumente

Parameter sind Variablen in der Funktionsdefinition, die Eingabewerte entgegennehmen. **Argumente** sind die konkreten Werte, die beim Funktionsaufruf übergeben werden.

[!NOTE] **Parameter vs. Argumente:** **Parameter** sind Platzhalter in der Funktionsdefinition. **Argumente** sind die tatsächlichen Werte, die beim Aufruf übergeben werden. Oft werden die Begriffe synonym verwendet, technisch gibt es aber einen Unterschied.

Positional Arguments (Positionale Argumente) Standardmäßig werden Argumente in der Reihenfolge zugeordnet, in der sie übergeben werden:

[!WARNING] **Reihenfolge ist entscheidend:** Wenn du die Reihenfolge vertauschst, kann das zu unerwarteten Ergebnissen führen:

Keyword Arguments (Benannte Argumente) Argumente können auch mit ihrem Parameter-Namen übergeben werden. Das macht den Code lesbarer und unabhängig von der Reihenfolge:

Best Practice: Verwende Keyword Arguments für bessere Lesbarkeit, besonders bei vielen Parametern:

Rückgabewerte mit `return`

Funktionen können Werte mit dem `return`-Statement zurückgeben. Das Programm springt nach `return` sofort aus der Funktion heraus.

[!NOTE] **return-Statement:** Beendet die Funktionsausführung und gibt optional einen Wert an den Aufrufer zurück. Ohne `return` gibt eine Funktion implizit `None` zurück.

[!TIP] **Beispiel: Funktion mit Rückgabewert**

Mehrere Rückgabewerte Python ermöglicht es, mehrere Werte gleichzeitig zurückzugeben (als Tupel):

[!TIP] **Unpacking bei mehreren Rückgabewerten:**

Funktionen ohne explizites `return` Wenn eine Funktion kein `return`-Statement hat oder `return` ohne Wert aufgerufen wird, gibt sie implizit `None` zurück:

[!WARNING] **Häufiger Fehler:** `print()` ist kein `return`!

Default-Parameter (Standardwerte)

Funktionen können Default-Werte für Parameter definieren. Diese werden verwendet, wenn beim Aufruf kein Argument übergeben wird.

[!NOTE] **Default-Parameter:** Parameter mit vordefinierten Werten in der Funktionsdefinition. Wenn beim Funktionsaufruf kein Argument für diesen Parameter übergeben wird, wird der Default-Wert verwendet.

[!TIP] **Beispiel: Funktion mit Default-Parametern**

Wichtige Regeln: 1. Parameter mit Default-Werten müssen nach Parametern ohne Default stehen:

2. Default-Werte werden nur einmal bei Funktionsdefinition ausgewertet:

[!WARNING] **Mutable Default-Werte vermeiden:** Verwende niemals mutable Objekte (Listen, Dictionaries) als Default-Werte:

Praktisches Beispiel: Funktion mit mehreren Default-Parametern

Funktionen als First-Class-Objekte

In Python sind Funktionen “First-Class-Objekte” (First-Class Citizens). Das bedeutet, dass Funktionen wie jede andere Variable behandelt werden können:

[!NOTE] **First-Class-Objekte:** Objekte, die zur Laufzeit dynamisch erzeugt, als Argument übergeben, als Rückgabewert zurückgegeben und Variablen zugewiesen werden können. In Python sind Funktionen First-Class-Objekte.

1. **Funktionen können Variablen zugewiesen werden:**
2. **Funktionen können als Argumente übergeben werden:**
3. **Funktionen können als Rückgabewerte zurückgegeben werden:**

[!TIP] **Praktische Anwendung – Sortierung mit key-Parameter:** Die `sorted()`-Funktion und `.sort()`-Methode akzeptieren einen `key`-Parameter, der eine Funktion ist:

(Lambda-Funktionen werden in V11 ausführlich behandelt)

Scope und Variablen-Sichtbarkeit

Variablen innerhalb einer Funktion sind **lokal** und außerhalb nicht sichtbar. Variablen außerhalb von Funktionen sind **global**.

[!NOTE] **Lokaler Scope:** Variablen, die innerhalb einer Funktion definiert werden, existieren nur während der Funktionsausführung und sind außerhalb nicht sichtbar.

[!NOTE] **Globaler Scope:** Variablen, die außerhalb aller Funktionen definiert werden, sind im gesamten Programm sichtbar.

[!TIP] **Beispiel: Lokale vs. Globale Variablen**

LEGB-Regel Python sucht Variablen in folgender Reihenfolge: 1. **Local:** Lokaler Scope (innerhalb der aktuellen Funktion) 2. **Enclosing:** Umschließender Scope (bei verschachtelten Funktionen) 3. **Global:** Globaler Scope (Modul-Ebene) 4. **Built-in:** Eingebaute Namen (z.B. `print`, `len`)

[!WARNING] **global-Keyword sparsam verwenden:** Wie in V03 erwähnt, sollten globale Variablen vermieden werden. Bevorzuge Parameter und Rückgabewerte:

Docstrings: Funktionen dokumentieren

Docstrings sind mehrzeilige Strings, die direkt nach der Funktionsdefinition stehen und die Funktion dokumentieren. Sie sind über `help(funktionsname)` oder `funktionsname.__doc__` abrufbar.

[!NOTE] **Docstring:** Ein String-Literal, das direkt nach der Funktionsdefinition steht und die Funktion dokumentiert. Docstrings verwenden Triple Quotes (`"""..."""`) und beschreiben, was die Funktion macht, welche Parameter sie hat und was sie zurückgibt.

[!TIP] **Beispiel: Gut dokumentierte Funktion**

Best Practices für Docstrings: 1. Beschreibe, **was** die Funktion macht (nicht **wie**) 2. Liste alle Parameter mit Typ und Bedeutung 3. Beschreibe den Rückgabewert 4. Füge bei Bedarf Beispiele hinzu 5. Erwähne mögliche Exceptions (z.B. `ValueError` bei ungültigen Eingaben)

Praktische Anwendung: Algorithmik mit Funktionen

Funktionen sind das zentrale Werkzeug zur Strukturierung von Algorithmen. Betrachten wir die Sortieralgorithmen aus der Theorie als Python-Implementierungen:

Bubble Sort als Funktion

Binäre Suche als Funktion

Häufige Fehler und Lösungen

[!WARNING] **Fehler 1: Funktion aufrufen, bevor sie definiert ist**

Lösung: Definiere Funktionen immer vor ihrer Verwendung, oder rufe sie innerhalb anderer Funktionen auf (die erst später aufgerufen werden).

[!WARNING] **Fehler 2: return vergessen**

Lösung: Verwende `return`, um Werte zurückzugeben.

[!WARNING] **Fehler 3: Funktion mit und ohne return wechseln**

Lösung: Unterscheide klar zwischen Funktionen, die Werte zurückgeben, und solchen, die nur Seiteneffekte haben.

[!WARNING] **Fehler 4: Mutable Default-Werte**

Lösung: Verwende `None` als Default und erstelle neue Objekte im Funktionskörper:

[!WARNING] **Fehler 5: Falsche Parameterreihenfolge**

Lösung: Positionale Argumente müssen vor Keyword-Argumenten stehen:

Zusammenfassung Python

Funktionen sind das zentrale Werkzeug zur Code-Strukturierung in Python. Sie ermöglichen wiederverwendbaren, lesbaren und testbaren Code. Eine Funktion wird mit `def` definiert, hat einen Namen, kann Parameter entgegennehmen und Werte mit `return` zurückgeben.

Parameter sind Platzhalter in der Funktionsdefinition, Argumente sind die konkreten Werte beim Aufruf. Python unterscheidet zwischen positionalen Argumenten (Reihenfolge wichtig) und Keyword-Argumenten (benannt, Reihenfolge egal). Default-Parameter ermöglichen optionale Argumente mit vordefinierten Werten, müssen aber nach Parametern ohne Default stehen.

Funktionen geben mit `return` Werte zurück. Mehrere Werte können als Tupel zurückgegeben werden. Ohne `return` gibt eine Funktion implizit `None` zurück. Wichtig: `print()` ist kein `return` – `print()` gibt nur aus, aber gibt keinen Wert zurück.

Python-Funktionen sind First-Class-Objekte: Sie können Variablen zugewiesen, als Argumente übergeben und als Rückgabewerte zurückgegeben werden. Dies ermöglicht mächtige Programmier Techniken wie Higher-Order Functions.

Variablen haben einen Scope: Lokale Variablen existieren nur innerhalb der Funktion, globale Variablen sind überall sichtbar. Die LEGB-Regel beschreibt die Reihenfolge der Namensauflösung. Vermeide das `global`-Keyword und bevorzuge Parameter/Rückgabewerte.

Docstrings dokumentieren Funktionen: Sie beschreiben, was die Funktion macht, welche Parameter sie hat und was sie zurückgibt. Gute Dokumentation ist essenziell für wartbaren Code.

Häufige Fehler umfassen vergessenes `return`, Verwechslung von Funktionen mit/ohne Rückgabewert, mutable Default-Werte und falsche Parameterreihenfolge. Diese lassen sich durch sorgfältige Planung und Tests vermeiden.

Neue Python-Funktionen/Methoden

In dieser Vorlesung wurden folgende Python-Konzepte **neu eingeführt**:

Schlüsselwörter und Syntax

- **def-Statement** (Python Keyword)
 - Definiert eine Funktion
 - Syntax: `def funktionsname(parameter):`
 - Signatur: `def name(param1, param2=default):`
 - Beispiel: `def addiere(a, b): return a + b`
- **return-Statement** (Python Keyword)
 - Beendet Funktionsausführung und gibt Wert zurück
 - Syntax: `return wert` oder `return wert1, wert2` (Tupel)
 - Ohne Wert: `return` gibt None zurück
 - Signatur: `return [expression]`
 - Beispiel: `return x ** 2`

Konzepte

- **Funktionsdefinition:** Erstellen wiederverwendbarer Codeblöcke mit `def`
- **Parameter:** Platzhalter in der Funktionsdefinition
- **Argumente:** Konkrete Werte beim Funktionsaufruf
- **Positionale Argumente:** Reihenfolge beim Aufruf ist entscheidend
- **Keyword Arguments:** Benannte Argumente (Reihenfolge egal)
- **Default-Parameter:** Parameter mit Standardwerten (`def func(x=10)`)
- **Mehrere Rückgabewerte:** Return als Tupel (`return a, b`)
- **First-Class-Objekte:** Funktionen als Variablen, Argumente, Rückgabewerte
- **Lokaler vs. Globaler Scope:** Variablen-Sichtbarkeit
- **LEGB-Regel:** Local → Enclosing → Global → Built-in
- **Docstrings:** Funktionsdokumentation mit Triple Quotes

[!NOTE] Lambda-Funktionen, `*args`, `**kwargs`, Type Hints und weitere fortgeschrittene Konzepte werden in V11 (Methoden/Funktionen – Teil 2) behandelt.

Weiterführende Ressourcen

Theorie

- **Introduction to Algorithms** (Cormen et al., MIT Press): Das Standardwerk zur Algorithmik
- **Grokking Algorithms** (Aditya Bhargava): Visuell ansprechende Einführung in Algorithmen
- **Big-O Cheat Sheet:** <https://www.bigocheatsheet.com/> – Übersicht über Komplexitäten
- **VisuAlgo:** <https://visualgo.net/> – Interaktive Visualisierungen von Algorithmen

Python

- **Python Documentation - Defining Functions:** <https://docs.python.org/3/tutorial/controlflow.html#functions>
- **Real Python - Defining Your Own Python Function:** <https://realpython.com/defining-your-own-python-function/>
- **PEP 257 - Docstring Conventions:** <https://peps.python.org/pep-0257/>
- **Python Function Arguments:** <https://www.programiz.com/python-programming/function-argument>

Übungsmaterial

- **LeetCode:** <https://leetcode.com/> – Algorithmus-Challenges (auch für Einsteiger)
 - **HackerRank:** <https://www.hackerrank.com/domains/python> – Python-spezifische Übungen
 - **Project Euler:** <https://projecteuler.net/> – Mathematisch orientierte Programmier-Challenges
-

Datei: lessons/V11-GPTs-LLMs-KI/01_theorie.ipynb

V11 – GPTs, LLMs & KI: Theorie

Lernziele

Nach diesem Notebook kannst du ... - den Begriff **Künstliche Intelligenz (KI)** einordnen und von klassischer Programmierung abgrenzen, - die drei großen Phasen der KI-Geschichte benennen (symbolische KI → Machine Learning → Deep Learning), - **Machine Learning** und **Deep Learning** voneinander unterscheiden, - die Grundidee eines **künstlichen Neurons** und eines **neuronalen Netzes** erklären, - **GPT**, **BERT** und **LLaMA** als LLM-Familien einordnen und Pre-Training vs. Fine-Tuning unterscheiden, - typische **Anwendungen** von LLMs im Ingenieur-Alltag nennen, - **Halluzinationen** und **Bias** als strukturelle Grenzen verstehen und Gegenmaßnahmen nennen.

Zeitbudget

Ca. 30 Minuten. Du darfst die Mathe-Formeln (§4, §5) in Ruhe zweimal lesen.
Ca. 25 Minuten. Du darfst die Mathe-Formeln in §4 in Ruhe zweimal lesen.
TL;DR Ein **LLM** wie ChatGPT ist im Kern eine **Wort-Vorhersage-Maschine**: auf Basis eines riesigen Trainings rät es Token für Token das wahrscheinlichste nächste Wort. Möglich wurde das durch Trainings rät es Token für Token das wahrscheinlichste nächste Wort – möglich gemacht durch **Deep Learning** und den **Transformer** (2017). LLMs sind beeindruckend

flexibel, aber sie **wissen** nichts – deshalb **halluzinieren** sie und übernehmen **Vorurteile** aus ihren Trainingsdaten.

Voraussetzungen

- Python-Recap aus 00_python_recap.ipynb.
- Keine mathematischen Vorkenntnisse über Vektoren hinaus – Formeln sind als Intuition gemeint.

1. Was ist Künstliche Intelligenz?

Künstliche Intelligenz (KI) ist ein Sammelbegriff für Computerprogramme, die Aufgaben erledigen, für die normalerweise menschliche Intelligenz nötig wäre – Texte schreiben, Bilder erkennen, Sprache verstehen, Muster in Daten finden.

Wichtig: KI „denkt“ nicht wie ein Mensch. KI ist ein Werkzeug, das Muster in Daten erkennt und darauf basierend Vorhersagen macht.

Beispiele aus dem Alltag: - **Sprachassistenten** (Siri, Alexa): Sprache erkennen, Antworten geben. - **Navigation** (Google Maps): beste Route unter Staus berechnen. - **Empfehlungen** (Netflix, Spotify): „Das könnte dir auch gefallen“. - **ChatGPT / Copilot:** Texte schreiben, Fragen beantworten, Code generieren.

Für Ingenieur:innen wichtig ist ein zweiter Blickwinkel: KI ist das **Umkehren** der klassischen Programmierung. Statt **Regeln + Daten → Ergebnis** gibt man **Daten + Ergebnisse → Regeln (Modell)** hinein – und lässt die Maschine die Regeln selbst lernen.

Diagramm-Quelle

2. Kurze Geschichte der KI

Die KI-Forschung lässt sich grob in drei Phasen einteilen, die jeweils von einem anderen Grundgedanken geprägt waren.

2.1 Symbolische KI und Expertensysteme (1950er–1980er)

Die frühe KI glaubte, Intelligenz lasse sich durch **logische Regeln** abbilden. **Expertensysteme** wie MYCIN (medizinische Diagnose) oder DENDRAL (chemische Analyse) kodierten menschliches Expertenwissen in Hunderten von „Wenn X, dann Y“-Regeln.

Das funktionierte in eng umrissenen Domänen – und scheiterte kläglich, sobald Sprache, Bilder oder echter Alltag ins Spiel kamen. Es ist schlicht unmöglich, „gesunder Menschenverstand“ in Regeln zu gießen.

2.2 Klassisches Machine Learning (1990er–2010er)

Mit wachsender Rechenleistung setzte sich ein anderer Gedanke durch: **lass die Maschine aus Daten lernen**. Algorithmen wie **Entscheidungsbäume**, **Support Vector Machines (SVMs)** und **Random Forests** wurden zum Standard für Klassifikation und Regression.

Der Haken: Diese Verfahren brauchten **Feature Engineering** – Menschen mussten die relevanten Merkmale (z. B. „Kantenlänge“, „Rauheit“, „Farbwert“) aus den Rohdaten vorab extrahieren. Aufwendig, domänenspezifisch, fehleranfällig.

2.3 Deep-Learning-Revolution (ab 2012)

Drei Faktoren kamen zusammen: **riesige Datensätze** (z. B. ImageNet mit Millionen Bildern), **spezialisierte Hardware** (GPUs, TPUs) und **verbesserte Algorithmen**. **Tiefe neuronale Netze** lernten, relevante Merkmale **selbst** aus Rohdaten zu extrahieren.

Meilensteine: - **2012**: AlexNet gewinnt den ImageNet-Wettbewerb – Durchbruch für Convolutional Neural Networks. - **2014**: Generative Adversarial Networks (GANs) erzeugen realistische Bilder. - **2017**: Das **Transformer**-Modell revolutioniert Sprachverarbeitung. - **2018**: **BERT** setzt neue Maßstäbe im Sprachverständnis. - **2020**: **GPT-3** mit 175 Mrd. Parametern zeigt Few-Shot-Lernen ohne Fine-Tuning. - **2022/2023**: **ChatGPT** und **GPT-4** bringen LLMs in die breite Öffentlichkeit.

Diagramm-Quelle

3. Machine Learning vs. Deep Learning

Deep Learning ist **eine Teilmenge** von Machine Learning, unterscheidet sich aber in mehreren praktischen Aspekten. Die Tabelle fasst das zusammen.

Aspekt	Klassisches ML	Deep Learning
Feature Engineering	manuell, aufwendig	automatisch aus Rohdaten
Datenmenge	Tausende Beispiele reichen	Millionen bis Milliarden
Hardware	Standard-CPU	GPU / TPU
Interpretierbarkeit	oft gut (z. B. Entscheidungsbaum)	meist Black-Box
Trainingszeit	Minuten bis Stunden	Stunden bis Wochen
Stärke bei	Tabellendaten	Bilder, Text, Audio

Faustregel: Strukturierte Tabellendaten und Interpretierbarkeit → klassisches ML. Unstrukturierte Daten (Bild, Text, Audio) und viele Beispiele → Deep Learning.

4. Neuronale Netze – die Grundidee

Ein **neuronales Netz** besteht aus vielen kleinen Recheneinheiten – **Neuronen** – die in Schichten organisiert sind. Der Aufbau ist von biologischen Nervenzellen inspiriert, die Mathematik dahinter ist aber simpel.

4.1 Ein künstliches Neuron

Ein Neuron nimmt mehrere Eingaben $x_1, x_2, \dots, x_n$ entgegen, gewichtet sie mit $w_1, w_2, \dots, w_n$, addiert einen **Bias** b und schickt das Ergebnis durch eine **Aktivierungsfunktion** f :

$$a = f\left(\sum_{i=1}^n w_i x_i + b\right)$$

Die Aktivierungsfunktion bringt **Nicht-Linearität** ins Spiel – erst dadurch kann das Netz komplexe Zusammenhänge lernen. Die drei häufigsten:

- **ReLU**: $f(x) = \max(0, x)$ – Standard in modernen Netzen, einfach und effizient.
- **Sigmoid**: $f(x) = \frac{1}{1 + e^{-x}}$ – Ausgabe zwischen 0 und 1.
- **Tanh**: $f(x) = \tanh(x)$ – Ausgabe zwischen -1 und 1 .

4.2 Vom Neuron zum Netz

Viele Neuronen zusammen bilden ein **Netz** aus drei Arten von Schichten:

- **Input Layer** – nimmt die Rohdaten auf (z. B. Pixelwerte, Messwerte).
- **Hidden Layers** – extrahieren stufenweise abstrakte Merkmale; je tiefer, desto abstrakter.
- **Output Layer** – liefert die Vorhersage (z. B. Klasse, Wahrscheinlichkeit, Zahl).

Jedes Neuron einer Schicht ist mit allen Neuronen der nächsten Schicht verbunden (im einfachsten Fall). Die **Gewichte** dieser Verbindungen sind genau das, was das Netz im Training lernt.

4.3 Wie lernt ein Netz? – Backpropagation

Das Training läuft in vier Schritten, die für jeden Trainingsdatensatz wiederholt werden:

1. **Forward Pass**: Eingabe durchs Netz schicken, Vorhersage berechnen.
2. **Loss berechnen**: Abweichung zur Wahrheit messen (z. B. quadratischer Fehler).
3. **Backward Pass**: Mit der Kettenregel zurückrechnen, wie stark jedes Gewicht am Fehler beteiligt war.

4. **Gewichte aktualisieren:** $w_{\text{neu}} = w_{\text{alt}} - \eta \cdot \frac{\partial L}{\partial w}$ (Gradientenabstieg, η = Lernrate).

Das Ganze läuft über viele **Epochen** (komplette Durchläufe des Datensatzes), bis das Netz konvergiert.

Overfitting: Wird ein Netz zu lange auf zu wenigen Daten trainiert, lernt es die Trainingsbeispiele **auswendig**, statt zu generalisieren. Gegenmittel: mehr Daten, **Dropout**, Regularisierung, Early Stopping.

5. Large Language Models: GPT, BERT, LLaMA

Transformer sind der **gemeinsame Baustein** moderner LLMs. Unterschiedliche Architektur-Varianten zielen auf unterschiedliche Aufgaben.

5.1 GPT – die Wort-Vorhersage-Maschine

GPT (Generative Pre-trained Transformer, OpenAI) ist ein **Decoder-only**-Transformer, trainiert mit **Causal Language Modeling**: das Modell lernt, **das nächste Token** aus allen vorherigen vorherzusagen.

Das Grundprinzip: Ein LLM liest den bisherigen Text und rät das wahrscheinlichste nächste Wort – wieder und wieder, bis ein ganzer Text entsteht.

Beispiel:

```
Eingabe: "Der Himmel ist ___"  
LLM rät: "blau"      (80 %)  
         "bewölkt"   (10 %)  
         "klar"      ( 5 %)  
         ...
```

GPT-Familie im Zeitraffer: - GPT-1 (2018): 117 Mio. Parameter. - GPT-2 (2019): 1,5 Mrd. - GPT-3 (2020): 175 Mrd. – zeigt **Emergent Abilities** (Fähigkeiten, die bei kleinen Modellen fehlen). - GPT-4 (2023): multimodal (Text + Bild), deutlich besseres Reasoning.

Diagramm-Quelle

5.2 BERT – bidirektionales Verstehen

BERT (Bidirectional Encoder Representations from Transformers, Google 2018) ist ein **Encoder-only**-Transformer, trainiert mit **Masked Language Modeling**: 15 % der Tokens werden maskiert, das Modell lernt, sie aus dem **beidseitigen** Kontext zu rekonstruieren.

Dadurch ist BERT stark bei Aufgaben, bei denen der **ganze Satz** verstanden werden muss: Sentiment-Analyse, Named Entity Recognition, Klassifikation,

Question Answering. Für das **Generieren** langer Texte ist BERT weniger geeignet – dafür gibt es GPT.

5.3 LLaMA – Open Source

LLaMA (Meta, 2023) ist eine Familie **Open-Source-LLMs** mit Größen von 7 bis 65 Mrd. Parametern, trainiert ausschließlich auf öffentlichen Daten. Abgeleitete Modelle wie **Alpaca**, **Vicuna** und **Llama 2** haben die Open-Source-Community massiv vorangebracht und ermöglichen Forschung und lokale Deployments, ohne auf OpenAI & Co. angewiesen zu sein.

5.4 Pre-Training, Fine-Tuning, RLHF

Moderne LLMs entstehen in **zwei oder drei Stufen**:

1. **Pre-Training** – unüberwachtes Training auf Milliarden Tokens (Web, Wikipedia, Bücher, Code). Hier lernt das Modell **Sprache und Weltwissen**. Kosten für GPT-3: geschätzt 4–12 Mio. USD.
2. **Fine-Tuning** – gezieltes Training auf kleineren, oft gelabelten Datensätzen für bestimmte Aufgaben (Übersetzung, Frage-Antwort, Code).
3. **RLHF** – *Reinforcement Learning from Human Feedback*: Menschen bewerten Modell-Antworten, das Modell lernt, **bevorzugte** Antworten häufiger zu geben. Genau diese Stufe macht aus dem rohen GPT-3 ein hilfsbereites ChatGPT.

Parameter-effiziente Verfahren wie **LoRA** (Low-Rank Adaptation) erlauben es, LLMs mit nur wenigen Prozent zusätzlicher Parameter an eigene Aufgaben anzupassen – Fine-Tuning wird damit auch auf Consumer-Hardware möglich.

6. Anwendungen von LLMs

LLMs sind **generalistisch** einsetzbar. Für Ingenieur:innen relevant sind vor allem diese fünf Felder:

- **Textgenerierung & Zusammenfassungen** – Wartungsprotokolle, Besprechungsnotizen, Berichte.
- **Übersetzung** – Datenblätter, Normen-Auszüge, E-Mails an internationale Lieferanten.
- **Code-Assistenz** – GitHub Copilot, Code-Reviews, Test-Generierung, Sprach-Migration.
- **Wissensextraktion** – Strukturierte Daten aus Freitext (z. B. Fehlerbeschreibungen → Datenbank).
- **Konversation** – Chatbots für Kundenservice, Tutorsysteme, FAQs.

In der Praxis nutzt man LLMs **im Loop**: Entwurf erzeugen lassen, prüfen, ggf. neu prompten, Fakten gegenchecken, freigeben.

Diagramm-Quelle

7. Halluzinationen – wenn die KI überzeugend lügt

LLMs können **plausibel klingende, aber faktisch falsche** Antworten geben. Dieses Phänomen heißt **Halluzination**.

Halluzination: Das LLM erfindet Informationen, die sich sprachlich gut in den Kontext einfügen, aber nicht stimmen.

Typische Beispiele: - Erfundene **Studien** mit realistisch klingenden Autoren und Journal-Namen. - Falsche **technische Daten:** „Die Zugspitze ist 3.472 m hoch“ (richtig: 2.962 m). - Nicht existierende **Python-Funktionen:** `str.reverse()` gibt es so nicht. - Logisch wirkende, aber **falsche Rechnungen**.

Ursachen: 1. LLMs sind auf *wahrscheinlichen* Text optimiert, nicht auf *wahren* Text. 2. Trainingsdaten enthalten Fehler und Widersprüche. 3. Das Modell hat keinen Live-Zugriff auf aktuelle Fakten. 4. Ein zu vager Prompt zwingt das Modell zum Raten.

Gegenmittel: - **RAG** (*Retrieval Augmented Generation*): relevante Dokumente vor der Antwort in den Prompt laden. - **Quellen anfordern:** „Gib jede Aussage mit Quelle an.“ – hilft, Lücken sichtbar zu machen. - **Menschliche Verifikation** bei allem, was technisch, rechtlich oder medizinisch relevant ist.

Goldene Regel: Vertraue KI-Ausgaben nie blind. Prüfe DIN-Normen, Grenzwerte und Werkstoffkennwerte **immer** gegen eine vertrauenswürdige Quelle.

Diagramm-Quelle

8. Bias – Vorurteile in der KI

Ein LLM übernimmt die **Vorurteile seiner Trainingsdaten**. Da die Daten aus dem Internet stammen, spiegeln sie gesellschaftliche Stereotype wider.

Dokumentierte Beispiele: - Frag nach „einem typischen Ingenieur“ → häufig männliche Beschreibung. - Frag nach „einer Krankenschwester“ → häufig weibliche Beschreibung. - „CEO“ wird tendenziell mit männlichen Pronomen verknüpft, „Sekretär:in“ mit weiblichen. - Westliche Perspektiven dominieren, nicht-westliche Kulturen sind unterrepräsentiert.

Ursprünge: 1. **Trainingsdaten** spiegeln historische Vorurteile. 2. **Label-Bias:** beim Annotieren fließen Vorurteile der Annotator:innen mit ein. 3. **Repräsentations-Ungleichgewicht:** manche Gruppen überrepräsentiert, andere unterrepräsentiert.

Gegenmittel: - Diversere Trainingsdaten und Annotator-Pools. - Automatisierte **Bias-Tests** und **Red Teaming**. - **Debiasing-Techniken** beim Training und Nachtraining. - **Transparenz** über bekannte Limitationen (Model Cards).

Für Maschinenbauer:innen relevant: Wird KI in **Qualitätskontrolle** oder **Predictive Maintenance** eingesetzt, muss man prüfen, ob die Trainingsdaten systematische Verzerrungen enthalten (z. B. nur eine Maschinengeneration, nur eine Schicht, nur eine Werkstoffcharge). Sonst wird der Bias in die Produktion übernommen.

Diagramm-Quelle

9. Ausblick

Weiterführend (optional, nicht prüfungsrelevant)

- **Multimodalität:** LLMs verstehen nicht nur Text, sondern auch Bilder, Audio und Video (GPT-4V, Google Gemini). Anwendung: Bauteil-Foto → Fehlerdiagnose.
- **Agentic AI:** LLMs werden zu **autonomen Agenten**, die Werkzeuge nutzen (Taschenrechner, Web-Suche, APIs) und komplexe Aufgaben in Teilschritten abarbeiten (AutoGPT, LangChain).
- **Effizienz:** Quantisierung, Pruning, LoRA und **On-Device-LLMs** machen Modelle klein genug für Smartphones und Edge-Geräte in der Produktion.
- **Alignment & AGI:** Je leistungsfähiger Modelle werden, desto wichtiger wird die Frage, ob sie menschliche Werte respektieren. **Artificial General Intelligence** – KI auf menschlichem Niveau in allen Bereichen – ist Forschungs- und Debattenthema.

Selbst-Check

Frage 1: Wie funktioniert ein LLM im Kern?

Es ist eine **Wort-Vorhersage-Maschine**: Es liest den bisherigen Text und wählt Token für Token das wahrscheinlichste nächste Wort auf Basis seiner Trainingsdaten. Es „versteht“ den Text nicht, es rechnet mit Wahrscheinlichkeiten.

Frage 2: Was unterscheidet Deep Learning von klassischem Machine Learning?

Deep Learning lernt **Features automatisch** aus Rohdaten, während klassisches ML **manuelles Feature Engineering** braucht. Deep Learning braucht dafür viel mehr Daten und spezialisierte Hardware (GPUs/TPUs), schlägt aber klassische Verfahren bei Bildern, Text und Audio.

Frage 3: Was ist die Grundidee von Self-Attention?

Jedes Token in einem Satz darf **auf alle anderen Tokens gleichzeitig schauen** und entscheiden, welche für sein Verständnis wichtig sind. So werden langreichweitige Bezüge erfasst, und das Training lässt sich parallelisieren.

Frage 4: Was sind Halluzinationen, und warum entstehen sie?

Halluzinationen sind **plausibel klingende, aber falsche** Antworten. Sie entstehen, weil LLMs auf wahrscheinlichen Text optimiert sind, nicht auf Wahrheit – und weil sie keinen Live-Zugriff auf aktuelle Fakten haben.

Frage 5: Warum ist Bias in KI-Modellen ein Problem?

Weil das Modell gesellschaftliche Stereotype und Ungleichgewichte aus seinen Trainingsdaten **reproduziert und verstärkt**. In Anwendungen wie Bewerbungs-Vorauswahl, Kreditvergabe oder auch Qualitätskontrolle kann das zu **systematisch unfairen oder falschen Entscheidungen** führen.

Zusammenfassung

- **KI** = Computerprogramme, die menschenähnliche Aufgaben erledigen; kein „Denken“, sondern Mustererkennung.
- Drei Phasen: **Symbolische KI** → **Machine Learning** → **Deep Learning**.
- **Deep Learning** lernt Features automatisch aus Rohdaten; neuronale Netze bestehen aus Neuronen mit **Gewichten, Bias** und **Aktivierungsfunktion** und werden per **Backpropagation** trainiert.
- Der **Transformer** (2017) mit **Self-Attention** ist die Grundlage moderner LLMs.
- **GPT** (decoder-only, generativ), **BERT** (encoder-only, verstehend), **LLaMA** (open source) sind die bekanntesten Familien.
- Training: **Pre-Training** → **Fine-Tuning** → **RLHF**.
- Grenzen: **Halluzinationen, Bias**, veraltete Daten, Datenschutz – **immer prüfen**.

Nächster Schritt

Weiter mit 02_praxis.ipynb – dort üben wir Python-Funktionen (Default-Parameter, ***args**, ****kwargs**, Docstrings), mit denen wir später LLM-APIs ansprechen können.

Datei: lessons/V12-Prompt-Engineering-Best-Practices/01_theorie.ipynb

V12 – Prompt Engineering: Theorie

Lernziele

Nach diesem Notebook kannst du ... - in eigenen Worten erklären, was ein **Prompt** ist und warum die Formulierung über die Antwortqualität entscheidet, - die **Bausteine** eines guten Prompts benennen (**Rolle, Kontext, Aufgabe**,

Format, Beispiele), - die Prompt-Muster **Zero-Shot**, **Few-Shot** und **Chain-of-Thought** unterscheiden, - den Unterschied zwischen **System-Prompt** und **User-Prompt** erklären, - fünf typische Prompt-Fehler erkennen und vermeiden, - drei konkrete Prompt-Vorlagen aus dem Ingenieur-Alltag (Wartungsprotokoll, Datenblatt, Code-Review) selbst erzeugen, - einen Prompt **iterativ verbessern**, statt den ersten Wurf zu akzeptieren, - die **Grenzen** eines LLM benennen (Halluzinationen, veraltete Daten, Datenschutz).

Zeitbudget

Ca. 30 Minuten. Lies ruhig zweimal – die Bausteine wirst du im Praxis- und Aufgaben-Notebook direkt brauchen.

TL;DR

Ein guter Prompt ist kein Glückstreffer, sondern ein handwerklich sauber zusammengesetzter Text aus **Rolle**, **Kontext**, **Aufgabe**, **Format** und ggf. **Beispielen**. Je präziser du formulierst, desto präziser (und weniger halluziniert) antwortet das Modell. **Prompt Engineering** heißt: diesen Text bewusst bauen, prüfen und iterativ verbessern.

Voraussetzungen

- V11 (was ein LLM ist und wie Token generiert werden).
- Das Recap aus 00_python_recap.ipynb.

1. Was ist ein Prompt?

Ein **Prompt** ist der Text, den du einem Sprachmodell (ChatGPT, Copilot, Claude, Gemini ...) als Eingabe gibst. Formal ist er die Folge von **Tokens**, auf die das Modell seine Antwort Token für Token vorhersagt (das Stichwort kennst du aus V11). Alltagssprachlich: dein **Auftrag** an die KI.

Weil das Modell **nur** aus dem Prompt (und seinem Training) schöpft, entscheidet dein Text darüber, ob du eine klare, brauchbare Antwort bekommst – oder eine vage, falsche oder am Thema vorbei formulierte.

1.1 Warum Prompt-Qualität so wichtig ist

Das Modell rät. Es rät **sehr gut**, aber es rät. Je weniger du vorgibst, desto freier rät es – und desto wahrscheinlicher trifft es nicht das, was du wirklich brauchst. Gibst du dagegen Rolle, Kontext und Format explizit vor, schränkst du den Raum möglicher Antworten ein. Das ist das Grundprinzip von **Prompt Engineering**: Du steuerst das Modell durch sprachliche Präzision.

Eine Faustregel: **Wenn ein Mensch die Aufgabe auch nicht lösen könnte, weil Informationen fehlen, kann sie die KI erst recht nicht lösen.**

2. Die Bausteine eines guten Prompts

Jeder brauchbare Prompt enthält – explizit oder implizit – bis zu fünf Bausteine. Je anspruchsvoller die Aufgabe, desto wichtiger ist, sie **alle** auszuschreiben.

2.1 Rolle

Mit der **Rolle** weist du dem Modell eine Perspektive zu: „Du bist Fertigungsingenieur“, „Du bist Python-Tutor für Anfänger“, „Du bist technischer Redakteur“. Die Rolle wirkt wie ein Filter auf Wortwahl und Detailtiefe – der gleiche Inhalt erklärt als „Tutor für Anfänger“ klingt ganz anders als „Fachkollege in der Konstruktionsabteilung“.

2.2 Kontext

Der **Kontext** sagt, unter welchen Voraussetzungen die Aufgabe zu lösen ist: „Wir fertigen Aluminium-Bauteile in Einzelstückzahlen“, „Der Studierende hat noch keine Schleifen gelernt“, „Die Zielgruppe sind Werkstatt-Mitarbeitende ohne Hochschulabschluss“. Kontext ersetzt Annahmen, die das Modell sonst selbst treffen würde (und oft falsch).

2.3 Aufgabe

Die **Aufgabe** ist der Kern: Was genau soll herauskommen? Formuliere sie mit einem Verb („erkläre“, „fasse zusammen“, „vergleiche“, „extrahiere“), und benenne Gegenstand und Ziel konkret. Statt „Schreib was über CNC“ besser: „Erkläre die Funktionsweise einer 3-Achs- CNC-Fräse anhand eines Beispiel-Werkstücks“.

2.4 Format

Das **Format** beschreibt, wie die Antwort strukturiert sein soll: Länge („maximal 5 Sätze“), Struktur („als Bullet-Liste“, „als Markdown-Tabelle mit Spalten ...“), Sprache („auf Deutsch“), Stil („Fachbegriffe erlaubt, aber jeder einmal erklärt“). Ohne Format bekommst du irgendetwas – meist einen Fließtext, der mittelmäßig zu deinem Zweck passt.

2.5 Beispiele

Beispiele (ein oder mehrere Input-Output-Paare) sind das stärkste Steuerungsmittel, wenn du eine **konsistente Form** brauchst. „Übersetze in einfache Sprache. Beispiel: ‘Spindeldrehzahl’ → ‘wie schnell sich das Werkzeug dreht’. Jetzt: ‘Vorschubgeschwindigkeit’ →“. Das Modell erkennt das Muster und zieht es durch.

2.6 Übersicht als Diagramm

Die fünf Bausteine fließen zu einem Prompt zusammen, aus dem das LLM eine Antwort erzeugt.

Diagramm-Quelle

3. Typische Prompt-Muster

Je nach Aufgabentyp haben sich drei Muster als besonders nützlich herausgestellt. Alle drei lassen sich beliebig kombinieren.

3.1 Zero-Shot

Zero-Shot bedeutet: Du gibst **keine Beispiele**, sondern formulierst nur Rolle/Kontext/ Aufgabe/Format. Das reicht für viele Standardaufgaben: „Fasse diesen Text in 3 Sätzen zusammen“. Der Vorteil: kurzer Prompt. Der Nachteil: bei komplexen Aufgaben rät das Modell mehr.

3.2 Few-Shot

Few-Shot heißt: Du zeigst **zwei bis fünf Beispiele**, wie Eingabe und Ausgabe aussehen sollen, und gibst dann die echte Eingabe. Das Modell erkennt das Muster und ahmt es nach. Gerade bei **Formatierungsaufgaben** (CSV → JSON, Fließtext → Stichpunkte) schlägt Few-Shot Zero-Shot fast immer.

3.3 Chain-of-Thought (CoT)

Bei **Chain-of-Thought** bittest du das Modell explizit, **Schritt für Schritt** zu denken, bevor es das Endergebnis ausgibt. Das verbessert die Qualität bei Rechen-, Logik- und Planungsaufgaben deutlich, weil das Modell seine eigenen Zwischenergebnisse lesen kann. Typische Formulierung: „Denke laut Schritt fuer Schritt. Zeige jede Rechnung. Gib am Ende das Endergebnis in einer Zeile.“

3.4 Vergleich der drei Muster

Diagramm-Quelle

4. System-Prompt vs. User-Prompt

Moderne Chat-Oberflächen kennen zwei Arten von Prompts: den **System-Prompt** und den **User-Prompt**. Der Unterschied ist wichtig, wenn du KI in eigene Anwendungen einbaust oder wenn du z. B. einen „Custom GPT“ konfigurierst.

4.1 System-Prompt

Der **System-Prompt** legt **Rolle und Regeln** fest, die für das **gesamte Gespräch** gelten: „Du bist ein hilfreicher Python-Tutor. Antworte immer auf Deutsch. Gib Code immer als Markdown-Codeblock aus. Erfinde keine Funktionsnamen.“ Der Nutzer sieht ihn meist nicht – er ist die stille Grundhaltung.

4.2 User-Prompt

Der **User-Prompt** ist das, was der Mensch konkret fragt: „Erkläre mir, was ein Dictionary ist“. Er ändert sich in jedem Schritt des Gesprächs, während der System-Prompt konstant bleibt. Das LLM sieht beide, gewichtet den System-Prompt aber tendenziell höher – Regeln dort sind also stabiler als Regeln, die der Nutzer mitten im Chat einstreut.

4.3 Schematische Darstellung

Diagramm-Quelle

5. Typische Prompt-Fehler

Die meisten enttäuschenden KI-Antworten liegen nicht am Modell, sondern am Prompt. Fünf Fehlermuster tauchen besonders oft auf.

5.1 Zu vage

„Erkläre Stahl“ ist keine Aufgabe, das ist ein Stichwort. Das Modell weiß nicht, ob es eine Werkstoffkunde-Einleitung, eine Liste von Stahlsorten oder einen Vergleich mit Aluminium schreiben soll. **Fix:** Verb + konkreter Gegenstand + Zielgruppe + Länge.

5.2 Zu lang, alles auf einmal

„Fasse das Dokument zusammen, übersetze es ins Englische, erstelle daraus Folien, formuliere noch drei Prüfungsfragen und gib mir Python-Code, der das automatisiert.“ In einem Prompt untergehen alle Teilergebnisse. **Fix:** Eine Aufgabe pro Prompt; das Ergebnis einer Runde ist der Input für die nächste.

5.3 Widersprüchlich

„Antworte sehr kurz, aber gehe auf jedes Detail ein.“ Das Modell entscheidet sich für eins von beidem – meist das falsche. **Fix:** Prompt Korrektur lesen und interne Widersprüche auflösen, bevor du ihn abschickst.

5.4 Keine Beispiele bei strukturierten Outputs

Wenn du eine **Tabelle**, ein **JSON-Objekt** oder ein **bestimmtes Template** brauchst, reicht die Beschreibung oft nicht. **Fix:** Ein bis zwei Mini-Beispiele in

den Prompt einbauen (Few-Shot).

5.5 Kein Format vorgegeben

Ohne Formatvorgabe bekommst du Fließtext. Wenn du aber gleich weiterarbeiten willst (Copy-Paste in ein Dokument, Einlesen als CSV), kostet dich das Nacharbeit. **Fix:** Format explizit ansagen („Bullet-Liste mit maximal 5 Punkten“, „Gib die Antwort als einzigen JSON-Block ohne erklärenden Text“).

6. Drei Prompt-Vorlagen aus dem Ingenieur-Alltag

Die folgenden drei Vorlagen sind bewusst **wiederverwendbar** gebaut: Du änderst nur die Platzhalter, das Grundgerüst bleibt.

6.1 Wartungsprotokoll strukturieren

Situation: Ein Kollege hat einen **Freitext-Eintrag** ins Wartungsbuch geschrieben („War heute an der Fraese, Spannfutter hat geeiert, ausgetauscht, laeuft wieder“). Du willst daraus einen **strukturierten Datensatz** machen (Datum, Maschine, Befund, Maßnahme, Status).

6.2 Datenblatt zusammenfassen

Situation: Du hast ein mehrseitiges Datenblatt eines Werkstoffs und willst die **wichtigsten Kennwerte** als Mini-Tabelle. Das ist ein klassischer Fall für einen **Zero-Shot-Prompt mit klarer Formatvorgabe**.

6.3 Code-Review-Prompt

Situation: Du hast ein Python-Snippet aus einer früheren Vorlesung und willst ein **strukturiertes Code-Review**. Hier lohnt sich eine strenge **Formatvorgabe** plus eine **Rolle**, die den Ton setzt.

7. Iterieren und verfeinern

Der erste Prompt ist selten der beste. Gute Ergebnisse entstehen in 2–4 Runden, in denen du den Prompt gezielt schärfst, statt die KI anzubetteln, es „besser“ zu machen.

7.1 Der Iterations-Loop

Drei Fragen helfen bei jeder Runde:

1. **Was fehlt?** Welcher Baustein (Rolle, Kontext, Aufgabe, Format, Beispiel) war nicht präzise genug?
2. **Was stört?** Gibt es einen Satz, der widersprüchlich oder überflüssig ist?
3. **Was zeigt die Antwort?** Enthält sie Halluzinationen, oder interpretiert sie die Aufgabe nur anders als gewünscht?

Diagramm-Quelle

7.2 Vorher/Nachher in drei Runden

Beobachte, wie der Prompt bei jeder Runde einen der Bausteine nachzieht.

8. Grenzen: was Prompt Engineering nicht löst

Auch der beste Prompt beseitigt die **strukturellen Grenzen** eines LLM nicht. Aus V11 kennst du die drei wichtigsten.

8.1 Halluzinationen

Ein LLM erfindet selbstbewusst Fakten, Normen, Quellen. Ein präziser Prompt **reduziert** das, er schaltet es nicht ab. **Regel:** Technische Angaben (DIN-Normen, Grenzwerte, Werkstoffkennwerte) **immer** gegen eine vertrauenswürdige Quelle prüfen.

8.2 Veraltete Daten

Das Modell kennt nur das, was in seinen Trainingsdaten stand. Neue Normen, gerade erschienene Studien oder Firmen-interne Dokumente sind **nicht** Teil des Wissens. Kompensieren kannst du das nur, indem du die relevanten Infos **mit in den Prompt** legst.

8.3 Datenschutz und Geheimnisse

Was du in einen Cloud-Chatbot schreibst, verlässt dein Unternehmen. Konstruktions- zeichnungen, Mitarbeiterdaten, Kundeninformationen gehören **nicht** in Prompts an externe Dienste. Für solche Fälle nutzt man entweder lokale Modelle oder vertraglich abgesicherte Business-Tenants.

Weiterführend (optional)

Moderne Assistenten erweitern den Prompt automatisch um **Tools** (Taschenrechner, Web-Suche) und um **Retrieval Augmented Generation (RAG)**: vor der Antwort werden passende Dokumente gesucht und an den Prompt angehängt. Das Grundprinzip – gute Prompts – bleibt gleich.

9. Best-Practice-Checkliste

Bevor du einen Prompt abschickst, geh kurz diese Liste durch:

- ☐ Rolle ist explizit genannt.
- ☐ Kontext (Zielgruppe, Vorwissen, Randbedingungen) steht im Prompt.
- ☐ Aufgabe beginnt mit einem klaren Verb.
- ☐ Format ist spezifiziert (Länge, Struktur, Sprache).
- ☐ Bei strukturierten Outputs: mindestens ein Beispiel dabei.

- ☐ Keine Widersprüche (z. B. „kurz und ausführlich“).
- ☐ Keine sensiblen Daten enthalten.
- ☐ Ich habe eine Vorstellung, wie ich die Antwort **prüfen** werde.

Selbst-Check

Frage 1: Was sind die fünf Bausteine eines guten Prompts?

Rolle, Kontext, Aufgabe, Format und (optional, aber sehr wirksam) **Beispiele**.

Frage 2: Wann lohnt sich Few-Shot statt Zero-Shot?

Immer dann, wenn du eine **konsistente Form** brauchst (Tabelle, JSON, Übersetzungs-Muster). Zwei bis fünf Beispiele reichen meist.

Frage 3: Was ist der Unterschied zwischen System-Prompt und User-Prompt?

Der **System-Prompt** legt Rolle und Regeln für das ganze Gespräch fest. Der **User-Prompt** ist die konkrete Frage in einer einzelnen Runde. Das Modell gewichtet den System-Prompt tendenziell höher.

Frage 4: Warum löst ein besserer Prompt keine Halluzinationen?

Das Modell rät immer – auch bei einem perfekten Prompt. Bessere Prompts **reduzieren** die Rate, schalten sie aber nicht ab. Technische Fakten immer prüfen.

Frage 5: Woran erkennst du einen zu vagen Prompt?

Er enthält nur ein Stichwort oder eine Ja/Nein-Frage, ohne Rolle, Format oder Zielgruppe. Beispiel: „Erkläre Stahl.“ – das kann alles und nichts bedeuten.

Zusammenfassung

- Ein guter Prompt besteht aus **Rolle + Kontext + Aufgabe + Format (+ Beispiele)**.
- **Zero-Shot**, **Few-Shot** und **Chain-of-Thought** sind drei sich ergänzende Muster.
- **System-Prompt** setzt die Grundhaltung, **User-Prompt** die konkrete Frage.
- Typische Fehler: zu vage, zu lang, widersprüchlich, ohne Beispiele, ohne Format.
- Iteriere: der zweite Prompt ist fast immer besser als der erste.
- Grenzen bleiben: Halluzinationen, veraltete Daten, Datenschutz.

Nächster Schritt

Weiter mit 02_praxis.ipynb – dort bauen wir Prompt-Vorlagen in Python mit **f-Strings** und String-Methoden.

Datei: lessons/V13-Betriebssysteme-Rechnerarchitektur-Teil1/01_theorie.ipynb

V13 – Theorie: Rechnerarchitektur (Teil 1)

Lernziele

Nach diesem Notebook kannst du ... - die **Von-Neumann-Architektur** mit ihren fünf Bausteinen benennen und erklären, - den **Fetch-Decode-Execute-Zyklus** einer CPU in eigenen Worten beschreiben, - die Rolle von **Registern** und der **Speicherhierarchie** (L1/L2/L3, RAM, SSD, HDD) einordnen, - **Daten-, Adress- und Steuerbus** voneinander unterscheiden, - den Unterschied zwischen einer klassischen **CPU**, einem **Mehrkern-Prozessor** und einer **GPU** skizzieren, - begründen, warum diese Hardware-Ebene für **Simulation, Messtechnik und CNC** ingenieursrelevant ist.

Zeitbudget

Rund 35 Minuten.

TL;DR

Ein Rechner besteht aus **CPU, Speicher, Bus-System** und **I/O**. Die CPU führt Befehle im **Fetch-Decode-Execute-Zyklus** aus, greift dafür auf eine hierarchisch gestaffelte Speicherlandschaft zu und kommuniziert über Busse mit Peripherie. Wer diese Grundlagen versteht, kann Performance-Engpässe in Ingenieur-Anwendungen gezielt analysieren.

Voraussetzungen

- Grundkenntnisse Binärzahlen (V01), Fließkommazahlen (V02)
- Python-Kenntnisse V05–V06 (siehe 00_python_recap.ipynb)

Warum Rechnerarchitektur für Ingenieurinnen und Ingenieure?

Moderne Maschinenbau-Anwendungen sind ohne Rechner nicht denkbar. Eine **FEM-Simulation** zur Bauteilauslegung rechnet stundenlang auf spezialisierter Hardware, eine **CNC-Steuerung** muss Befehle in Mikrosekunden an Servomotoren weiterleiten, und eine **Messwerterfassung** an einem Prüfstand liest gleichzeitig hunderte Sensoren aus. All diese Anwendungen teilen sich dieselbe grundlegende Architektur – und wer sie versteht, erkennt schneller, **warum** ein Programm langsam läuft oder **welche** Hardware für eine Aufgabe geeignet ist.

In diesem Notebook steigst du deshalb von der obersten Ebene (Von-Neumann-Modell) schrittweise nach unten, bis hin zur Cache-Hierarchie und zum Bus-System. Wir bleiben dabei bewusst auf einem Niveau, das für die späteren Aufgaben und die Betriebssystem-Kapitel in V14 ausreicht.

1. Kurzer historischer Blick

Die erste Generation elektronischer Rechner (ENIAC, 1945) wurde noch durch manuelles Umstecken von Kabeln programmiert. Jede neue Aufgabe bedeutete tagelangen Umbau, und die physische Trennung zwischen „Programm“ und „Daten“ war eine Selbstverständlichkeit.

Im Jahr 1945 formulierte John **von Neumann** gemeinsam mit Kollegen am Institute for Advanced Study das Konzept eines **gespeicherten Programms**: Die Befehlsfolge sollte nicht länger über Schalter, sondern **als Zahlenfolge im selben Speicher** liegen wie die Daten. Diese Idee ist so einflussreich, dass nahezu jeder heutige Rechner – vom Smartphone bis zum Supercomputer – als Variante dieser Architektur gelten kann.

2. Die Von-Neumann-Architektur im Überblick

Die **Von-Neumann-Architektur** beschreibt ein Rechnerkonzept, in dem **Programm und Daten gemeinsam** in einem einheitlichen Speicher liegen. Die zentrale Recheneinheit (CPU) holt sich sowohl den nächsten Befehl als auch die zu verarbeitenden Werte aus diesem gemeinsamen Speicher. Verbunden sind alle Bausteine über ein zentrales **Bussystem**, das sowohl Adressen als auch Daten und Steuersignale überträgt.

Das nachfolgende Diagramm zeigt die fünf Hauptkomponenten – **Eingabewerk, Ausgabewerk, Speicher, Rechenwerk** und **Steuerwerk** – sowie ihre Verbindung über den **Systembus**.

Diagramm-Quelle: `diagramme/01_von_neumann.mmd`

2.1 Die fünf Bausteine im Detail

Damit aus dem abstrakten Diagramm ein konkretes mentales Modell entsteht, schauen wir uns jeden Baustein einzeln an. Jeder hat eine **klar umrissene Aufgabe** und einen typischen Industriebezug, den du dir als Anker merken kannst.

Eingabewerk

Das **Eingabewerk** überträgt Daten aus der Außenwelt in den Speicher des Rechners. Im Büro-Alltag sind das Tastatur und Maus, im Ingenieur-Kontext dagegen **Messkarten** (z. B. National Instruments oder Beckhoff), die Spannungen von Dehnungsmessstreifen, Thermoelementen oder Beschleunigungssensoren digitalisieren. Alle diese Geräte haben eine Gemeinsamkeit: Sie liefern

einen **Strom von Zahlen**, der in den Hauptspeicher geschrieben wird, bevor die CPU damit rechnen kann.

Ausgabewerk

Analog zum Eingabewerk leitet das **Ausgabewerk** Daten aus dem Rechner nach außen. Das klassische Beispiel ist der Bildschirm, im Maschinenbau sind es jedoch häufig **Aktoren**: Servomotoren einer CNC-Fräse, Magnetventile einer Hydraulikanlage oder Stellglieder eines Regelkreises. Die CPU schreibt dafür Zahlen in eine Speicheradresse, die physikalisch mit dem Ausgabegerät verknüpft ist („memory-mapped I/O“), und das Gerät übersetzt diese Zahl in eine Spannung oder einen digitalen Impuls.

Speicher

Der **Hauptspeicher** (RAM) hält während des Programmlaufs sowohl die Befehle als auch die bearbeiteten Daten. Er ist als lineares Array von **Zellen** organisiert, die jeweils eine Adresse haben und typischerweise **1 Byte** (8 Bit) groß sind. Die CPU adressiert eine Zelle über ihren Index, liest den Inhalt oder schreibt ihn zurück.

[!NOTE] Wenn du im Python-Debugger eine Variable siehst, liegt ihr Wert tatsächlich an einer ganz konkreten Adresse im RAM. Die Funktion `id(x)` aus V03 liefert dir in CPython sogar einen Zeiger darauf.

Rechenwerk (ALU)

Das **Rechenwerk**, oft als **ALU** (Arithmetic Logic Unit) bezeichnet, führt die eigentlichen Berechnungen aus: Addition, Subtraktion, Multiplikation, logische AND/OR/NOT-Verknüpfungen und Vergleiche. Für Fließkomma-Operationen existiert üblicherweise ein zusätzliches **Gleitkommawerk** (FPU), das beispielsweise die in V02 besprochenen IEEE-754-Zahlen in wenigen Nanosekunden multipliziert. Moderne CPUs kombinieren mehrere ALUs und spezielle **Vektoreinheiten** (SIMD, z. B. AVX-512), die dieselbe Operation gleichzeitig auf 8 oder 16 Werte anwenden – essenziell für FEM-Löser und Signalverarbeitung.

Steuerwerk

Das **Steuerwerk** ist der Dirigent der CPU: Es liest Befehle aus dem Speicher, entschlüsselt sie und aktiviert die passenden Leitungen, damit die ALU rechnet, Register gelesen oder geschrieben werden und gegebenenfalls der nächste Befehl aus einer anderen Speicherstelle geholt wird. In modernen Prozessoren ist das Steuerwerk erstaunlich komplex – es beherrscht **Pipelining**, **Sprungvorhersage** und **Out-of-Order-Execution**, um die ALUs möglichst selten leer laufen zu lassen.

Für die Vorlesung genügt die Vorstellung: Das Steuerwerk weiß in jedem Taktzyklus genau, **welcher Befehl gerade dran ist**, und schickt den Rest der CPU entsprechend an die Arbeit.

[!NOTE] **Weiterführend: Harvard- vs. Von-Neumann-Architektur.** In der Von-Neumann-Variante teilen sich Programm und Daten denselben Speicher und denselben Bus. Die **Harvard-Architektur** trennt beides physisch, was in kleinen Mikrocontrollern (z. B. AVR in Arduino-Boards) und digitalen Signalprozessoren häufig vorkommt. Der Vorteil: Befehle und Daten können parallel geholt werden. Der Nachteil: Unflexiblere Speicherverwaltung. Moderne Desktop-CPU's nutzen einen Hybrid – getrennte L1-Caches für Befehle und Daten, aber gemeinsamer Hauptspeicher.

3. Die CPU im Detail

Die **Central Processing Unit** (CPU) ist das Herzstück jedes Rechners und vereint Rechenwerk und Steuerwerk auf einem einzigen Chip. Moderne CPUs enthalten außerdem mehrere Cache-Ebenen, Speicher-Controller und Schnittstellen zu Peripherie-Bussen direkt auf dem Silizium-Die. Was du als Endnutzerin oder Endnutzer kaufst – einen Intel Core i7 oder einen AMD Ryzen 7 – ist also bereits ein kleines System-on-a-Chip.

In diesem Abschnitt konzentrieren wir uns auf die **logische Funktionsweise**: Wie verarbeitet die CPU einen einzelnen Befehl, und warum ist dieser Ablauf für das Verständnis von Performance so zentral?

Der Fetch-Decode-Execute-Zyklus

Jeder Befehl durchläuft in der CPU denselben drei- bis vierstufigen Ablauf, den man **Fetch-Decode-Execute-Zyklus** (oft erweitert um einen Write-Back-Schritt) nennt. Die folgende Abbildung zeigt die vier Phasen und den Schleifencharakter: Sobald ein Befehl fertig ist, wird der **Program Counter** (PC) erhöht und der nächste Befehl geladen.

Diagramm-Quelle: `diagramme/04_cpu_zyklus.mmd`

Im Detail laufen die vier Phasen wie folgt ab:

1. **Fetch** – Das Steuerwerk liest die im Program Counter hinterlegte Adresse, holt den dort liegenden Befehl aus dem Speicher und legt ihn ins **Befehlsregister** (Instruction Register, IR).
2. **Decode** – Der Befehl wird analysiert: Welche Operation (ADD, LOAD, JUMP) ist gemeint? Welche Register oder Speicheradressen sind Operanden?
3. **Execute** – Die ALU führt die eigentliche Rechenoperation aus oder das Steuerwerk initiiert einen Speicherzugriff.

4. **Write-Back** – Das Ergebnis wird zurück in ein Register oder in den Hauptspeicher geschrieben, und der Program Counter wird auf den nächsten Befehl erhöht.

Diese vier Schritte wiederholen sich milliardenfach pro Sekunde – eine CPU mit 3 GHz arbeitet rund 3 Milliarden solcher Zyklen pro Sekunde ab, wobei durch **Pipelining** mehrere Befehle gleichzeitig in unterschiedlichen Phasen bearbeitet werden.

Takt und Taktfrequenz

Die **Taktfrequenz** gibt an, wie viele elementare Zyklen pro Sekunde ablaufen, und wird in Hertz gemessen. Eine Angabe wie **3,6 GHz** bedeutet $3,6 \times 10^9$ Zyklen pro Sekunde. Nicht jeder Befehl ist in einem Zyklus fertig – eine Division benötigt typisch 10–40 Zyklen, ein einfacher Register-Vergleich nur einen.

Die folgende kleine Simulation zeigt dir plastisch, wie viele Operationen moderne CPUs in Sekundenbruchteilen erledigen: Wir messen, wie lange Python für eine Million einfacher Additionen braucht.

4. Register – das schnellste Gedächtnis der CPU

Register sind winzige Speicherplätze **direkt in der CPU**, typischerweise 64 Bit breit. Sie sind so schnell, dass die ALU in einem einzigen Taktzyklus darauf zugreifen kann. Eine typische x86-64-CPU hat rund 16 allgemein nutzbare Register für Ganzzahlen und weitere 16–32 für Fließkomma- und Vektoroperationen.

Wichtige Register-Typen

Auch wenn du in Python selten direkt mit Registern zu tun hast, ist es hilfreich, ihre Rollen zu kennen:

- **Program Counter (PC)** – enthält die Adresse des nächsten auszuführenden Befehls.
- **Instruction Register (IR)** – speichert den aktuell dekodierten Befehl.
- **Akkumulator / Universalregister** – nehmen Zwischenergebnisse auf (heute meistens RAX, RBX, ... auf x86-64).
- **Stackpointer (SP)** – zeigt auf das obere Ende des Call-Stacks und steuert Funktionsaufrufe.
- **Status- oder Flag-Register** – halten einzelne Bits für Ergebnisinformationen wie „Resultat ist Null“, „Überlauf aufgetreten“, „Vorzeichen negativ“.

Warum Register so wichtig sind

Ein Registerzugriff dauert typisch **unter 1 Nanosekunde**, ein RAM-Zugriff dagegen **rund 100 Nanosekunden**. Der Unterschied liegt also bei Faktor 100.

Wenn die ALU in jedem Taktzyklus einen neuen Operanden aus dem RAM holen müsste, liefe sie 99 % der Zeit leer. Genau deshalb versucht der **Compiler** (und bei interpretierten Sprachen wie Python der Python-Interpreter samt JIT oder C-Backend), möglichst viele Operanden in Registern zu halten. In NumPy oder SciPy-Code, der die meisten Ingenieur-Probleme trifft, geschieht das bereits automatisch.

Datei: lessons/V14-Betriebssysteme-Rechnerarchitektur-Teil2/01_theorie.ipynb

V14 – Betriebssysteme Teil 2: Theorie

Lernziele

Nach diesem Notebook kannst du ... - den Unterschied zwischen **Prozess** und **Thread** präzise erklären und Beispiele aus dem Ingenieurs-Alltag zuordnen, - den **Prozess-Lebenszyklus** mit den Zuständen *Neu* → *Bereit* → *Laufend* → *Wartend* → *Beendet* beschreiben, - die wichtigsten **Scheduling-Strategien** (FCFS, SJF, Round-Robin, Prioritäts-Scheduling) benennen und einordnen, wann welche sinnvoll ist, - die Grundidee von **virtuellem Speicher** und **Paging** nachvollziehen und verstehen, was ein Page Fault ist, - die **Speicherhierarchie** (Register → Cache → RAM → SSD → HDD) in ihrer Größenordnung einordnen, - die Struktur eines **hierarchischen Dateisystems** und die Bedeutung der **rwX-Rechte** erklären.

Zeitbudget

Ca. 60 Minuten Lesezeit. Nimm dir Zeit für die Diagramme — sie sind der eigentliche Inhalt, der Text ist nur die Begleitmusik.

TL;DR

Das Betriebssystem jongliert für dich tausende **Prozesse**, teilt CPU-Zeit über einen **Scheduler** zu, simuliert jedem Programm einen eigenen Adressraum über **virtuellen Speicher** und organisiert alle persistenten Daten in einem **Dateisystem-Baum**. Als Ingenieurin oder Ingenieur triffst du diese Konzepte überall, wo mehrere Messungen, Simulationen oder Steuerungen parallel laufen müssen — vom Hintergrund-Logger über den FEM-Solver auf 16 Kernen bis zum Web-Dashboard, das deine Produktionslinie überwacht.

Voraussetzungen

- V13 — Betriebssysteme & Rechnerarchitektur Teil 1 (CPU, RAM, von-Neumann-Architektur)

- V00-Recap dieses Ordners (Schleifensteuerung, List Comprehensions, Listen)

0. Warum brauchen wir ein Betriebssystem?

In V13 hast du die Architektur eines Rechners auf der Hardware-Ebene betrachtet: eine **CPU**, die Instruktionen abarbeitet, ein **RAM** als flüchtiger Arbeitsspeicher, eine **SSD** für persistente Daten sowie die Busse, die beides verbinden. Diese Hardware allein ist aber nicht benutzbar. Sie weiß nichts davon, dass du gleichzeitig einen Python-Datenlogger, einen Browser und einen FEM-Solver laufen lassen willst, und sie hat keine Ahnung, wie aus dem flachen Bytestrom einer SSD eine Datei mit dem Namen `messung_03.csv` werden soll.

Genau diese Lücke schließt das **Betriebssystem** (kurz **OS** für *Operating System*). Es ist die Schicht zwischen Hardware und Anwendung, und es erfüllt im Kern vier Aufgaben: Es verteilt die CPU-Zeit auf alle laufenden Programme, verwaltet den Arbeitsspeicher, bietet ein strukturiertes Dateisystem und steuert den Zugriff auf Ein-/Ausgabegeräte wie Netzwerk, Tastatur und Drucker. Für dich als Anwender fühlt es sich so an, als hätte jedes deiner Programme den Rechner für sich allein — und das ist kein Zufall, sondern das erklärte Designziel jedes modernen Betriebssystems.

In diesem Notebook betrachten wir die vier zentralen Konzepte, die diese Illusion möglich machen: **Prozesse**, **Threads**, **virtueller Speicher** und **Dateisystem**. Alle vier werden dir später in der Praxis wiederbegegnen — spätestens dann, wenn du deinen ersten Mess-Daemon im Hintergrund laufen lassen willst oder der RAM auf deinem Laptop beim nächsten ANSYS-Lauf knapp wird.

1. Prozesse — die ausführbare Einheit

Wenn du ein Programm startest — etwa `python messung.py` — erzeugt das Betriebssystem daraus einen **Prozess**. Ein Prozess ist nicht dasselbe wie das Programm auf der Festplatte: Das Programm ist eine Datei, ein Prozess ist eine konkrete *laufende Instanz* dieser Datei mit eigenen Ressourcen.

[!NOTE] Ein **Prozess** ist eine in Ausführung befindliche Programm-Instanz. Er umfasst den geladenen Programmcode, seinen **eigenen Adressraum** im RAM, einen eigenen **Stack** für Funktionsaufrufe, offene Datei-Handles, Netzwerkverbindungen und eine eindeutige **Prozess-ID (PID)**.

Zwei Prozesse, die aus demselben Programm gestartet wurden, sind trotzdem vollkommen voneinander isoliert. Stürzt einer ab, läuft der andere unbeeinträchtigt weiter. Liest einer an Adresse `0x7fff00a8`, hat das mit dem Speicher des anderen nichts zu tun. Diese Isolation ist die wichtigste Sicherheitsgarantie, die ein modernes OS liefert.

Ingenieur-Beispiel: der Datenlogger im Hintergrund

Stell dir vor, an deinem Prüfstand läuft ein Python-Skript `datenlogger.py`, das alle 100 ms einen Drehmoment-Sensor ausliest und in eine CSV schreibt. Parallel sitzt du selbst an der grafischen Oberfläche und wertest die Daten des Vortages in Jupyter aus. Dritter im Bunde ist vielleicht ein Webserver, der die aktuelle Messung als Live-Chart ins Intranet streamt.

Drei unabhängige **Prozesse**, drei verschiedene PIDs, drei separate Speicherbereiche. Dein Jupyter-Absturz bringt den Logger nicht zu Fall, und wenn du dem Webserver den Saft abdrehst, geht die Messung trotzdem weiter. Genau dafür ist die Prozess-Isolation da — und genau deswegen ist die Frage „wer läuft gerade als welcher Prozess?“ eine der wichtigsten, die du dir in produktiven Umgebungen stellen kannst.

1.1 Der Prozess-Lebenszyklus

Ein Prozess durchläuft während seines Lebens mehrere Zustände. Wenn du ein Programm startest, entsteht er zunächst im Zustand **Neu**. Sobald alle Ressourcen geladen sind, wechselt er nach **Bereit** — er wartet darauf, dass der Scheduler ihm CPU-Zeit zuteilt. Bekommt er die CPU, wechselt er in den Zustand **Laufend**. Von dort kann er dreierlei tun: er läuft zu Ende und wechselt nach **Beendet**; der Scheduler entzieht ihm die CPU und setzt ihn zurück nach **Bereit**; oder er muss auf etwas warten (z. B. Tastatureingabe, Netzwerkantwort, Festplatten-I/O) und wechselt nach **Wartend** / **Blockiert**. Ist die Wartebedingung erfüllt, wandert er aus **Wartend** wieder nach **Bereit** — *nicht* direkt nach **Laufend**, denn erst muss der Scheduler ihm wieder eine Zeitscheibe zuteilen.

Dieser Zyklus ist der Grund, warum ein Texteditor, der auf deine Tastatureingabe wartet, keine CPU-Last erzeugt: er steht im Zustand **Wartend** und verbraucht exakt null Rechenzeit, bis du eine Taste drückst.

Das folgende Diagramm fasst die Zustandsübergänge zusammen. Die Quelle liegt als Mermaid-Datei unter `diagramme/01_prozess_lifecycle.mmd`.

2. Der Scheduler — wer darf wann auf die CPU?

Ein moderner Laptop hat typischerweise 8 oder 16 **Kerne**, aber hunderte Prozesse, die gleichzeitig „laufen“ wollen. Irgendjemand muss entscheiden, welcher Prozess im aktuellen Augenblick die CPU bekommt. Diese Entscheidungen trifft der **Scheduler** — eine Komponente des Betriebssystem-Kerns.

[!NOTE] Der **Scheduler** wählt aus der Menge der bereiten Prozesse denjenigen aus, der als Nächstes die CPU erhält. Die Zeit, für die ein Prozess sie erhält, nennt man eine **Zeitscheibe** (engl. *time slice* oder *quantum*). Typische Werte liegen bei 1–100 ms.

Weil das Umschalten zwischen Prozessen selbst Rechenzeit kostet (ein **Kontextwechsel** muss alle CPU-Register sichern und durch die des neuen Prozesses ersetzen), ist die Wahl der Zeitscheibe ein Balanceakt: Zu kurze Scheiben produzieren zu viel Overhead; zu lange Scheiben machen das System träge und unfair.

Scheduling-Strategien im Überblick

In der Theorie der Betriebssysteme existieren mehrere klassische Strategien, nach denen der Scheduler seine Entscheidungen treffen kann. Jede hat ihre Stärken und Schwächen, und moderne Systeme kombinieren meist mehrere davon.

- **FCFS** (*First Come First Served*) — Prozesse werden in der Reihenfolge ihrer Ankunft bedient. Sehr einfach, aber ein einziger langer Prozess blockiert alle anderen (das sogenannte **Convoy-Problem**).
- **SJF** (*Shortest Job First*) — Der kürzeste bereite Prozess zuerst. Optimal bezüglich mittlerer Wartezeit, aber in der Praxis kaum umsetzbar, weil man die Laufzeit im Voraus kennen müsste.
- **Round-Robin (RR)** — Jeder bereite Prozess bekommt reihum eine feste Zeitscheibe. Extrem fair, typisch für interaktive Systeme wie Linux oder Windows.
- **Prioritäts-Scheduling** — Jeder Prozess trägt eine Priorität, die höchste gewinnt. Wichtig für Echtzeit-Systeme (Messdatenerfassung, Robotsteuerung); Gefahr des **Starvation**, wenn niedrige Prioritäten niemals drankommen.

In der Realität verwenden Linux (*CFS* — *Completely Fair Scheduler*) und Windows hybride Verfahren, die Round-Robin mit dynamisch angepassten Prioritäten kombinieren.

Round-Robin in der Praxis

Round-Robin ist das Arbeitspferd interaktiver Betriebssysteme. Die Idee ist bestechend einfach: Alle bereiten Prozesse werden in eine Warteschlange eingereiht. Der erste bekommt die CPU für genau *eine* Zeitscheibe — zum Beispiel 10 ms. Ist er fertig oder blockiert, nimmt der nächste Prozess seinen Platz ein; ist die Zeitscheibe aber abgelaufen und der Prozess hat noch zu tun, wandert er ganz ans Ende der Warteschlange. So kommt garantiert jeder dran, egal ob fünf oder fünfhundert Prozesse warten.

[!TIP] Öffne auf deinem Rechner einmal den Task-Manager (Windows) oder `htop` (Linux/macOS). Was du dort unter *CPU-Auslastung pro Prozess* siehst, ist direkt das Ergebnis der Scheduler-Entscheidungen der letzten Sekunde. Ein Prozess, der konstant bei 100 % steht, hat in jeder Zeitscheibe voll gerechnet.

[!WARNING] Round-Robin garantiert *Fairness im Durchschnitt*,

aber **keine Echtzeit**. Wenn du eine Robotersteuerung schreibst, die in exakt 1 ms antworten muss, reicht das nicht — dafür gibt es spezielle Echtzeit-Betriebssysteme (RTOS).

Das folgende Diagramm zeigt schematisch, wie ein Scheduler aus einer Ready-Queue nach verschiedenen Strategien auswählt. Quelle: `diagramme/03_scheduler.mmd`.

3. Threads — mehrere Ausführungspfade in einem Prozess

Prozesse sind teuer. Einen neuen Prozess zu erzeugen heißt, einen komplett neuen Adressraum anzulegen, alle Register zu initialisieren und das OS über den neuen Eintrag in der Prozess-Tabelle zu informieren. Wenn du innerhalb desselben Programms zwei Dinge parallel tun willst — etwa einen Sensor lesen *und* gleichzeitig die Werte ins Netz senden — lohnt sich dieser Aufwand nicht. Die leichtgewichtige Alternative heißt **Thread**.

[!NOTE] Ein **Thread** (deutsch: Ausführungsfaden) ist eine parallel ausführbare Einheit innerhalb eines Prozesses. Alle Threads eines Prozesses **teilen sich denselben Adressraum** und damit auch alle Variablen und offenen Dateien. Sie haben aber jeweils einen eigenen **Stack** und eigene Register.

Threads sind der Grund, warum dein Browser gleichzeitig eine Seite rendern, einen Download verarbeiten und ein Video abspielen kann, ohne dass sich die drei Aufgaben gegenseitig blockieren. Jeder der drei läuft in einem eigenen Thread desselben Browser-Prozesses.

Prozess vs. Thread im direkten Vergleich

Merkmal	Prozess	Thread
Adressraum	eigener, isoliert	gemeinsam mit anderen Threads des Prozesses
Speicher pro Instanz	hoch (MB)	niedrig (KB)
Kommunikation untereinander	nur über OS (Pipes, Sockets, Shared Memory)	direkt über gemeinsame Variablen
Erzeugung	teuer	billig
Absturz eines Exemplars	nur dieser Prozess betroffen	ggf. gesamter Prozess stürzt ab
Typische Verwendung	unabhängige Programme	nebenläufige Teilaufgaben

Die letzte Zeile ist der entscheidende Trade-off: Threads sind billig und schnell, aber weil sie sich Speicher teilen, können Fehler in einem Thread (etwa eine Speicher-Korruption) den gesamten Prozess zum Absturz bringen. Prozesse

isolieren stärker — dafür kostet jede Inter-Prozess-Kommunikation den Umweg über das Betriebssystem.

[!WARNING] Zwei Threads, die dieselbe Variable gleichzeitig ändern, können zu einer **Race Condition** führen — einem Fehler, der mal auftritt und mal nicht, je nach Scheduling-Zufall. In Python mildert der *Global Interpreter Lock* (GIL) dieses Problem ab, entfernt es aber nicht vollständig.

Ingenieur-Beispiel: paralleler Mess-Daemon mit Threads

Ein typisches Setup an einem modernen Prüfstand: ein einziger Prozess `messdaemon.py` startet drei Threads. **Thread A** liest im Takt von 1 ms den Kraftsensor. **Thread B** puffert die Werte in einer gemeinsamen Liste und fasst sie sekundenweise zusammen. **Thread C** schickt die Zusammenfassungen alle fünf Sekunden per HTTP an einen Server im Intranet.

Weil alle drei Threads denselben Adressraum teilen, können sie direkt auf einen gemeinsamen Ring-Puffer zugreifen, ohne die Werte über einen Socket oder eine Datei umkopieren zu müssen. Für die FEM-Simulation parallel auf demselben Rechner verwendet man dagegen oft *mehrere Prozesse* (etwa mit **multiprocessing**): Hier wiegen die Isolation und die Nutzung mehrerer CPU-Kerne gleichzeitig (umgeht den GIL) den höheren Kommunikations-Overhead auf.

Das folgende Diagramm zeigt dieses Muster: ein Mess-Prozess mit mehreren Threads, die gemeinsamen Speicher nutzen — daneben ein FEM-Prozess mit eigener Speicherwelt. Quelle: `diagramme/04_prozess_vs_thread.mmd`.

4. Virtueller Speicher und Paging

Wir haben bereits gesagt, dass jeder Prozess einen „eigenen Adressraum“ bekommt. Das klingt zunächst, als würde das OS für jeden Prozess einen Teil des RAM reservieren. In Wirklichkeit ist die Lösung weit eleganter — und für Ingenieurinnen und Ingenieure, die mit großen Simulationsdaten arbeiten, eine essenzielle Grundlage.

[!NOTE] **Virtueller Speicher** ist ein Abstraktionsmechanismus, bei dem jeder Prozess Adressen in einem fiktiven, scheinbar exklusiven Adressraum (z. B. 48 Bit = 256 TB) sieht. Das Betriebssystem und die **Memory Management Unit (MMU)** der CPU übersetzen diese virtuellen Adressen transparent in die tatsächlichen physikalischen RAM-Adressen.

Der große Vorteil: Das OS kann einzelne Teile eines Prozesses sogar auf die SSD auslagern (**Swap**), wenn der RAM knapp wird. Der Prozess merkt davon nichts — außer, dass der Zugriff dann spürbar langsamer wird.

Paging — der Speicher wird in Seiten zerteilt

Technisch teilt das OS den virtuellen und den physikalischen Speicher in gleich große Blöcke zu typischerweise 4 KiB, genannt **Seiten** (engl. *pages*) bzw. **Rahmen** (engl. *frames*). Eine **Page Table** hält für jeden Prozess fest, welche virtuelle Seite gerade in welchem physikalischen Rahmen liegt — oder ob sie auf die SSD ausgelagert wurde.

Greift ein Prozess auf eine Adresse in einer gerade ausgelagerten Seite zu, meldet die MMU einen **Page Fault**. Das OS springt ein, lädt die fehlende Seite von der SSD in einen freien Rahmen und lässt den Prozess weiterlaufen. Dieser ganze Vorgang ist transparent — der Prozess sieht nur einen etwas langsameren Speicherzugriff.

[!WARNING] Wenn zu viele Prozesse zu viel Speicher wollen und das OS ständig Seiten zwischen RAM und SSD hin- und herschauft, spricht man von **Thrashing**. Das System wird dabei dramatisch langsam, obwohl die CPU-Auslastung niedrig bleibt — ein klassisches Symptom, das jede Simulations-Ingenieurin schon einmal erlebt hat, wenn das FEM-Modell knapp über die RAM-Grenze hinausging.

Das folgende Diagramm zeigt den Übersetzungspfad einer virtuellen Adresse: MMU schaut in der Page Table nach, findet den Rahmen im RAM — oder triggert bei Valid=0 einen Page Fault, nach dem das OS die Seite von der SSD nachlädt. Quelle: `diagramme/05_paging.mmd`.

4.1 Speicherhierarchie — Größenordnungen im Vergleich

Virtueller Speicher funktioniert nur deswegen gut, weil die Speicherebenen eines Rechners dramatisch unterschiedlich schnell sind. Je näher der Speicher an der CPU liegt, desto schneller ist er zugreifbar — und desto teurer und kleiner ist er.

Die Hierarchie umfasst typischerweise fünf Ebenen: **Register** im CPU-Kern (Bruchteile einer Nanosekunde), gefolgt von mehreren **Cache-Stufen** (L1 1 ns, L2 3 ns, L3 10 ns), dann der **Hauptspeicher (RAM)** im Bereich von 100 ns, darunter die **SSD** bei 50 μ s und ganz unten die **HDD** bei ca. 5 ms. Jede Ebene ist ungefähr um einen Faktor 10–100 langsamer als die darüber — und gleichzeitig größer. Ein L1-Cache fasst vielleicht 64 KiB, eine SSD fast beliebig viel.

Aus dieser Hierarchie folgt eine Faustregel für effizienten Code: *Wenn du etwas oft brauchst, halte es nahe an der CPU*. Das ist der eigentliche Grund, warum datenlokale Algorithmen — die nacheinander auf benachbarte Speicherstellen zugreifen — so viel schneller sind als sprunghafte. Cache-Effekte können einen FEM-Löser um Faktor 5 bis 10 beschleunigen, ohne dass der Algorithmus inhaltlich geändert wurde.

Quelle des Diagramms: `diagramme/06_speicherhierarchie_os.mmd`.

5. Das Dateisystem

Die CPU-Konzepte waren der eine Aufgabenbereich des Betriebssystems. Der andere, für dich als Anwender mindestens so sichtbare Bereich ist das **Dateisystem**. Hier kümmert sich das OS darum, dass aus dem flachen Byte-Strom einer SSD oder Festplatte eine hierarchische Struktur aus Ordern und Dateien wird, und dass Zugriffsrechte durchgesetzt werden.

5.1 Baum-Struktur

Jedes moderne Dateisystem ist ein **Baum**. An der Spitze steht ein einzelner Wurzelknoten (`/` unter Linux/macOS, `C:\` unter Windows). Unterhalb liegen **Verzeichnisse** (auch *Ordner*), die wiederum Verzeichnisse oder **Dateien** enthalten. Dateien sind Blätter des Baums, Verzeichnisse innere Knoten.

Ein typischer Pfad wie `/home/student/fertigung/messdaten.csv` ist nichts anderes als eine Wegbeschreibung durch diesen Baum, beginnend an der Wurzel. Unter Windows mit `C:\Users\Anna\messung.csv` funktioniert das gleich — nur mit Backslashes und einem laufwerksspezifischen Wurzelknoten.

[!TIP] Python glättet diese Windows-vs-Linux-Unterschiede für dich, wenn du `os.path.join` oder (moderner) `pathlib.Path` benutzt. Niemals solltest du Pfade per String-Konkatenation zusammenbauen — das bricht spätestens beim ersten Plattformwechsel.

5.2 Zugriffsrechte: das `rwX`-Modell

Unter Unix-artigen Systemen trägt jede Datei und jedes Verzeichnis drei Gruppen von **Rechten**: Lesen (**r**), Schreiben (**w**) und Ausführen (**x**). Die drei Rechte werden getrennt für *Besitzer*, *Gruppe* und *alle anderen* festgelegt. Eine Datei mit den Rechten `rwXr-Xr--` ist vom Besitzer voll nutzbar, von seiner Gruppe les- und ausführbar, von allen anderen nur lesbar.

Für *Verzeichnisse* bedeuten die Rechte etwas leicht anderes: **r** erlaubt das *Auflisten* des Inhalts, **w** das Anlegen und Löschen von Einträgen, **x** das *Betreten*. Ein Verzeichnis ohne **x**-Recht ist wie ein verschlossener Raum — man sieht vielleicht, was drin steht, aber kann nicht hineingehen.

Unter Windows existiert ein ähnliches, aber feingranulareres Modell über **Access Control Lists (ACLs)**, das aber denselben Grundgedanken umsetzt: *Wer darf was mit welchem Objekt tun?*

Quelle des Baum-Diagramms: `diagramme/02_dateisystem_baum.mmd`.

6. Zwei Ingenieur-Beispiele zum Einordnen

Um die bisher abstrakt erklärten Konzepte fest zu verankern, schauen wir uns zwei sehr reale Szenarien aus dem Maschinenbau-Alltag an. In beiden treffen Prozesse, Threads, Scheduler und Dateisystem zusammen.

Beispiel 1: Datenlogger als Hintergrund-Prozess

Stell dir vor, du willst einen Temperatursensor über 24 Stunden aufzeichnen. Du startest dein Python-Skript `datenlogger.py` und willst danach den Rechner *trotzdem* weiter normal benutzen können. Das Skript wird dafür als eigener **Prozess** gestartet, typischerweise mit niedriger Priorität. Der **Scheduler** gibt ihm regelmäßig kleine Zeitscheiben, damit er alle Sekunde eine Messung machen kann. Die CPU-Auslastung bleibt dabei praktisch bei 0 %, weil der Prozess die meiste Zeit im Zustand **Wartend** (auf den Sensor-Timer) ruht.

Der Datenlogger öffnet eine Logdatei im Dateisystem und schreibt regelmäßig neue Zeilen. Weil die Datei vom Betriebssystem gepuffert wird, geschieht die eigentliche Festplatten-Schreibung nur alle paar Kilobyte — ein schönes Beispiel dafür, wie das OS im Hintergrund optimiert, ohne dass dein Code etwas davon mitbekommt.

Wird zwischendrin der Arbeitsspeicher knapp, weil du parallel ein Video schaust und der RAM fast voll ist, wandern Teile deines Logger-Speichers möglicherweise per **Paging** auf die SSD. Beim nächsten Zugriff entsteht ein kurzer **Page Fault**, und die Seite kommt zurück in den RAM. Der Logger merkt davon nichts — außer einer minimalen Verzögerung.

Beispiel 2: FEM-Simulation auf vielen Kernen

Nun der Gegenpol: eine Finite-Elemente-Simulation mit 10 Millionen Knoten. Hier willst du *maximale* CPU-Last — und zwar auf allen 16 Kernen gleichzeitig. Typisch startet der Solver **mehrere Prozesse** (oft einen pro Kern) und teilt das Gitter zwischen ihnen auf. Innerhalb jedes Prozesses laufen wiederum **Threads**, die stark parallelisierte Matrizen-Operationen ausführen.

Der **Scheduler** hat es hier einfach: so viele Prozesse wie Kerne, alle rechnen, keine wartet. Kritisch wird der **Speicher**: 10 Millionen Knoten mit je ein paar Freiheitsgraden belegen schnell 20 GiB RAM. Passt das nicht mehr rein, setzt **Thrashing** ein — und deine Simulation wird um Faktor 100 langsamer. Die wichtige Lehre aus dieser Beobachtung: kenne deine RAM-Grenze, bevor du den Solver startest.

Die Zwischenergebnisse landen währenddessen im Dateisystem, oft in einem dedizierten Arbeitsverzeichnis mit entsprechenden **rwX-Rechten** — schließlich soll niemand sonst auf deinem Rechencluster in deine Zwischendateien schauen können.

7. Selbst-Check

Prüfe dein Verständnis mit den folgenden Fragen. Klicke auf „Antwort anzeigen“, um die Lösung zu sehen — aber erst, nachdem du selbst eine Antwort formuliert hast.

Frage 1: Worin besteht der fundamentale Unterschied zwischen einem Programm und einem Prozess?

Ein Programm ist eine *statische Datei* auf der Festplatte — Bytecode, der ausgeführt werden *könnte*. Ein Prozess ist eine konkrete *laufende Instanz* dieses Programms im Arbeitsspeicher, mit eigener PID, eigenem Adressraum, eigenem Stack, offenen Datei-Handles und eigenem Zustand im Scheduler.

Frage 2: In welchen Zustand wechselt ein Prozess, der gerade auf eine Tastatureingabe wartet — und warum verbraucht er dabei keine CPU?

Er wechselt in den Zustand **Wartend** / **Blockiert**. Der Scheduler vergibt CPU-Zeit nur an Prozesse im Zustand **Bereit**. Ein wartender Prozess wird vom Scheduler bewusst übergangen, bis die Wartebedingung erfüllt ist (Eingabe kommt an), worauf ihn das OS zurück nach *Bereit* setzt.

Frage 3: Warum ist Round-Robin für einen Echtzeit-Regelkreis mit 1 ms Antwortzeit nicht geeignet?

Round-Robin garantiert Fairness, aber **keine obere Schranke für die Antwortzeit**. Sind z. B. 50 andere Prozesse bereit, kann es 50 Zeitscheiben dauern, bis unser Regler wieder die CPU bekommt. Für harte Echtzeitbedingungen braucht man einen RTOS-Scheduler mit festen Prioritäten und deterministischen Antwortzeiten.

Frage 4: Zwei Threads eines Prozesses arbeiten parallel auf derselben Liste. Welche Gefahr droht, und was hat sie mit dem gemeinsamen Adressraum zu tun?

Es droht eine **Race Condition**. Weil beide Threads denselben Adressraum und damit dieselbe Liste sehen, können gleichzeitige Schreibzugriffe zu inkonsistenten Zwischenzuständen führen — z. B. verlorene Einträge oder korruptierte interne Datenstrukturen. Man braucht Synchronisations-Primitive wie Locks, um den kritischen Bereich zu schützen.

Frage 5: Was passiert technisch bei einem Page Fault, und warum ist er meistens kein Fehler?

Bei einem Page Fault versucht ein Prozess auf eine virtuelle Seite zuzugreifen, die momentan nicht im physikalischen RAM liegt. Die MMU löst eine Ausnahme aus, das OS übernimmt, lädt die Seite von der SSD (oder initialisiert sie neu, falls sie noch nie gelesen wurde) in einen freien Rahmen und setzt den Prozess fort. Das ist kein Fehler im Programmiersinn, sondern ein normaler Mechanismus des virtuellen Speichers. Nur wenn die Seite überhaupt nicht existieren darf (Null-Pointer, Zugriff außerhalb des Adressraums), ist es ein echter Absturz-Grund.

Frage 6: Warum ist ein Zugriff auf den L1-Cache ungefähr 100-mal schneller als auf den RAM, obwohl beide „elektronischer Speicher“ sind?

L1-Caches sitzen *direkt im CPU-Kern* und sind mit SRAM-Zellen gebaut, die in wenigen Takten ansprechbar sind. Der RAM-Chip hängt am Speicherbus, muss Zeilen in DRAM-Matrizen aktivieren, und die Signallaufzeit über Platinen-Leiterbahnen ist bereits physikalisch nicht unter ein paar Nanosekunden zu bringen. Die Hierarchie gibt es genau deswegen: schnell muss nahe sein, nahe muss klein sein.

Frage 7: Ein Ordner hat die Rechte `r-xr-x—`. Was darfst du als nicht-Besitzer, aber Mitglied der Gruppe, damit tun?

Du darfst den Ordner **betreten** (`x`) und den Inhalt **auflisten** (`r`), aber keine Dateien darin anlegen, umbenennen oder löschen (das wäre `w`). Einzelne Dateien darin könntest du durchaus lesen — vorausgesetzt, die Dateien selbst gewähren das entsprechende Recht.

Frage 8: Warum startet ein FEM-Solver typischerweise mehrere Prozesse und nicht nur viele Threads innerhalb eines Prozesses?

In Python gibt es den **GIL** (Global Interpreter Lock), der verhindert, dass mehrere Threads gleichzeitig Python-Bytecode ausführen — bei reiner CPU-Last bringt Threading dann keinen Gewinn auf mehreren Kernen. Auch in nativen Sprachen ist die Prozess-Isolation für rechenlastige, unabhängige Arbeits-Pakete oft robuster: ein Absturz eines Prozesses beeinträchtigt nicht die anderen, und das OS kann sie sauber auf verschiedene Kerne verteilen.

Zusammenfassung

- Ein **Prozess** ist eine in Ausführung befindliche Programm-Instanz mit eigenem, isoliertem Adressraum und eigener PID.
- Prozesse durchlaufen den Lebenszyklus *Neu* → *Bereit* → *Laufend* → *Wartend* → *Beendet* unter Regie des **Schedulers**.
- Gängige Scheduling-Strategien sind **FCFS**, **SJF**, **Round-Robin** und **Prioritäts-Scheduling**; moderne Systeme kombinieren diese.
- **Threads** sind leichtgewichtige Ausführungspfade innerhalb eines Prozesses; sie teilen sich den Adressraum und damit alle Variablen — mit allen Vor- und Nachteilen, die daraus folgen.
- **Virtueller Speicher** und **Paging** verschaffen jedem Prozess den Anschein, den gesamten Speicher für sich zu haben. Ein **Page Fault** lädt fehlende Seiten von der SSD.
- Die **Speicherhierarchie** (Register < Cache < RAM < SSD < HDD) bestimmt die Laufzeit realer Programme oft stärker als der Algorithmus selbst.
- Das **Dateisystem** ist ein Baum aus Ordnern und Dateien; **rwX-Rechte** (oder ACLs unter Windows) regeln, wer was darf.

Nächster Schritt

Weiter mit 02_praxis.ipynb — dort lernst du, mit dem `os`-Modul in Python das Dateisystem praktisch zu bearbeiten.

Datei: lessons/V15-Netzwerktechnik-Grundlagen-Protokolle-Teil1/01_theorie.ipynb

V15 – Netzwerktechnik Grundlagen & Protokolle: Theorie

Lernziele

Nach diesem Notebook kannst du ... - erklären, **warum Rechnernetze in Schichten** modelliert werden und welches Problem das löst, - die sieben **OSI-Schichten** und die vier Schichten des **TCP/IP-Stacks** benennen, gegeneinander abgrenzen und typische Aufgaben zuordnen, - **IP-Adressen**, **MAC-Adressen** und **Ports** sicher voneinander unterscheiden und ihre jeweilige Rolle beim Paketversand beschreiben, - den grundsätzlichen **Aufbau eines Netzwerkpakets** (Header/Payload, Kapselung) erklären, - den Zweck von **DNS** in einem Satz erklären, - die industriellen Protokolle **Modbus TCP**, **OPC UA**, **Profinet** und **EtherNet/IP** auf konzeptioneller Ebene einordnen.

Zeitbudget

Ca. 60 Minuten in Ruhe durcharbeiten.

TL;DR

Netzwerke übertragen Daten nicht in einem Rutsch, sondern in kleinen **Paketen** und durch **mehrere aufeinander aufbauende Schichten**, die je eine Teilaufgabe lösen (Verkabelung, Adressierung, Wegewahl, Anwendungsprotokoll ...). In der industriellen Fertigung werden darauf weitere Protokolle aufgesetzt (Modbus TCP, OPC UA, Profinet), die den besonderen Echtzeit- und Robustheitsanforderungen gerecht werden. In diesem Notebook lernst du die konzeptionelle Grundlage – ohne selbst echte Sockets zu programmieren.

Voraussetzungen

- 00_python_recap.ipynb
- Grundverständnis: „Ein Computer hat eine Adresse, Programme tauschen Daten aus , , ,

: , : {}, : [1 , , , , . Ein **Rechnernetz** ist deshalb immer eine sorgfältig abgestimmte Kombination aus Hardware (Kabel, Switches, Netzwerkkarten)

und Software (Protokollen), die zusammen den zuverlässigen Datenaustausch ermöglichen.

[!NOTE] Ein **Protokoll** ist ein verbindliches Regelwerk, das festlegt, **wie** zwei Teilnehmer Daten formatieren, austauschen und bestätigen. Ohne Protokoll wüsste der Empfänger nicht, wo eine Nachricht anfängt, was sie bedeutet und ob sie vollständig angekommen ist.

1.1 Das Grundproblem: Komplexität beherrschen

Die Kommunikation zwischen zwei Maschinen enthält eine überraschend große Zahl an Teilproblemen: Wie erkennen die elektrischen Pegel auf dem Kabel eine Null von einer Eins? Wie finden Daten durch ein Netz mit mehreren Wegen ihren Empfänger? Wie wird sichergestellt, dass alle Pakete in der richtigen Reihenfolge und vollständig ankommen? Wie entscheidet die Anwendung, was die empfangenen Bytes eigentlich bedeuten?

Würde man all das in einer einzigen monolithischen Software lösen, wäre der Code unverständlich und unwartbar. Die Informatik nutzt deshalb ein bewährtes Werkzeug: **Abstraktion durch Schichten**. Jede Schicht löst genau eine Teilaufgabe und stellt der darüberliegenden Schicht eine klar definierte Schnittstelle zur Verfügung. Die höhere Schicht muss nicht wissen, wie die tieferen arbeiten – sie verlässt sich nur auf deren Dienste.

2. Das OSI-7-Schichten-Modell

Das **OSI-Referenzmodell** (Open Systems Interconnection) der ISO ist der Klassiker unter den Schichtenmodellen. Es ist heute **kein real implementiertes System**, aber das gemeinsame Vokabular, mit dem Netzwerktechniker und -technikerinnen über Probleme sprechen. Wer „Layer 2 ist tot , , , , 2 , 3 , 4 , 5 , 6 , 7

: , : null, : {}, : [], : [, diagramme/01_osi_modell.mmd”, encoding=“utf-8”) as f: display(Markdown(f”mermaid\n{f.read()}\n“))

2.1 Merkhilfe und pragmatische Einordnung

Ein verbreiteter Merksatz für die sieben Schichten von unten nach oben lautet „Please Do Not Throw Sausage Pizza Away 5 , , ,

2.2 Was OSI nicht leistet

Das OSI-Modell ist bewusst **abstrakt** gehalten und sagt nichts darüber, wie die einzelnen Schichten konkret zu implementieren sind. Es ist ein **Denkwerkzeug**, kein Bauplan. In der Praxis gibt es daher auch Unschärfen: TLS etwa wird je nach Lehrbuch mal Schicht 6, mal Schicht 5 zugeordnet, und viele Anwendungsprotokolle überspringen die Schichten 5 und 6 schlicht. Lass dich

von solchen Einordnungsdebatten nicht verunsichern – das Modell hilft beim Strukturieren, nicht beim Haarspalten.

3. Der TCP/IP-Stack – das Modell der realen Welt

Das **TCP/IP-Referenzmodell** ist das Schichtenmodell, mit dem das heutige Internet tatsächlich gebaut ist. Es ist **pragmatischer** als OSI und fasst die sieben Schichten in vier zusammen:

1. **Netzzugangsschicht (Link Layer)** – entspricht OSI 1+2. Kümmt sich um das konkrete Übertragungsmedium: Ethernet, WLAN, Glasfaser, Mobilfunk.
2. **Internet-Schicht (Internet Layer)** – entspricht OSI 3. Beherbergt das **Internet Protocol (IP)** in den Varianten IPv4 und IPv6. Hier werden Pakete über Grenzen einzelner Netze hinweg zugestellt.
3. **Transportschicht (Transport Layer)** – entspricht OSI 4. Die beiden wichtigen Vertreter sind **TCP** (verbindungsorientiert, zuverlässig, geordnet) und **UDP** (verbindungslos, schnell, ohne Garantie).
4. **Anwendungsschicht (Application Layer)** – entspricht OSI 5–7 zusammengefasst. Hier leben alle konkreten Protokolle: HTTP, HTTPS, DNS, SMTP, Modbus TCP, OPC UA.

[!NOTE] Das **TCP/IP-Modell** ist nicht nur ein Modell, sondern auch eine Familie tatsächlich spezifizierter Protokolle. Es ist damit konkreter und näher an der Implementierung als OSI.

3.1 TCP vs. UDP – zwei sehr verschiedene Dienste

Auf der Transportschicht stehen zwei grundlegend unterschiedliche Werkzeuge zur Verfügung. **TCP (Transmission Control Protocol)** baut vor dem Senden eine logische Verbindung auf (Handshake), sorgt für die Reihenfolge der Pakete, fordert verlorene Pakete erneut an und regelt die Senderate abhängig von der Netzauslastung. TCP ist damit **zuverlässig, aber mit Overhead** verbunden und eignet sich für Dateien, Web-Zugriffe, Datenbankverbindungen und industrielle Steuerungsprotokolle wie Modbus TCP oder OPC UA.

UDP (User Datagram Protocol) verschickt einzelne Datagramme ohne Verbindungsaufbau und ohne jede Garantie: Pakete können verloren gehen, doppelt ankommen oder in falscher Reihenfolge eintreffen. Dafür ist UDP **extrem schnell und ressourcenschonend** – ideal für Live-Video, Audio, Gaming, DNS-Anfragen oder sensornahe IoT-Protokolle wie MQTT-SN, bei denen ein einzelnes verlorenes Paket weniger wichtig ist als geringe Latenz.

[!WARNING] „UDP ist unzuverlässig

: , : {}, : [3.2

: , : {}, : [4

255 4 3 , 128 , , 8 12 16

```
: , : {}, : [ 4.1 , , 24 , , , 24 24 256
: , : null, : {}, : [], : [ , 192.168.10.100" oktette = [int(x) for x in ip.split(":")]
print(f'Oktette: {oktette}') print(f'Gehört zum Netz 192.168.10.0/24: {oktette[:3]} == [192, 168, 10]')
```

5. MAC-Adressen: die unveränderliche Geräte-Kennung

Eine **MAC-Adresse (Media Access Control)** identifiziert eine Netzwerkkarte auf der Sicherungsschicht. Sie ist 48 Bit lang und wird üblicherweise als sechs hexadezimale Paare geschrieben, etwa 00:1A:2B:3C:4D:01. Die ersten drei Byte sind die **OUI (Organizationally Unique Identifier)** und identifizieren den Hersteller, die letzten drei Byte sind eine herstellerinterne Gerätenummer.

Im Gegensatz zur IP-Adresse, die sich ändert, sobald ein Gerät in ein anderes Netz wandert, ist die MAC-Adresse in der Regel **werkseitig fest** und bleibt gleich, solange die Netzwerkkarte existiert. Deshalb nutzt man MAC-Adressen in industriellen Netzen häufig als eindeutige Kennung für Geräte, z. B. um einem bestimmten Roboter per DHCP-Reservierung immer dieselbe IP-Adresse zuzuteilen.

[!WARNING] MAC-Adressen lassen sich softwareseitig ändern („MAC-Spoofing“)

```
: , : {}, : [ 5.1
```

6. Ports: mehrere Programme auf einem Rechner

Ein Rechner hat eine IP-Adresse, aber meist laufen darauf **viele Programme parallel**, die alle ins Netz kommunizieren wollen: ein Webserver, ein Datenbankserver, ein OPC-UA-Server. Damit die Transportschicht weiß, an welches dieser Programme ein eingehendes Paket gerichtet ist, nutzt sie **Ports** – 16-Bit-Zahlen von 0 bis 65535, die innerhalb eines Rechners wie interne Hausnummern wirken.

[!NOTE] Ein **Port** ist eine Zahl, die zusammen mit der IP-Adresse einen konkreten **Endpunkt** (Socket) auf einem Rechner bezeichnet. Ohne Port wüsste der Empfangs-Stack nicht, welches der laufenden Programme die Daten bekommen soll.

Die **IANA** teilt den Portbereich in drei Zonen ein:

- **Well-Known Ports (0–1023)** – fest zugeordnete Ports für Standarddienste, benötigen auf Unix-Systemen Root-Rechte zum Öffnen.
- **Registered Ports (1024–49151)** – halbwegs gebräuchliche Ports für bekannte Anwendungen, formal bei der IANA registriert.
- **Dynamic / Private Ports (49152–65535)** – freie Ports für kurzlebige Verbindungen, werden vom Betriebssystem automatisch vergeben.

6.1 Wichtige Ports für dich

Port	Protokoll	Dienst
22	TCP	SSH (Fernzugriff)
53	UDP/TCP	DNS
80	TCP	HTTP (Web, unverschlüsselt)
443	TCP	HTTPS (Web, TLS-verschlüsselt)
502	TCP	Modbus TCP (Industrie-SPS)
1883	TCP	MQTT (IoT, unverschlüsselt)
4840	TCP	OPC UA (Standardport)
44818	TCP	EtherNet/IP (Rockwell)

7. Aufbau eines Netzwerkpakets: Header und Payload

Jedes Paket, das über ein Netz läuft, folgt demselben Grundschemata: Zuerst kommen ein oder mehrere **Header**, dann die **Nutzdaten (Payload)**. Die Header enthalten Steuerinformationen (Absender, Empfänger, Reihenfolge, Prüfsumme), die Payload enthält die eigentliche Nachricht der Anwendung.

Durch die Kapselung entsteht eine geschachtelte Struktur: Eine HTTP-Anfrage wird in einen TCP-Segment-Header eingebettet, das wiederum in einen IP-Paket-Header eingebettet, das wiederum in einen Ethernet-Rahmen eingebettet wird. Jeder Header addiert einige Byte Overhead, weshalb die **Nutzdatenrate** immer etwas niedriger ist als die **Bruttorate** des Mediums.

[!TIP] Eine typische Ethernet-MTU (maximale Paketgröße) liegt bei 1500 Byte. Davon gehen etwa 20 Byte für den IP-Header und 20 Byte für den TCP-Header ab, sodass für deine Anwendungsdaten pro Paket rund 1460 Byte bleiben.

7.1 Was steht in einem IP-Header?

Der IPv4-Header enthält unter anderem die **Quell-IP**, die **Ziel-IP**, die **Paketlänge**, ein **Time-to-Live-Feld (TTL)**, das bei jedem Router dekrementiert wird und verhindert, dass Pakete ewig kreisen, sowie eine **Header-Prüfsumme**. Diese Felder zusammen sind rund 20 Byte groß und ermöglichen es jedem Zwischenknoten (Router), das Paket auszuwerten und weiterzuleiten, ohne die Nutzdaten interpretieren zu müssen.

7.2 Was steht in einem TCP-Header?

Der TCP-Header enthält neben **Quell- und Ziel-Port** auch die **Sequenznummer**, die **Bestätigungsnummer**, **Flags** für den Verbindungsaufbau (SYN, ACK, FIN) und ein **Fenstergröße-Feld** zur Flusskontrolle. Die Sequenznummer ist der Mechanismus, mit dem TCP Pakete in der richtigen Reihenfolge wieder zusammensetzt, auch wenn sie auf dem Weg durcheinander geraten sind.

8. DNS – das Telefonbuch des Internets

Menschen merken sich `beispiel.de` leichter als `93.184.216.34`. Das **Domain Name System (DNS)** übersetzt einen menschenlesbaren Namen in eine IP-Adresse. Die Abfrage läuft hierarchisch: Dein Rechner fragt einen lokalen Resolver, der fragt wiederum die zuständigen Nameserver, bis am Ende eine IP zurückkommt – alles üblicherweise innerhalb weniger Millisekunden.

In Industrienetzen spielt DNS eine untergeordnete Rolle, weil Maschinenadressen meist fest konfiguriert sind. Für die Anbindung an übergeordnete IT-Systeme (ERP, MES, Cloud) ist DNS aber genauso relevant wie im klassischen Büro-Netz.

[!NOTE] **DNS** ist selbst ein Anwendungsprotokoll, das meist über UDP auf Port 53 läuft und Namensauflösungen bearbeitet.

9. Industrienetze: Besonderheiten in der Fertigung

Ein Büro-Netz transportiert E-Mails und Webseiten – wenn ein Paket 200 ms später ankommt, merkt es niemand. In einer Produktionsanlage darf ein Not-Aus-Signal **nicht** 200 ms brauchen. Industrielle Netze stellen deshalb ergänzende Anforderungen:

- **Echtzeitfähigkeit** – garantierte, deterministische Latenz (bei Profinet IRT bis in den Mikrosekunden-Bereich),
- **Robustheit** – tolerant gegenüber elektromagnetischer Störung, Staub, Vibration und Temperatur,
- **Diagnosefähigkeit** – jeder Teilnehmer meldet seinen Gesundheitszustand zur frühzeitigen Fehlererkennung,
- **Interoperabilität** – Geräte verschiedener Hersteller müssen dieselbe Sprache sprechen.

Die dafür entwickelten Protokolle werden meistens **auf** das klassische TCP/IP **aufgesetzt**, so dass dieselbe Ethernet-Infrastruktur genutzt werden kann:

- **Modbus TCP** – simples Client/Server-Protokoll auf Port 502, 1979 ursprünglich für RS-485 entwickelt, heute in einer TCP-Variante allgegenwärtig in kleinen SPS. Vorteil: Extrem einfach, viele Bibliotheken. Nachteil: Keine Typinformation, keine Verschlüsselung, veraltetes Datenmodell.
- **OPC UA** (Open Platform Communications – Unified Architecture) – moderner Industrie-4.0-Standard, Port 4840. Beschreibt Maschinen als hierarchischen **Informationsraum** aus Knoten und Variablen, bringt Typsystem, Verschlüsselung, Authentifizierung und plattformunabhängige Clients mit.
- **Profinet** – Siemens-geprägter Echtzeit-Standard auf Basis von Ethernet. In der Variante **Profinet IRT** mit Hardware-Zeitsynchronisation für harte Echtzeitanforderungen in Motion Control.

- **EtherNet/IP** – Rockwell-geprägter US-Standard, nutzt das **Common Industrial Protocol (CIP)** über TCP/UDP auf Port 44818.
- **MQTT** – Publish-Subscribe-Protokoll auf Port 1883, beliebt in der Sensorik/IoT wegen extrem geringer Datenmengen.

[!WARNING] Nur weil ein Protokoll **Ethernet als Medium** nutzt, ist es noch kein Büro-Ethernet. Profinet IRT beispielsweise taktet Switches hardwareseitig und verträgt sich nicht beliebig mit allgemeiner IT-Hardware.

9.1 Warum OPC UA die Zukunft ist

OPC UA ist das einzige der industriellen Protokolle, das von Grund auf für die Anforderungen der vernetzten Fabrik entworfen wurde. Es bringt von Haus aus ein objektorientiertes **Informationsmodell** mit, in dem Maschinen ihre Variablen selbst beschreiben („Drehzahl in U/min, Datentyp Double, nur lesbar

, , , , ,

: , : {}, : [10 , , 2. **Transportschicht** – TCP bekommt dieses Byte-Paket und setzt einen Header davor, in dem Ziel-Port 4840 und Quell-Port (z. B. 52341) stehen, dazu Sequenznummern zur Reihenfolgesicherung. 3. **Internet-Schicht** – IP ergänzt einen weiteren Header mit Quell-IP 192.168.10.100 und Ziel-IP 192.168.10.200. 4. **Netzzugangsschicht** – Ethernet fügt schließlich einen Rahmen mit Quell-MAC und Ziel-MAC hinzu und verschickt das Ganze als elektrisches Signal über das Kabel.

Am Empfänger laufen dieselben Schritte rückwärts: Ethernet packt aus und reicht nach oben, IP packt aus und reicht nach oben, TCP rekonstruiert die ursprüngliche Reihenfolge, und der OPC-UA-Server auf Port 4840 interpretiert schließlich die Nachricht.

11. Weiterführend

[!TIP] Wer sich tiefer einlesen möchte, dem seien die Standards **IEEE 802.3 (Ethernet)**, **RFC 791 (IP)**, **RFC 793 (TCP)** und **IEC 62541 (OPC UA)** empfohlen. Diese sind die Primärquellen – alles andere (Wikipedia, Lehrbücher) ist Sekundärliteratur.

12. Selbst-Checks

Beantworte dir die folgenden Fragen im Kopf und klappe erst dann die Musterantwort auf.

1. Wieso teilt man Netzwerke überhaupt in Schichten?

Um die überwältigende Komplexität der Datenkommunikation in handhabbare Teilprobleme zu zerlegen. Jede Schicht löst eine klar abgegrenzte Aufgabe und bietet der darüberliegenden Schicht einen wohldefinierten Dienst an, ohne dass die höhere Schicht Details der tieferen kennen muss.

2. Welche vier Schichten hat der TCP/IP-Stack?

Netzzugang, Internet, Transport, Anwendung.

3. Welcher Unterschied besteht zwischen einer IP-Adresse und einer MAC-Adresse?

Die IP-Adresse ist logisch und wechselt beim Umziehen in ein anderes Netz; die MAC-Adresse ist hardwareseitig (meist) fest und identifiziert die Netzwerkkarte selbst. IP wirkt auf der Vermittlungsschicht über Netze hinweg, MAC nur auf der Sicherungsschicht im lokalen Segment.

4. Welches Protokoll spricht die Maschine, wenn sie Port 502 nutzt?

Modbus TCP – das klassische Industrieprotokoll zur Abfrage von SPS-Registern.

5. Wann ist UDP die bessere Wahl als TCP?

Immer dann, wenn niedrige Latenz wichtiger ist als die garantierte Zustellung einzelner Pakete: Echtzeit-Audio, Video, manche Sensor- und Telemetrie-Protokolle. Verlorene Pakete werden dort gar nicht erst erneut angefordert.

6. Warum hat jedes Paket einen Header und Payload?

Der Header enthält die Steuerinformation (wer, wohin, wie lang, in welcher Reihenfolge, welche Prüfsumme), die Payload die eigentlichen Anwendungsdaten. Ohne Header wüsste weder ein Zwischenknoten noch der Empfänger, was mit dem Paket zu tun ist.

7. Was übernimmt DNS konkret?

Die Übersetzung menschenlesbarer Namen (z. B. **beispiel.de**) in maschinenlesbare IP-Adressen. Ohne DNS müsstest du dir für jeden Dienst die IP merken.

8. Warum kann man OPC UA guten Gewissens als Industrie-4.0-Standard bezeichnen?

Weil OPC UA Plattformunabhängigkeit, ein objektorientiertes Informationsmodell, Authentifizierung, Verschlüsselung und Skalierbarkeit von der Sensorik bis zur Cloud mitbringt – Eigenschaften, die ältere Protokolle wie Modbus TCP nicht von Haus aus haben.

Zusammenfassung

- OSI und TCP/IP sind zwei Schichtenmodelle mit demselben Grundgedanken: Komplexität in klar umrissene Teilprobleme aufteilen.
- IP-Adresse, MAC-Adresse und Port beantworten drei verschiedene Fragen: welches logische Netz, welches Gerät auf dem Kabel, welches Programm auf dem Gerät.
- Pakete bestehen immer aus Header(n) und Payload und werden beim Versenden Schicht für Schicht eingepackt.

- Industrienetze setzen spezielle Protokolle (Modbus TCP, OPC UA, Profinet, EtherNet/IP) auf klassisches TCP/IP auf, um Echtzeit-, Robustheits- und Diagnoseanforderungen zu erfüllen.

Nächster Schritt

Weiter mit 02_praxis.ipynb.

Datei: lessons/V16-Netzwerktechnik-Grundlagen-Protokolle-Teil2/01_theorie.ipynb

V16 – Netzwerktechnik Teil 2: Theorie

Lernziele

Nach diesem Notebook kannst du ... - ein **Pandas-DataFrame** aus CSV- und JSON-Dateien laden und die Struktur mit `head`, `shape`, `dtypes`, `describe` inspizieren, - einzelne Spalten als **Series** ansprechen und mit Aggregations-Methoden (`mean`, `sum`, `max`, `count`) auswerten, - Daten mit **Boolean-Masken** filtern (`df[df["spalte"] > wert]`), - mit **groupby** Gruppen bilden und pro Gruppe aggregieren (Split-Apply-Combine), - eine **Series** direkt mit `.plot()` als Linien- oder Balkendiagramm visualisieren, - erklären, wie eine **HTTP-Anfrage** mit **Statuscodes** (200, 404, 500) funktioniert, - die Grundidee einer **REST-API** und das Zusammenspiel mit dem Datenformat **JSON** beschreiben, - eine hartcodierte JSON-Antwort in ein Python-Dictionary parsen und ein DataFrame daraus bauen.

Zeitbudget

Ca. 45 Minuten in Ruhe durcharbeiten.

TL;DR

Wir verbinden zwei Welten: Auf der einen Seite **Pandas** als Werkzeug, um Tabellen effizient zu analysieren – ein DataFrame ist im Kern eine beschriftete Tabelle, auf der du mit wenigen Zeilen filtern, gruppieren und aggregieren kannst. Auf der anderen Seite die **Netzwerk-Schicht**: Moderne Maschinen liefern ihre Daten heute meistens über eine **HTTP-basierte REST-API** im **JSON-Format**. Wer beides versteht, kann aus einem Maschinen-Endpunkt in wenigen Sekunden einen auswertbaren DataFrame bauen.

Voraussetzungen

- 00_python_recap.ipynb
- V15 (Netzwerk-Grundlagen: OSI/TCP/IP, IP-Adressen, Ports)

1. Warum Pandas?

In V09 hast du gesehen, wie man eine CSV-Datei **zeilenweise** liest und eine Liste von Dictionaries aufbaut. Das funktioniert, ist aber für größere Datensätze mühsam: Schon eine einfache Frage wie „Welcher Sensor-Typ hat die höchste mittlere Temperatur?“ kostet zehn Zeilen Schleifen-Code. **Pandas** macht solche Auswertungen zu Einzeilern – und im Hintergrund laufen die Operationen in hochoptimiertem C-Code deutlich schneller als reine Python-Schleifen.

Pandas ist nicht magisch: Ein DataFrame ist im Kern ein Wörterbuch aus Spalten-Arrays plus einem Index. Wer die drei Kern-Operationen **Laden**, **Filtern**, **Gruppieren** beherrscht, deckt damit 80 % aller Daten-Fragen im Engineering-Alltag ab.

2. DataFrame und Series – die zwei Kernobjekte

Pandas kennt zwei zentrale Datenstrukturen. Eine **Series** ist eine eindimensionale beschriftete Werte-Reihe; du kannst sie dir als eine einzelne Excel-Spalte vorstellen. Ein **DataFrame** ist eine zweidimensionale Tabelle mit mehreren Spalten unterschiedlicher Datentypen; jede Spalte ist intern eine Series. Beide teilen den **Index** (standardmäßig 0, 1, 2, ...), der jede Zeile eindeutig adressiert.

Die folgende Skizze zeigt, wie ein DataFrame aus einer CSV in Spalten-Series zerfällt:

2.1 Ein DataFrame aus CSV laden

Die Funktion `pd.read_csv(pfad)` liest eine komma-getrennte Datei ein, nimmt die erste Zeile als Spaltennamen und rät die Datentypen pro Spalte. Das Ergebnis ist ein DataFrame, das du sofort benutzen kannst.

[!NOTE] Die Funktion `read_csv` hat viele optionale Parameter (`sep`, `decimal`, `encoding`, `parse_dates`), die du brauchst, wenn die Datei nicht UTF-8 / Komma-Standard ist. Für die V16-Testdaten reichen die Defaults.

2.2 Struktur inspizieren – `dtypes`, `describe`, `info`

Bevor du rechnest, schau dir das DataFrame an: Welche Spalten gibt es, welchen Datentyp haben sie, wie sehen typische Werte aus? Diese drei Methoden sind dein Standard-Repertoire.

`describe()` liefert **nur für numerische Spalten** Statistiken (Anzahl, Mittelwert, Std.-Abweichung, Min/Max, Quartile). Das ist der schnellste Weg, auffällige Messwerte zu erkennen: Wenn `max` deutlich über 75 % liegt, hast du wahrscheinlich Ausreißer.

3. Spaltenzugriff und Aggregation

Eine einzelne Spalte sprichst du mit `df["spaltenname"]` an – das Ergebnis ist eine Series. Auf dieser kannst du direkt aggregieren: `.mean()`, `.sum()`, `.max()`, `.min()`, `.count()`, `.std()`. Diese Methoden ignorieren fehlende Werte (NaN) standardmäßig, was in realen Messreihen der Normalfall ist.

3.1 Mehrere Spalten gleichzeitig

Mit einer **Liste von Spaltennamen** holst du dir mehrere Spalten als Sub-DataFrame. Das ist nützlich, um die Sicht vor weiteren Operationen gezielt zu verkleinern.

4. Filtern mit Boolean-Masken

Eine **Boolean-Maske** ist eine Series aus **True/False**-Werten, mit der du ein DataFrame indizieren kannst. Ein Ausdruck wie `df["Temperatur_C"] > 80` liefert genau solche Maske zurück. Setzt du diese in eckige Klammern des DataFrames, behältst du nur die Zeilen, für die die Maske **True** ist.

[!NOTE] Eine **Boolean-Maske** ist ein Filter: Für jede Zeile wird eine Bedingung ausgewertet und das Ergebnis (**True/False**) merkt sich Pandas. Nur Zeilen mit **True** bleiben im Ergebnis übrig.

4.1 Kombinierte Bedingungen

Mit `&` (und), `|` (oder) und `~` (nicht) verknüpfst du mehrere Masken. **Achtung:** Jede Teilbedingung muss wegen der Operator-Vorrangregeln in Klammern stehen.

[!WARNING] Verwende für Masken **nicht** `and/or` – das funktioniert nur für skalare Wahrheitswerte, nicht für Series. Pandas wirft dann einen **ValueError**. Immer `&`, `|`, `~` mit Klammern pro Teil-Bedingung.

5. Gruppieren mit `groupby` – Split-Apply-Combine

`groupby` ist das mächtigste Werkzeug von Pandas. Die Idee heißt **Split-Apply-Combine**: Das DataFrame wird nach einer Schlüsselspalte in Gruppen geteilt, auf jeder Gruppe wird dieselbe Aggregation ausgeführt, und die Ergebnisse werden wieder zu einem Objekt zusammengeführt.

Das klassische Beispiel: Du willst pro Sensor-Typ den mittleren Temperatur-Wert. Statt für jeden Typ eine Boolean-Maske anzulegen, schreibst du eine Zeile.

Das Ergebnis ist eine Series mit den Sensor-Typen als Index und dem Mittelwert als Werte. Du kannst sie weiterverarbeiten wie jede andere Series: Maximum finden, sortieren, plotten.

5.1 idxmax – die Gruppe mit dem höchsten Wert

`.idxmax()` liefert den **Index**, an dem der höchste Wert steht – also den Gruppennamen. Das ist das Standard-Werkzeug für Fragen wie „Welche Kategorie ist am teuersten?“ oder „Welche Linie produziert am meisten?“.

5.2 Zählen pro Gruppe

Für Fragen wie „Wie viele Sensoren pro Status?“ nimmst du `size()` oder `value_counts()`. Beide sind mit einer Zeile erledigt.

6. Einfache Plots mit `.plot()`

Jede Series und jedes DataFrame hat eine `.plot()`-Methode, die direkt auf Matplotlib aufsetzt. Ohne weitere Argumente gibt es ein Liniendiagramm; mit `kind="bar"` oder `kind="hist"` bekommst du Balken oder Histogramm. In Notebooks wird die Grafik automatisch inline angezeigt.

[!TIP] Für einen schnellen Überblick reicht `df["spalte"].plot()`. Erst wenn du Achsenbeschriftungen, Titel oder mehrere Layer willst, lohnt der Griff zu Matplotlib direkt.

7. JSON laden – `read_json` und `json.load`

Pandas kann JSON direkt lesen, wenn die Struktur bereits eine Liste von flachen Objekten ist. Bei verschachtelten JSON-Dateien (wie unseren Wartungsaufträgen mit einer Root-Ebene `"wartungsauftraege": [...]`) ist der sicherste Weg der Zwei-Schritt: erst `json.load`, dann `pd.DataFrame(liste)`.

8. XML in ein DataFrame überführen

Für XML gibt es keine so elegante Einzeiler-Funktion, weil XML freier strukturiert ist als JSON. Der Standard-Ansatz: Mit `xml.etree.ElementTree` durch den Baum iterieren, pro Eintrag ein Dictionary bauen, die Liste der Dictionaries an `pd.DataFrame(...)` übergeben.

Das folgende Muster brauchst du in A4:

9. Netzwerk-Teil: HTTP, Statuscodes, REST

In V15 hast du die Schichten OSI/TCP/IP kennengelernt. Auf der **Anwendungsschicht** läuft heute fast jede Maschinen-zu-Maschinen-Kommunikation über das **Hypertext Transfer Protocol (HTTP)**. Ein Client (dein Python-Skript) sendet eine **Anfrage** (Request) an einen Server und bekommt eine **Antwort** (Response) zurück. Jede Antwort trägt einen **Statuscode**, der in einer Zahl ausdrückt, ob alles geklappt hat.

[!NOTE] Ein **HTTP-Statuscode** ist eine dreistellige Zahl, die den Ausgang der Anfrage beschreibt. Die erste Ziffer kategorisiert grob: 2xx Erfolg, 3xx Umleitung, 4xx Client-Fehler, 5xx Server-Fehler.

9.1 Die drei wichtigsten Codes

Code	Bedeutung	Wer ist schuld?
200 OK	Alles gut, Daten im Body	niemand
404 Not Found	Ressource gibt's nicht	Client (falsche URL)
500 Internal Server Error	Der Server hat Mist gebaut	Server

Wer eine API anspricht, prüft als erstes den Code und erst dann den Body. Ein Body mit HTML-Fehlermeldung hilft keinem Auswerte-Skript.

9.2 REST-APIs und Endpunkte

REST (REpresentational State Transfer) ist kein Protokoll, sondern ein **Architekturstil**, nach dem viele moderne Web-APIs aufgebaut sind. Die Kernidee: Jede **Ressource** (Sensoren, Aufträge, Produkte) bekommt eine eindeutige URL, einen sogenannten **Endpunkt**, und die **HTTP-Methode** entscheidet, was passiert: **GET** liest, **POST** legt an, **PUT** ersetzt, **DELETE** löscht.

Für eine fiktive Fabrik-API könnten die Endpunkte so aussehen:

9.3 requests.get – konzeptionell

Die Python-Bibliothek **requests** macht HTTP-Aufrufe zu einem Einzeiler. Wir führen sie in diesem Notebook **bewusst nicht live** aus (keine Netzwerk-Verbindung nötig), du sollst aber das Muster erkennen:

```
# Nur konzeptionell - wird hier nicht ausgeführt!
# import requests
# antwort = requests.get("https://fabrik.example.com/api/sensors")
# if antwort.status_code == 200:
#     daten = antwort.json()    # JSON → Python-Liste/Dict
#     df = pd.DataFrame(daten)
# else:
#     print("Fehler:", antwort.status_code)
```

Der wichtige Punkt: Das Ergebnis von `.json()` ist ein gewöhnliches Python-Dictionary oder eine Liste davon. Ab da ist alles Pandas-Routine.

9.4 Eine hartcodierte JSON-Antwort parsen

Statt einen echten Server zu fragen, simulieren wir die Antwort. Das Ergebnis eines `requests.get(...).json()`-Aufrufs wäre ein Python-Objekt wie dieses – und genau damit können wir weiterarbeiten, als wäre es live geholt worden.

10. Industrie-Beispiel: OPC-UA-Wertstrom → CSV → Pandas

In einer typischen Produktionshalle läuft folgender Datenpfad: Sensoren melden ihre Werte an einen **OPC-UA-Server**, ein **Historian** (Archiv-Dienst) speichert sie als CSV- oder Parquet-Datei, und das Engineering-Team wertet die Datei offline mit Pandas aus. Vorteil: Die Auswertung belastet den OPC-UA-Server nicht und ist beliebig reproduzierbar.

Die Tabelle im Historian sieht üblicherweise so aus:

Zeitstempel	NodeId	Wert	Qualität
2024-12-09 08:00:01	ns=2;s=Spindel.Temp	78.3	Good
2024-12-09 08:00:02	ns=2;s=Spindel.Drehzahl	8500	Good
...	...	...	...

Mit Pandas kannst du in drei Zeilen „Wie hoch ist die mittlere Spindel-Temperatur pro Stunde?“ beantworten: `read_csv`, Boolean-Maske auf `NodeId`, `groupby` auf die Stunde, `mean` auf `Wert`. Genau diese Schritte übst du in Praxis und Aufgaben.

11. Selbst-Check – aufklappbar

F1: Was ist der Unterschied zwischen einer Series und einem DataFrame?

Eine **Series** ist eindimensional (eine Spalte mit Werten plus Index). Ein **DataFrame** ist zweidimensional (mehrere Spalten unterschiedlicher Typen mit gemeinsamem Index). Technisch ist jede Spalte eines DataFrames eine Series.

F2: Wie filterst du alle Zeilen, in denen `Temperatur_C` größer als 50 ist?

```
df[df["Temperatur_C"] > 50]
```

F3: Welche drei Schritte sind das Split-Apply-Combine-Muster von groupby?

1. **Split**: Das DataFrame wird nach einer Schlüsselspalte in Gruppen geteilt.
2. **Apply**: Auf jede Gruppe wird dieselbe Aggregation angewendet (z. B. `mean`, `sum`).
3. **Combine**: Die Ergebnisse werden zu einer neuen Series/einem neuen DataFrame zusammengefügt.

F4: Welchen HTTP-Statuscode erwartest du bei einer falsch geschriebenen URL?

404 Not Found. Der Server existiert, aber die angegebene Ressource gibt es dort nicht.

F5: Warum nicht `and/or` für Boolean-Masken in Pandas?

and/or erwarten **skalare** Wahrheitswerte. Eine Series ist mehrwertig – Pandas weiß nicht, ob sie als Ganzes wahr ist, und wirft einen **ValueError**. Für elementweise Kombination nutzt du **&**, **|**, **~** und klammerst jede Teilbedingung.

F6: Was liefert `df.groupby("Kategorie")["Preis"].mean().idxmax()`?

Den **Namen der Kategorie** mit dem höchsten Durchschnittspreis. `idxmax` gibt den Index zurück, an dem das Maximum steht.

F7: Warum rufen wir `requests.get` in diesem Kurs nicht live auf?

Weil die Übungen **offline** durchlaufen sollen (keine Abhängigkeit von externen Servern, reproduzierbar). Das Prinzip bleibt identisch: Eine REST-Antwort ist am Ende nur ein Python-Dict/eine Liste – und genau damit arbeiten wir aus den mitgelieferten Testdaten.

F8: Du hast eine verschachtelte JSON-Datei mit einer Root-Ebene. Wie kommst du zum DataFrame?

`json.load` liefert das Python-Objekt. Die relevante Liste adressierst du mit dem Root-Key (`daten["wartungsauftraege"]`) und übergibst sie an `pd.DataFrame(...)`.

Zusammenfassung

- Ein **DataFrame** ist eine Tabelle, eine **Series** eine einzelne Spalte.
- **Laden** über `pd.read_csv`, `pd.read_json`, oder `json.load + pd.DataFrame(...)`.
- **Filtern** mit Boolean-Masken (**&**, **|**, **~**, immer klammern).
- **Gruppieren** mit `groupby("spalte")["zielspalte"].aggregation()` – Split-ApPLY-Combine.
- **Plotten** mit `.plot()` direkt auf Series/DataFrame.
- **HTTP** nutzt Statuscodes: 200 OK, 404 Not Found, 500 Server-Fehler.
- **REST-APIs** liefern meist JSON – `.json()` → Python-Objekt → `pd.DataFrame`.

Nächster Schritt

Weiter mit `02_praxis.ipynb` – Hands-on mit `sensoren_daten.csv`.

Datei: `lessons/V17-Kryptografie-Teil1/01_theorie.ipynb`

V17 – Kryptografie (Teil 1): Theorie

Lernziele

Nach diesem Notebook kannst du ... - die Grundbegriffe **Klartext**, **Geheimtext**, **Schlüssel**, **Verschlüsseln** und **Entschlüsseln** präzise verwenden, - das

Kerckhoffs-Prinzip erklären und auf moderne Verfahren übertragen, - die **Caesar-Chiffre** manuell und per Formel $(pos + k) \% 26$ anwenden, - einen **Brute-Force-Angriff** auf Caesar gedanklich und in Python durchspielen, - **symmetrische** Verfahren (z. B. AES) von **asymmetrischen** Verfahren (z. B. RSA) unterscheiden und das **Hybrid-Verfahren** bei HTTPS einordnen, - den Unterschied zwischen **Block-Chiffre** und **Stream-Chiffre** auf Schlagwortniveau benennen.

Zeitbudget

Ca. 30 Minuten konzentriertes Lesen und Ausprobieren.

TL;DR

Kryptografie sorgt dafür, dass Daten zwischen Alice und Bob vertraulich bleiben. Die **Caesar-Chiffre** ist das einfachste Verfahren, per Brute-Force aber in Sekunden gebrochen. Moderne Verfahren unterscheiden **symmetrische** (beide nutzen denselben Schlüssel, z. B. AES) und **asymmetrische** Verfahren (öffentlicher und privater Schlüssel, z. B. RSA). HTTPS kombiniert beides zu einem **Hybrid-Verfahren**.

Voraussetzungen

- 00_python_recap.ipynb (Strings, %, ord/chr)
- Grundkenntnisse Funktionen/Schleifen (V06, V10)

1. Warum Verschlüsselung? Drei ingenieurnahe Szenarien

Drei Szenarien aus dem Maschinenbau-Alltag machen klar, warum Kryptografie keine reine Theorie-Disziplin ist, sondern eine Kernkompetenz moderner Ingenieurinnen und Ingenieure.

Szenario 1 – Industriespionage. Ein Konstruktionsbüro entwickelt einen neuen Getriebetyp. Die CAD-Zeichnungen werden per E-Mail zwischen Standorten ausgetauscht. Ohne Verschlüsselung liegen sie auf jedem Mailserver im Klartext vor – ein einziger korrupter Administrator genügt, und ein Wettbewerber ist zwei Jahre Entwicklungsvorsprung billig eingekauft.

Szenario 2 – DSGVO und Personendaten. Eine Maschine protokolliert, welcher Mitarbeiter wann welche Schicht gefahren hat. Diese Log-Dateien enthalten **personenbezogene Daten** im Sinne der **DSGVO** und müssen bei Übertragung und Lagerung verschlüsselt sein. Verstoß = Bußgeld bis 4 % des Jahresumsatzes.

Szenario 3 – Maschinenkommandos. Eine CNC-Fräse empfängt Befehle über das Werksnetz. Wenn die Kommandos unverschlüsselt und ungesichert laufen, kann ein Angreifer Vorschübe ändern und damit **Ausschuss erzeugen** oder sogar Menschen gefährden. Die Verschlüsselung schützt hier nicht nur

die Vertraulichkeit, sondern über digitale Signaturen auch die **Integrität** der Kommandos.

Drei Schutzziele auf einen Blick

Die klassische **Kryptografie** verfolgt traditionell drei zentrale Schutzziele, die in der Literatur oft als **CIA-Triade** (Confidentiality, Integrity, Authenticity) abgekürzt werden:

1. **Vertraulichkeit** – Nur Berechtigte können den Inhalt lesen.
2. **Integrität** – Eine Veränderung der Nachricht unterwegs fällt auf.
3. **Authentizität** – Der Empfänger ist sicher, wer der Absender war.

In V17 konzentrieren wir uns auf das erste Ziel, die **Vertraulichkeit**. Die beiden anderen Ziele tauchen in V18 (Hashes, Signaturen) wieder auf.

2. Die Grundbegriffe

Um sauber über Kryptografie sprechen zu können, brauchst du fünf Begriffe, die in jeder Vorlesung und jedem Fachbuch vorkommen:

- **Klartext** (englisch *plaintext*) – Die lesbare Originalnachricht, z. B. "HALLO WELT".
- **Geheimtext** (englisch *ciphertext*) – Die unleserliche, verschlüsselte Version, z. B. "KDOOR ZHOW".
- **Schlüssel** (englisch *key*) – Ein Parameter, der aus Klartext Geheimtext macht (und umgekehrt). Bei Caesar ist der Schlüssel einfach eine Zahl zwischen 1 und 25.
- **Verschlüsseln** – Der Vorgang $Klartext + Schlüssel \rightarrow Geheimtext$.
- **Entschlüsseln** – Der Vorgang $Geheimtext + Schlüssel \rightarrow Klartext$.

Die beiden Personen, zwischen denen die Nachricht läuft, heißen in der Fachliteratur traditionell **Alice** (Senderin) und **Bob** (Empfänger). Die Angreiferin heißt **Eve** (von *eavesdropper*, Lauscherin) oder **Mallory** (aktiver Angreifer).

Der Schlüssel – das eigentliche Geheimnis

Wichtig: **Das Geheime ist nicht der Algorithmus, sondern der Schlüssel.** Bei Caesar wissen alle, wie das Verfahren funktioniert (Buchstaben verschieben). Trotzdem kann Eve die Nachricht nicht lesen, wenn sie den **Schlüsselwert** (also die Anzahl der Verschiebeschritte) nicht kennt. Die Sicherheit hängt ausschließlich daran, dass der Schlüssel geheim bleibt.

Diese Trennung zwischen öffentlichem Verfahren und geheimem Schlüssel ist so fundamental, dass sie einen eigenen Namen hat.

3. Das Kerckhoffs-Prinzip (1883)

Der niederländische Kryptograf **Auguste Kerckhoffs** formulierte bereits 1883 ein Prinzip, das heute in jedem sicheren System gilt:

Ein kryptografisches Verfahren muss auch dann sicher bleiben, wenn alles außer dem Schlüssel öffentlich bekannt ist.

Das klingt erst paradox – warum sollte man einem Angreifer verraten, wie das Verfahren arbeitet? Die Begründung ist pragmatisch: **Ein Algorithmus lässt sich nicht dauerhaft geheim halten**. Er ist in Software implementiert, läuft auf hunderten Rechnern und wird früher oder später reverse-engineered. Der **Schlüssel** hingegen ist nur eine kurze Zahlenfolge, die sich leicht schützen, austauschen und bei Verdacht neu setzen lässt.

Der Gegensatz dazu heißt *Security through Obscurity* – Sicherheit durch Verschleierung. Wer darauf setzt, hat früher oder später ein Problem, denn aus Obscurity wird irgendwann eine veröffentlichte CVE-Meldung.

Konsequenzen für moderne Systeme

Aus Kerckhoffs' Prinzip folgen drei praktische Regeln, an die sich jedes seriöse Security-Team hält:

1. **Nutze nur standardisierte, öffentlich geprüfte Algorithmen** wie AES, RSA, SHA-256. Eigenbau-Krypto ist fast immer angreifbar.
2. **Halte die Schlüssel geheim**, nicht den Code. Quelltext darf offen sein – Schlüssel gehören in ein **Key-Management-System** oder ein Hardware-Sicherheitsmodul (HSM).
3. **Rotiere Schlüssel regelmäßig**. Je länger ein Schlüssel in Gebrauch ist, desto größer die Angriffsfläche.

4. Die Caesar-Chiffre

Die **Caesar-Chiffre** ist das älteste dokumentierte Verschlüsselungsverfahren und geht auf den römischen Feldherrn **Gaius Julius Caesar** zurück, der damit militärische Befehle gegen neugierige Boten schützte. Das Prinzip ist verblüffend einfach: Jeder Buchstabe wird im Alphabet um eine feste Anzahl Stellen nach rechts verschoben.

Bei einer Verschiebung um drei Stellen wird aus A ein D, aus B ein E, aus C ein F und so weiter. Am Ende des Alphabets **rollt die Zählung rund**: Aus X wird A, aus Y wird B, aus Z wird C. Genau dieses „Rundrollen“ ist der Grund, warum wir später den Modulo-Operator brauchen.

Die Verschiebe-Tabelle (Schlüssel $k = 3$)

Für einen Schlüssel $k = 3$ ergibt sich folgende Ersetzungstabelle. Jede Spalte listet den Klartext-Buchstaben oben und den zugehörigen Geheimtext-Buchstaben

unten:

Klartext	A	B	C	D	E	F	G	H	I	J	K	L	M
Geheimtext	D	E	F	G	H	I	J	K	L	M	N	O	P

Klartext	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Geheimtext	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Beachte die drei letzten Spalten: Hier passiert das Umlaufen am Alphabet-Ende.

Die Formel in einer Zeile

Mathematisch ausgedrückt läuft Caesar so: Jedem Buchstaben wird zunächst seine **Position im Alphabet** `pos` zugewiesen (`A=0`, `B=1`, ..., `Z=25`). Dann gilt

$$\text{neue_pos} = (\text{pos} + k) \bmod 26$$

Der Modulo-Operator `% 26` sorgt dafür, dass Werte `26` korrekt auf `0–25` zurückgefaltet werden. Anschließend wird `neue_pos` wieder in einen Buchstaben zurückübersetzt.

Beispiel: "HALLO" + Schlüssel 3

Wir verschlüsseln das Wort `HALLO` Buchstabe für Buchstabe:

Klartext	Position <code>pos</code>	<code>pos + 3</code>	<code>% 26</code>	Neue Position	Geheimtext
H	7	10	10	10	K
A	0	3	3	3	D
L	11	14	14	14	O
L	11	14	14	14	O
O	14	17	17	17	R

Aus `HALLO` wird also `KDOOR`. Die Leerzeichen in längeren Sätzen lassen wir unverändert – sie sind keine Buchstaben `A–Z`.

Caesar live in Python

Die Formel aus dem vorigen Abschnitt übersetzt sich direkt in eine kurze Python-Funktion. Wir arbeiten konsequent mit **Großbuchstaben** und ignorieren alle anderen Zeichen (Leerzeichen, Ziffern, Satzzeichen).

Um einen ganzen Text zu verschlüsseln, wenden wir `caesar_zeichen` Buchstabe für Buchstabe an und sammeln das Ergebnis in einem neuen String.

Entschlüsseln = Verschlüsseln mit negativem Schlüssel

Wenn man um k nach rechts verschoben hat, muss man um k nach links zurückschieben. In der Formel ist das einfach $(pos - k) \% 26$, und Python kann Modulo auch mit negativen Zahlen, sodass wir genauso `caesar_text(geheim, -k)` schreiben dürfen.

5. Warum Caesar unsicher ist: Brute-Force

Die Caesar-Chiffre hat genau **25 mögliche Schlüssel** ($k = 1$ bis $k = 25$; $k = 0$ oder $k = 26$ wären identisch mit dem Klartext). Das bedeutet: Ein Angreifer probiert einfach alle 25 Schlüssel durch und schaut, welches Ergebnis lesbar aussieht. Dieses Vorgehen nennt man **Brute-Force-Angriff** – rohe Rechengewalt statt cleverer Mathematik.

Bei Caesar genügt dafür ein Mensch mit 25 Sekunden Zeit, oder ein Computer mit wenigen Mikrosekunden. Caesar ist damit **kryptografisch nutzlos** und dient heute nur noch als Lehrbeispiel – genau dafür verwenden wir es in dieser Vorlesung.

In der Ausgabe oben siehst du für $k = 3$ sofort den lesbaren Klartext HALLO WELT. Alle anderen 24 Zeilen sind Buchstabensalat. Das zeigt zwei Dinge:

1. **Brute-Force funktioniert bei kleinem Schlüsselraum trivial.**
2. Die Auswahl des richtigen Ergebnisses ist bei reinem ASCII-Text leicht, weil nur eine Zeile sinnvolle deutsche Wörter enthält.

Moderne Verfahren wie AES haben einen Schlüsselraum von 2^{128} oder 2^{256} – dagegen ist auch ein Rechenzentrum chancenlos.

Weiterführend: Eine raffiniertere Variante ist die **Häufigkeitsanalyse**. Wer weiß, dass im Deutschen der Buchstabe E am häufigsten vorkommt, schaut im Geheimtext nach dem häufigsten Buchstaben und kennt sofort den Schlüssel. Das funktioniert bei allen **monoalphabetischen** Chiffren und war über Jahrhunderte das Standardwerkzeug von Kryptoanalytikern.

6. Symmetrisch vs. asymmetrisch – der große Unterschied

Alle modernen Verfahren lassen sich in zwei Familien einteilen, die sich darin unterscheiden, **wie viele Schlüssel im Spiel sind und wer sie kennt**. Der Unterschied hat massive Folgen für Schlüsselverwaltung, Performance und Einsatzgebiet.

Symmetrische Verfahren

Ein **symmetrisches Verfahren** nutzt **einen einzigen Schlüssel K** , den sowohl Alice als auch Bob kennen. Alice verschlüsselt mit K , Bob entschlüsselt

mit K . Die Caesar-Chiffre ist ein (unsicheres) symmetrisches Verfahren, denn der Verschiebewert ist für beide Seiten derselbe.

Analogie Vorhängeschloss. Stell dir einen verschließbaren Koffer vor, für den es nur einen Schlüssel gibt, von dem Alice und Bob jeweils eine Kopie besitzen. Alice packt die Nachricht hinein, schließt ab, Bob schließt auf. Das geht schnell, ist simpel – aber wie bekommt Bob überhaupt die Schlüsselkopie, wenn er in einem anderen Land sitzt?

Praxis-Beispiel: AES. Der *Advanced Encryption Standard* (AES) ist das meistgenutzte symmetrische Verfahren und arbeitet mit Schlüssellängen von 128, 192 oder 256 Bit. AES ist extrem schnell (moderne CPUs haben eigene AES-Instruktionen) und wird überall eingesetzt: Festplattenverschlüsselung, WLAN (WPA2), HTTPS-Datentransfer nach dem Handshake.

Asymmetrische Verfahren

Ein **asymmetrisches Verfahren** nutzt **zwei zusammengehörende Schlüssel**: einen **öffentlichen** (*public key*) und einen **privaten** (*private key*). Was mit dem einen verschlüsselt wurde, lässt sich nur mit dem anderen wieder entschlüsseln. Bob veröffentlicht seinen Public Key für alle Welt, behält den Private Key ausschließlich bei sich.

Analogie Briefkasten. Bobs Briefkasten hat einen Schlitz (öffentlich, jeder kann einwerfen) und eine Klappe mit Schloss (privat, nur Bob hat den Schlüssel). Alice wirft ihre Nachricht ein – rausholen kann sie nur Bob. Der geniale Effekt: **Alice und Bob müssen vorher nie einen gemeinsamen Schlüssel ausgetauscht haben.**

Praxis-Beispiel: RSA. Der Algorithmus von *Rivest, Shamir und Adleman* (1977) beruht darauf, dass das Produkt zweier großer Primzahlen leicht zu berechnen, aber praktisch nicht zu faktorisieren ist. RSA-Schlüssel sind typischerweise 2048 oder 4096 Bit lang – und deutlich langsamer als AES.

Direkter Vergleich

Eigenschaft	Symmetrisch (AES)	Asymmetrisch (RSA)
Anzahl Schlüssel	1 gemeinsamer	2 pro Person (public/private)
Geschwindigkeit	sehr schnell	deutlich langsamer (~1000×)
Schlüssellänge	128–256 Bit	2048–4096 Bit
Schlüsselaustausch	schwierig (Henne-Ei)	einfach (Public Key ist öffentlich)
Einsatzgebiet	große Datenmengen	kleine Daten, Schlüsselaustausch, Signaturen

Aus der Tabelle liest man das **Henne-Ei-Problem** heraus: Symmetrisch ist schnell, aber der Austausch des Schlüssels ist heikel. Asymmetrisch löst den Austausch, ist aber zu langsam für GB-große Dateien. Die Lösung kombiniert beide Welten.

Das Hybrid-Verfahren (z. B. bei HTTPS)

In der Praxis nutzt man ein **hybrides Verfahren**: Zu Beginn einer Verbindung wird per **asymmetrischer** Kryptografie ein zufälliger **Sitzungsschlüssel** ausgehandelt, mit dem dann per **symmetrischer** Kryptografie die eigentlichen Daten schnell verschlüsselt werden.

Bei **HTTPS** / **TLS** läuft das grob so ab:

1. Der Browser holt das TLS-Zertifikat des Servers – darin steht der öffentliche RSA-Schlüssel des Servers.
2. Der Browser würfelt einen AES-Sitzungsschlüssel und verschlüsselt ihn mit dem öffentlichen Server-Schlüssel.
3. Der Server entschlüsselt den AES-Schlüssel mit seinem privaten Schlüssel.
4. Ab jetzt läuft die gesamte Kommunikation per AES – schnell und trotzdem sicher, weil den AES-Schlüssel nur diese beiden Parteien kennen.

Moderne TLS-Versionen (1.3) ersetzen RSA durch **Diffie-Hellman mit elliptischen Kurven (ECDHE)**, das Grundprinzip bleibt aber dasselbe: asymmetrischer Austausch, dann symmetrische Massendaten.

7. Block- versus Stream-Chiffre (Kurzüberblick)

Innerhalb der symmetrischen Welt unterscheidet man zwei Bauarten:

- Eine **Block-Chiffre** verarbeitet den Klartext in Blöcken fester Größe (AES z. B. 128 Bit = 16 Byte pro Block). Kleinere Reststücke werden mit *Padding* aufgefüllt. Beispiele: AES, DES (veraltet), 3DES (veraltet).
- Eine **Stream-Chiffre** verschlüsselt Bit für Bit bzw. Byte für Byte, indem sie aus dem Schlüssel einen Pseudozufallsstrom ableitet und diesen per XOR mit dem Klartext kombiniert. Beispiele: ChaCha20, RC4 (veraltet).

In V17 implementieren wir keine dieser Familien – die Caesar-Chiffre arbeitet zeichenweise und ist am ehesten eine uralte Stream-Chiffre mit einem konstanten „Schlüsselstrom“. Wichtig ist, dass du die Begriffe einordnen kannst, wenn sie in Dokumentationen auftauchen.

8. Maschinenbau-Beispiel: CAD-Datei sicher übertragen

Stell dir vor, ein Ingenieurbüro in Stuttgart schickt einer Fertigung in Shanghai eine CAD-Zeichnung eines neuen Turbinenschaufelprofils. Die Datei ist 80 MB groß und streng vertraulich. So läuft der Versand in der Praxis:

1. Das Büro holt sich den **öffentlichen RSA-Schlüssel** der Fertigung aus deren Unternehmensverzeichnis.
2. Es würfelt einen zufälligen **AES-256-Schlüssel** (32 Byte).
3. Die CAD-Datei wird per **AES-GCM** verschlüsselt – in Sekunden, weil AES hardwarebeschleunigt ist.
4. Der AES-Schlüssel wird per **RSA** mit dem öffentlichen Schlüssel der Fertigung verschlüsselt.
5. Beides (verschlüsselte Datei + verschlüsselter Schlüssel) wird übertragen.
6. Die Fertigung entschlüsselt den AES-Schlüssel per **privatem RSA-Schlüssel** und danach die Datei per AES.

Das ist exakt das Hybrid-Prinzip aus dem vorigen Abschnitt – angewandt auf einen realen Industrie-Workflow. Genau so arbeiten auch Tools wie **PGP**, **S/MIME**, **SFTP mit Public-Key-Auth** und die meisten B2B-Transferportale.

9. Hinweis zu den testprogramme/

Im Ordner testprogramme/ liegen Python-Skripte aus V16, die per **Socket** kommunizieren (P1_echo_client_TEST.py etc.). Diese Programme **führen wir in V17 nicht aus**, weil sie nichts mit Kryptografie zu tun haben. Sie bleiben erhalten, damit du bei Bedarf zu V16 zurückspringen kannst. Alle Krypto-Beispiele in V17 laufen offline und erzeugen keinen Netzwerkverkehr.

10. Selbst-Check – Verständnisfragen

Beantworte die Fragen für dich selbst, bevor du die Lösungen aufklappst.

Frage 1: Was genau ist bei Caesar der Schlüssel?

Der Schlüssel ist die ganze Zahl k zwischen 1 und 25, um die jeder Buchstabe verschoben wird. Bei $k = 3$ wird aus A ein D, bei $k = 10$ wird aus A ein K.

Frage 2: Was besagt das Kerckhoffs-Prinzip in einem Satz?

Ein Verfahren muss sicher bleiben, selbst wenn der Algorithmus öffentlich bekannt ist – nur der Schlüssel darf geheim sein.

Frage 3: Warum brauchen wir $\% 26$ in der Caesar-Formel?

Weil das Alphabet nach 26 Buchstaben endet und wir bei Überlauf wieder bei A anfangen müssen. $\% 26$ faltet jede Zahl auf den Bereich 0–25 zurück.

Frage 4: Warum ist Caesar heute unsicher?

Weil der Schlüsselraum nur 25 Werte umfasst. Ein Angreifer probiert alle durch (Brute-Force) oder nutzt Häufigkeitsanalyse. In beiden Fällen ist die Chiffre in Sekunden gebrochen.

Frage 5: Wie viele Schlüssel hat ein symmetrisches, wie viele ein asymmetrisches Verfahren?

Symmetrisch: **einen** gemeinsamen Schlüssel. Asymmetrisch: **zwei zusammengehörige** Schlüssel pro Person (einen öffentlichen und einen privaten).

Frage 6: Warum nutzt HTTPS ein Hybrid-Verfahren statt nur RSA?

RSA ist für große Datenmengen zu langsam. Man nutzt RSA daher nur, um am Anfang einen kurzen AES-Schlüssel sicher auszutauschen, und verschlüsselt den eigentlichen Datenstrom schnell mit AES.

Frage 7: Welche Rolle spielt ein Public Key bei RSA?

Mit dem Public Key verschlüsselt man Nachrichten an den Besitzer des zugehörigen Private Keys. Er darf und soll öffentlich zugänglich sein – das ist gerade der Witz des Verfahrens.

Frage 8: Worin unterscheiden sich Block- und Stream-Chiffre?

Eine Block-Chiffre arbeitet auf Blöcken fester Größe (z. B. AES mit 128 Bit), eine Stream-Chiffre verarbeitet den Klartext bitweise/bytewise mit einem Pseud Zufallsstrom (z. B. ChaCha20).

Zusammenfassung

- **Kryptografie** schützt Vertraulichkeit, Integrität und Authentizität – im Ingenieur-Alltag z. B. bei CAD-Transfer und Maschinenkommandos.
- **Klartext**, **Geheimtext**, **Schlüssel**, **Verschlüsseln**, **Entschlüsseln** sind die Grundbegriffe.
- Das **Kerckhoffs-Prinzip** trennt öffentlichen Algorithmus von geheimem Schlüssel.
- Die **Caesar-Chiffre** folgt der Formel $(pos + k) \% 26$, ist aber per Brute-Force in Sekunden gebrochen.
- **Symmetrisch** (ein Schlüssel, schnell, AES) und **asymmetrisch** (zwei Schlüssel, langsam, RSA) ergänzen sich im **Hybrid-Verfahren** (HTTPS).
- **Block-** und **Stream-Chiffren** sind die zwei Bauarten innerhalb der symmetrischen Welt.

Nächster Schritt

Weiter mit 02_praxis.ipynb – Caesar-Chiffre selbst programmieren.

Datei: lessons/V18-Kryptografie-Teil2/01_theorie.ipynb

V18 – Kryptografie Teil 2: Theorie

Lernziele

Nach diesem Notebook kannst du ... - die drei zentralen Eigenschaften kryptografischer Hash-Funktionen benennen (**deterministisch**, **einweg**, **Lawineneffekt**) und jede davon an einem Beispiel erklären, - erklären, warum Passwörter niemals im Klartext in einer Datenbank liegen dürfen und wie Hash-Funktionen dieses Problem lösen, - die Grundidee eines **Wörterbuch-Angriffs** beschreiben und mit dem Konzept **Salt** die wichtigste Gegenmaßnahme erläutern, - die Sicherheitsunterschiede zwischen MD5, SHA-1 und SHA-256 einordnen, - das Prinzip einer **digitalen Signatur** als Kombination aus Hash und asymmetrischer Verschlüsselung skizzieren und am Beispiel eines Firmware-Updates beschreiben, wann ein Roboterarm eine Signatur prüft.

Zeitbudget

Ca. 30 Minuten für die reine Theorie.

TL;DR

Eine **Hash-Funktion** erzeugt aus beliebig großen Eingaben einen festen, scheinbar zufälligen Kurz-Output. Gute Hash-Funktionen sind **einweg** (der Weg zurück ist praktisch unmöglich) und reagieren auf die kleinste Änderung der Eingabe mit einem komplett anderen Ergebnis (**Lawineneffekt**). Passwörter werden deshalb gehasht gespeichert. Angreifer probieren dagegen mit Wortlisten sehr viele Hash-Berechnungen durch – die Gegenmaßnahme heißt **Salt**. Wird der Hash zusätzlich mit einem privaten Schlüssel verschlüsselt, entsteht eine **digitale Signatur**, mit der ein Empfänger die Echtheit einer Datei überprüfen kann.

Voraussetzungen

- V17 (Caesar-Chiffre, symmetrische vs. asymmetrische Verschlüsselung).
- 00_python_recap.ipynb.

1. Was ist eine Hash-Funktion?

Stell dir einen digitalen **Fingerabdruck** vor. Egal wie groß ein Mensch ist – sein Fingerabdruck ist immer gleich klein und trotzdem praktisch eindeutig. Genau das leistet eine **Hash-Funktion** mit beliebigen Daten: Sie nimmt eine Eingabe beliebiger Länge – ein einzelnes Wort, einen ganzen Roman oder eine mehrere Gigabyte große Firmware-Datei – und erzeugt daraus einen **kurzen, festen** Wert, den **Hash** oder **Hashwert**. Typische Längen sind 128 Bit (MD5),

160 Bit (SHA-1) oder 256 Bit (SHA-256). Dargestellt werden diese Bits meist als **Hexadezimal-Zeichenkette**.

[!NOTE] Eine **Hash-Funktion** ist eine mathematische Abbildung von einer beliebig langen Eingabe auf einen Ausgabewert fester Länge. Eine **kryptografische** Hash-Funktion erfüllt darüber hinaus zusätzliche Sicherheitseigenschaften, auf die wir gleich im Detail eingehen.

Anders als die Caesar-Chiffre aus V17 ist eine Hash-Funktion **kein Verschlüsselungsverfahren**. Bei einer Verschlüsselung willst du die ursprüngliche Nachricht später wieder entschlüsseln können. Bei einer Hash-Funktion ist das Gegenteil das Ziel: Aus dem Hash soll sich die Eingabe **nicht** wiederherstellen lassen. Eine Hash-Funktion wirft also absichtlich Information weg.

Die folgende Skizze zeigt die drei Kern-Eigenschaften einer guten kryptografischen Hash-Funktion auf einen Blick.

2. Die drei zentralen Eigenschaften

2.1 Deterministisch

Eine Hash-Funktion liefert für dieselbe Eingabe **immer** denselben Hash-Wert, egal wann und auf welchem Rechner sie ausgeführt wird. Diese Eigenschaft klingt banal, ist aber die Grundlage jeder Anwendung: Nur weil der Hash reproduzierbar ist, kann ich zwei Dateien vergleichen, indem ich ihre Hashes vergleiche.

2.2 Einweg-Eigenschaft

Gegeben der Hash-Wert **h**, ist es **praktisch unmöglich**, eine Eingabe **m** zu finden, deren Hash genau **h** ergibt. Formal spricht man von **Preimage Resistance**. „Praktisch unmöglich“ heißt: Mit den schnellsten heute verfügbaren Computern würde eine Brute-Force-Suche über alle denkbaren Eingaben länger dauern als das Universum existiert. Diese Asymmetrie ist der Kern: In die eine Richtung ist die Berechnung billig, in die andere Richtung astronomisch teuer.

Weiterführend: Es gibt drei verwandte Sicherheitsbegriffe – *Preimage Resistance* (zu gegebenem Hash keinen Input finden), *Second Preimage Resistance* (zu gegebenem Input keinen zweiten Input mit gleichem Hash finden) und *Collision Resistance* (gar kein Paar mit gleichem Hash finden). Für Passwort-Hashing reicht die erste Eigenschaft, für digitale Signaturen brauchst du die dritte.

2.3 Lawineneffekt

Selbst eine **winzige Änderung** an der Eingabe – ein umgedrehtes Bit, ein geändertes Komma, ein zusätzliches Leerzeichen – führt dazu, dass sich etwa die Hälfte aller Bits im Hash-Wert ändert. Der Hash sieht danach aus wie

ein komplett anderer Zufallswert. Dank des Lawineneffekts erkennst du selbst winzige Manipulationen sofort – ideal für Integritätsprüfungen.

Das folgende Beispiel zeigt den Effekt. Die beiden Strings unterscheiden sich nur in einem Zeichen, die Hashes sind trotzdem völlig unterschiedlich.

3. Die wichtigsten Hash-Algorithmen im Überblick

Im Laufe der letzten 30 Jahre hat die Kryptografie-Community mehrere Hash-Funktionen entwickelt und wieder verworfen. Der Grund: Sobald effiziente Angriffe bekannt werden, gilt ein Algorithmus als **gebrochen** und darf nicht mehr für Sicherheits-kritische Zwecke verwendet werden. Die folgende Tabelle gibt einen Überblick.

Algorithmus	Output-Länge	Status	Verwendung
MD5	128 Bit (32 Hex)	gebrochen seit 2004	nur noch für Checksummen, nicht sicherheitskritisch
SHA-1	160 Bit (40 Hex)	gebrochen seit 2017	Legacy-Systeme, wird ausgemustert
SHA-256	256 Bit (64 Hex)	sicher	Industrie-Standard, z. B. Bitcoin, TLS, Git
SHA-3	variabel	sicher	modernes Alternativ-Design (seit 2015)
bcrypt / Argon2	variabel	sicher	speziell für Passwort -Hashing

3.1 MD5 – der Klassiker, heute unsicher

MD5 wurde 1991 von Ron Rivest entwickelt und war jahrelang der De-facto-Standard. 2004 zeigte ein chinesisches Forscher-Team, wie man **Kollisionen** effizient erzeugen kann, also zwei verschiedene Eingaben mit demselben MD5-Hash. Für Passwörter und digitale Signaturen ist MD5 heute tabu. Als schnelle **Checksumme** gegen zufällige Übertragungsfehler (nicht gegen böswillige Manipulation) ist er aber noch in Gebrauch – zum Beispiel wenn eine Linux-Distribution einen `.iso`-Download mit MD5-Prüfsumme anbietet.

3.2 SHA-1 – ebenfalls unsicher

SHA-1 wurde 1995 von der NSA entworfen und lange als Nachfolger von MD5 eingesetzt, etwa in den Versionskontroll-Systemen Git oder in alten TLS-Zertifikaten. 2017 veröffentlichte Google den **SHAttered**-Angriff und zeigte zwei unterschiedliche PDF-Dateien mit identischem SHA-1-Hash. Seitdem gilt auch SHA-1 als gebrochen und wird in neuen Systemen nicht mehr eingesetzt.

3.3 SHA-256 – der heutige Standard

SHA-256 gehört zur SHA-2-Familie und erzeugt einen 256 Bit langen Hash. Es gilt nach aktuellem Stand als sicher und wird praktisch überall eingesetzt: in TLS-Zertifikaten, bei Bitcoin-Transaktionen, in Docker-Image-IDs, in Git-Commit-Hashes (neue Repositories) und eben auch in digitalen Signaturen von Firmware-Updates. Wenn du heute eine Hash-Funktion auswählen musst und kein Passwort-Hashing betreibst, ist SHA-256 meistens die richtige Wahl.

4. Warum Passwörter niemals im Klartext gespeichert werden

Stell dir vor, ein Online-Shop speichert alle Kunden-Passwörter im Klartext in einer Datenbank-Tabelle `users(email, password)`. Wird dieser Shop gehackt und die Datenbank kopiert – das passiert regelmäßig und nennt sich **Database Dump** – hat der Angreifer auf einen Schlag Zugriff auf alle Passwörter. Da viele Menschen Passwörter **wiederverwenden**, kann er sich anschließend bei E-Mail-Konten, Online-Banking und Social-Media-Profilen derselben Nutzer einloggen.

Die saubere Lösung: Der Server speichert niemals das Passwort selbst, sondern nur dessen **Hash**. Bei jedem Login rechnet der Server den Hash der eingegebenen Eingabe und vergleicht ihn mit dem gespeicherten Hash. Stimmt beides überein, ist die Authentifizierung erfolgreich. Wird die Datenbank gestohlen, hat der Angreifer nur die Hashes – und die Einweg-Eigenschaft macht das Zurückrechnen auf das Klartext-Passwort praktisch unmöglich.

Login-Ablauf im Detail

Die Reihenfolge ist wichtig: Der Hash wird **auf dem Server** gebildet. In der Datenbank liegt nur der Hash. Selbst die Administratoren sehen das Passwort niemals im Klartext. Moderne Frameworks wie Django, Rails oder Spring Security erledigen diesen Ablauf automatisch – du musst ihn in der Praxis selten selbst implementieren. Verstehen solltest du ihn trotzdem, damit du bei der Architektur eigener Systeme die richtigen Fragen stellen kannst.

5. Angriffe auf Passwort-Hashes

Die Einweg-Eigenschaft schützt **nicht** vor allen Angriffen. Ein Angreifer kann die Rechnung umdrehen: Statt aus dem Hash auf das Passwort zu schließen,

berät er sich selbst viele Kandidat-Passwörter zusammen und hasht sie durch. Passt einer der berechneten Hashes zum gestohlenen Hash, hat er das Passwort gefunden. Dieser Ansatz ist besonders effektiv, weil Menschen extrem schlechte Passwörter wählen: `123456`, `passwort`, `qwerty`, `admin` oder der Vorname des Haustiers.

5.1 Wörterbuch-Angriff

Beim **Wörterbuch-Angriff** nimmt der Angreifer eine große Liste häufiger Passwörter – oft mehrere Millionen Einträge, zum Beispiel die berühmte `rockyou.txt` aus einem alten Datenleck – und hasht jeden Eintrag. Jeder berechnete Hash wird mit dem gestohlenen Hash verglichen. Moderne GPUs schaffen Milliarden SHA-256-Berechnungen pro Sekunde, einfache Passwörter sind damit in wenigen Sekunden geknackt.

5.2 Rainbow-Tables

Rainbow-Tables sind vorausberechnete Tabellen, die Millionen von Hashes fertig auf der Festplatte ablegen. Statt bei jedem Angriff neu zu rechnen, schlägt man den gestohlenen Hash einfach in der Tabelle nach. Der Angreifer tauscht Rechenzeit gegen Speicherplatz und ist bei einem Angriff extrem schnell.

5.3 Die Gegenmaßnahme: Salt

Ein **Salt** ist ein zufälliger Wert, den der Server bei der Registrierung pro Nutzer erzeugt und zusammen mit dem Hash speichert. Beim Login wird nicht mehr nur das Passwort gehasht, sondern die Kombination `passwort + salt`. Der gespeicherte Datensatz sieht dann etwa so aus: `(email, salt, hash_von(passwort + salt))`.

Das hat zwei Effekte: Erstens wird eine vorgerechnete Rainbow-Table wertlos, denn der Angreifer müsste für jeden Salt eine eigene Tabelle anlegen. Zweitens haben zwei Nutzer mit dem gleichen Passwort trotzdem unterschiedliche Hashes, denn ihr Salt ist unterschiedlich. Ein Angriff muss also immer **pro Nutzer einzeln** laufen, was den Aufwand massiv erhöht.

[!NOTE] Ein **Salt** ist ein zufälliger Zusatzwert, der vor dem Hashen an das Passwort angehängt wird. Er wird unverschlüsselt mitgespeichert – sein Nutzen liegt nicht in der Geheimhaltung, sondern darin, dass jeder Datensatz einen **individuellen** Hash erzeugt.

5.4 Langsame Passwort-Hashes: `bcrypt`, `scrypt`, `Argon2`

SHA-256 ist **zu schnell** für Passwort-Hashing. Genau diese Eigenschaft, die bei der Datei-Integritätsprüfung ein Vorteil ist, hilft dem Angreifer. Deshalb wurden spezielle **Passwort-Hash-Funktionen** entwickelt, die absichtlich langsam sind und einen einstellbaren Kostenfaktor haben: **bcrypt** (1999), **scrypt** (2009) und **Argon2** (2015, Sieger der *Password Hashing Competition*). In Python nutzt

du dafür die Pakete `bcrypt` oder `argon2-cffi`. In dieser Vorlesung bleiben wir bei SHA-256, damit du die Grundprinzipien ohne zusätzliche Bibliothek verstehst. In echter Produktions-Software solltest du immer einen spezialisierten Passwort-Hasher verwenden.

6. Kollisionen – wenn zwei Eingaben denselben Hash erzeugen

Da eine Hash-Funktion aus beliebig langen Eingaben einen fest langen Output macht, muss es logischerweise **Kollisionen** geben: zwei verschiedene Eingaben mit demselben Hash-Wert. Bei SHA-256 gibt es 2^{256} mögliche Hash-Werte – eine astronomisch große, aber endliche Zahl. Die entscheidende Frage lautet also nicht „Gibt es Kollisionen?“ sondern „Wie schwer ist es, eine Kollision **gezielt** zu finden?“.

Das **Geburtstagsparadoxon** aus der Wahrscheinlichkeitstheorie sagt: Wenn du 23 Personen in einen Raum steckst, ist die Wahrscheinlichkeit, dass zwei von ihnen am selben Tag Geburtstag haben, schon über 50 %. Übertragen auf Hash-Funktionen heißt das: Mit etwa $2^{(n/2)}$ zufälligen Versuchen findest du eine Kollision in einem n-Bit-Hash. Bei SHA-256 sind das 2^{128} Versuche – immer noch jenseits aller technischen Möglichkeiten. Bei MD5 sind es nur 2^{64} , und genau dieser Angriff ist 2004 gelungen.

7. Integritätsprüfung von Dateien

Bevor wir zur digitalen Signatur kommen, noch ein einfacherer Anwendungsfall: die **Integritätsprüfung**. Wenn du eine große Datei über das Internet herunterlädst, willst du sicherstellen, dass sie unterwegs nicht beschädigt wurde – weder durch Übertragungsfehler noch durch einen Angreifer, der die Datei manipuliert hat. Der Hersteller veröffentlicht dazu zusammen mit der Datei einen SHA-256-Hash. Nach dem Download rechnest du lokal den Hash und vergleichst ihn.

Eine **Integritätsprüfung** schützt nur gegen zufällige Fehler, nicht gegen einen Angreifer. Wenn derselbe Angreifer sowohl die Datei als auch den publizierten Hash manipulieren kann, merkst du nichts. Deshalb braucht es für echte Sicherheit eine **signierte** Angabe des Hashes – und damit sind wir bei digitalen Signaturen.

8. Digitale Signaturen

Erinnere dich an V17: Bei der **asymmetrischen Kryptografie** hat jeder Teilnehmer ein Schlüsselpaar – einen **privaten Schlüssel** (geheim, nur der Besitzer kennt ihn) und einen **öffentlichen Schlüssel** (darf jeder kennen). Daten, die mit dem privaten Schlüssel verschlüsselt wurden, lassen sich nur mit dem passenden öffentlichen Schlüssel wieder entschlüsseln. Diese Asymmetrie nutzt die **digitale Signatur**.

8.1 Signatur erstellen

Der Hersteller einer Firmware möchte einem Kunden beweisen, dass die Datei tatsächlich von ihm stammt und nicht verändert wurde. Er geht in zwei Schritten vor:

1. Er berechnet den **SHA-256-Hash** der Firmware-Datei. Das Ergebnis ist ein kompakter Fingerabdruck der Datei.
2. Er **verschlüsselt diesen Hash mit seinem privaten Schlüssel**. Das Ergebnis ist die **digitale Signatur** und wird zusammen mit der Firmware veröffentlicht.

8.2 Signatur prüfen

Der Empfänger – zum Beispiel der Roboterarm, der das Update installieren soll – geht die beiden Schritte rückwärts:

1. Er berechnet selbst den **SHA-256-Hash** der empfangenen Firmware-Datei.
2. Er **entschlüsselt die Signatur mit dem öffentlichen Schlüssel des Herstellers**. Heraus kommt der Hash, den der Hersteller damals berechnet hat.
3. Er **vergleicht** beide Hashes. Sind sie identisch, ist die Firmware unverändert **und** tatsächlich vom Hersteller. Unterscheiden sich die Hashes, wird das Update **abgelehnt**.

8.3 Warum überhaupt der Umweg über den Hash?

Theoretisch könnte der Hersteller die komplette Firmware direkt mit seinem privaten Schlüssel verschlüsseln und das Ergebnis als Signatur mitsenden. In der Praxis scheitert das an zwei Punkten: Asymmetrische Verschlüsselung ist **sehr langsam** und nur für kleine Datenmengen praktikabel, und eine Firmware hat oft mehrere Megabyte. Der Trick mit dem Hash reduziert das Signieren auf 256 Bit – das geht in Millisekunden.

8.4 Wer garantiert den öffentlichen Schlüssel?

Die ganze Konstruktion hängt an einem Faden: Der Empfänger muss sich sicher sein, dass der **öffentliche Schlüssel** tatsächlich zum Hersteller gehört. Ein Angreifer könnte sonst einen eigenen Schlüssel als „Herstellerschlüssel“ verbreiten und selbst gültige Signaturen ausstellen. Dieses Problem löst eine **Certificate Authority (CA)** – eine vertrauenswürdige Instanz, die Zertifikate ausstellt und ihrerseits wieder signiert. Die Details (Zertifikatskette, Root CAs) schauen wir uns in V19/V20 im Kontext von TLS nicht erneut an, der wichtige Punkt für heute ist: In der Praxis wird nie ein nackter öffentlicher Schlüssel ausgetauscht, sondern ein **Zertifikat**, das den Schlüssel an eine Identität bindet.

9. HMAC – Hash mit Schlüssel

Zwischen reinem Hash und voller digitaler Signatur gibt es noch einen Mittelweg: **HMAC** (*Hash-based Message Authentication Code*). Dabei wird der Hash nicht nur aus der Nachricht gebildet, sondern aus der Kombination **Nachricht** + **gemeinsamem Geheimschlüssel**. Zwei Parteien, die den Schlüssel vorab ausgetauscht haben, können so die Echtheit einer Nachricht prüfen, ohne asymmetrische Kryptografie zu brauchen. HMAC ist schnell, leicht zu implementieren und in vielen Protokollen (z. B. JWT, TLS, API-Signaturen wie AWS) im Einsatz.

10. Maschinenbau-Beispiel: Firmware-Update für einen Roboterarm

Ein Industrie-Roboter bekommt regelmäßig **Over-The-Air-Updates** für seine Firmware. Das Szenario ist sicherheitskritisch: Eine manipulierte Firmware könnte den Roboter dazu bringen, unkontrollierte Bewegungen auszuführen, wodurch Menschen verletzt oder Maschinen beschädigt werden. Genau deshalb prüft die Steuerungseinheit **vor** jeder Installation die digitale Signatur des Herstellers.

Der typische Ablauf:

1. Der Hersteller veröffentlicht eine neue Firmware `firmware_v2.3.0.bin` zusammen mit einer JSON-Datei, die unter anderem den **SHA-256-Hash** der Firmware und eine **digitale Signatur** enthält. Ein solcher Datensatz liegt im Ordner `testdaten/` bereit und sieht in verkürzter Form etwa so aus: `{"sha256_hash": "...", "signature": "...", "signed_by": "RobotCorp CA"}`.
2. Der Roboter lädt beide Dateien herunter.
3. Er berechnet selbst den SHA-256-Hash der `.bin`-Datei und vergleicht ihn mit dem Feld `sha256_hash`.
4. Er prüft die Signatur mit dem öffentlichen Schlüssel der Herstellers-CA.
5. Nur wenn beide Prüfungen erfolgreich sind, wird das Update installiert und anschließend neu gestartet.

Im Praxis-Notebook bauen wir Schritt 3 konkret nach – die Signaturprüfung mit echten RSA-Schlüsseln sparen wir uns, sie läuft konzeptionell aber genauso.

11. Selbst-Check

Bevor du weitermachst, beantworte die folgenden Fragen. Klicke erst danach auf das Dreieck, um die Antwort aufzuklappen.

1. Was liefert eine Hash-Funktion?

Einen **fest langen** Wert (z. B. 256 Bit bei SHA-256), der aus einer beliebig großen Eingabe berechnet wird. Er ist deterministisch, aber praktisch nicht umkehrbar.

2. Warum darf MD5 nicht mehr für Signaturen eingesetzt werden?

Weil seit 2004 effiziente **Kollisionsangriffe** bekannt sind. Ein Angreifer kann zwei Dokumente erzeugen, die zwar unterschiedlichen Inhalt haben, aber denselben MD5-Hash – und damit wäre jede auf MD5 aufbauende Signatur austauschbar.

3. Warum speichert ein Server Passwörter als Hash und nicht im Klartext?

Damit ein Datenbank-Diebstahl den Angreifer nicht sofort in den Besitz aller Passwörter bringt. Die Einweg-Eigenschaft des Hashes macht es praktisch unmöglich, aus dem gestohlenen Hash das Klartext-Passwort zu rekonstruieren.

4. Wie funktioniert ein Wörterbuch-Angriff?

Der Angreifer nimmt eine Liste häufig verwendeter Passwörter, hasht jedes davon und vergleicht das Ergebnis mit dem gestohlenen Hash. Ist ein Passwort einfach oder in der Liste enthalten, wird es in Sekunden geknackt.

5. Was bringt ein Salt?

Ein Salt ist ein zufälliger Zusatzwert pro Nutzer, der vor dem Hashen an das Passwort angehängt wird. Dadurch werden **vorgerechnete Rainbow-Tables wertlos**, und zwei Nutzer mit identischem Passwort bekommen trotzdem unterschiedliche Hashes.

6. Welche beiden Werte vergleicht der Roboter bei der Firmware-Prüfung?

Den **selbst berechneten** SHA-256-Hash der heruntergeladenen `.bin`-Datei und den Hash, den er aus der mitgelieferten Signatur nach Entschlüsselung mit dem öffentlichen Schlüssel herausbekommt. Nur bei Gleichheit wird das Update installiert.

7. Warum reicht es nicht, eine Datei mit ihrem MD5-Hash als „sicher“ zu betrachten, wenn beides vom selben Server kommt?

Weil ein Angreifer, der den Server kontrolliert, sowohl die Datei als auch den publizierten Hash austauschen kann. Der MD5-Hash schützt nur gegen **zufällige** Übertragungsfehler, nicht gegen einen böswilligen Akteur. Für echten Schutz brauchst du eine **Signatur**, die mit einem unabhängig vertrauenswürdigen Schlüssel erstellt wurde.

8. Warum wird beim Signieren zuerst gehasht und dann verschlüsselt, statt direkt die ganze Datei zu verschlüsseln?

Weil asymmetrische Verschlüsselung sehr langsam ist und bei einer mehrere MB großen Firmware unpraktikabel wäre. Der Hash reduziert die Eingabe auf 256 Bit, und nur dieser kompakte Fingerabdruck muss mit dem privaten Schlüssel verschlüsselt werden.

Zusammenfassung

- Eine **Hash-Funktion** erzeugt aus beliebig langen Eingaben einen fest langen, deterministischen, einweg-gerichteten Fingerabdruck mit starkem **Lawineneffekt**.
- **MD5** und **SHA-1** gelten als gebrochen und dürfen nicht mehr für Sicherheit eingesetzt werden. **SHA-256** ist der aktuelle Standard.
- **Passwörter** werden niemals im Klartext gespeichert, sondern als Hash – idealerweise mit **Salt** und einer langsamen Passwort-Hash-Funktion (bcrypt, Argon2).
- Ein **Wörterbuch-Angriff** probiert eine Liste von Kandidaten durch und funktioniert nur gegen schwache Passwörter.
- Eine **digitale Signatur** = Hash(Datei) + asymmetrische Verschlüsselung mit dem privaten Schlüssel. Empfänger prüfen sie mit dem öffentlichen Schlüssel – Standardverfahren für Firmware-Updates.

Nächster Schritt

Weiter mit 02_praxis.ipynb – dort lernst du **hashlib** in Python praktisch kennen.

Datei: lessons/V19-Datenbanken-Teil1/01_theorie.ipynb

V19 – Datenbanken Teil 1: Theorie

Lernziele

Nach diesem Notebook kannst du ... - den Unterschied zwischen einer flachen Datei (CSV/JSON) und einer relationalen Datenbank konkret benennen, - das **relationale Modell** (Tabelle, Zeile, Spalte, Schlüssel) in eigenen Worten erklären, - die **SQL-Grundbefehle** CREATE, INSERT, SELECT, UPDATE, DELETE lesen und aufschreiben, - **SQLite** einordnen und den Unterschied zu ServerDBs wie PostgreSQL oder MySQL skizzieren, - das Python-Modul **sqlite3** mit **Parameter-Binding** korrekt und sicher gegen **SQL-Injection** einsetzen.

Zeitbudget

Ca. 30 Minuten.

TL;DR

Eine **Datenbank** strukturiert Daten in **Tabellen** mit festen Spalten und ausdrucksstarken Abfragen. **SQLite** ist eine Datei-basierte Datenbank, die Python im Standardmodul **sqlite3** mitliefert. Die fünf wichtigsten SQL-Befehle sind

CREATE TABLE, INSERT, SELECT, UPDATE und DELETE. Parameter-Binding mit ? ist die einzige sichere Methode, Nutzereingaben in Queries einzubauen.

Voraussetzungen

- 00_python_recap.ipynb.
- Grundverständnis von CSV-Dateien aus V09.

1. Warum überhaupt eine Datenbank?

Solange du einzelne Messreihen oder Konfigurations-Dateien verwaltest, reicht eine **CSV**- oder **JSON**-Datei häufig aus. Sobald du aber mehrere Tabellen verknüpfst, gleichzeitig von verschiedenen Programmen aus liest und schreibst oder konsistent auf Teilmengen zugreifst, stößt du an die Grenzen flacher Dateien sehr schnell.

Stell dir eine Fertigungshalle vor: Du erfasst **Maschinen**, die **Werkstücke** produzieren, zusätzlich **Wartungen** pro Maschine und **Sensormesswerte** im Sekundentakt. Eine einzelne CSV-Datei würde sich schnell aufblähen. Zwei oder drei CSV-Dateien ließen sich zwar organisieren, doch jeder Zugriff erfordert, die komplette Datei zu laden, zu filtern und anschließend wieder zu schreiben – ohne jede Garantie, dass zwei Programme sich nicht gegenseitig überschreiben.

Konkrete Vorteile einer Datenbank

Eine **relationale Datenbank** bietet vier Eigenschaften, die flachen Dateien fehlen. Erstens garantiert sie **Konsistenz**: Zwei Zeilen mit identischer Produkt-ID werden durch einen **Primärschlüssel** verhindert, ein falsches Datumsformat durch eine **CHECK**-Bedingung abgefangen. Zweitens erlaubt sie **gleichzeitige Nutzung** durch mehrere Programme, ohne dass sich die Änderungen gegenseitig überschreiben. Drittens stellt sie **ausdrucksstarke Abfragen** zur Verfügung: „Alle Werkstücke, deren Produktion länger als 20 Minuten dauert und mehr als 2 kg Material benötigt“ ist in SQL ein Einzeiler. Viertens lassen sich **Integritätsbedingungen** (z. B. „jeder Sensor muss zu einer existierenden Maschine gehören“) direkt in der Datenbank erzwingen.

[!NOTE] Eine **relationale Datenbank** speichert Daten in Form von **Relationen** (Tabellen) mit festen **Attributen** (Spalten) und erlaubt es, Beziehungen zwischen Tabellen über Schlüssel abzubilden.

2. Das relationale Modell

Das relationale Modell wurde 1970 von **Edgar F. Codd** beschrieben und hat bis heute die Praxis der meisten Datenbanken geprägt. Die zentralen Begriffe sind schnell erklärt, weil sie Alltagsvokabeln aus jeder Tabelle aufgreifen.

Eine **Tabelle** (formal: Relation) ist eine zweidimensionale Struktur mit einem festen **Schema**. Jede **Zeile** (auch **Datensatz** oder **Tupel**) beschreibt genau

ein Objekt der realen Welt, etwa ein Produkt oder eine Messung. Jede **Spalte** (auch **Attribut**) legt einen Namen und einen Datentyp fest, zum Beispiel `Produktname TEXT` oder `Produktionszeit_Minuten INTEGER`.

Primärschlüssel und Fremdschlüssel (Kurz-Version)

Ein **Primärschlüssel** ist eine Spalte (oder Spaltenkombination), die jede Zeile der Tabelle eindeutig identifiziert. In der Produkt-Tabelle wäre das die `Produkt_ID`. Ein **Fremdschlüssel** verweist von einer Tabelle auf den Primärschlüssel einer anderen und stellt so **Beziehungen** zwischen Tabellen her, etwa „dieser Sensor gehört zu dieser Maschine“.

[!NOTE] Ein **Primärschlüssel** (Primary Key) ist eine Spalte, deren Werte niemals NULL sein dürfen und pro Zeile eindeutig sein müssen. SQLite erzeugt für `INTEGER PRIMARY KEY`-Spalten automatisch fortlaufende IDs, falls du keinen Wert angibst.

Die Feinheiten von Fremdschlüsseln, Joins und Normalformen sind das Kernthema von **V20**. In V19 konzentrieren wir uns bewusst auf **eine einzelne Tabelle**, damit die SQL-Syntax im Vordergrund bleibt.

3. SQLite – Datenbank als Datei

SQLite ist eine Datenbank, die sich erheblich von den großen, bekannten Namen wie **PostgreSQL**, **MySQL** oder **Oracle** unterscheidet. Während diese Systeme als **Server-Prozesse** laufen, die Clients über ein Netzwerkprotokoll bedienen, ist SQLite eine reine **C-Bibliothek**, die eine einzige **Datei** (Endung `.db` oder `.sqlite`) als Datenbank behandelt.

[!NOTE] **SQLite** ist eine Embedded-Datenbank ohne Server. Die komplette Datenbank liegt in einer einzigen Datei auf der Festplatte und wird direkt von der Anwendung über eine Bibliothek geöffnet.

Wann SQLite, wann ein Server-DBMS?

SQLite spielt seine Stärken überall dort aus, wo eine **eingebettete** Datenbank gebraucht wird: in Browsern, Smartphones, Desktop-Programmen, kleinen Web-Anwendungen mit wenigen gleichzeitigen Schreibern und – für uns besonders wichtig – in **Lernumgebungen** und kleinen Ingenieur-Tools. Jede laufende Android- oder iOS-App nutzt SQLite intern; viele Messgeräte und Steuerungssysteme schreiben ihre Daten ebenfalls in SQLite-Dateien.

Sobald jedoch **hunderte gleichzeitige Schreiber**, **strikte Zugriffsrechte** pro Benutzer oder **Replikation** über mehrere Rechner gefordert sind, sind Server-Datenbanken wie PostgreSQL oder MySQL die bessere Wahl. Der Umstieg ist nicht dramatisch, weil die SQL-Grundsyntax sehr ähnlich bleibt – die Konzepte dieser Vorlesung sind eins zu eins übertragbar.

[!TIP] Faustregel: Für Lernen, Desktop-Tools, Prototypen und kleine Web-Apps ist **SQLite** erste Wahl. Für Mehrbenutzer-Systeme mit nennenswertem Schreib-Aufkommen nimmt man **PostgreSQL**.

Die In-Memory-Datenbank

SQLite kann eine Datenbank auch komplett im Arbeitsspeicher halten. Der Verbindungsaufwurf lautet dann `sqlite3.connect(":memory:")`. Das Ergebnis verhält sich wie eine normale Datenbank, verschwindet aber mit dem Programm wieder. Für Unit-Tests, Lern-Notebooks oder kurzlebige Analysen ist das ideal – und wir nutzen das in V19 durchgehend, damit dein Projektordner sauber bleibt.

4. SQL – die Sprache der Datenbanken

SQL (Structured Query Language) ist eine standardisierte Sprache, um relationale Datenbanken zu erstellen, zu befüllen und abzufragen. Ihre Befehle lassen sich grob in zwei Gruppen einteilen. Die **DDL** (Data Definition Language) definiert die Struktur der Datenbank: **CREATE TABLE**, **ALTER TABLE**, **DROP TABLE**. Die **DML** (Data Manipulation Language) arbeitet mit den eigentlichen Daten: **SELECT**, **INSERT**, **UPDATE**, **DELETE**.

[!NOTE] **SQL** ist eine deklarative Sprache. Du beschreibst, **was** du möchtest (alle Produkte mit Gewicht > 2 kg), nicht **wie** die Datenbank das intern umsetzt. Der **Query-Planner** der Datenbank wählt den schnellsten Weg selbst.

Die Groß-/Kleinschreibung der SQL-Schlüsselwörter ist im Standard **nicht** relevant, **SELECT** und **select** funktionieren gleich. Üblich ist die Konvention, SQL-Schlüsselwörter **groß** und eigene Namen (Tabellen, Spalten) in **Unterstrich-** oder **PascalCase**-Schreibweise zu schreiben, damit Queries auf einen Blick lesbar bleiben.

5. CREATE TABLE – Tabelle anlegen

Mit **CREATE TABLE** legst du eine neue Tabelle an. Für jede Spalte gibst du einen **Namen**, einen **Datentyp** und optional **Constraints** (Bedingungen) an. SQLite kennt nur eine Handvoll Typen: **INTEGER** für ganze Zahlen, **REAL** für Fließkommazahlen, **TEXT** für Zeichenketten, **BLOB** für rohe Binärdaten und **NULL** für den „fehlenden Wert“.

Die wichtigsten Constraints

PRIMARY KEY markiert die eindeutige Identität jeder Zeile. **NOT NULL** verbietet einen fehlenden Wert. **UNIQUE** erlaubt jeden Wert nur einmal. **CHECK** (<Bedingung>) prüft beim Einfügen oder Ändern, ob die Bedingung erfüllt ist.

DEFAULT <wert> setzt einen Standardwert, wenn beim Einfügen kein Wert für diese Spalte angegeben wurde. Für automatisch hochzählende Primärschlüssel reicht in SQLite INTEGER PRIMARY KEY – das Schlüsselwort AUTOINCREMENT ist nur in Sonderfällen nötig und kostet Performance.

[!WARNING] TEXT ohne NOT NULL erlaubt, dass in dieser Spalte NULL stehen kann. NULL verhält sich anders als der leere String "" und ist **nicht** gleich NULL. Der Vergleich wert = NULL liefert immer NULL (nicht TRUE), korrekt ist wert IS NULL.

6. INSERT INTO – Daten einfügen

Ein INSERT-Befehl fügt eine oder mehrere Zeilen in eine Tabelle ein. Die einfachste Form listet erst die Spalten, dann die passenden Werte auf.

```
INSERT INTO Produkte (Produkt_ID, Produktname, Produktionszeit_Minuten, Material_pro_Stueck,  
VALUES (1, 'Zahnrad Z42', 25, 1.8);
```

Lässt du eine Spalte weg, für die es einen DEFAULT oder eine AUTOINCREMENT-Regel gibt, wählt die Datenbank den Wert selbst. Strings stehen in **einfachen** Anführungszeichen, Zahlen ohne.

Mehrere Zeilen auf einmal

Du kannst in einem einzigen INSERT auch mehrere Zeilen durch Kommas getrennt einfügen, was bei großen Daten-Imports deutlich schneller ist als viele einzelne Inserts.

```
INSERT INTO Produkte (Produkt_ID, Produktname, Produktionszeit_Minuten, Material_pro_Stueck,  
    (2, 'Welle W-18', 35, 3.2),  
    (3, 'Flansch F-90', 18, 2.5),  
    (4, 'Buchse B-25', 12, 0.8);
```

In Python verwendet man dafür bevorzugt cursor.executemany(sql, seq_of_params), wobei seq_of_params eine Liste von Tupeln ist. Dazu kommen wir im Praxis-Notebook.

7. SELECT – Daten lesen

SELECT ist der vielseitigste und am häufigsten verwendete SQL-Befehl. Die Grundform lautet:

```
SELECT <spalten>  
FROM   <tabelle>  
WHERE  <bedingung>  
ORDER BY <spalte> [ASC|DESC]  
LIMIT <anzahl>;
```

SELECT * liefert alle Spalten – in Produktionscode solltest du die benötigten Spalten immer **explizit** aufzählen, weil sich sonst bei späteren Schema-Änderungen unerwartete Seiteneffekte ergeben.

Beispiele

```
-- Alle Produktnamen
SELECT Produktname FROM Produkte;

-- Alle Produkte mit mehr als 2 kg Material, sortiert nach Gewicht absteigend
SELECT Produkt_ID, Produktname, Material_pro_Stueck_kg
FROM   Produkte
WHERE  Material_pro_Stueck_kg > 2.0
ORDER BY Material_pro_Stueck_kg DESC;

-- Die zwei schnellsten Produkte (kleinste Produktionszeit)
SELECT Produktname, Produktionszeit_Minuten
FROM   Produkte
ORDER BY Produktionszeit_Minuten ASC
LIMIT 2;
```

WHERE akzeptiert die üblichen Vergleichsoperatoren (=, !=, <, >, <=, >=) sowie AND, OR, NOT, IN (...), BETWEEN ... AND ... und LIKE '%muster%'. Wichtig: Zeichenvergleiche verwenden in SQL **ein** Gleichheitszeichen (=), nicht == wie in Python.

8. UPDATE und DELETE

Mit UPDATE änderst du bestehende Zeilen, mit DELETE löschst du sie. Beide Befehle haben eine WHERE-Klausel – vergisst du diese, werden **alle** Zeilen verändert bzw. gelöscht.

```
UPDATE Produkte
SET   Produktionszeit_Minuten = Produktionszeit_Minuten + 5
WHERE Produkt_ID = 3;
```

```
DELETE FROM Produkte WHERE Produkt_ID = 6;
```

[!WARNING] Vergiss niemals die **WHERE-Klausel**! Ein UPDATE Produkte SET Produktionszeit_Minuten = 0; ohne WHERE setzt die Zeit **aller** Produkte auf 0. Ein DELETE FROM Produkte; leert die komplette Tabelle. Beides ist ohne Backup nicht mehr umkehrbar.

Zum Üben kannst du in SQLite vor einem heiklen UPDATE eine **Transaktion** starten (BEGIN) und im Fehlerfall mit ROLLBACK alles zurückdrehen. Dazu gleich mehr.

9. Transaktionen – zwei Zeilen über Commit

Jede Änderung (INSERT, UPDATE, DELETE) läuft in SQLite innerhalb einer **Transaktion**. Die Änderungen sind erst nach einem COMMIT dauerhaft gespeichert; bis dahin sieht sie nur deine eigene Verbindung. Das Gegenteil ist ROLLBACK: Alle seit Transaktionsbeginn ausgeführten Änderungen werden verworfen.

In Python macht das `sqlite3`-Modul das komfortabel: Nach einer ändernden Operation rufst du `conn.commit()` auf, wenn alles in Ordnung war, oder `conn.rollback()` im Fehlerfall. Ein `with conn:-`Block übernimmt das automatisch.

[!NOTE] Eine **Transaktion** ist eine logische Einheit von Datenbank-Operationen, die entweder vollständig (COMMIT) oder gar nicht (ROLLBACK) angewendet wird. Dieses **Alles-oder-Nichts**-Prinzip ist eine zentrale Eigenschaft zuverlässiger Datenbanken (das **A** in **ACID**).

10. Das Python-Modul `sqlite3`

Python liefert in der Standardbibliothek das Modul `sqlite3` mit. Damit sind vier Objekte wichtig: die **Connection**, der **Cursor**, einzelne **Queries** mit `execute` und das **Ergebnis-Objekt** mit `fetchone`/`fetchall`.

connect, cursor, execute

`sqlite3.connect(pfad)` öffnet (oder erzeugt) eine Datenbank-Datei und liefert eine **Connection**. `:memory:` erzeugt eine reine RAM-Datenbank. Auf der Connection rufst du `cursor()` auf und erhältst einen **Cursor**, der deine SQL-Befehle entgegennimmt. Mit `cursor.execute(sql)` oder `cursor.execute(sql, parameter)` läuft eine Query; mit `cursor.executemany(sql, liste)` gleich viele Inserts auf einmal.

Lese-Queries liefern den Cursor selbst zurück, über den du entweder iterieren kannst (`for row in cursor.execute(...)`) oder die Zeilen explizit mit `fetchall()` (alle als Liste) bzw. `fetchone()` (nächste einzelne Zeile) abholst.

11. Parameter-Binding und SQL-Injection

Kommen wir zur **wichtigsten Regel** dieser Vorlesung: Nutzereingaben dürfen **niemals** per String-Konkatenation in eine SQL-Query eingebaut werden. Das öffnet Angreifern Tür und Tor zu einer **SQL-Injection**.

[!WARNING] **SQL-Injection** ist eine der häufigsten Sicherheitslücken überhaupt. Der Angreifer schleust über ein Eingabefeld Teile einer SQL-Query ein und übernimmt so die Kontrolle über deine Datenbank – bis hin zu kompletter Datenlöschung.

Die einzige korrekte Methode ist das **Parameter-Binding**. Du schreibst in die Query ein Fragezeichen `?` als Platzhalter und übergibst die Werte als Tupel in einem zweiten Argument an `execute`. Der Datenbank-Treiber kümmert sich um das sichere Escaping.

[!TIP] **Merksatz:** Niemals SQL per `+`, `%` oder f-String zusammenbauen, sobald **irgendein** Wert aus Nutzereingabe, Datei oder Netz stammt. Immer `?`-Platzhalter und Tupel-Argument verwenden. Das kostet keine Zeile Code mehr und schützt dich und deine Nutzer.

12. CRUD im Überblick

Die vier grundlegenden Operationen auf Daten heißen oft nach ihren englischen Anfangsbuchstaben **CRUD: Create, Read, Update, Delete**. Jede fachliche Anforderung an eine Datenbank lässt sich letztlich auf diese vier Operationen (plus gezielte Abfragen mit `WHERE/ORDER BY`) zurückführen.

13. Maschinenbau-Beispiel: Eine Produktionsdatenbank

Damit die Theorie nicht abstrakt bleibt, denken wir uns eine kleine **Produktionsdatenbank** aus. Wir haben Tabellen für **Maschinen** (Drehmaschinen, Fräsen, 3D-Drucker), für **Produkte** (Zahnrad, Welle, Flansch) und für **Wartungen** (Wer? Wann? Was wurde gemacht?). Eine typische Frage an diese Datenbank wäre: „Welche Maschine hat im letzten Monat die meisten Werkstücke produziert?“ oder „Welche Maschinen haben seit über 500 Betriebsstunden keine Wartung mehr gesehen?“.

In V19 beschränken wir uns bewusst auf **eine einzelne Tabelle** (unsere **Produkte**-Tabelle aus `produkte.csv`), damit wir den CRUD-Kern sauber üben können. Die Welt der Joins, Fremdschlüssel-Constraints und Normalformen folgt in V20.

Vorschau: Tabellen-Schema der Übungsdaten

Die Datei `testdaten/produkte.csv` enthält sechs Produkte mit Produktionszeit und Materialbedarf. Die Tabelle, die wir gleich anlegen, sieht so aus:

Spalte	Typ	Constraint	Bedeutung
Produkt_ID	INTEGER	PRIMARY KEY	Eindeutige Identität
Produktname	TEXT	NOT NULL	Bezeichnung des Produkts
Produktionszeit_Minuten	INTEGER	–	Zeit pro Stück in Minuten
Material_pro_Stueck_kg	REAL	–	Materialbedarf in kg

14. Mini-Demo: SQL in wenigen Zeilen

Die folgende Zelle zeigt das komplette CRUD-Spektrum in etwa zehn Zeilen Code. Lies sie entspannt, jede Zeile dürfte dir mit dem bisherigen Wissen sofort einleuchten.

15. Selbst-Check – kurze Verständnis-Fragen

Wodurch unterscheidet sich eine Datenbank von einer CSV-Datei?

Eine Datenbank erzwingt ein **festes Schema**, bietet **Integritätsbedingungen** wie **PRIMARY KEY**, erlaubt **ausdrucksstarke Abfragen** per SQL und ermöglicht **gleichzeitigen Zugriff** durch mehrere Programme. Eine CSV-Datei ist dagegen nur eine unstrukturierte Text-Datei ohne jede dieser Garantien.

Was ist ein Primärschlüssel, was ein Fremdschlüssel?

Ein **Primärschlüssel** identifiziert jede Zeile einer Tabelle eindeutig. Ein **Fremdschlüssel** ist eine Spalte, deren Werte auf die Primärschlüssel-Werte einer anderen Tabelle verweisen und so eine Beziehung herstellen.

Warum ist `SELECT * FROM ...` in produktivem Code problematisch?

Sobald die Tabelle um Spalten erweitert oder reduziert wird, ändert sich stillschweigend auch das Ergebnis der Query. Code, der auf eine bestimmte Reihenfolge oder Anzahl der Spalten vertraut, bricht plötzlich oder liefert falsche Daten. Besser: die benötigten Spalten **explizit** auflisten.

Was passiert bei einem `UPDATE` ohne `WHERE`-Klausel?

Ohne `WHERE` gilt das `UPDATE` für **alle** Zeilen der Tabelle. Jede Zeile wird mit den angegebenen Werten überschrieben. Ohne Backup ist das in der Regel nicht rückgängig zu machen.

Wie heißt die einzige sichere Methode, Nutzereingaben in eine SQL-Query einzubauen?

Parameter-Binding mit `?`-Platzhaltern. Die Werte werden als Tupel im zweiten Argument von `cursor.execute(sql, params)` übergeben und vom Datenbank-Treiber sicher escaped. Nie per String-Konkatenation, f-String oder `%`-Operator zusammenbauen.

Wann solltest du SQLite statt PostgreSQL wählen?

Für **eingebettete** Anwendungen, **Lernumgebungen**, **Desktop-Tools** und **Prototypen** mit einem einzelnen oder wenigen gleichzeitigen Nutzern. Sobald viele gleichzeitige Schreiber, Netzwerk-Zugriff oder feingranulare Rechte nötig sind, ist ein Server-DBMS wie PostgreSQL die bessere Wahl.

Was bedeutet `conn.commit()`?

Alle seit dem letzten Commit ausgeführten Änderungen werden **dauerhaft** in die Datenbank geschrieben. Ohne Commit bleiben sie innerhalb der aktuellen

Transaktion und sind für andere Verbindungen unsichtbar. Bei In-Memory-Datenbanken ist das Commit oft verzichtbar, bei Datei-DBs ist es Pflicht, damit die Änderungen den Prozess überleben.

Welche fünf SQL-Befehle bilden den Kern dieser Vorlesung?

CREATE TABLE, INSERT, SELECT, UPDATE und DELETE. Zusammen bilden sie die **CRUD**-Operationen (Create, Read, Update, Delete).

Zusammenfassung

- Eine **relationale Datenbank** organisiert Daten in Tabellen mit festen Spalten, Typen und Integritätsbedingungen.
- **SQLite** ist eine Datei-basierte Embedded-Datenbank, ideal für Lernen, Prototypen und kleine Tools.
- Die **fünf SQL-Kernbefehle** sind CREATE TABLE, INSERT, SELECT, UPDATE, DELETE.
- **Parameter-Binding** mit `?` ist die einzige sichere Methode gegen **SQL-Injection**.
- `sqlite3.connect` → `cursor()` → `execute(sql, params)` → `fetchall()/fetchone()` ist das Grundmuster in Python.

Nächster Schritt

Weiter mit 02_praxis.ipynb – dort setzt du alles an einer In-Memory-Datenbank um.

Datei: lessons/V20-Datenbanken-Teil2/01_theorie.ipynb

V20 – Datenbanken Teil 2: Theorie

Lernziele

Nach diesem Notebook kannst du ... - die Grundidee der **Normalisierung** (1NF, 2NF, 3NF) erklären und Redundanzen erkennen, - **Fremdschlüssel** sinnvoll einsetzen und **ON DELETE**-Konsequenzen abschätzen, - **INNER JOIN** und **LEFT JOIN** richtig anwenden und den Unterschied benennen, - **GROUP BY** zusammen mit den Aggregatfunktionen **COUNT**, **SUM**, **AVG**, **MIN**, **MAX** lesen und schreiben, - den Unterschied zwischen **WHERE** und **HAVING** sauber erklären, - einfache **Subqueries**, **Views** und **Indizes** erkennen und verstehen, wofür sie gut sind, - **Transaktionen** mit **BEGIN/COMMIT/ROLLBACK** einsetzen und die **ACID**-Eigenschaften benennen.

Zeitbudget

Ca. 35 Minuten.

TL;DR

In V19 haben wir einzelne Tabellen erzeugt und abgefragt. Jetzt verbinden wir **mehrere** Tabellen über Fremdschlüssel, fassen Ergebnisse mit **GROUP BY** zusammen und lernen Werkzeuge kennen, mit denen echte Produktions-Datenbanken arbeiten: Views, Indizes und Transaktionen.

Voraussetzungen

- V19 (Tabellen, **SELECT/INSERT/UPDATE/DELETE**, **WHERE**, Parameter-Binding).
- 00_python_recap.ipynb dieser Vorlesung.

1. Wo wir stehen – und was jetzt kommt

In V19 hast du mit **einer** Tabelle gearbeitet – zum Beispiel **Produkte** mit **Produkt_ID**, **Name**, **Produktionszeit**. Alle Informationen zu einem Produkt lagen in einer einzigen Zeile, und die Welt war einfach.

In der Praxis hat man aber nie nur eine Tabelle. Jede Maschine hat viele Wartungen, jede Wartung gehört zu einem Techniker, jeder Techniker zu einer Abteilung. Wenn wir diese Information in eine einzige Tabelle pressen, entstehen **Redundanzen**: derselbe Techniker-Name steht hundertfach in der Wartungs-Tabelle, und wenn er heiratet, müssen wir hundert Zeilen ändern. Genau dort setzt dieses Notebook an.

2. Normalisierung – der Grundgedanke

Normalisierung ist die systematische Zerlegung einer großen, breiten Tabelle in mehrere kleine, schmalere Tabellen, sodass jede Information nur einmal gespeichert wird. Die drei üblichen Normalformen 1NF, 2NF und 3NF bauen aufeinander auf und wurden in den 1970er Jahren von Edgar F. Codd formuliert, dem Erfinder des relationalen Modells.

Wir schauen sie nur kurz an, damit du die Grundidee erkennst: **Jede Information gehört genau an einen Ort**. Wer das einhält, hat später weniger Ärger mit Inkonsistenzen, Tippfehlern und vergessenen Änderungen.

2.1 Erste Normalform (1NF)

Eine Tabelle ist in **1NF**, wenn jede Zelle genau **einen atomaren Wert** enthält. Listen oder komma-getrennte Felder wie "Teil A, Teil B, Teil C" in einer einzigen Spalte sind verboten; sie müssen in eigene Zeilen oder eigene Tabellen ausgelagert werden. Der Hintergedanke ist, dass man sonst keine saubere **WHERE**-Suche nach einzelnen Teilen formulieren kann.

2.2 Zweite Normalform (2NF)

Eine Tabelle ist in **2NF**, wenn sie zusätzlich zur 1NF die Eigenschaft erfüllt, dass jedes Nicht-Schlüssel-Attribut vom **gesamten** Primärschlüssel abhängt – nicht nur von einem Teil davon. Diese Forderung greift vor allem, wenn der Primärschlüssel aus mehreren Spalten zusammengesetzt ist.

Beispiel: In einer Tabelle **Bestellung** (**Bestell_ID**, **Produkt_ID**, **Produktname**, **Menge**) wäre **Produktname** bereits durch **Produkt_ID** allein festgelegt. Solche „teil-abhängigen“ Spalten gehören in eine eigene Tabelle **Produkte**.

2.3 Dritte Normalform (3NF)

Eine Tabelle ist in **3NF**, wenn sie in 2NF ist und zusätzlich keine **transitiven Abhängigkeiten** enthält. Konkret heißt das: Ein Nicht-Schlüssel-Attribut darf nicht indirekt über ein anderes Nicht-Schlüssel-Attribut vom Schlüssel abhängen.

Stünde in **Maschinen** (**Maschinen_ID**, **Name**, **Abteilung**, **Abteilungsleiter**) der **Abteilungsleiter** nur deshalb drin, weil er zur **Abteilung** gehört, dann ist das eine transitive Abhängigkeit: **Maschinen_ID** $\rightarrow$ **Abteilung** $\rightarrow$ **Abteilungsleiter**. Die Lösung: eine eigene Tabelle **Abteilungen** (**Abteilung**, **Abteilungsleiter**).

[!NOTE] **Faustregel:** 3NF ist für unsere Zwecke „gut genug“. Es gibt noch Boyce-Codd-Normalform und 4NF/5NF – für den Ingenieur-Alltag reicht 3NF in über 95 % der Fälle.

3. Fremdschlüssel (Foreign Key)

Sobald wir mehrere Tabellen haben, brauchen wir Verweise zwischen ihnen. Das ist die Aufgabe eines **Fremdschlüssels**. Ein Fremdschlüssel ist eine Spalte (oder Spaltenkombination) in einer Tabelle A, deren Werte auf den **Primärschlüssel** einer Tabelle B verweisen.

Durch diesen Mechanismus stellt die Datenbank **referentielle Integrität** sicher: Man kann nicht versehentlich eine Wartung für Maschine 99 eintragen, wenn es Maschine 99 gar nicht gibt. Der Fremdschlüssel wird mit **FOREIGN KEY (spalte) REFERENCES ziel_tabelle(ziel_spalte)** deklariert.

[!WARNING] SQLite prüft Fremdschlüssel standardmäßig **nicht!** Erst nach **PRAGMA foreign_keys = ON;** pro Verbindung werden die Einschränkungen wirklich durchgesetzt. Für unsere Notebooks ist das oft nebensächlich, aber in produktiven Anwendungen ist es Pflicht.

3.1 ON DELETE – was passiert, wenn die Quelle verschwindet?

Beim Anlegen des Fremdschlüssels kann man festlegen, wie die Datenbank reagiert, wenn eine referenzierte Zeile gelöscht wird. Die häufigsten Varianten sind `ON DELETE CASCADE` (alle abhängigen Zeilen werden mitgelöscht), `ON DELETE SET NULL` (die Fremdschlüssel-Spalte wird auf `NULL` gesetzt) und `ON DELETE RESTRICT` (das Löschen wird abgelehnt, solange noch etwas referenziert). Welches Verhalten sinnvoll ist, hängt von der Domäne ab: Für Wartungen zu einer verschrotteten Maschine ist `CASCADE` meist richtig, für Rechnungen zu einem Kunden meist nicht.

4. JOINS – mehrere Tabellen zusammenführen

Ein **JOIN** ist die SQL-Operation, die Zeilen zweier Tabellen anhand einer Bedingung kombiniert – fast immer entlang eines Fremdschlüssels. Das Ergebnis ist wieder eine Tabelle, in der Spalten beider Quelltabellen nebeneinander stehen.

Die beiden wichtigsten Varianten sind `INNER JOIN` und `LEFT JOIN`. Sie unterscheiden sich nur in einer einzigen, aber entscheidenden Frage: **Was passiert mit Zeilen, für die es keinen Partner in der anderen Tabelle gibt?**

4.1 INNER JOIN

Ein `INNER JOIN` liefert **nur** die Kombinationen, bei denen in beiden Tabellen ein passender Eintrag existiert. Alles ohne Gegenstück fällt weg. Anschaulich: Es ist die **Schnittmenge** der beiden Tabellen entlang der Join-Bedingung.

Syntaktisch ist ein `INNER JOIN` eine Erweiterung von `FROM`:

```
SELECT M.Name, W.Wartungstyp, W.Kosten
FROM Maschinen M
INNER JOIN Wartungen W ON M.Maschinen_ID = W.Maschinen_ID;
```

Hier werden für jede Kombination aus passender Maschine und Wartung eine neue Ergebniszeile gebildet. Die Aliase `M` und `W` machen den Code nur kürzer lesbar, sie sind nicht verpflichtend.

4.2 LEFT JOIN

Ein `LEFT JOIN` liefert **alle** Zeilen der linken Tabelle. Wo rechts kein passender Eintrag existiert, werden die rechten Spalten mit `NULL` aufgefüllt. Damit kann man zum Beispiel beantworten: „*Welche Maschinen haben noch nie eine Wartung erhalten?*“ – das wäre mit `INNER JOIN` gar nicht möglich, weil genau diese Maschinen wegfallen würden.

```
SELECT M.Name, W.Wartungstyp
FROM Maschinen M
```

```
LEFT JOIN Wartungen W ON M.Maschinen_ID = W.Maschinen_ID
WHERE W.Wartung_ID IS NULL;
```

Der Trick hier ist die WHERE W.Wartung_ID IS NULL-Klausel: Sie filtert gezielt jene Zeilen heraus, für die der LEFT JOIN kein Gegenstück gefunden hat. Dieses Muster werden wir in Aufgabe 5 verwenden.

4.3 RIGHT JOIN und FULL OUTER JOIN – nur konzeptuell

Ein RIGHT JOIN ist spiegelbildlich zum LEFT JOIN: Er behält alle Zeilen der rechten Tabelle. Da man jeden RIGHT JOIN durch Vertauschen der Tabellen als LEFT JOIN schreiben kann, wird er in der Praxis selten verwendet.

Ein FULL OUTER JOIN behält Zeilen aus **beiden** Seiten, auch wenn sie keinen Partner haben. SQLite unterstützt diesen erst seit Version 3.39 (Juni 2022). In MySQL fehlt er bis heute und muss durch UNION zweier JOINS emuliert werden.

5. GROUP BY und Aggregatfunktionen

Bisher haben wir immer einzelne Zeilen zurückbekommen. GROUP BY ändert das fundamental: Die Datenbank bildet **Gruppen** von Zeilen mit gleichem Wert in einer Gruppierungs-Spalte und liefert **pro Gruppe eine Ergebniszeile**. Zusammen mit den **Aggregatfunktionen** COUNT, SUM, AVG, MIN, MAX kann man so Kennzahlen pro Gruppe berechnen.

5.1 Die fünf wichtigsten Aggregatfunktionen

Funktion	Bedeutung
COUNT(*)	Anzahl Zeilen in der Gruppe
SUM(x)	Summe der Werte in Spalte x
AVG(x)	arithmetischer Mittelwert
MIN(x)	kleinster Wert
MAX(x)	größter Wert

Diese Funktionen verlangen, dass jede **andere** Spalte in der SELECT-Liste entweder in GROUP BY auftaucht oder selbst aggregiert ist. Sonst ist unklar, welchen Wert die Datenbank liefern soll.

5.2 Ein komplettes GROUP-BY-Beispiel

```
SELECT Wartungstyp, COUNT(*) AS anzahl, AVG(Kosten) AS mittel
FROM Wartungen
GROUP BY Wartungstyp
ORDER BY anzahl DESC;
```

Diese Abfrage liefert für jeden Wartungstyp die Anzahl der Einträge und die durchschnittlichen Kosten. **AS anzahl** vergibt einen **Alias**, damit man die Spalte im **ORDER BY** – und später in Python – bequem ansprechen kann.

6. WHERE vs. HAVING

WHERE und **HAVING** filtern beide Zeilen heraus, arbeiten aber an **unterschiedlichen Stellen** der Ausführungskette:

- **WHERE** filtert **vor** der Gruppierung. Es sieht einzelne Original-Zeilen und kann keine Aggregate nutzen.
- **HAVING** filtert **nach** der Gruppierung. Es sieht bereits berechnete Gruppen mit ihren Aggregaten und darf auf **SUM**, **COUNT** etc. zugreifen.

6.1 Konkret

```
-- Nur Wartungen aus dem Jahr 2024, dann gruppieren, dann nur
-- Maschinen mit Gesamtkosten > 1000 behalten:
SELECT Maschinen_ID, SUM(Kosten) AS gesamt
FROM Wartungen
WHERE Datum LIKE '2024-%'           -- Einzelzeilen-Filter
GROUP BY Maschinen_ID
HAVING gesamt > 1000;                -- Gruppen-Filter
```

Wer eine aggregierte Bedingung in **WHERE** schreibt, bekommt von SQLite einen Fehler: Die Aggregate existieren an dieser Stelle noch gar nicht.

[!WARNING] Die Versuchung, einfach immer **HAVING** zu nehmen, ist groß – es scheint ja „mehr zu können“. Tu das nicht: **WHERE** wird **vor** der Gruppierung ausgeführt und kann Indizes nutzen. Das ist bei großen Tabellen ein Riesenunterschied in der Laufzeit.

7. Subqueries – Abfragen in Abfragen

Eine **Subquery** ist ein **SELECT**, das innerhalb einer anderen SQL-Anweisung steht. Subqueries erlauben es, ein Zwischenergebnis zu berechnen und direkt weiterzuverwenden, ohne es in einer temporären Tabelle zu speichern.

7.1 WHERE ... IN (SELECT ...)

```
SELECT Name
FROM Maschinen
WHERE Maschinen_ID IN (
    SELECT Maschinen_ID FROM Wartungen WHERE Kosten > 1000
);
```

Die innere Abfrage liefert eine Liste von IDs, die äußere filtert darauf. Gleich nützlich lässt sich das Muster auch mit **NOT IN** einsetzen – etwa um Maschinen zu finden, zu denen es **keine** Wartung mit teurer Reparatur gibt.

7.2 Skalare Subquery

Eine **skalare Subquery** liefert genau einen einzelnen Wert und kann überall stehen, wo ein Wert erwartet wird – etwa im **SELECT** oder in einer **WHERE**-Bedingung:

```
SELECT Name
FROM Maschinen
WHERE Maschinen_ID = (
    SELECT Maschinen_ID FROM Wartungen
    ORDER BY Kosten DESC LIMIT 1
);
```

Das liefert die Maschine mit der **teuersten einzelnen** Wartung. Wichtig: Liefert die innere Abfrage mehr als eine Zeile, bricht SQLite mit einem Fehler ab.

8. Views – gespeicherte Abfragen

Ein **View** ist eine unter einem Namen abgelegte **SELECT**-Abfrage. Aus Sicht des Nutzers verhält er sich wie eine Tabelle: Man kann **SELECT * FROM view_name** schreiben, obwohl hinter den Kulissen jedes Mal die gespeicherte Abfrage ausgeführt wird.

Views sind nützlich, um oft gebrauchte Abfragen nicht ständig neu zu tippen und um komplexe SQL vor den Endanwendern zu verbergen. Sie werden mit **CREATE VIEW name AS SELECT ...** angelegt.

```
CREATE VIEW Wartungsuebersicht AS
SELECT M.Name, W.Wartungstyp, W.Kosten
FROM Maschinen M
INNER JOIN Wartungen W ON M.Maschinen_ID = W.Maschinen_ID;
```

```
-- Jetzt kann man einfach abfragen:
SELECT * FROM Wartungsuebersicht WHERE Kosten > 500;
```

9. Indizes – Beschleuniger für große Tabellen

Ein **Index** ist eine zusätzliche Datenstruktur, die die Datenbank anlegt, damit bestimmte Abfragen schneller werden. Ohne Index muss die DB bei einer **WHERE**-Suche jede Zeile anschauen (**Full Table Scan**); mit einem passenden Index – meist ein B-Baum – findet sie die gesuchten Zeilen in logarithmischer Zeit.

Der Primärschlüssel erhält automatisch einen Index. Für zusätzliche Spalten, nach denen häufig gefiltert oder gejoint wird, kann man mit **CREATE INDEX** nachrüsten:

```
CREATE INDEX idx_wartungen_maschine ON Wartungen(Maschinen_ID);
```

Indizes sind aber nicht kostenlos: Sie brauchen Speicher und müssen bei jedem INSERT/UPDATE/DELETE mitgepflegt werden. Für kleine Tabellen oder reine Lesetabellen ist das meist egal; bei Millionen Zeilen wird es zur Designfrage.

10. Transaktionen und ACID

Eine **Transaktion** bündelt mehrere SQL-Anweisungen zu einer logischen Einheit, die entweder **komplett** ausgeführt oder **gar nicht** wirksam wird. Das ist zum Beispiel beim Umbuchen von Geld zwischen zwei Konten lebenswichtig: Entweder wird bei A abgebucht **und** bei B gutgeschrieben – oder nichts passiert.

10.1 Die ACID-Eigenschaften

- **Atomicity (Atomarität):** Alle Anweisungen der Transaktion wirken zusammen oder gar nicht.
- **Consistency (Konsistenz):** Nach der Transaktion ist die DB wieder in einem gültigen Zustand (alle Constraints erfüllt).
- **Isolation (Isolation):** Parallele Transaktionen beeinflussen sich nicht sichtbar – jede sieht eine konsistente Version der Daten.
- **Durability (Dauerhaftigkeit):** Nach COMMIT sind die Änderungen auch einen Stromausfall später noch da.

10.2 BEGIN / COMMIT / ROLLBACK

In SQL startet man eine Transaktion mit **BEGIN**, beendet sie erfolgreich mit **COMMIT** oder verwirft sie mit **ROLLBACK**. In Python mit `sqlite3` ist das meist eleganter über den `with`-Kontext der Verbindung: Bei erfolgreichem Block-Ende wird automatisch committet, bei einer Exception automatisch zurückgerollt.

11. Maschinenbau-Beispiel – alles auf einmal

Angenommen, die Fertigungsleitung fragt: „*Welche Maschine hat uns im letzten Jahr am meisten Wartungskosten verursacht?*“ Die passende SQL kombiniert genau die Bausteine, die wir gerade kennengelernt haben:

```
SELECT M.Name, SUM(W.Kosten) AS gesamt
FROM Maschinen M
INNER JOIN Wartungen W ON M.Maschinen_ID = W.Maschinen_ID
GROUP BY M.Maschinen_ID
ORDER BY gesamt DESC
LIMIT 1;
```

Der **INNER JOIN** holt Name und Wartungskosten zusammen, **GROUP BY** macht aus vielen Wartungszeilen pro Maschine eine einzige Zeile, **SUM** addiert die Kosten, **ORDER BY ... DESC LIMIT 1** liefert den teuersten Fall. **Dieses Muster** (**JOIN** → **GROUP BY** → **ORDER BY** → **LIMIT**) wirst du in Aufgabe 4 wiedersehen.

12. Selbstcheck

Wie unterscheiden sich `INNER JOIN` und `LEFT JOIN`?

`INNER JOIN` liefert nur Zeilen, die in beiden Tabellen Partner haben. `LEFT JOIN` behält **alle** Zeilen der linken Tabelle; fehlt rechts ein Partner, stehen dort `NULL`-Werte.

Warum darf man `COUNT(*) > 5` nicht in `WHERE` schreiben?

`WHERE` filtert einzelne Zeilen **vor** der Gruppierung, da existiert `COUNT(*)` noch gar nicht. Solche Bedingungen gehören in `HAVING`, das nach `GROUP BY` greift.

Was bringt ein Index auf einer Spalte?

Er beschleunigt `WHERE`-Suchen und `JOINS` auf dieser Spalte von linearer zu logarithmischer Komplexität, kostet aber Speicher und macht Schreiboperationen etwas langsamer.

Wozu dient ein Fremdschlüssel?

Er stellt sicher, dass jeder Verweis in einer Tabelle auf einen existierenden Eintrag in einer anderen Tabelle zeigt – das nennt man referentielle Integrität.

Was bedeutet die „A“ in ACID?

Atomarität: Eine Transaktion wird entweder komplett wirksam oder gar nicht – niemals halb.

Wann nehme ich einen View?

Wenn dieselbe komplizierte Abfrage öfter gebraucht wird und man sie unter einem sprechenden Namen einmal definieren möchte, statt sie überall neu zu tippen.

Warum braucht man die 3NF?

Sie vermeidet transitive Abhängigkeiten. Dadurch verhindert sie, dass dieselbe Information (z. B. der Abteilungsleiter) in mehreren Zeilen auftaucht und bei Änderungen inkonsistent wird.

Was macht `HAVING gesamt > 1000` nach `GROUP BY`?

Es behält nur jene Gruppen, deren aggregierter Wert **gesamt** größer als 1000 ist; alle anderen Gruppen verschwinden aus dem Ergebnis.

[!TIP] **Weiterführend:** Für sehr große Analysen setzt man statt Subqueries häufig **CTEs** (`WITH name AS (...)`) ein, weil sie lesbarer sind. SQLite unterstützt CTEs seit Version 3.8. In Kurs und Praxis reichen Subqueries aber völlig aus.

Zusammenfassung

- **Normalisierung** bis 3NF vermeidet Redundanz und Inkonsistenz.

- **Fremdschlüssel** verknüpfen Tabellen; `ON DELETE CASCADE/SET NULL/RESTRICT` steuert die Reaktion auf Löschungen.
- `INNER JOIN` = Schnittmenge, `LEFT JOIN` = alles links + NULLs rechts.
- `GROUP BY` plus Aggregate `COUNT/SUM/AVG/MIN/MAX` liefern Kennzahlen pro Gruppe.
- `WHERE` filtert Zeilen, `HAVING` filtert Gruppen – der Unterschied ist die Ausführungsreihenfolge.
- Subqueries, Views und Indizes sind Werkzeuge der echten Praxis.
- Transaktionen (`BEGIN/COMMIT/ROLLBACK`) garantieren ACID.

Nächster Schritt

Weiter mit 02_praxis.ipynb.
